Scientific Data Analysis and
Performance Optimization in Python

Pythonによる
データ解析と高速化

Ichiro KOMAE

2026/10/24

目次

はじめに ・・・

3.5 章末課題：保存したプログラムを実行する ・・・・・・・・・・・・・・・・・・・・ 36
第 4 章 Python の基本 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 37
4.1 データ型と基本操作 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 37
4.2 リスト・タプル・辞書・集合 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 41
4.3 条件分岐と反復処理 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 44
4.4 関数とモジュール ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 48
4.5 例外・docstring・命名・設定値 ・・・・・・・・・・・・・・・・・・・・・・・・・・ 52
4.6 章末課題：記録を選び、平均と時刻差を求める ・・・・・・・・・・・・・・・・・・ 55
第 5 章 入出力とファイル ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 57
5.1 テキストファイルの入出力と探索 ・・・・・・・・・・・・・・・・・・・・・・・・・ 57
5.2 文字列データの解析と整形 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 61
5.3 オブジェクトと構造化データの保存 ・・・・・・・・・・・・・・・・・・・・・・・・ 63
5.4 日時データの変換と保存形式の選択 ・・・・・・・・・・・・・・・・・・・・・・・・ 64
5.5 章末課題：CSV を読み、要約を保存する ・・・・・・・・・・・・・・・・・・・・・ 66
第 6 章 NumPy による数値計算 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 69
6.1 配列を作って間隔を求める ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 69
6.2 配列の生成・演算・集計 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 71
6.3 型・軸・ビューに関する注意点 ・・・・・・・・・・・・・・・・・・・・・・・・・・ 78
6.4 差分・ヒストグラム・線形代数への応用 ・・・・・・・・・・・・・・・・・・・・・ 82
6.5 章末課題：発生間隔を計算して集計する ・・・・・・・・・・・・・・・・・・・・・ 85
第 7 章 pandas による表形式データ解析 ・・・・・・・・・・・・・・・・・・・・・・・・ 87
7.1 DataFrame の構造と列指向処理 ・・・・・・・・・・・・・・・・・・・・・・・・・ 87
7.2 抽出・変換・集計・入出力 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 89
7.3 コピー・型・行処理に関する注意点 ・・・・・・・・・・・・・・・・・・・・・・・ 98
7.4 観測表の集計と可視化への受け渡し ・・・・・・・・・・・・・・・・・・・・・・・ 101
7.5 章末課題：検出器ごとの信号を集計する ・・・・・・・・・・・・・・・・・・・・・ 102
第 8 章 Polars による式ベースの表処理 ・・・・・・・・・・・・・・・・・・・・・・・・ 103
8.1 式と遅延実行による表処理 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 103
8.2 列変換・集計・結合・入出力 ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 105
8.3 pandas との違いと遅延実行の注意点 ・・・・・・・・・・・・・・・・・・・・・・・ 111
8.4 大規模表処理と可視化への受け渡し ・・・・・・・・・・・・・・・・・・・・・・・ 112
8.5 章末課題：表の処理方法と結果を比較する ・・・・・・・・・・・・・・・・・・・・ 112
第 9 章 Matplotlib による可視化 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 113
9.1 線図・散布図・ヒストグラムの基本 ・・・・・・・・・・・・・・・・・・・・・・・ 113
9.2 分布・相関・密度に応じた図の選択 ・・・・・・・・・・・・・・・・・・・・・・・ 119
9.3 オブジェクト指向 API による複雑な図の管理 ・・・・・・・・・・・・・・・・・・ 127
9.4 ラベル・凡例・色・スタイルの管理 ・・・・・・・・・・・・・・・・・・・・・・・ 132

9.5 図中の数式表記 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 136
9.6 pyplot と Axes・カテゴリ軸・画像保存の注意点 ・・・・・・・・・・・・・・・・・ 137
9.7 章末課題：分布の特徴が伝わる図を作る ・・・・・・・・・・・・・・・・・・・・・・ 138
第 10 章 SciPy による数値計算とフィット ・・・・・・・・・・・・・・・・・・・・・・・・ 141
10.1 データに直線を当てはめる ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 141
10.2 分布と要約統計 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 143
10.3 モデル選択・重み付け・残差評価 ・・・・・・・・・・・・・・・・・・・・・・・・ 144
10.4 モデル・制約・外挿に関する注意点 ・・・・・・・・・・・・・・・・・・・・・・・ 150
10.5 発生間隔分布のモデル比較 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 150
10.6 章末課題：モデルを当てはめて妥当性を確かめる ・・・・・・・・・・・・・・・・・ 152
第 11 章 Astropy による時刻・単位・天体データ処理 ・・・・・・・・・・・・・・・・・・ 153
11.1 数値と単位・時刻・座標の一元的な管理・・・・・・・・・・・・・・・・・・・・・・ 153
11.2 単位・時刻・座標・表・FITS の操作・・・・・・・・・・・・・・・・・・・・・・・ 154
11.3 単位情報・時刻系・座標系を失わないための注意 ・・・・・・・・・・・・・・・・・ 159
11.4 SciPy・pandas とのデータ受け渡し・・・・・・・・・・・・・・・・・・・・・・・・ 160
11.5 章末課題：単位と時刻の意味を保って計算する ・・・・・・・・・・・・・・・・・・ 160
第 12 章 自作モジュールの作成と再利用 ・・・・・・・・・・・・・・・・・・・・・・・・ 163
12.1 平均の関数を別のファイルから使う ・・・・・・・・・・・・・・・・・・・・・・・ 163
12.2 解析処理を再利用可能な部品へ分ける理由 ・・・・・・・・・・・・・・・・・・・・ 164
12.3 引数・設定ファイルとコードを分ける ・・・・・・・・・・・・・・・・・・・・・・ 164
12.4 uv によるパッケージ開発の初期化 ・・・・・・・・・・・・・・・・・・・・・・・ 166
12.5 具体例: ヒストグラムとフィットをまとめたパッケージ ・・・・・・・・・・・・・ 167
12.6 docstring、型ヒント、README・・・・・・・・・・・・・・・・・・・・・・・・・ 176
12.7 uv pip による通常インストールと editable install ・・・・・・・・・・・・・・・ 178
12.8 Git リポジトリによるパッケージの共有 ・・・・・・・・・・・・・・・・・・・・・ 179
12.9 実行時依存関係と開発用依存関係・・・・・・・・・・・・・・・・・・・・・・・・ 179
12.10 章末課題：平均を求めるパッケージを作る ・・・・・・・・・・・・・・・・・・・ 180
第 13 章 Python 解析の高速化 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 181
13.1 処理時間の基準測定 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 181
13.2 計算量の見積もり ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 182
13.3 ベクトル化・型・逐次処理による改善 ・・・・・・・・・・・・・・・・・・・・・ 185
13.4 処理時間の内訳と発展的な改善 ・・・・・・・・・・・・・・・・・・・・・・・・ 187
13.5 高速化前後の比較と測定条件の報告 ・・・・・・・・・・・・・・・・・・・・・・ 190
13.6 章末課題：正しさと処理時間を比較する・・・・・・・・・・・・・・・・・・・・・ 191
第 14 章 総合課題：同時観測イベントの探索 ・・・・・・・・・・・・・・・・・・・・・ 193
14.1 入力データと実行環境の準備・・・・・・・・・・・・・・・・・・・・・・・・・・ 193
14.2 検証・修正・高速化の要件 ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 194

14.3 結果のまとめと発表 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 197
付録 A 実行環境の補足 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 199
A.1 既存の Python から仮想環境を作る ・・・・・・・・・・・・・・・・・・・・・・・・ 199
A.2 uv による環境の記録と再現 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 200
A.3 pyenv による複数バージョンの管理 ・・・・・・・・・・・・・・・・・・・・・・・・ 203
A.4 Notebook でコードと結果をまとめる ・・・・・・・・・・・・・・・・・・・・・・・ 207
A.5 演習：環境の作成方法と切り替えを比較する ・・・・・・・・・・・・・・・・・・・ 208
付録 B SSH, 鍵認証, ファイル転送 ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 209
B.1 サーバーへの SSH 接続と鍵認証 ・・・・・・・・・・・・・・・・・・・・・・・・・ 209
B.2 接続設定と GitHub 連携 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 211
B.3 scp・rsync によるファイル転送 ・・・・・・・・・・・・・・・・・・・・・・・・・ 212
付録 C Git と GitHub の基本 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 215
C.1 リポジトリ・ステージング・コミットの関係 ・・・・・・・・・・・・・・・・・・・ 215
C.2 ローカル履歴の作成 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 216
C.3 GitHub への接続 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 219
C.4 認証と最小ワークフロー ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 221
付録 D pybind11 による C++ 拡張モジュール ・・・・・・・・・・・・・・・・・・・・・ 223
D.1 C++ 拡張を選ぶ判断基準 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 223
D.2 pybind11 パッケージの最小構成 ・・・・・・・・・・・・・・・・・・・・・・・・・ 223
D.3 拡張モジュールのビルドとインストール ・・・・・・・・・・・・・・・・・・・・・ 226
D.4 Python からの C++ 関数の呼び出し ・・・・・・・・・・・・・・・・・・・・・・・ 227
D.5 C++ へ移す処理の選び方 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 227
付録 E 正規表現の基本 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 229
E.1 文字列パターンを表す正規表現 ・・・・・・・・・・・・・・・・・・・・・・・・・ 229
E.2 アンカー・文字クラス・量指定子 ・・・・・・・・・・・・・・・・・・・・・・・・ 230
E.3 run 名・時刻・数値ログの抽出 ・・・・・・・・・・・・・・・・・・・・・・・・・ 232
E.4 Python の re と grep/awk の使い分け ・・・・・・・・・・・・・・・・・・・・・・ 233
付録 F シェルスクリプトによる自動化と一括処理 ・・・・・・・・・・・・・・・・・・ 235
F.1 シェルスクリプトの基礎構造 ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 235
F.2 変数、演算、条件式 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 236
F.3 条件分岐 (if, case) ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 238
F.4 反復処理と一括自動化 (for) ・・・・・・・・・・・・・・・・・・・・・・・・・・ 239
F.5 sed による条件別設定ファイルの生成 ・・・・・・・・・・・・・・・・・・・・・・ 241
参考文献 ・・・ 243
索引 ・・ 247

はじめに

本書は、Python を使って自然科学のデータを解析するための入門書である。ファイルから測定値を読み、平均や発生間隔を計算し、結果を図にするところから始める。短い例を実行し、入力した値と表示された答えを見比べながら、処理の意味を学ぶ。

前半では、ファイルの場所を確認する操作、Python の準備、プログラムの書き方、データの読み書きを学ぶ。コードをどこへ保存し、どう実行するかも順に説明する。

中盤では、多数の数値をまとめて計算する NumPy、表を整理する pandas と Polars、図を作る Matplotlib を使う。さらに、SciPy でデータに合う式を求め、Astropy で単位や天体の座標を扱う。それぞれの道具が何を受け取り、どのような結果を返すかを確かめる。

後半では、同じ処理を別のデータにも使うために、関数を別ファイルへまとめる。計算結果を自動で確認する方法や、処理時間を測って改善する方法も学ぶ。最後の総合課題では、複数の観測装置がほぼ同時に記録した事象を探し、ここまでの内容を一つの解析へまとめる。

計算結果が正しいかを確かめることも、学習の一部である。まず手計算できる少数の値で試し、答えが合うことを確認してから、多くのデータへ進む。得られた値だけでなく、使ったデータと計算の手順を説明できるようになることを目指す。

Python の言語仕様と標準ライブラリの説明には、公式チュートリアル [1] と公式ライブラリリファレンス [2] を参照した。これらの資料は、各ライブラリの公式資料とともに巻末の参考文献にまとめた。

第 1 章

本書の構成と実行例の読み方

1.1 対象と到達目標

本書では、Python を使ってデータを読み、計算し、図を作るところから学ぶ。Python やターミナル操作が初めての読者は、前から順に読み、短い例を一つずつ実行する。既に解析コードを書いている読者は、必要な章から読み始めてもよい。

学習を進めると、次の作業ができるようになる。

- データが保存されている場所を確認し、プログラムを実行する。
- ファイルから数値や表を読み、平均や発生間隔を求める。
- 計算結果を図にし、ラベルや単位を付けて保存する。
- 小さな入力で答えを確かめ、同じ計算を別のデータにも使う。
- 正しい結果を保ちながら、処理時間とメモリ使用量を減らす。

後半の例は、それ以前の章で導入した用語とファイルを使う。分からない用語は索引から説明箇所を探せる。索引の太字のページ番号は主な説明箇所を、矢印は別の見出しへの参照を示す。

1.2 ファイルを開くところから解析までの順序

章は、実際の作業を次の順で進められるように並んでいる。

1. **ファイルの場所を確かめて Python を起動する**。シェルの章でフォルダーの移動とファイルの表示を試す。次の環境構築の章で Python を用意し、短いプログラムをファイルへ保存して実行する。
2. **短いプログラムを書いてデータを読む**。基本構文の章で値、条件分岐、反復、関数を学

ぶ。入出力の章では、ファイルの文字列を数値や表として読み込む。

3. **数値や表を使って計算し、図にする**。NumPy で配列を計算し、pandas と Polars で表を整理する。Matplotlib で図を作り、SciPy でモデルを当てはめる。Astropy では単位、時刻、天体の座標を扱う。
4. **同じ処理を繰り返し使い、処理時間を減らす**。関数を別のファイルへ分け、結果を自動で確認する方法を学ぶ。処理時間を測り、正しい答えを保ちながら計算方法を改善する。最後の総合課題では、これらを一つの解析へまとめる。

最初は、第 2 章から第 5 章までを順に進め、ファイルを読み書きする短いプログラムを実行できるようにする。その後は、扱うデータと目的に応じて各ライブラリの章へ進む。例えば、数値の列を計算して図にしたい場合は NumPy と Matplotlib、表を集計したい場合は pandas が出発点になる。Polars は別の表処理方法として、Astropy は単位や天体データを扱うときに参照できる。「補足」と「発展課題」は、基本の例を理解してから取り組む。

例えば、CSV に保存された数値の平均を求めたいときは、まずファイルの場所と内容を確認し、それから読み込んで計算する。結果だけを見て判断せず、入力が何件あり、どの値を計算に使ったかも確かめる。

1.3　コード例と実行結果の読み方

コード例の最初に、入力する場所を確認する。ターミナルに入力する命令と、Python に入力するコードは別である。ターミナルの例では行頭に `$` を付けるが、この印自体は入力しない。Python の例は、原則として拡張子が `.py` のテキストファイルへ保存して実行する。このファイルを Python スクリプトという。保存と実行の手順は第 3 章で説明する。

ここでは、コードと表示結果の区別を見ておく。次は、計算する内容を記したコードである。

```
1 count = 3
2 print(count + 2)
```

表示結果は次のようになる。

```
5
```

1 行目では、`3` という値に `count` という名前を付ける。2 行目では、その値に `2` を足し、`print()` で結果を画面に表示する。丸括弧の中に、表示したい値や計算式を書く。この例では `5` が表示されれば、同じ計算を実行できている。

`count = 3` を `count = 10` へ変えると、計算結果は `12` になる。ファイルを書き換えた場合は、保存してからもう一度実行し、表示が変わることを確認する。

第 4 章には、Python を起動して 1 行ずつ試す「対話モード」の例もある。この場合は入力行に `>>>`、続きの行に `...` を示す。どちらの印も入力しない。コードの左に付いた行番号も入力しない。

同じ変数を使う続きの例は、その変数を作ったコードの下へ追記して実行する。作業フォルダーは章ごとに分け、仮想環境は共通のものを使う。その仮想環境を有効にした端末で例を試す。表示結果や `$`、`>>>` の印はファイルに書かない。ファイルを読む例では、先に示すデータを指定した名前で保存する。図を作る例では、保存したファイルを開き、軸や値を確かめる。

パス、バージョン番号、処理時間は環境によって異なる。例えば `Python 3.12.14` と表示される箇所は、手元では別の番号になる場合がある。これらは番号や文字列全体の一致を求めず、各例で示した確認事項に沿って読む。

1.4　章末課題の進め方

章末課題では、その章の内容を、それ以前に学んだ操作と組み合わせて使う。前章のファイルを引き継ぐ課題では、指定された入力と作業ディレクトリを確認する。地震カタログの課題は、原則として第 2 章で作る `earthquake_practice` を作業ディレクトリとする。本文中の少数行の CSV は動作確認用の人工データであり、USGS の実際の観測値とは区別する。小さな検証用データで処理を確かめた後、配布データまたは読者自身のデータへ適用する流れが基本となる。

課題に示す数値の目安は、コードの誤りに気付くために使う。自分の答えと違うときは、使った入力と計算の手順を本文の例と比べる。図を提出する問題では、保存した図を開き、軸の名前と単位が付いていることや、どのデータを描いたかが分かることも確認する。

第 14 章の総合課題は、複数の章で学んだ処理を組み合わせる課題である。課題用スクリプトとデータの入手先は同章に記載した。作業順序は、配布コードによる入出力の把握、小規模データによる正しさの検証、大規模データの処理、性能改善である。

課題の結果は、使った入力、行った処理、結果の表や図、そこから分かったことの順にまとめる。発表ではコードの全行を読み上げるのではなく、選んだ方法と結果の意味を説明する。

1.5　入力・実行条件・結果の検証記録

エラーが出ずにプログラムが終わっても、意図した計算ができているとは限らない。各章では、次の点を確認する。

- 少数の値を使い、手計算の答えと表示結果を比べる。
- ファイルを読むときは、先頭の数行と件数を確認する。
- 図や計算方法を変えたときは、同じ入力で結果を比べる。

使った入力ファイル、実行した命令、表示結果や保存先をメモしておくと、後から同じ作業を繰り返せる。型、単位、計算条件など、その章で必要になる確認事項は、各例の説明で

扱う。

1.6　公式資料・付録・参考文献の参照方法

本文で取り上げるコマンドやライブラリの使い方は、ソフトウェアの更新によって変わる。本文では概念と代表的な使用例を説明する。指定できる項目の全一覧、関数が返す値、バージョンごとの違いを調べる場合は、公式資料を参照せよ。引用資料は巻末の参考文献にまとめた。

Python の言語仕様と標準ライブラリについては、Python 公式チュートリアル [1] と公式ライブラリリファレンス [2] を参照した。環境構築には uv の公式資料 [4]、主要な解析ライブラリには各プロジェクトの公式資料 [6–11] を用いた。

OS は、macOS、Windows、Linux など、コンピュータの基本的な動作を管理するソフトウェアである。同じ名前のシェルコマンドでも、OS やソフトウェアのバージョンによって書き方が異なる場合がある。手元のコンピュータに入っているコマンドの使い方を表示する方法は、第 2 章で具体例とともに説明する。付録は、環境の切り替え、別の計算機への接続、変更履歴の管理などが必要になったときに参照する。本文の基本例を進める前に、付録をすべて読む必要はない。

次章では、まずターミナルを開き、ファイルがどこにあるかを確認する。

第 2 章

シェルとファイル操作

この章では、作業するフォルダーへ移動し、ファイルの一覧と内容を確認する。まず `pwd`、`ls`、`cd` の役割を区別し、どの場所で操作しているかを確かめられるようにする。取得、圧縮、ログの保存は、その基本操作を使う例として学ぶ。

2.1 ターミナル・シェル・WSL の役割

本書では、ファイル操作やスクリプト実行をターミナルから行う。初めに、ターミナルとシェルの役割を区別する。

macOS では「ターミナル」を開いて以下の例を試す。Windows では、先に本節後半の WSL の手順を行い、Ubuntu の端末を開く。命令を入力したら Enter を押す。

- ターミナル: 文字で命令を入力するためのアプリケーション
- シェル: ターミナルの中で動き、入力されたコマンドを解釈して実行するプログラム

ターミナルを開くと、シェルはコマンドを入力できる位置に短い文字列を表示する。この文字列をプロンプトという。表示内容は環境によって異なるが、末尾には `$` が使われることが多い。例えば、実際の画面には次のように表示される。

```
student@host ~ $
```

この表示は、シェルが入力を待っていることを意味する。本書の実行例では、実際のプロンプト全体を省略し、読者が入力する行の先頭に `$` だけを置く。`$` は入力行を見分けるための印であり、読者が入力する文字には含まれない。

本書では、macOS の例に `zsh`、Linux と WSL 上の Ubuntu の例に `bash` を用いる。どちらも本章の基本コマンドを実行できる。

2.1.1 Windows における WSL の利用

本書のコマンド例は macOS や Linux のシェルを前提にしている。Windows では、WSL (Windows Subsystem for Linux) 上の Ubuntu などを利用すると、同じコマンド例を実行しやすい。WSL は、管理者権限で開いた PowerShell から次のコマンドで導入できる。PowerShell のプロンプトは、本書の UNIX シェル用プロンプトと区別して示す。

```
PS C:\> wsl --install
```

導入時に再起動を求められたら、再起動して Ubuntu を開き、画面の案内に従って利用者名とパスワードを設定する。その後の本書のコマンドは Ubuntu の端末へ入力する。

WSL 上では、本書の Linux 用コマンドとディレクトリ構造を Windows 上で利用できる。また、後半で扱う並列処理や周辺ツールには Linux 系環境を前提とするものがあるため、Windows のコマンドプロンプトや PowerShell とは実行環境を分けて説明する。

2.2 コマンド・オプション・引数とヘルプ

2.2.1 コマンド、オプション、引数

まず、今いるフォルダーの内容を詳しく表示する。

```
$ ls -l .
```

`ls` は実行するコマンド、`-l` は詳しい表示へ切り替えるオプション、`.` は対象を指定する引数である。`.` は現在のフォルダーを表す。このように、コマンドの後ろへ動作の指定や対象を続けて書く。表示される項目の読み方は、次の節で説明する。

2.2.2 説明書を開く

使い方を調べるには、`man` の後ろへコマンド名を書く。例えば次を実行すると、`ls` の説明書が開く。

```
$ man ls
```

`NAME` は名前と機能、`SYNOPSIS` は命令の書き方、`DESCRIPTION` は詳しい説明を示す。書式の角括弧 `[...]` は省略できる部分を表す。上下キーや Space で移動し、`q` で閉じる。

コマンドによっては、`--help` や `-h` を付けると短い使い方を表示できる。ただし、受け付ける指定はコマンドや OS によって異なる。手元の動作がインターネット上の例と違うときは、`man` などでその環境の説明を確認する。

2.3　現在位置・一覧・パス・ディレクトリ操作

2.3.1　現在位置の確認と項目の一覧: pwd と ls

シェルでは、現在作業しているディレクトリがパス解釈の基準となる。このディレクトリをカレントディレクトリという。現在位置は pwd、その場所にある項目は ls で確認できる。

```
$ pwd
$ ls
$ ls -l
$ ls -lh
$ ls -a
$ ls -lah
```

pwd は、現在のカレントディレクトリの絶対パスを表示する。ls は、そのディレクトリにあるファイルやディレクトリの一覧を表示する。たとえば pwd の結果が次のようであれば、

```
/Users/student/analysis
```

現在の作業場所は /Users/student/analysis である。この状態で ls を実行すると、その中にある項目が表示される。

```
$ ls
data    scripts
```

この結果から、/Users/student/analysis の中には data と scripts という 2 つのディレクトリがあることが分かる。

ls の表示内容はオプションで変更できる。-l は、各項目の種別、アクセス権、リンク数、所有者、グループ、サイズ、最終更新日時、名前を表示する。

```
$ ls -l
total 0
drwxr-xr-x  3 student  staff  96  4月 12 23:19 data
drwxr-xr-x  3 student  staff  96  4月 12 23:19 scripts
```

ls -l の先頭にある total は、表示対象へ割り当てられたファイルシステムブロック数の合計である。1 ブロックを表示上何バイトと数えるかは実装や環境変数で異なるため、ファイルごとのバイト数には各行のサイズ欄、ディスク使用量には du を用いる。

ls -l の出力では、その下の各行が 1 つのファイルまたはディレクトリに対応しており、左から順に次の情報が表示される。

- drwxr-xr-x：ファイル種別とアクセス権
- 3：リンク数
- student：所有者

- `staff`：グループ
- `96`：サイズ
- `4 月 12 23:19`：最終更新日時
- `data`：名前

先頭の 1 文字は種別を表し、`d` ならディレクトリ、`-` なら通常のファイルである。続く `rwxr-xr-x` はアクセス権を表している。アクセス権の詳しい意味は第 2.10.1 節で説明する。

リンク数は、同じ inode を参照するハードリンクの数を表す。通常ファイルを新規作成した直後は一般に 1 であり、`ln` でハードリンクを追加すると増える。ディレクトリのリンク数はファイルシステムとサブディレクトリ構造に依存し、ディレクトリ内の項目数そのものではない。

`ls -l` のサイズ欄はバイト単位である。`-h` を併用すると、同じ値が `K`、`M`、`G` などの接頭辞付きで表示される。表示の丸めを避けて正確なバイト数を記録する場合は、`-h` を付けない出力を用いる。

```
$ ls -lh
total 0
drwxr-xr-x  3 student  staff    96B  4月 12 23:19 data
drwxr-xr-x  3 student  staff    96B  4月 12 23:19 scripts
```

この例の `96B` はディレクトリ項目そのものの大きさであり、配下のファイルサイズの合計ではない。ディレクトリ以下のディスク使用量を調べる場合は、`du` を用いる。

通常のファイルに対して `ls -lh` を使うと、バイト数は `12K` や `3.4M` のように SI 接頭語に似た接尾辞付きで表示される。正確なバイト数が必要な場合は、OS に応じて `stat` または `wc -c` を用いる。

一方、`-a` オプションを付けると、ドットで始まる隠し項目も含めて表示できる。

```
$ ls -a
.       ..      .config data    scripts
```

名前が `.`（ドット）で始まるファイルやディレクトリは隠し項目として扱われ、通常の `ls` では表示されない。たとえば `.config`、`.gitignore`、`.bashrc` などがこれにあたる。設定ファイルなど、通常の一覧に常時表示する必要のない項目にこの命名が用いられる。

上の `ls -a` に現れる `.` と `..` にも意味がある。`.` は現在のディレクトリ自身を表し、`..` は 1 つ上の親ディレクトリを表す。これらは後でディレクトリ移動をするときにも重要になる。

`-lah` は、詳細表示の `-l`、隠し項目を含める `-a`、サイズに接尾辞を付ける `-h` を組み合わせた指定である。

```
$ ls -lah
total 96
drwxr-xr-x  5 student  staff   160B  4月 12 23:20 .
drwxr-xr-x  5 student  staff   160B  4月 12 23:18 ..
-rw-r--r--  1 student  staff    47K  4月 12 23:22 .config
drwxr-xr-x  3 student  staff    96B  4月 12 23:19 data
```

```
drwxr-xr-x  3 student  staff    96B  4月 12 23:19 scripts
```

この例では `.config` という通常のファイルも表示されており、`ls -l` の場合とは `total` の値が異なる。`total` は各行のサイズ欄の合計ではなく、表示対象へ割り当てられたファイルシステムブロック数に基づく。

`pwd` は現在位置、`ls` は項目名、`ls -l` は権限や更新時刻を含む詳細、`ls -a` は隠し項目を含む一覧を表示する。`-h` を `-l` と組み合わせると、ファイル容量が単位付きで表示される。必要な情報に応じて、これらのオプションを選択する。

2.3.2　ディレクトリの移動: `cd`

`cd` はカレントディレクトリを変更するコマンドである。`ls`、`cat`、`cp` などへ相対パスを渡すと、そのパスはカレントディレクトリを基準に解釈される。次のディレクトリ構成を例に、移動前後でパスの指す対象を確認する。

```
analysis/
├── data/
│   └── events.txt
└── scripts/
    └── plot.py
```

いま `/Users/student/analysis` にいるとする。まず `pwd` と `ls` を実行すると、現在位置とその場にある項目は次のように分かる。

```
$ pwd
/Users/student/analysis
$ ls
data    scripts
```

この状態で `cd data` を実行すると、`analysis` の下にある `data` ディレクトリへ移動する。

```
$ cd data
$ pwd
/Users/student/analysis/data
$ ls
events.txt
```

`data` は相対パスなので、カレントディレクトリ `/Users/student/analysis` を基準とする `/Users/student/analysis/data` を指す。

次に `cd ..` を実行すると、1 つ上の親ディレクトリへ戻る。

```
$ cd ..
$ pwd
/Users/student/analysis
```

ここで使っている `..` は、`ls -a` でも見た「親ディレクトリ」を表す特別な名前である。

カレントディレクトリに依存しない指定で直接移動することもできる。`~` は、シェルが

ホームディレクトリのパスへ展開する記号である。次の指定では、現在位置に関係なく `scripts` へ移動できる。

```
$ cd ~/analysis/scripts
$ pwd
/Users/student/analysis/scripts
$ ls
plot.py
```

また、`cd -` は 1 つ前にいた場所へ戻る。たとえば上の状態で実行すると、直前にいた `/Users/student/analysis` へ戻る。

```
$ cd -
/Users/student/analysis
```

引数なしの `cd` は、現在位置にかかわらずホームディレクトリへ移動する。

```
$ cd
$ pwd
/Users/student
```

`cd` には、相対パスによる移動、絶対パスによる移動、`..` による親ディレクトリへの移動、`-` による直前のディレクトリへの移動がある。移動後に `pwd` と `ls` を実行すれば、現在位置とその内容を確認できる。

2.3.3　絶対パスと相対パス

ファイルやディレクトリの場所は、パスで表す。パスは階層を `/` でつないだ表記であり、最初の書き方によって意味が変わる。大きく分けると絶対パスと相対パスがある。

表 2.1　よく出てくるパス表記

表記	意味
`/abs/data.txt`	絶対パス。`/` から始まり、どこにいても同じ場所を指す
`~/analysis`	ホームディレクトリからのパス。実行前にシェルが展開する
`data.txt`	相対パス。今いるディレクトリを基準に解釈される
`./x.txt`	相対パス。`.` は現在のディレクトリ
`../x.txt`	相対パス。`..` は 1 つ上のディレクトリ

表 2.1 は、シェルを使い始めるときに区別しておきたい表記をまとめたものである。絶対パスは `/` から始まる。`~/analysis` はシェルによる展開後にホームディレクトリ下の絶対パスとなり、カレントディレクトリには依存しない。

次のディレクトリ構成を考える。

```
analysis/
├── data/
│   └── events.txt
```

```
`-- scripts/
    `-- plot.py
```

現在位置が `/Users/student/analysis` なら、次の 2 つのコマンドは同じファイルを指している。

```
$ pwd
/Users/student/analysis
$ cat data/events.txt
$ cat /Users/student/analysis/data/events.txt
```

1 つ目の `data/events.txt` は、カレントディレクトリ `/Users/student/analysis` を基準とする相対パスである。2 つ目の `/Users/student/analysis/data/events.txt` は絶対パスであり、カレントディレクトリに依存しない。この状態では、両者とも同じファイルを指す。

一方で、現在位置が `/Users/student/analysis/scripts` に変わると、同じ相対パスでも意味が変わる。

```
$ pwd
/Users/student/analysis/scripts
$ cat data/events.txt
cat: data/events.txt: No such file or directory
$ cat ../data/events.txt
```

この場所で `data/events.txt` と指定すると、シェルは `scripts/data/events.txt` を探す。そのファイルは存在しないため、`data/events.txt: No such file or directory` というエラー文が表示される。相対パスで `events.txt` を指定するには、1 つ上へ戻る `..` を使って `../data/events.txt` と書く必要がある。

同様に、`./plot.py` は現在のディレクトリにある `plot.py`、`../data` は一つ上のディレクトリにある `data` を表す。相対パスが指す対象はカレントディレクトリによって変わるため、コマンドの実行前に `pwd` と対象パスを確認する。

2.3.4　ディレクトリの作成: `mkdir`

`mkdir` は新しいディレクトリを作るコマンドである。解析では、生データ、途中結果、図、ログなどを別々の場所へ整理したいので、作業の最初に出力先のディレクトリを作ることが多い。

たとえば、最初に次のような構成になっているとする。

```
analysis/
`-- data/
    `-- events.txt
```

`output` というディレクトリを 1 つ作るだけなら、次のように実行すればよい。

```
$ mkdir output
$ ls
data    output
```

1 行目では、現在のカレントディレクトリの直下に output が新しくできる。2 行目の ls で、それが確認できる。

一方、output/2026/04 のような深い階層をまとめて作る場合もある。途中の output/2026 がまだ無ければ、mkdir output/2026/04 だけではエラーになる。-p オプションを付けると、存在しない親ディレクトリも含めて作成される。

```
$ mkdir output/2026/04
mkdir: output/2026: No such file or directory
$ mkdir -p output/2026/04
```

実行後は、次のようなディレクトリ構成になる。

```
analysis/
¦-- data/
¦   `-- events.txt
`-- output/
    `-- 2026/
        `-- 04/
```

観測日ごとの出力先や figures/summary/ のような階層を一度に作る場合は、mkdir -p を用いる。-p は必要な親ディレクトリも作成し、指定したディレクトリがすでに存在する場合にはエラーにしない。

2.4　コピー、移動、削除

2.4.1　ファイル・ディレクトリのコピー: cp

cp は元のファイルやディレクトリを残したまま複製を作るコマンドである。最後の引数が既存のディレクトリならその中へ同じ名前でコピーされ、ファイル名ならその名前で複製が作られる。以下のディレクトリ構造を例とする。

```
work/
¦-- backup/
`-- raw/
    `-- events.txt
```

ここで raw/events.txt を backup へコピーするには、たとえば次のように実行できる。

```
$ cp raw/events.txt backup/
$ cp raw/events.txt backup/events_20260412.txt
```

1 行目では、backup/ が既存のディレクトリなので、その中に同じ名前の events.txt が作られる。2 行目では、コピー先の名前まで明示しているので、同じ内容を events_20260412.txt という別名で保存できる。

```
work/
```

```
¦-- backup/
¦   ¦-- events.txt
¦   `-- events_20260412.txt
`-- raw/
    `-- events.txt
```

cp の実行後も raw/events.txt は残り、コピー先に独立したファイルが作成される。元データを保持したまま加工用ファイルを作る場合は、入力と出力のパスを分ける。

ディレクトリごと複製したい場合は -r を付ける。

```
$ cp -r raw raw_backup
```

```
work/
¦-- backup/
¦   ¦-- events.txt
¦   `-- events_20260412.txt
¦-- raw/
¦   `-- events.txt
`-- raw_backup/
    `-- events.txt
```

-r は、指定したディレクトリ以下を再帰的にコピーする。コピー先に同名の項目が存在する可能性がある場合、-i を指定すると上書き前に確認が表示される。自動処理では対話入力に依存させず、出力先の存在を事前に検査する。

2.4.2　移動と名前の変更: mv

mv は「別の場所へ移す」と「名前を変える」の両方に使う。考え方は cp と似ており、最後の引数が既存ディレクトリならその中へ移動し、そうでなければその名前へ変更する。ただし、cp と違って元の場所には残らない。

まず、同じディレクトリの中で名前だけを変える例を見る。

```
work/
`-- result.txt
```

```
$ mv result.txt result_old.txt
```

```
work/
`-- result_old.txt
```

この場合は、同じ場所にあるファイルの名前だけが result_old.txt へ変わる。

次に、別のディレクトリへ移動しながら名前も変える例を考える。

```
work/
¦-- output/
`-- result_old.txt
```

```
$ mv result_old.txt output/final.txt
```

```
work/
`-- output/
    `-- final.txt
```

`mv result_old.txt output/final.txt` では、`result_old.txt` が `output` の中へ移り、名前も `final.txt` に変わる。もし `mv result_old.txt output/` と書けば、名前はそのままで場所だけが変わる。

このように、`mv` は「移動」と「改名」を 1 つのコマンドで行えるのが特徴である。整理のためにファイルを別ディレクトリへ移したり、解析段階に応じて `tmp.txt` を `final.txt` に改名したりするときに使う。`mv -i` を使うと、すでにファイルが存在した場合に上書きするかどうかを確認しながら進められる。

2.4.3　ファイル・ディレクトリの削除: rm

`rm` はファイルやディレクトリを削除するコマンドである。通常の `rm` は対象をゴミ箱へ移さず、その場で削除する。そのため、実行前に `pwd` や `ls` で場所と対象を確認する。

たとえば、次のような状態を考える。

```
work/
¦-- old_output/
¦   `-- plot.png
`-- result_old.txt
```

ここで使う基本形は次の通りである。

```
$ rm -i result_old.txt
$ rm -ri old_output
```

`rm -i result_old.txt` は通常のファイルを削除する前に確認を表示する。`rm -ri old_output` は、ディレクトリの内部を再帰的にたどり、各対象の削除前に確認する。確認が不要で対象を確定できている場合は、それぞれ `-i` を外した `rm result_old.txt` または `rm -r old_output` を用いる。これらは別々の初期状態に対する代替例であり、同じ対象に順番に実行するものではない。

`rm -r` は、指定したディレクトリ以下を再帰的に削除する。対象パスの誤指定による削除範囲の拡大を防ぐため、実行前に `ls old_output` や `ls result_old.txt` で対象を確認する。内容を個別に確認する場合は `rm -ri` を用いる。

2.4.4　ワイルドカード

ワイルドカードは、パターンに一致する複数のパスを指定する記法である。`*` は、ディレクトリ区切りを除く 0 文字以上の文字列に一致する。`cat` や `rm` の実行前にシェルがパターンを実際のパスへ展開し、その結果を各コマンドの引数として渡す。

たとえば、次のような構成を考える。

```
data/
├── run01.csv
├── run02.csv
├── run03.txt
├── test_20260411.csv
├── test_20260412.csv
├── test_20260413.csv
└── subdir/
    └── run04.csv
```

このとき、次のような書き方ができる。

```
$ ls data/*.csv
$ cp data/*.csv backup/
$ rm data/test_*.csv
```

`data/*.csv` は、`data` ディレクトリ直下にある `.csv` ファイルすべてを意味する。したがって、この例では `run01.csv` と `run02.csv` は対象になるが、`subdir/run04.csv` は含まれない。`*` は `/` をまたいで階層全体を探すわけではないからである。

`cp data/*.csv backup/` とすると、条件に合う複数の CSV をまとめてコピーできる。`rm data/test_*.csv` なら、`data/test_001.csv` や `data/test_20260413.csv` のように同じ接頭辞を持つファイルを一括削除できる。

ただし、ワイルドカードは想定より多くのファイルに一致することがある。特に削除系のコマンドでは、先に `ls data/test_*.csv` で実際の対象を表示してから `rm data/test_*.csv` を実行する。また、一致する項目がない場合の振る舞いはシェル設定によって異なるため、削除前の一覧表示はその確認にもなる。

2.5　ファイルの内容確認とデータ抽出

データを処理する前には、ファイル形式と内容の一部を確認する。表 2.2 に、内容表示、行抽出、列処理に用いるコマンドと主要なオプションを示す。

表 2.2 は、ファイルの確認と抽出に用いるコマンドを役割別に示したものである。「内容の表示」「条件に合う行の抽出」「列単位の整形」を区別し、必要な操作に対応するコマンドを選ぶ。以下では、各コマンドの入力、出力、主要なオプションを順に説明する。

2.5.1　標準入力・標準出力・標準エラー出力

コマンドには、データを受け渡すための三つの標準的な経路がある。標準入力（standard input, stdin）は入力データを受け取る経路、標準出力（standard output, stdout）は通常の結果を送る経路、標準エラー出力（standard error, stderr）はエラーや警告、進捗などの診断情報を送る経路である。Unix 系の OS では、これらをそれぞれファイル記述子という番号 0、

表 2.2　内容表示・行抽出・列処理のコマンドと主要オプション

コマンド	主なオプションや書式	主な用途
`cat`	`-n`	短いファイルをそのまま読む、複数ファイルを連結する
`less`	`-N`, `/pattern`, `q`	長いファイルをスクロールしながら読む
`head`	`-n N`	先頭の数行だけを見る
`tail`	`-n N`, `-f`	末尾の数行を見る、ログの追記を監視する
`wc`	`-l`, `-w`, `-c`	行数、単語数、バイト数を数える
`grep`	`-i`, `-v`, `-n`, `-A N`, `-B N`	条件に合う行だけを抽出する
`awk`	`-F`, `if (...)`, `sum += ...`	列単位の抽出、整形、集計を行う

1、2 で識別する。

端末で通常どおり実行すると、標準入力はキーボードからの入力に、二つの出力は端末の画面に接続される。このため、画面の表示だけでは stdout と stderr を区別できない。経路はファイルや別のコマンドにも接続でき、必ずしもキーボードや画面を意味しない。結果と診断情報を分けておけば、数値データのファイルに警告文が混入するのを防げる。

引数と標準入力も別物である。`wc -l data.txt` では、文字列 `data.txt` を引数として渡し、`wc` がそのファイルを開く。`wc -l < data.txt` では、シェルが `<` によってファイルを標準入力へ接続し、`wc` はそこからデータを読む。どちらも同じファイルの改行数を数えるが、後者の出力には入力ファイル名が付かない。

2.5.2　cat

`cat` は、指定したファイルの内容を順に標準出力へ送る。複数のファイルを渡すと、引数の順に内容を連結する。

```
$ cat data/events.txt
$ cat header.txt body.txt
$ cat log/*.log
$ cat -n config.txt
```

3 行目のように、ワイルドカードを使えば複数のログファイルを連結して表示できる。ただし、長いログや観測データでは出力が画面の範囲を超える。短いファイルの全体表示には `cat`、長いファイルの閲覧や一部表示には `less`、`head`、`tail` を用いる。

`-n`　各行に行番号を付ける。設定ファイルやログの位置を行番号で参照できる。

2.5.3　less

`less` は、ファイルを画面単位で表示し、開いたまま移動と検索を行うコマンドである。

```
$ less data/events.txt
$ less -N data/events.txt
```

less では、ファイルを開いたまま検索と移動を行える。主要な操作を次に示す。

-N　行番号を表示する。後で「どの行に何があったか」を参照しやすくなる。

Space　1 画面ぶん先へ進む。

b　1 画面ぶん戻る。

/pattern　指定した文字列を前方検索する。たとえば /ERROR と入力すると、ERROR を含む次の行が表示位置となる。

n　直前の検索条件の次の一致へ進む。

q　表示を終了してターミナルへ戻る。

2.5.4　head

head は、ファイルの先頭から指定した行数を表示する。ヘッダ、コメント行、区切り文字の確認に用いる。

```
$ head data/events.txt
$ head -n 20 data/events.txt
```

-n N　先頭から N 行を表示する。指定しない場合は 10 行である。

2.5.5　tail

tail はファイル末尾の行を表示する。ログの直近部分の確認や、末尾へ追記される計算結果の監視に用いる。

```
$ tail log.txt
$ tail -n 30 log.txt
$ tail -f log.txt
```

-n N　末尾から N 行を表示する。

-f　ファイルの末尾を監視し、追記された内容を継続して表示する。実行中の処理がログへ追記する内容を確認できる。

2.5.6　wc

wc は、ファイルの行数、単語数、バイト数を数える。1 行が 1 レコードに対応する形式では、-l の結果からヘッダ行とコメント行を除くことでレコード数を確認できる。

```
$ wc data/events.txt
$ wc -l data/events.txt
```

```
$ wc -w memo.txt
$ wc -c output.bin
```

-l　改行の数を行数として表示する。末尾に改行がない最終行の扱いに注意する。
-w　空白で区切られた単語数を表示する。
-c　入力のバイト数を表示する。

2.5.7　grep

grep は、指定したパターンに一致する行を出力する。大文字・小文字の無視、非一致行の出力、行番号や前後行の付加をオプションで指定できる。

```
$ grep "ERROR" log.txt
$ grep -n "ERROR" log.txt
$ grep -A 2 -B 1 "ERROR" log.txt
$ grep -i "warning" log.txt
$ grep -v "^#" data.txt
$ grep -rn "TODO" src/
```

-i　大文字と小文字を区別せずに検索する。たとえば、error と ERROR の両方に一致する。
-v　一致しなかった行を出す。コメント行や除外対象の行を取り除ける。
-n　一致した行とともに、元ファイル内の行番号を表示する。
-A N　一致した行に続く N 行も表示する。エラー行と直後の補足メッセージを同時に確認できる。
-B N　一致した行より前の N 行も表示する。エラー行に先行するログを同時に確認できる。
-r　指定したディレクトリ以下を再帰的に検索する。grep -rn "TODO" src/ は、src/ 以下で TODO を含む行を、ファイル名と行番号付きで表示する。

grep では正規表現も使用できる。^ は行頭、$ は行末を表すため、grep "^#" data.txt は # で始まる行だけを抽出する。grep -r "ERROR" log/ は、log/ 以下を再帰的に検索する。[]、.、* を含む正規表現の記法は、付録 E で説明する。

2.5.8　awk

awk は、入力行を区切り文字で列へ分け、列条件による抽出、列の並べ替え、集計を行うコマンドである。既定の区切りは連続する空白であり、-F で別の区切り文字を指定できる。

```
$ awk '{print $1}' data.txt
$ awk '{print $1, $3}' data.txt
$ awk -F, '{print $2}' data.csv
$ awk '{ if ($3 > 10) print $1, $3 }' data.txt
$ awk '{ sum += $3 } END { print sum }' data.txt
```

既定では空白やタブが区切り文字として使われる。そのため、空白区切りの観測データでは `$1` が 1 列目、`$2` が 2 列目を表す。条件分岐を `if` で明示すれば、「3 列目が 10 より大きい行だけを表示する」という処理を記述できる。各行の値を加算し、最後に合計や平均を出力することもできる。

`-F` 区切り文字を指定する。CSV のようにカンマ区切りなら `-F,` のように書く。

`{print $1}` 1 列目を表示する最も基本的な形である。`{print $1, $3}` のようにすれば複数列を並べて出せる。

`{ if ($3 > 10) print $1, $3 }` 3 列目が 10 より大きい行について、1 列目と 3 列目を表示する。

`{ sum += $3 } END { print sum }` 各行で 3 列目を足し上げ、入力をすべて処理した後に合計を表示する。

ここでの `awk -F,` は、カンマを単純な区切りとして扱う。引用符で囲まれた値にカンマや改行を含む一般の CSV は正しく解析できない。その場合は第 5.2.1 節の `csv` モジュールや pandas を使う。

2.5.9 パイプ ¦

パイプ（¦）は、左側のコマンドの標準出力を、右側のコマンドの標準入力へ接続する。中間ファイルを作らずに、抽出、並べ替え、行数の集計などを連結できる。

```
$ grep "ERROR" log.txt ¦ head -n 10
$ grep "ERROR" log.txt ¦ wc -l
$ grep -v "noise" events.txt ¦ awk '{print $2}'
```

1 行目は `ERROR` を含む行を抽出し、その先頭 10 行だけを表示する。2 行目は同じ抽出結果を `wc -l` へ渡し、件数を数える。3 行目はノイズ行を除外したうえで、2 列目だけを取り出す。

パイプによって、行の抽出、件数の集計、列の選択を左から右の順に一つのコマンド列として記述できる。

2.6 curl・wget による URL からのファイル取得

解析では、公開データ、設定例、インストール用スクリプトを URL から取得することがある。以下では、受信内容を既定で標準出力へ送る `curl` と、URL の末尾から決めた名前でファイルへ保存する `wget` を扱う。macOS の標準環境では `curl` を利用できるが、`wget` の利用には Homebrew などによる追加が必要である。

2.6.1 curl

curl は、URL から受信した内容を標準出力へ出すか、指定したファイルへ保存する。標準出力、URL 末尾の名前、指定した名前という 3 通りの出力先を次に示す。

```
$ curl https://example.com/config.txt ¦ head -n 2
$ curl -O https://example.com/data/run001.csv
$ curl -o events.csv https://example.com/data/run001.csv
$ curl -L -O https://example.com/latest.csv
$ curl -fsSL -o pyenv-installer.sh https://pyenv.run
$ less pyenv-installer.sh
```

1 行目では、取得した内容をファイルへ保存せず、そのまま標準出力へ送っている。配布元の config.txt の先頭が次の 2 行であれば、同じ内容が表示される。

```
threshold=5.0
window_ns=250
```

2 行目と 3 行目はファイル保存である。2 行目の -O は URL の末尾にあるファイル名をそのまま使う。3 行目の -o events.csv は、保存名を自分で決めたいときに使う。4 行目の -L はリダイレクト先までたどるオプションである。5 行目は取得したスクリプトをファイルへ保存し、6 行目で実行前に内容を確認する例である。取得内容を直接シェルへ渡すと、確認していないコードが実行されるため、配布元と内容の確認が必要である。-fsSL の各文字は、-f が HTTP エラーを失敗として扱い、-sS が通常の進捗を抑制しつつエラーを表示し、-L がリダイレクトをたどることを表す。

-O URL の末尾にある名前で保存する。保存名を URL と一致させる場合に用いる。

-o 保存名を引数で指定する。URL から保存名を決められない場合にも利用できる。

-L HTTP のリダイレクト応答で示された URL をたどる。リダイレクトを返す URL から転送先の内容を取得する場合に必要となる。

-f HTTP 400 以上の応答を受けた場合に、終了状態を失敗とする。スクリプトから終了状態を検査すれば、エラーページを正常な取得結果として後続処理へ渡すことを防げる。

-sS 進捗表示を抑制し、エラーメッセージは表示する。スクリプト内で出力を制御する場合に用いる。

2.6.2 wget

wget は、URL を指定すると既定でファイルへ保存する。-c を指定すれば、サーバーが範囲指定による取得に対応している場合に、途中まで保存したファイルの続きから取得できる。

```
$ wget https://example.com/data/run001.csv
$ wget -O events.csv https://example.com/data/run001.csv
$ wget -c https://example.com/archive.tar.gz
```

1 行目では、URL の末尾にある `run001.csv` という名前で保存される。2 行目の `-O` は保存名を指定する。3 行目の `-c` は、途中まで保存されたファイルがある場合に、その続きから取得を再開する。

`-O`　保存名を自分で指定する。既定の名前を使いたくないときに使う。
`-c`　途中まで保存したファイルの続きから取得を試みる。サーバーが範囲指定による取得に対応していなければ、先頭から取得し直す場合がある。
`-q`　通常のメッセージを表示しない。
`--show-progress`　quiet モードを指定していても、進捗表示を有効にする。

`wget` が手元にない環境でも、`curl -O` または `curl -o` でファイルへ保存できる。受信内容をパイプで次のコマンドへ渡す場合は、既定で標準出力へ送る `curl` をオプションなしで利用できる。

2.7　gzip・tar・zip による圧縮と展開

2.7.1　gzip: 1 ファイルの圧縮

`gzip` は、1 つのファイルを gzip 形式の 1 つの圧縮ファイルへ変換する。ディレクトリ構造や複数のファイルを一つにまとめる機能は持たない。

```
$ gzip -k events.csv
$ ls -lh events.csv*
$ gzip -cd events.csv.gz ¦ head -n 2
$ gzip -cd events.csv.gz > events_restored.csv
```

1 行目の `-k` は元ファイルを残す指定である。付けない場合は、通常 `events.csv` が `events.csv.gz` に置き換わる。2 行目では圧縮前後のサイズを見比べている。3 行目の `-cd` は、展開した内容をファイルへ戻さず標準出力へ流す指定であり、`head` や `grep` と組み合わせられる。4 行目は同じ出力をリダイレクトし、元の `events.csv` を残したまま `events_restored.csv` へ展開する。`gunzip events.csv.gz` で元の名前へ戻すには、同名の `events.csv` が存在しないことが必要である。

`-k`　圧縮または展開の完了後も入力ファイルを残す。
`-d`　gzip 形式のファイルを展開する。
`-c`　圧縮または展開した結果を標準出力へ送る。ファイルを作らずに `head` や `grep` へ渡せる。
`-1` から `-9`　圧縮にかける時間と出力サイズの兼ね合いを指定する。数字を大きくすると、通常は処理時間が長くなる代わりに出力ファイルが小さくなる。実際の差は入力内容に依存するため、必要な場合に同じ入力で処理時間と容量を比較する。

gzip 自体は複数ファイルを 1 つにまとめない。複数ファイルやディレクトリ構造を保持する場合は、次の tar でアーカイブを作成し、そのアーカイブを gzip で圧縮する。

2.7.2　tar: tar アーカイブの作成と gzip 圧縮

tar は、複数のファイルやディレクトリ構造を一つのファイルへ格納する。この格納先を tar アーカイブという。-z を指定すると、作成した tar アーカイブを gzip で圧縮した .tar.gz ファイルを作る。ここで用いる -c、-t、-x、-f、-z の意味は GNU tar 公式マニュアル [12] に基づく。

```
$ tar -czf archive.tar.gz raw/ notes.txt
$ tar -tzf archive.tar.gz
$ tar -xzf archive.tar.gz
```

1 行目は、raw/ と notes.txt を archive.tar.gz へ格納し、gzip で圧縮する。2 行目の -t は、展開せずに収録項目を表示する。3 行目の -x は内容を展開する。

-c　新しくアーカイブを作る。
-t　中身一覧だけを見る。
-x　展開する。
-z　gzip 圧縮を併用する。
-f　直後の文字列をアーカイブ名として扱う。

tar -czf archive.tar.gz ... の -c は tar アーカイブの作成、-z は gzip 圧縮、-f は直後の archive.tar.gz をアーカイブ名として扱う指定である。内容一覧には -t、展開には -x を用いる。

2.7.3　zip: ZIP アーカイブの作成と展開

zip は、複数のファイルやディレクトリを圧縮し、1 つの ZIP アーカイブへ格納する。tar とはオプションと展開コマンドが異なるため、ファイル名の拡張子に対応するコマンドを選ぶ。

```
$ zip -r archive.zip raw/ notes.txt
$ unzip -l archive.zip
$ unzip archive.zip
```

1 行目の -r は、指定したディレクトリ以下を再帰的に格納する。2 行目の unzip -l は展開せずに収録項目を表示し、3 行目は内容を展開する。形式に応じて、.zip には unzip、.tar.gz には tar -xzf、単一の .gz ファイルには gunzip または gzip -d を用いる。

展開後のファイルが元と同じかを確認するには、cmp を使う。例えば cmp raw/events.csv restored/events.csv は 2 ファイルを全バイト比較する。一致すれば何も表示せず終了ステータス 0 を返し、不一致なら 1、読み込みなどのエラーなら 2 を返す。先頭数行が同じだけでは、ファイル全体の一致は確認できない。

2.8　環境変数と PATH

2.8.1　環境変数の定義と参照

シェルやプログラムには、「名前」と「値」の組として渡される設定がある。これが環境変数である。環境変数は、シェルの中だけで完結するものではなく、そこから起動した Python や各種コマンドにも引き継がれる。現在設定されている主な環境変数の値は、次のコマンドで表示できる。

```
$ echo $HOME
$ echo $SHELL
$ echo $PATH
```

実行結果は、たとえば次のようになる。

```
/Users/student
/bin/zsh
/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

1 行目はホームディレクトリ、2 行目はログインシェル、3 行目は実行ファイルを探索するディレクトリの一覧である。`PATH` の値はコロン `:` で区切られ、シェルは左から順に同名の実行ファイルを探索する。`PYENV_ROOT` や `DATA_ROOT` も環境変数であり、起動するプログラムへ名前付きの値として引き継がれる。新たな環境変数は次の形で設定する。

```
$ export DATA_ROOT="$HOME/data/events"
$ echo $DATA_ROOT
```

結果の例は次の通りである。

```
/Users/student/data/events
```

`export` 自体は成功しても通常は何も表示しない。続けて `echo $DATA_ROOT` を実行し、意図した値が入ったことを確認する。ここでは `$HOME` が先に展開され、その結果として絶対パスが保存されている。

ただし、この設定は通常そのターミナルを閉じると消える。毎回使う値なら、対話シェルが読み込む設定ファイルに `export ...` を書いておく。第 2.1 節で述べたように、`echo $SHELL` でログインシェルを、`ps -p $$ -o comm=` で現在のシェルを確認できる。zsh では通常 `~/.zshrc`、bash では環境に応じて `~/.bashrc` または `~/.bash_profile` に追記する。

2.8.2　展開と引用

シェルはコマンドの実行前に、`$HOME` のような変数展開と、`$(pwd)` のようなコマンド置換を行う。展開の有無と、展開後の値を一つの引数として保つかどうかは引用符によって異

なる。

```
$ echo $HOME
$ echo "$HOME"
$ echo '$HOME'
$ name="analysis result"
$ echo $name
$ echo "$name"
$ echo "$(pwd)"
$ echo "`pwd`"
```

1 行目と 2 行目は、どちらも `$HOME` を展開する。`HOME` に空白がなければ表示は同じになる。しかし、クォートなしの変数展開は、展開後にフィールド分割とファイル名展開の対象になる。`"$HOME"` は展開結果を一つの引数として保つため、文字列として渡す変数は原則としてダブルクォートで囲む。

3 行目のシングルクォート `'...'` 内では変数展開が行われないため、`$HOME` という文字列がそのまま表示される。Bash では、`name="analysis result"` の後に引用符なしで `echo $name` と書くと、既定の設定では空白によって 2 引数へ分割される。zsh の既定の変数展開では、この分割は起こらない [13]。シェルの設定に依存しないよう、変数の値を一つの引数として渡すときは引用符を付ける。`echo "$name"` では空白を含む値が一つの引数として渡る。`$(pwd)` とバッククォートは、いずれも `pwd` の標準出力によるコマンド置換を表す。`$(...)` は入れ子を明示できるため、本書ではこちらを用いる。

この違いは、設定ファイルへ 1 行追記するときに特に重要である。たとえば `echo 'export PATH="$HOME/bin:$PATH"' >> ~/.zshrc` のようにシングルクォートで囲むと、`$HOME` や `$PATH` を展開せず、その文字列をそのまま `~/.zshrc` へ追記できる。もしダブルクォートで囲むと、その場で現在の値へ展開されて書き込まれるので、意図とずれることがある。

また、空白と改行の扱いにも注意が必要である。クォートの外では、空白やタブや改行が単語の区切りになる。そのため、空白を含むファイル名を引用符で囲まないと、複数の引数に分かれてしまう。

```
$ mkdir "analysis result"
$ cp "analysis result/data.txt" backup/
```

1 つの長いコマンドを複数行へ分けたいときは、行末に `\` を置いて次の行へ続ける。

```
$ python analyze.py \
  --input data/events.txt \
  --output result.txt
```

引用符や括弧の外にある改行は、通常はコマンドの終端として解釈される。行末の `\` は直後の改行を除去し、次の物理行を同じコマンドとして継続する。

2.8.3 PATH と実行ファイル

シェルが python3 や ls を実行できるのは、それらが実行ファイルとして保存されており、しかもシェルがその場所を知っているからである。その探索に使う環境変数が $PATH である。実際にどの実行ファイルが使われるかを調べるには、次のコマンドを用いる。

```
$ command -v python3
$ command -v ls
$ echo $PATH
```

出力の例は次のようになる。

```
/opt/homebrew/bin/python3
/bin/ls
/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

command -v python3 は、現在のシェルが python3 という名前をどのコマンドに解決するかを表示する。例のようにパスが表示される場合もあれば、エイリアスやシェル関数として定義されている場合もある。PATH で /opt/homebrew/bin が /usr/bin より左にあるため、同名の実行ファイルが両方にあれば前者が優先される。Python 自身が認識する実行ファイルは、python3 -c "import sys; print(sys.executable)" で別に確認できる。

bin は実行ファイルを置くディレクトリに用いられる慣習的な名前である。仮想環境の .venv/bin にも Python と、その環境へインストールしたコマンドの起動ファイルが置かれる。

独自の実行ファイルを置く ~/my_scripts をコマンド探索の対象にする例を示す。既存の探索先を保持するために現在の $PATH を右辺へ含め、新しいディレクトリを先頭へ追加する。対話シェルの起動時に同じ設定を適用する場合は、利用中のシェルが読み込む設定ファイルに記述する。

```
$ export PATH="$HOME/my_scripts:$PATH"
```

2.8.4 補足：シェルの種類と設定ファイル

利用者がログインするときに起動するよう設定されたシェルを、ログインシェルという。その名前は、次のコマンドで確認できる。

```
$ echo $SHELL
```

この実行例で読者が入力するのは、echo $SHELL の部分だけである。行頭の $ は入力しない。echo は後ろに続く値を表示し、$SHELL はシェルの設定値を取り出す。このような、プログラムへ引き継がれる名前付きの設定値を環境変数という。
実行結果の例は次のようになる。

```
/bin/zsh
```

zsh と bash では、起動時に読む設定ファイルや一部のシェル構文は異なる。上の /bin/zsh は、ログインシェルとして zsh が設定されていることを表す。

ただし、SHELL は現在実行中のシェルを必ず示すわけではない。別のシェルを起動した場合などは値が一致しない。現在コマンドを解釈しているシェルは、次のコマンドで確認できる。

```
$ ps -p $$ -o comm=
zsh
```

設定ファイルは、シェル名だけでなく OS と起動方法によっても異なる。zsh の対話シェルでは通常 ~/.zshrc を使う。bash では Linux の対話シェルなら ~/.bashrc、macOS のログインシェルなら ~/.bash_profile が使われることが多い。

2.9　リダイレクトとログ保存

2.9.1　echo、標準出力、標準エラー

第 2.5.1 節で説明した標準出力と標準エラー出力は、それぞれ別の保存先へ接続できる。まず、短い文字列を出力する echo で標準出力の行き先を確かめる。

echo は、引数として受け取った文字列を標準出力へ送る。

```
$ echo "hello"
$ echo "$HOME"
```

1 行目は hello、2 行目は変数展開後の $HOME の値を標準出力へ送る。端末上では標準出力が画面へ接続されているが、次節のリダイレクトによってファイルへ変更できる。

2.9.2　> と >>

> は標準出力をファイルへ書き出す。既存のファイルがあれば中身を上書きする。

```
$ echo "alpha" > notes.txt
$ cat notes.txt
alpha
```

この例では、echo の結果が画面ではなく notes.txt に保存される。notes.txt がすでに存在する場合、その内容は新しい出力で置き換えられる。既存ファイルを保持する必要がある場合は、出力先を確認してから実行する。

>> は追記であり、既存の内容を残したまま末尾に足す。

```
$ echo "alpha" > notes.txt
$ echo "beta" >> notes.txt
$ cat notes.txt
alpha
```

```
beta
```

`>>` は既存ファイルを保持して末尾へ追記する。`>` は既存内容を置き換えるため、設定ファイルへ行を追加する操作には `>>` を用いる。いずれの場合も、実行前に対象パスを確認する。

2.9.3　1> と 2>

`>` は実際には `1>` と同じ意味であり、標準出力だけをファイルへ送る。標準エラーを別ファイルへ分けたいときは `2>` を使う。

```
$ python analyze.py > output.log
$ python analyze.py 1> output.log 2> error.log
```

1 行目では通常の出力だけが `output.log` に入り、エラーメッセージは画面に残る。2 行目では、通常の出力を `output.log` に、エラーメッセージを `error.log` に分けて保存している。長時間の処理では、両者を分けると失敗原因を追跡しやすい。

標準出力と標準エラーを同じファイルへまとめたいときは、`2>&1` を使う。

```
$ python analyze.py > run.log 2>&1
```

リダイレクトは左から順に適用される [14]。`> run.log 2>&1` では、まず標準出力をファイルへ接続し、次に標準エラー出力をその時点の標準出力と同じ行き先へ接続する。順序を逆にした `2>&1 > run.log` では、標準エラー出力は先に元の標準出力、通常は端末へ接続されるため、ファイルに入るのは標準出力だけである。

`¦` が通常接続するのは標準出力だけである。標準エラー出力も次のコマンドへ渡す場合は `command 2>&1 ¦ next_command` とする。両経路をまとめても、プログラム内の出力の一時蓄積（バッファリング）により、実際に処理した順序とログの行順が一致するとは限らない。

2.9.4　tee による画面表示と保存

`>` や `>>` で出力をファイルへ送ると、その出力は画面に表示されない。`tee` は標準入力を標準出力へ複製すると同時に、指定したファイルへ保存する。

```
$ python analyze.py ¦ tee stdout.log
$ python analyze.py 2>&1 ¦ tee combined.log
```

1 行目は標準出力だけを画面と `stdout.log` の両方へ流す。2 行目は標準エラーもまとめ、画面と `combined.log` へ同じ内容を出力する。二つの行は保存対象の違いを示す代替例である。

`tee` に `-a` を付けると追記になる。

```
$ python analyze.py 2>&1 ¦ tee -a run.log
```

この場合は、既存の `run.log` を消さずに末尾へ足していく。毎日の実行結果を 1 つのログへ蓄積したい場合に向く。

2.9.5　終了ステータスによる成功・失敗の確認

コマンドは終了時に整数の終了ステータスを返す。通常は 0 が成功、0 以外が失敗などの状態を表す。直前の値は `$?` で確認できる。

```
ls existing_file.txt
echo "$?"
```

`echo` など別のコマンドを実行すると `$?` も更新されるので、調べたいコマンドの直後に確認する。非零の意味はコマンドごとに異なり、`grep` の 1 は検索条件に一致する行がないことを表す。標準エラー出力に警告があっても成功するプログラムがあり、逆に何も表示せず失敗するものもある。

Bash と zsh のパイプラインは、既定では最後のコマンドの終了ステータスを返す。`set -o pipefail` を設定すると、途中のコマンドの失敗も検出できる。ただし、`head` が必要な行だけ読んで終了すると、前段が書き込み先を失って非零になる場合がある。終了ステータスは、そのコマンドと処理手順の意味に沿って解釈する。

2.10　権限とスクリプトの実行

2.10.1　ファイル権限の表記と解釈

ファイルにはアクセス権がある。読み取れるか、書き込めるか、実行できるかを `rwx` で表す。`ls -l` を使うと確認しやすい。

```
-rwxr-xr-x  1 user  staff  1234 Apr  8 10:00 run_analysis.sh
-rw-r--r--  1 user  staff   411 Apr  8 10:05 notes.txt
-rw-------  1 user  staff   411 Apr  8 10:05 id_ed25519
```

最初の 10 文字のうち、先頭 1 文字はファイル種別であり、残り 9 文字が権限である。3 文字ずつに区切って、所有者、グループ、その他の利用者に対する権限を表す。文字と権限の対応は表 2.3 のとおりである。

権限の変更には、`chmod` コマンドを用いる。所有者（`user`）に実行権限（`x`）を加える指定は `u+x` である。その他の利用者（`other`）から読み取り権限（`r`）と書き込み権限（`w`）を除く指定には `o-rw` を用いる。グループ（`group`）とその他の利用者の権限を読み取りだけにする指定は `go=r` である。`a+x` の `a`（`all`）は、すべての利用者を対象とする。

```
$ chmod u+x my_script.sh
$ chmod o-rw notes.txt
$ chmod go=r read_only.txt
$ chmod a+x run_analysis.sh
```

表 2.3 に示すように、権限は数字でも指定できる。各桁は読み取りを 4、書き込みを 2、実

表 2.3　権限の記号と数字

記号	数字	意味
r	4	読み取り
w	2	書き込み
x	1	実行

行を 1 として合計した値である。たとえば `7` は `rwx`、`6` は `rw-`、`5` は `r-x` に対応する。したがって、`755` は所有者が `rwx`、グループとその他の利用者が `r-x`、`644` は所有者が `rw-`、グループとその他の利用者が `r--` である。SSH の秘密鍵には、所有者だけに読み書きを許可する `600` を指定する。設定例を次に示す。

```
$ chmod 755 run_analysis.sh
$ chmod 644 notes.txt
$ chmod 600 ~/.ssh/id_ed25519
```

2.10.2　スクリプトの直接実行

複数の命令を保存して実行するファイルをスクリプトという。次をテキストファイル `hello.sh` として保存する。

```
1 #!/usr/bin/env bash
2
3 echo "hello"
```

`bash hello.sh` で実行すると `hello` が表示される。先頭の行は、このファイルを直接実行するときに使うプログラムを指定する行であり、シバンという。`/usr/bin/env bash` は `PATH` から Bash を探す指定である。

ファイル名を指定して直接実行する場合は、実行権限も付ける。

```
$ chmod u+x hello.sh
$ ./hello.sh
hello
```

`u+x` は所有者に実行権限を加える指定、`./` は現在のフォルダーを表す。引数や繰り返しを使うシェルスクリプトは付録 F で扱う。

2.11　章末課題：データを取得し、圧縮形式を比較する

1. **作業場所を用意する。**次を実行し、`pwd` と `ls` の結果を示せ。データ、コード、結果をどのフォルダーへ置くか説明せよ。

```
$ mkdir -p earthquake_practice/{raw,compressed,scripts,figures,results}
$ cd earthquake_practice
```

2. **地震カタログの内容を調べる。**次の URL から CSV を取得し、データ行数、先頭と末尾の時刻、日本を含む地名の例を表にせよ。`wc -l` ではヘッダも数えることに注意する。取得できない場合は、第 5 章の人工データで表示と集計を試せる。

```
$ curl -fL -o raw/usgs_global_2024_01.csv \
  'https://earthquake.usgs.gov/fdsnws/event/1/query?format=csv&starttime=2024-01-01&
    endtime=2024-01-31T23:59:59.999999&limit=20000&orderby=time-asc'
$ curl -fL -o results/usgs_count.txt \
  'https://earthquake.usgs.gov/fdsnws/event/1/count?starttime=2024-01-01&endtime
    =2024-01-31T23:59:59.999999'
$ cat results/usgs_count.txt
```

   `count` は同じ検索条件の総件数を返す。取得上限の 20000 件を超える場合は、期間を分割し、イベント ID の重複を除く [15]。取得日時と検索条件も記録する。
3. **3 種類の圧縮形式を比較する。**同じ CSV を gzip、tar.gz、zip へ圧縮せよ。元の容量、圧縮後の容量、容量比（圧縮後／元）を表にする。それぞれ別のフォルダーへ展開し、`cmp` 元の `csv` 展開後の `csv` で全内容の一致を確かめよ。最も小さかった形式と、格納方法の違いを説明せよ。
4. **発展：取得条件と URL の関係を調べる。**2 問目の URL を、取得先、パス、検索条件に分けて示せ。期間または地域を変え、取得件数と変更した項目を表にして、どの指定が結果を変えたか説明せよ。

第 3 章

Python 実行環境の構築

3.1 最初の計算を実行する

この章では、Python を準備し、短いコードをファイルへ保存して実行する。計算結果の `5` を表示し、コードの変更後に `12` へ変わることを確認するのが最初の目標である。準備にはインターネット接続を使う。

3.1.1 OS に合う端末を開く

macOS では「ターミナル」を開く。Windows では前章の手順で WSL 上の Ubuntu を用意し、Ubuntu の端末を開く。以下の命令はその端末に入力する。Windows の PowerShell へそのまま入力する命令ではない。

以下では、どちらの OS でも uv を使って Python 3.12 と作業用の環境を用意する。Python の導入と仮想環境の詳細は、最初の計算を試した後に説明する。

3.1.2 uv を用意する

`uv --version` で番号が表示される場合は、次の小節へ進む。`command not found` と表示される場合は、公式の導入スクリプト [18] を保存する。

```
$ curl -fLsS https://astral.sh/uv/install.sh -o uv-install.sh
$ less uv-install.sh
```

`less` の表示は `q` で閉じる。取得元とスクリプトの内容を確認したうえで、次を実行する。導入先や変更内容はスクリプトが表示する案内で確認する。

```
$ sh uv-install.sh
```

導入後は、新しい端末を開いて `uv --version` を実行する。`uv 0.x.y` のように番号が出れ

ば、次へ進める。番号自体は導入時期によって変わる。

3.1.3　作業用フォルダーと Python を用意する

次の命令を 1 行ずつ実行する。`analysis_practice` には、この章で作るファイルを保存する。

```
$ mkdir -p analysis_practice
$ cd analysis_practice
$ uv venv --python 3.12 --seed .venv
$ source .venv/bin/activate
$ python --version
```

`uv venv` は、使用する Python と追加ライブラリをまとめる作業用の環境を `.venv` に作る。指定した Python が手元になければ、uv が取得する。`--seed` は後で使う pip もこの環境へ入れる指定である。`source` の行は、この端末でその環境を使い始める命令である。

最後に `Python 3.12.x` と表示されることを確認する。`x` の部分は更新によって変わる。このように作業ごとに用意する環境を「仮想環境」という。

3.1.4　Python ファイルを保存して実行する

コードを書くには、文字だけのファイルを編集するテキストエディタを使う。ここでは、端末内で使えるエディタ `nano` を例にする。`analysis_practice` にいることを `pwd` で確認し、次を入力する。

```
$ nano first_calculation.py
```

編集画面が開いたら、次の 2 行を入力する。左側の行番号は入力しない。

first_calculation.py

```
1 count = 3
2 print(count + 2)
```

Ctrl キーを押しながら O を押すと、保存するファイル名の確認が出る。Enter で確定し、Ctrl+X でエディタを閉じる。`nano` を使えない場合や使い慣れたエディタがある場合は、そのエディタで同じ内容を保存してよい。ファイル形式は標準テキスト、文字コードは UTF-8 とし、名前の末尾が `.py.txt` にならないようにする。

ターミナルへ戻ったら、保存したファイルを実行する。

```
$ python first_calculation.py
5
```

`python` は Python を起動する命令であり、後ろの `first_calculation.py` は実行するファイル名である。`print()` は括弧内の計算結果を画面に表示する。`5` が表示されれば、保存したコードを実行できている。

もう一度 `nano first_calculation.py` で開き、`count = 3` を `count = 10` へ変えて保存する。同じ実行コマンドで `12` が表示されることを確認する。このように、編集、保存、実行、結果の確認を繰り返してプログラムを作る。

`can't open file` と表示されたら、`pwd` と `ls` で現在位置とファイル名を確かめる。`SyntaxError` と表示されたら、表示された行の括弧や引用符などを見本と比べる。エラーの詳しい読み方は第 4 章で説明する。

3.1.5　作業を終える・再開する

作業を終えたら、端末で `deactivate` を実行して仮想環境の利用を終える。作成したファイルや環境は削除されない。再開するときは、`analysis_practice` へ移動してから、次を実行する。環境を毎回作り直す必要はない。

```
$ source .venv/bin/activate
$ python first_calculation.py
12
```

3.2　使っている Python を確認する

ここからは、最初の手順で何を用意したかを確かめる。Python 本体はコードを実行する。仮想環境を使うと、作業ごとに追加ライブラリを分けて管理できる。

以下は、`analysis_practice` フォルダーへ移動し、`source .venv/bin/activate` で仮想環境を有効にした端末で実行する。

```
$ python --version
$ command -v python
$ python -c "import sys; print(sys.executable)"
```

1 行目は版、2 行目はシェルが見つけた命令の場所、3 行目は実際に起動した Python の実行ファイルの場所を表示する。`-c` は、後ろの引用符内を Python コードとして実行する指定である。ここでは `sys` という Python 付属の機能で、実行ファイルの場所を表示している。仮想環境を使っていれば、2 行目と 3 行目には通常 `analysis_practice/.venv/bin/python` を含むパスが出る。値が異なる場合は、現在のフォルダーと仮想環境の有効化を確認する。

3.3　解析用ライブラリを追加する

ライブラリは、計算や作図などの機能をまとめたものである。Python に最初から含まれる標準ライブラリと、自分で追加する外部ライブラリがある。例えば平方根を求める `math` は標準ライブラリであり、図を作る Matplotlib は外部ライブラリである。本書で使う主な外部ライブラリを表 3.1 に示す。それぞれの具体的な使い方は、対応する章で学ぶ。

表 3.1　本書で使う主なライブラリと役割

ライブラリ	主な役割
NumPy	多数の数値の計算
pandas、Polars	表の読み込み、加工、集計
Matplotlib	図の作成と保存
SciPy	データへの式の当てはめなどの数値計算
Astropy	単位、時刻、天体の座標の扱い
tqdm	処理の進み具合の表示

`analysis_practice` の仮想環境を有効にした端末で、次を実行する。`pip` は外部ライブラリを導入する道具である。`python -m pip` と書くことで、現在使っている Python に対応する pip を実行できる。

```
$ python -m pip install numpy pandas polars matplotlib scipy astropy tqdm
```

途中には取得状況やライブラリ名が表示される。最後にエラーが出ずに入力待ちへ戻ったら、次の命令で、導入した機能を Python から読み込めるか確認する。`import` はライブラリを使うための読み込み命令である。

```
$ python -c "import numpy, pandas, polars, matplotlib, scipy, astropy, tqdm; print('OK')"
OK
```

`ModuleNotFoundError` が出た場合は、表示された名前のライブラリを読み込めていない。仮想環境を有効にしたか、導入時の名前を間違えていないかを確認する。

3.4　次章へ進む前の確認

自分で保存した `first_calculation.py` を実行し、値を書き換えた後に答えが変わることを確認できれば、次章へ進める。作業を再開するときに、作業フォルダーへ移動して仮想環境を有効にする手順も確認しておく。

3.5　章末課題：保存したプログラムを実行する

1. **計算を変更して実行する。**`first_calculation.py` を実行し、`count` が 3 の場合と 10 の場合の表示を表にせよ。実行した命令と、編集・保存・実行の順序を説明せよ。
2. **使っている環境を確認する。**仮想環境の Python の版と場所、ライブラリの導入確認の結果を記録せよ。作業を終了して再開し、同じファイルを再実行できることを示せ。

別方式での環境作成や、ライブラリの版を記録・切り替える方法、環境管理の演習は付録 A にまとめた。

第 4 章

Python の基本

この章では、測定値に名前を付け、条件に合う値を選び、同じ処理を繰り返す方法を学ぶ。最後には、複数の測定値の平均を関数で求められるようにする。最初は短い式を対話モードで試し、処理が長くなったら Python ファイルへまとめる。

4.1 データ型と基本操作

前章で用意した仮想環境を有効にし、ターミナルで `python` と入力すると、Python をインタラクティブモードで起動できる。インタラクティブモードでは、Python のバージョン情報に続いて入力待ちを表す `>>>` が表示される。次は表示の一部を示した例である。

```
$ python
Python 3.12.14 ...
>>>
```

この章の前半には、インタラクティブモードへの入力と、その直後に表示される値を対にして示す。`$`、`>>>`、継続行の `...` は入力待ちを表すプロンプトであり、入力するコードには含めない。複数行の処理は空行で確定し、`exit()` で Python を終了する。スクリプトでは、式だけを書いても値は表示されず、`print()` などによる出力が必要になる。

4.1.1 変数と基本型

測定値を後の計算でも使いたいときは、値に名前を付ける。次の例では、最初の測定値を `first`、次の測定値を `second` という名前で使う。対話モードでは、`>>>` の後へ 1 行ずつ入力する。ファイルで試す場合は、4 行を `mean.py` へ保存して `python mean.py` で実行する。

```
first = 12.4
second = 15.1
mean = (first + second) / 2
print(mean)
```

```
13.75
```

mean には、2 つの測定値の平均が入る。このように、値を使うために付けた名前を「変数」という。= は、右側の値を左側の名前で使えるようにする代入の記号である。数学の等号とは意味が異なり、値を比較するときには == を使う。比較は第 4.3.1 節で扱う。

first = 12.4 を first = 10.0 に変え、4 行すべてをもう一度実行すると、表示は 12.55 になる。変数の値を変えた後には、その値を使う計算もやり直す必要がある。

4.1.1.1 数値と文字列を区別する

次に、数値の 12 と、引用符で囲んだ "12" を比べる。対話モードで入力すると、次のようになる。

```
>>> count = 12
>>> text = "12"
>>> count + 3
15
>>> text + "3"
'123'
```

count + 3 は数値を足す。一方、text + "3" は文字を後ろにつなげるため、'123' になる。引用符で囲んだ値を「文字列」という。ファイルから読み込んだ "12" を数値として計算したいときは、int(text) のように変換する。

```
>>> int(text) + 3
15
>>> type(count)
<class 'int'>
>>> type(text)
<class 'str'>
```

type() は、その値がどの種類かを返す。この種類を「型」という。int は整数、str は文字列を表す。

4.1.1.2 小数と真偽値を使う

小数を扱う型を浮動小数点型（float）という。最初の例の 12.4 と 15.1 はこの型である。条件を満たすかどうかを表す True と False は真偽値（bool）である。

```
>>> type(12.4)
<class 'float'>
>>> use_log = True
>>> type(use_log)
<class 'bool'>
```

件数には整数、平均値のように小数になり得る値には浮動小数点数、名前には文字列を使う。Python では、使い始める前に変数の型を宣言する必要はないが、値ごとの型は区別される。NumPy で大量の値を扱うときの型は、第 6.3.6 節で説明する。

`None` は「値がない」ことを表す特別な値であり、0、空文字列、`False` とは異なる。測定できなかった信号を表す場合などに使い、`value is None` で判定する。

4.1.2　数値と演算子

対話モードでは、式を入力するたびに評価結果が表示される。四則演算、べき乗、剰余、床除算の例を次に示す。

```
$ python
>>> 2 + 3
5
>>> 2 * 3
6
>>> 7 / 3
2.3333333333333335
>>> 7 // 3
2
>>> 7 % 3
1
>>> 2 ** 10
1024
```

`/` は通常の除算を行い、整数どうしでも浮動小数点数を返す。`//` は商を負の無限大方向へ切り下げる床除算であり、正の整数どうしでは小数部分を切り捨てた結果と一致する。特に負の値を含む場合、`-7 // 3` は `-3` であり、0 方向へ切り捨てる `int(-7 / 3)` の `-2` とは一致しない。計算の順序を明確にするため、必要な箇所を括弧で囲む。

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
>>> 7 / 3
2.3333333333333335
>>> 7 // 3
2
>>> int(7 / 3)
2
```

この例では、`7 // 3` と `int(7 / 3)` はどちらも `2` になるが、計算規則は異なる。`//` は床除

算であり、`int(7 / 3)` はまず浮動小数点数の除算を行い、その結果を 0 方向へ切り捨てて整数へ変換している。正の値だけでは差が見えないため、負の値を含む処理では両者を混同してはならない。

十進数の 0.1 などは、二進数では有限桁で表現できない。Python の通常の浮動小数点数は有限桁の二進数で保持されるため、こうした入力は近い値に丸められる [19]。たとえば `0.1 + 0.2 == 0.3` は `False` になる。この丸め誤差は、後の章で扱う統計量とフィット結果の比較にも影響する。

4.1.3　文字列

ファイル名、日付時刻、ヘッダ、ラベルなど、解析コードでは文字列を扱う場面が多い。Python では、`"` や `'` で挟まれたものを文字列として認識する。文字列には、分割、検索、置換、前後の空白除去などの操作を適用できる。次の例では、`name.upper()` で文字を大文字にする。このように、値の後ろへ `.操作名()` を付けて呼び出す関数をメソッドという。

```
>>> name = "detector_A"
>>> date = "2026-04-04"
>>> raw_line = "  detector_A,42,3.5  \n"
>>> name.upper()
'DETECTOR_A'
>>> date.replace("-", "/")
'2026/04/04'
>>> name.split("_")
['detector', 'A']
>>> raw_line.strip()
'detector_A,42,3.5'
>>> f"{name}_{date}.txt"
'detector_A_2026-04-04.txt'
```

`date.replace("-", "/")` は、`-` を `/` へ置き換えた文字列を返す。丸括弧の中に渡す `"-"` と `"/"` を引数、処理の結果として得る `'2026/04/04'` を返り値という。`upper()` のように引数がなくても、操作を実行するときは丸括弧を付ける。

文字列では、`split()`、`replace()`、`strip()` のような操作が、ファイル読み込みの前処理で頻繁に現れる。`split("_")` は `_` の位置で文字列を分ける。`strip()` は、行末の改行と前後の空白を取り除く。カンマ区切りの表には引用符の規則もあるため、ファイルを読む章では `csv` を使う。

f 文字列を用いると、変数の値と書式指定を文字列へ埋め込める。解析コードでは、ログ、図の保存名、結果ファイル名、単位付きの数値表示などに利用する。

4.2　リスト・タプル・辞書・集合

4.2.1　リスト、タプル、辞書

複数の値をまとめて保持するデータ構造として、リスト、タプル、辞書がある。三つの定義例を次に示す。

```
>>> labels = ["A", "B", "C", "D"]
>>> point = (12, 18)
>>> row = {"id": 42, "value": 3.5, "flag": True}
```

リストとタプルは要素の順序を保つ。リストの要素は変更できるが、タプルの要素は差し替えられない。辞書ではキーと値を対応付ける。実際に要素を取り出すときは、リストやタプルでは位置を、辞書ではキーを使う。対話モードで試すと次のようになる。

```
>>> labels[0]
'A'
>>> point[1]
18
>>> row["id"]
42
>>> row["value"]
3.5
```

この例では、`labels[0]` は最初のラベル、`point[1]` は 2 番目の座標成分、`row["id"]` は `"id"` というキーに対応する値を表す。位置を示す番号を添字といい、先頭を 0 として数える。辞書のキーは、`"id"` のように値を取り出すための名前である。

Python が扱う数値、文字列、リスト、辞書などの値をまとめてオブジェクトという。リストや辞書は作成後に中身を変更でき、この性質を「可変」という。大量の記録を扱う場合のメモリ使用量と処理時間は、後の表処理と高速化の章で確認する。

データ構造は、必要な操作に応じて選ぶ。位置で要素を取り出す場合はリストやタプル、キーによって値を取り出す場合は辞書が適する。選択によって、コードの可読性、変更可能性、メモリ使用量、処理時間が変わる。

リストは作成後に要素を変更できる。

```
>>> labels[1] = "B_detector"
>>> labels
['A', 'B_detector', 'C', 'D']
```

一方、タプルは作った後で要素を書き換えられない。たとえば次のようにするとエラーになる。

```
>>> point[0] = 15
Traceback (most recent call last):
```

```
  ...
TypeError: 'tuple' object does not support item assignment
```

座標のように、複数の成分を一組の値として保持し、作成後に要素を変更しない場合にはタプルが向く。ラベルや処理対象ファイルの一覧など、要素を追加、削除、更新する場合にはリストが向く。辞書は、列名や設定名などのキーによって値を参照する場合に、対応関係を明示できる。

4.2.2　集合と要素の所属

集合（set）は、重複を持たない要素の集まりである。要素の表示順序や添字による位置は保証されない。in は要素が含まれるかを判定し、len() は要素数を返す。両者はリストや辞書などにも使え、辞書に対する in はキーの存在を調べる。

```
1 detectors = {"A", "B", "A"}
2 print(len(detectors))                # 2
3 print("A" in detectors)               # True
4 print(detectors & {"B", "C"})         # {'B'}
5 empty_set = set()
```

& は共通部分、| は和集合、- は差集合を求める。空の集合は set() で作る。{} は空の辞書である。集合の要素と辞書のキーには、文字列や整数など、検索に使う数値（ハッシュ値）を計算できる値を使う。この性質をハッシュ可能という。リストはそのままでは使えない。

4.2.3　代入とコピー

b = a は、a が参照する値に別の名前を付ける代入であり、リストのコピーを作る操作ではない。

```
1 a = [1, 2]
2 b = a
3 b.append(3)
4 print(a)                      # [1, 2, 3]
5 c = a.copy()
6 c.append(4)
7 print(a)                      # [1, 2, 3]
```

copy() は外側のリストを複製する浅いコピーである。入れ子になったリストなどの要素は共有される。タプルは要素の差し替えができないが、その中にあるリスト自体の変更までは禁止しない。

4.2.4　リストのメソッド

リストの要素追加、並べ替え、取り出しに用いる主なメソッドを次に示す。

```
>>> values = [3, 1, 4]
```

```
>>> values
[3, 1, 4]
>>> values.append(1)
>>> values
[3, 1, 4, 1]
>>> values.sort()
>>> values
[1, 1, 3, 4]
>>> last = values.pop()
>>> last
4
>>> values
[1, 1, 3]
```

`append()` は末尾へ追加し、`sort()` はその場で並べ替え、`pop()` は末尾要素を取り出して削除する。

リストメソッドの多くは「その場で変更する」点に注意が必要である。新しいリストを返すのではなく、元のリスト自体を書き換える操作が多いので、どこで状態が変わるかを意識して読む必要がある。

特に `sort()` は、並べ替えたリストを返すのではなく、元のリストそのものを変更する。そのため、次のように書くと意図とずれることがある。

```
>>> values = [3, 1, 4]
>>> result = values.sort()
>>> print(result)
None
>>> values
[1, 3, 4]
```

元のリストを保持し、並べ替えた新しいリストを作る場合は `sorted()` を用いる。

```
>>> values = [3, 1, 4]
>>> new_values = sorted(values)
>>> values
[3, 1, 4]
>>> new_values
[1, 3, 4]
```

4.2.5　インデックスとスライス

解析では、配列や文字列の一部を取り出す操作が頻出する。Python では、`[]` の中に位置を指定して要素を取り出す。この位置を表す整数をインデックスとよぶ。`:` を用いて範囲を切り出す操作はスライスとよばれる。

```
>>> values = [10, 20, 30, 40, 50]
>>>
>>> values[0]
```

```
10
>>> values[-1]
50
>>> values[1:4]
[20, 30, 40]
>>> values[:3]
[10, 20, 30]
>>> values[2:]
[30, 40, 50]
>>> values[:-1]
[10, 20, 30, 40]
```

リストのインデックスは `0` から始まる。負のインデックスは末尾から数え、`-1` は最後の要素を表す。この例では、`values[1:4]` はインデックス 1 から 3 までの要素、`values[:3]` は `values[3]` より前の要素、`values[2:]` は `values[2]` 以降の要素を取り出す。リストのスライスでは、`:` の左側に指定した位置の要素は含まれるが、右側に指定した位置の要素は含まれない。

文字列に対しても同じ考え方が使える。例えば次のようになる。

```
>>> hhmmss = "123045"
>>> hhmmss[0:2]
'12'
>>> hhmmss[2:4]
'30'
>>> hhmmss[4:6]
'45'
```

`hhmmss[0:2]`、`hhmmss[2:4]`、`hhmmss[4:6]` は、それぞれ時、分、秒に対応する 2 文字を取り出す。開始位置を含み終了位置を含まないスライス規則は、いずれの部分文字列にも共通する。

4.3 条件分岐と反復処理

4.3.1 条件分岐

条件によって処理を変えるには `if` を使う。最も基本的な形は次の通りである。

```
>>> value = 2.5
>>> threshold = 3.0
>>> if value <= threshold:
...     print("small")
... else:
...     print("large")
...
small
```

比較に使う主な演算子には、==、!=、<、<=、>、>= がある。たとえば value <= threshold は「value が threshold 以下なら真」を意味する。条件式そのものの値は真偽値であり、True か False のどちらかになる。

```
>>> value = 5
>>> threshold = 3
>>> value > threshold
True
>>> value == threshold
False
```

条件が 3 通り以上に分かれる場合は、if に続けて elif を置く。条件は上から順に評価され、最初に真となった分岐だけが実行される。

```
>>> value = 3.0
>>> threshold = 3.0
>>> if value < threshold:
...     print("small")
... elif value == threshold:
...     print("equal")
... else:
...     print("large")
...
equal
```

解析コードでは、閾値判定、ファイル名の選別、欠損値の除外などに条件分岐を用いる。条件を決めたら、境界値、採用理由、条件を満たす場合と満たさない場合の処理を記録する。

条件分岐が増えて読みにくくなったときは、条件式そのものを変数へ切り出すと意図が明確になる。たとえば is_valid_row = value <= threshold のように名前を付けると、後続の条件分岐を短く記述できる。

複数の条件を同時に満たす場合には and、少なくとも一方を満たす場合には or、真偽を反転する場合には not を用いる。条件が長い場合は、物理的または解析上の意味ごとに変数へ分けると、各条件を個別に検証できる。

```
>>> value = 5
>>> lower = 3
>>> upper = 10
>>> use_log = False
>>> lower <= value and value <= upper
True
>>> value < lower or value > upper
False
>>> not use_log
True
```

否定が重なる条件や、複数の and と or が混在する条件では、括弧で評価順序を明示するか、意味を表す一時変数に分割する。条件式には、境界値と真偽の組み合わせを検証するテストを用意する。

`if` などの末尾のコロン `:` に続く処理のまとまりをブロックと呼ぶ。Python では、行頭の空白による字下げ（インデント）でブロックを表し、同じ深さの処理をそろえる。通常は 1 段につき空白 4 個を使い、タブと空白を混在させない。

4.3.2 反復処理

`for` 文は、複数の値を順に取り出して同じ処理を適用する構文である。

```
>>> for label in labels:
...     print(label)
...
A
B_detector
C
D
```

要素とその番号（添字）の両方が必要な場合は、`enumerate` を組み合わせる。

```
>>> for i, value in enumerate(labels):
...     print(i, value)
...
0 A
1 B_detector
2 C
3 D
```

一方、大規模なデータでは、Python レベルの `for` ループが処理時間のボトルネックになることがある。その回避方法は本書の後半で扱う。

まず、反復の対象と更新される値が明確な `for` 文を作る。その動作を検証した後で NumPy や pandas の配列演算へ置き換えると、処理内容を保ったまま高速化できる。

4.3.2.1 `range` と `while`

`for` 文と並んで、反復処理では `range` と `while` もよく用いる。`range` は連続した整数を順に生成し、`while` は条件が成り立つ間くり返す。記述例を次に示す。

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
>>> count = 0
>>> while count < 3:
...     print(count)
...     count += 1
```

```
...
0
1
2
```

`range(5)` は `0` から `4` までの整数を順に生成し、ファイル番号やビン番号を使う反復処理に利用できる。`while` は、各反復の開始時に条件を評価し、真である間だけ処理を続ける。時刻順のデータについて、時間差が閾値以下である間だけ比較する処理などに対応する。終了条件に関係する変数を更新しなければ反復が終了しないため、境界条件をテストする必要がある。

4.3.2.2　`break`, `continue`, `else`

反復処理では、途中で打ち切ったり、条件に合う要素の処理を飛ばしたりしたくなる。Python では `break` と `continue` がそのために使える。

```
>>> values = [-1, 1, 3, 5]
>>> threshold = 4
>>> for value in values:
...     if value < 0:
...         continue
...     if value > threshold:
...         break
...     print(value)
...
1
3
```

`continue` は現在の反復だけを打ち切り、`break` はループ全体を終了する。さらに `for ... else` 構文では、`break` せずに反復を終えた場合だけ `else` 節が実行される。探索対象が見つかった場合に `break` し、最後まで見つからなかった場合の処理を `else` 節へ置く用途などに使える。

4.3.2.3　内包表記

各要素に一つの変換を施す処理や、一つの条件で要素を抽出する処理は、内包表記によって一つの式として記述できる。複数の分岐や副作用を含む処理では、通常の `for` 文を用いる方が処理順序を明示しやすい。

```
>>> values = [1, 2, 3, 4, 5]
>>> squares = [x * x for x in values]
>>> large_values = [x for x in values if x >= 3]
>>> squares
[1, 4, 9, 16, 25]
>>> large_values
[3, 4, 5]
```

内包表記に複数段階の条件や複雑な変換を詰め込むと、処理順序を追いにくくなる。その場合は、複数行の `for` 文または名前を付けた補助関数へ分ける。

4.4 関数とモジュール

4.4.1 関数を分ける理由

入力に対する処理をひとまとめにし、名前を付けたものが関数である。処理の結果は返り値として渡せる。一例として、時刻を表す文字列から秒数を計算する関数を考える。

```
def hhmmss_to_seconds(hhmmss):
    hour = int(hhmmss[0:2])
    minute = int(hhmmss[2:4])
    second = int(hhmmss[4:6])
    return hour * 3600 + minute * 60 + second
```

この関数は短いが、「文字列で与えられた時刻を秒へ変換する」という役割が明確である。ここから先は、複数行の処理をまとまりとして読むため、短いスクリプトをファイルへ保存して実行する形も使う。たとえば次の内容を `time_conversion.py` という名前で保存すれば、ターミナルで `python3 time_conversion.py` と実行できる。

time_conversion.py

```
def hhmmss_to_seconds(hhmmss):
    hour = int(hhmmss[0:2])
    minute = int(hhmmss[2:4])
    second = int(hhmmss[4:6])
    return hour * 3600 + minute * 60 + second

print(hhmmss_to_seconds("013015"))
```

```
$ python3 time_conversion.py
4515
```

`return` に値を付けない場合や、`return` がない関数では、`None` が返る。平均のような結果を次の計算で使いたい場合は、返す値を明示する。

`def` は関数を定義するための語であり、続く名前が関数名である。`hhmmss` は受け取る文字列の名前である。`return` の後ろに、返す値や計算式を書く。`"013015"` は 1 時間 30 分 15 秒なので、$3600 + 1800 + 15 = 4515$ 秒になる。関数を定義しただけでは計算は始まらず、`hhmmss_to_seconds("013015")` と呼んだときに中の処理が実行される。

4.4.1.1　関数の引数

関数の引数には、値の意味と省略時の既定値を定める。Python では位置引数、既定値付き引数、キーワード引数を使用できる。

```
def histogram_path(name, suffix=".pdf"):
    return f"{name}{suffix}"

print(histogram_path("value_hist"))
print(histogram_path("value_hist", suffix=".png"))
```

```
value_hist.pdf
value_hist.png
```

引数に既定値（デフォルト値）を設定しておくと、呼び出し時に省略できる。呼び出し側は、その値を使う場合には引数を省略し、別の値が必要な場合には位置引数またはキーワード引数で上書きする。

ただし、可変オブジェクトをデフォルト引数に置くと、複数回の関数呼び出しで同じオブジェクトが共有される。Python 公式チュートリアル [1] でも説明されているように、リストや辞書をデフォルト値として直接指定する次の書き方は避ける。

```
def bad_append(value, buffer=[]):
    buffer.append(value)
    return buffer
```

この例では、`buffer` が呼び出しのたびに新しく作られるわけではない。そのため、前回の状態が次回にも残る。実際に呼ぶと次のようになる。

```
print(bad_append(1))
print(bad_append(2))
```

```
[1]
[1, 2]
```

可変オブジェクトを既定値として必要とする場合は、既定値を `None` とし、関数内部で新しいオブジェクトを作る。

```
def good_append(value, buffer=None):
    if buffer is None:
        buffer = []
    buffer.append(value)
    return buffer
```

```
print(good_append(1))
print(good_append(2))
```

```
[1]
[2]
```

`bad_append()` は引数を省略した呼び出しの間でリストを共有する。`good_append()` は省略時に毎回新しいリストを作るため、その共有を避けられる。ただし、呼び出し側がリストを明示的に渡した場合は、そのリストを変更する。

4.4.1.2 モジュールと `import`

別ファイルに書かれた関数を使うには次のように `import` する。たとえば標準ライブラリの `math` を使うと、円周率や平方根を次のように扱える。

```
import math

print(math.pi)
print(math.sqrt(2.0))
```

```
3.141592653589793
1.4142135623730951
```

Python の解析では、標準ライブラリと外部ライブラリを組み合わせて使う。各名前の定義元を確認すると、規模の大きいコードでも関数やクラスの役割を追いやすい。

同じ名前でも、標準ライブラリと外部ライブラリでは定義元や役割が異なる場合がある。インポート文は、そのファイルが依存するモジュールと、名前の定義元を示す。既存のライブラリだけでなく、自分が作成した Python ファイルも同じ仕組みで読み込める。

4.4.1.3 自作モジュールへの分離

解析コードの規模が大きくなると、一つのファイルでは機能ごとの境界を追いにくくなる。再利用する関数や定数を別ファイルへ分け、モジュールとして読み込む構成を次に示す。

```
analysis_project/
¦-- constants.py
¦-- physics.py
`-- main.py
```

この構成にすると、`physics.py` に書いた関数を `main.py` から読み込める。次の例は、その最小形である。

```
# physics.py
import math

def deg_to_rad(degrees):
    return degrees * math.pi / 180.0


# main.py
import physics

theta = physics.deg_to_rad(45.0)
```

この形にしておくと、座標変換、時刻変換、定数定義のような再利用しやすい処理を 1 か所へまとめられる。たとえば `main.py` 側で `print(theta)` と書けば、結果は次のようになる。

```
0.7853981633974483
```

同じ Python ファイルは、モジュールとして別のファイルから import することも、スクリプトとして直接実行することもできる。次に示す `if __name__ == "__main__":` は、直接実行した場合だけ処理を呼び出す条件である。

4.4.1.4　スクリプトとしての実行

Python ファイルは、他のファイルから読み込まれることもあれば、直接実行されることもある。その違いを区別する基本形が次である。

```
1 def main():
2     print("run")
3
4 if __name__ == "__main__":
5     main()
```

import した場合は関数定義だけが読み込まれ、`main()` は呼び出されない。ファイルを直接実行した場合は条件が真となり、`main()` が呼び出される。

4.4.2　実行時に入力先や出力先を指定する

入力ディレクトリや出力先が実行ごとに変わる解析スクリプトでは、それらをコードへ埋め込まず、コマンドライン引数として受け取る。Python 標準ライブラリの `argparse` を用いた最小構成を次に示す。

```
1 import argparse
2
3 parser = argparse.ArgumentParser()
4 parser.add_argument("input_dir")
5 parser.add_argument("--outdir", default="output")
6 args = parser.parse_args()
7
8 print(args.input_dir, args.outdir)
```

このスクリプトを `analyze.py` として保存した場合の実行例を次に示す。

```
$ python analyze.py data/run001
data/run001 output
$ python analyze.py data/run001 --outdir results/figures
data/run001 results/figures
```

1 行目では、位置引数 `input_dir` に `data/run001` が入り、`--outdir` は省略したため既定値 `output` が使われる。2 行目では、`--outdir` の指定によって `results/figures` が使われる。こ

の出力から、input_dir が必須の位置引数、--outdir が既定値を持つオプション引数であることを確認できる。

4.4.3 短い関数と値の展開

lambda 引数: 式 は、式の値を返す名前のない短い関数を作る。複数の処理が必要な場合は def で名前付きの関数を定義する。

```
rows = [("A", 3.0), ("B", 1.0)]
print(sorted(rows, key=lambda row: row[1]))
# [('B', 1.0), ('A', 3.0)]
name, value = rows[0]
print(name, value)                  # A 3.0
```

name, value = rows[0] は、二つの要素を二つの変数へ分けるアンパックである。また、関数呼び出しの f(*params) は、リストやタプルの要素を位置引数として展開する。たとえば params = (2, 3) なら f(*params) は f(2, 3) と同じ呼び出しになる。

4.5 例外・docstring・命名・設定値

4.5.1 例外名とトレースバックによる原因の特定

処理を続けられない問題が起きたことを知らせる仕組みを例外という。例外が起きると、通常はそこで実行が止まり、トレースバックというエラーの記録が表示される。この記録には、呼び出された処理、問題が起きた行、例外名、詳しい理由が示される。原因を調べるときは、末尾から順に確認する。

たとえば FileNotFoundError ならファイルパスが違うかもしれないし、ValueError なら想定した形式でデータを読めていないかもしれない。実際のエラー表示は次のようになる。

```
with open("missing.txt") as f:
    text = f.read()
```

```
Traceback (most recent call last):
  File "read_missing_file.py", line 1, in <module>
    with open("missing.txt") as f:
FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'
```

まず例外名と末尾の数行を確認し、発生したファイルと行番号を特定する。

解析コードでは、開発中にエラーが発生すること自体は珍しくない。問題は、エラーを無視したり、意味を確かめずに場当たり的な修正を加えたりすることである。例外名、発生行、直前に読み込んだデータの形を順に確認すると、原因を切り分けやすい。

4.5.1.1　docstring とヘルプ

関数定義の直後に置いた文字列は、docstring（ドキュメント文字列）として help() などから参照できる。役割、引数、返り値、例外条件を記述する。

```
def read_table(path):
    """空白区切りの表を読み、各行を辞書として返す。"""
    ...
```

help(read_table) を実行すると、この説明を後から確認できる。たとえば要約だけの docstring なら、次のように表示される。

```
help(read_table)
```

```
Help on function read_table:

read_table(path)
    空白区切りの表を読み、各行を辞書として返す。
```

docstring はコメントと異なり、実行時に help() から参照できる。関数名.__doc__ でも読み取れる。このように、値や関数に付随し、ドットと項目名で取り出す情報を属性という。公開関数には、入力、返り値、例外条件を記録する。

1 行の要約だけでは使い方を説明できない関数では、引数、返り値、例外条件を複数行の docstring に記述する。

```
def read_table(path):
    """空白区切りの表を読み込む。

    Args:
        path: 入力ファイルへのパス。

    Returns:
        各行を辞書として格納したリスト。

    Raises:
        FileNotFoundError: ファイルが存在しない場合。
    """
    ...
```

最初の 1 行には関数の要約を置き、空行の後に引数、返り値、例外条件などの詳細を置く。help() やエディタの補完表示は、この docstring を関数の説明として利用する。

4.5.1.2　名前付けと PEP 8

Python には PEP 8 という代表的なスタイルガイドがある。変数名、関数名、空白の入れ方をそろえるだけでも、コードの構造を追いやすくなる。

```
MAX_EVENTS = 1000
```

```
2
3 def read_event_table(path):
4     total_count = 0
5     ...
```

本書では、変数名と関数名に `snake_case`、定数名に `UPPER_CASE` を用いる。演算子の前後の空白や改行位置も統一し、同じ構造を同じ書式で表す。

PEP 8 のうち、本書の例で一貫して用いる規則を次に示す。

- タブではなく 4 スペースによるインデント
- 関数名・変数名には `snake_case`、クラス名には `CamelCase`、定数には `UPPER_CASE` という命名規則
- 演算子（`=`, `+`, `-` など）の前後と、カンマの後ろの空白
- トップレベルの関数定義・クラス定義を区切る空行
- 過度に長い行の回避（PEP 8 の目安は 79 文字）
- 標準ライブラリ、外部ライブラリ、自作モジュールの順にまとめた `import`

これらの規則をプロジェクト全体に適用すると、表記のばらつきを減らせる。自動整形ツールを利用する場合も、採用した規則と対象ファイルをプロジェクト内で統一する。

4.5.1.3　マジックナンバーを避ける

コード中に `250` や `3600` のような数値が説明なしに直接記述されていると、その意味や単位を読む側が推測しなければならない。このような数値をマジックナンバーと呼ぶ。次の例では、意味と単位を表す名前を付ける。

```
NS_PER_SECOND = 1_000_000_000
COINCIDENCE_WINDOW_NS = 250

second_diff = 1          # 秒の部分の差
ns_diff = -999_999_780   # 秒未満の部分の差
dt_ns = second_diff * NS_PER_SECOND + ns_diff
print(dt_ns)             # 220
is_coincident = abs(dt_ns) <= COINCIDENCE_WINDOW_NS
```

`COINCIDENCE_WINDOW_NS` という名前は、`250` が一致判定の時間窓であり、単位が ns であることを表す。同様に、`SECONDS_PER_HOUR` と `SECONDS_PER_DAY` は、`3600` と `86400` の意味を名前で表している。

物理定数や単位換算のように定義として固定する値は、名前と単位を明示したコード内定数にする。一方、入力パス、出力先、閾値、検出器番号のように実行条件として変わる値は、コマンドライン引数または設定ファイルへ分離する。値の変更頻度と共有範囲が、配置を決める基準となる。

4.5.1.4　例外処理

エラーは読むべき情報だが、予想される失敗を自分で受け止めたい場面もある。例えば、1行だけ形式が壊れているデータを見つけて記録し、解析全体は続けたいことがある。そのときに使うのが次のような `try` と `except` の組み合わせである。

```
try:
    value = float(text)
except ValueError:
    print("数値へ変換できませんでした")
```

たとえば `text = "abc"` のような文字列に対して `float(text)` を呼ぶと、`ValueError` が起きる。すべての例外を `except:` で捕捉すると、本来停止すべき不具合まで隠れる。捕捉する例外を明示し、失敗した入力の記録、処理を中止する条件、除外して継続する条件をあらかじめ定める。

入力が関数の条件を満たさない場合は、`raise` で例外を発生させる。

```
def positive_seconds(value):
    seconds = float(value)
    if seconds <= 0:
        raise ValueError("seconds must be positive")
    return seconds
```

この例は正の有限値だけを保証する検査ではなく、`nan` や `inf` も入力し得る場合には `math.isfinite(seconds)` による検査を追加する。命名や字下げの慣例は Python のスタイルガイド PEP 8 [20] にまとめられている。

4.6　章末課題：記録を選び、平均と時刻差を求める

次の 4 件を使う。`time_ns` はナノ秒単位の記録時刻、`signal` は信号値、`None` は欠測を表す。

```
events = [
    {"detector": "A", "time_ns": 1000, "signal": 12.4},
    {"detector": "B", "time_ns": 1180, "signal": 9.8},
    {"detector": "A", "time_ns": 1300, "signal": 15.1},
    {"detector": "B", "time_ns": 1510, "signal": None},
]
```

1. **欠測を除いて表示する。** `for` と `if` を使い、信号値が `None` でない 3 件を「検出器名、時刻、信号値」の順で表示せよ。表示結果を示し、除いた記録を説明せよ。
2. **検出器ごとの平均を求める。** `mean_signal(events, detector)` を作れ。欠測を除き、有効値がないときは `ValueError` を出すこと。A の平均 13.75 と B の平均 9.8 を表にし、平均に使った件数を併記せよ。関数には引数、返り値、例外条件を説明する docstring を付ける。

3. **隣り合う記録の時刻差を求める。**時刻を取り出し、昇順に並べて隣との差を計算せよ。結果 `[180, 120, 210]` を単位付きで示し、4 件の記録から差が 3 個得られる理由を説明せよ。
4. **実行時に抽出条件を変える。**検出器名を位置引数、信号の下限を `--min-signal`（既定値 0.0）で受け取る `filter_events.py` を作れ。`python filter_events.py A --min-signal 13` で時刻 1300 の 1 件が残ることを確かめ、下限を変えた結果と比較せよ。

第 5 章

入出力とファイル

この章では、ファイルの文字を読み、必要な値を数値へ変換し、計算結果を別のファイルへ保存する。データを読み込むことを入力、結果を書き出すことを出力という。まず 3 行のファイルを表示し、その後で表の読み込みと保存を学ぶ。読み戻した内容が保存前と一致するかを確かめられることが目標である。

5.1 テキストファイルの入出力と探索

5.1.1 ファイルのオープンと基本的な読み書き

Python でテキストファイルを読むときの基本は `open` である。さらに `with` を組み合わせると、処理が正常に終わった場合だけでなく、途中で例外が発生した場合にもファイルが閉じられる。まず、次の 3 行をテキストファイル `sample.txt` として保存する。保存先は、端末で `pwd` を実行して表示される作業フォルダーとする。

sample.txt

```
run 1: 12.4
run 2: 10.1
run 3: 14.8
```

次を同じ作業フォルダーの read_sample.py へ保存し、端末で python read_sample.py を実行する。

```
path = "sample.txt"

with open(path, "r", encoding="utf-8") as f:
    for line in f:
        print(line.rstrip("\r\n"))
```

実行結果は次の通りである。

```
run 1: 12.4
run 2: 10.1
run 3: 14.8
```

第 1 引数の path は開くファイルの場所、"r" は読み込みモード、encoding="utf-8" は文字を UTF-8 として解釈する指定である。as f によって、開いたファイルを表すオブジェクトを変数 f で参照する。for line in f は全行を一度にメモリへ置かず、1 行ずつ取り出す。

引数なしの read() は、現在の読み取り位置から末尾までを一度に読み込む。開いた直後ならファイル全体が対象になる。入力容量に比例したメモリを要するため、大きなテキストファイルは反復処理によって 1 行ずつ読み込む。

このとき、line には行末文字が含まれる。CR（\r）と LF（\n）は改行に使われる文字である。rstrip("\r\n") は CR と LF だけを末尾から取り除き、データに含まれる通常の空白は残す。引数なしの rstrip() は末尾の空白も削除するため、空白自体がデータである場合には区別が必要である。

5.1.1.1　pathlib によるパス操作

pathlib.Path は、パス要素の結合、存在確認、名前や拡張子の取得をメソッドと演算子で提供する。

```
from pathlib import Path

root = Path(".")
path = root / "sample.txt"

print(path.exists())
print(path.name)
```

この例でファイルが存在する場合、出力は次のようになる。

```
True
sample.txt
```

exists() はパスが存在するかを真偽値で返し、name は末尾のファイル名だけを返す。root / "sample.txt" の / は除算ではなく、Path オブジェクトに対してパス要素を結合する演算子である。例えば Path("data") / "2026" / "04" / "sample.txt" とすれば、複数階層のパスも

同じ方法で組み立てられる。

Path の / 演算子は、OS ごとの区切り文字を直接記述せずにパス要素を結合する。name、parent、with_suffix() は、それぞれ末尾名、親ディレクトリ、拡張子の置換を表す。pathlib を用いて出力ディレクトリとファイルパスを作る例を次に示す。

```
outdir = Path("output")
outdir.mkdir(parents=True, exist_ok=True)
outfile = outdir / "summary.txt"
```

parents=True は output より上位の未作成ディレクトリも必要に応じて作る指定であり、exist_ok=True は既存のディレクトリをエラーにしない指定である。ただし、同名の通常ファイルが存在する場合はエラーになる。

この指定により、出力ディレクトリが存在しない場合は作成し、既存のディレクトリであれば処理を継続する。出力ファイルの上書き可否は、mkdir とは別に検査する。

5.1.1.2　ファイルオブジェクトのメソッド

ファイルオブジェクトは、read、readline、write などのメソッドを持つ。ファイル全体を文字列として取得する処理と、現在位置から 1 行を取得する処理を区別する。Python 公式チュートリアル [1] に基づく最小例を次に示す。

```
with open(path, "r", encoding="utf-8") as f:
    first_line = f.readline()

with open("out.txt", "w", encoding="utf-8") as f:
    f.write("hello\n")
```

"w" は書き込みモードであり、同名ファイルがあれば内容を置き換える。この例は実習用の out.txt を作る。readline() の返り値には末尾の改行が含まれる。write() は書き込んだ文字数を返すが、改行を自動では補わないため、行末が必要なら "\n" を明示する。この例を実行すると、out.txt の内容は hello と改行になる。

readline は現在位置から 1 行を読み、write は指定した文字列をファイルへ書く。大きな入力では、全体を読み込む前に readline や第 2 章のシェルコマンドで先頭数行を確認できる。

5.1.1.3　複数ファイルの収集

ログや測定結果が複数ファイルに分かれている場合は、処理前に対象パスの一覧を確定する。glob.glob による再帰的な探索例を次に示す。

```
import glob

paths = sorted(glob.glob("data/**/*.txt", recursive=True))
print(f"count = {len(paths)}")
for path in paths[:3]:
```

```
    print(path)
```

例えば対象が 3 ファイルなら、出力は次のようになる。

```
count = 3
data/2025/run01.txt
data/2026/run01.txt
data/2026/run02.txt
```

パターン中の `*` は同じ階層の任意の文字列、`**` は `recursive=True` と組み合わせたときに複数階層へ一致する。`paths[:3]` は確認のために先頭 3 件だけを表示するスライスであり、処理対象を 3 件へ制限しているわけではない。

`glob.glob` はシェルのワイルドカードに近い書式で使える。複数ファイルを処理する前に、見つかった件数と先頭数件のパスを表示すれば、探索範囲の誤りを検出しやすい。

`glob.glob` の返す順序を処理条件として仮定せず、時系列や連番を順に処理する場合は `sorted` で順序を明示する。これにより、処理順序とログの順序を実行環境に依存させずに再現できる。

5.1.2　Python での標準入出力

Python では、`sys.stdin`、`sys.stdout`、`sys.stderr` が三つの標準的な経路に対応する [21]。`print()` は既定で標準出力へ書き込み、`file=` で別の経路を指定できる。次の内容を `uppercase.py` に保存する。

```
import sys

n_lines = 0
for line in sys.stdin:
    print(line.rstrip("\r\n").upper())
    n_lines += 1
print(f"Processed {n_lines} lines", file=sys.stderr)
```

```
python uppercase.py < sample.txt > converted.txt 2> conversion.log
```

変換した本文は `converted.txt` に、処理行数は `conversion.log` に保存される。`input()` は標準入力から 1 行を読み、末尾の改行を除いた文字列を返す関数である。引数に案内文を渡すと標準出力へ表示するため、データをパイプで受け渡すプログラムでは案内文の混入に注意する。

5.2　文字列データの解析と整形

5.2.1　CSV 規則に従った列名付きデータの読み込み

CSV は、列をカンマで区切って保存する表の形式である。1 件分をレコード、その中の各項目をフィールドという。引用符で囲まれたフィールド内には、カンマや改行を含められる。`line.split(",")` はこの規則を解釈できないため、標準ライブラリの `csv` でレコードとフィールドを読み取る。

次の内容を、現在の作業フォルダーへ `events.csv` という名前で保存する。

events.csv

```
time,mag,place
2026-04-04T12:00:00Z,3.2,"Offshore, Japan"
2026-04-04T12:05:00Z,2.8,Tokyo
```

保存後、次のコードを実行する。

```
1  import csv
2  from pathlib import Path
3
4  path = Path("events.csv")
5
6  with path.open("r", encoding="utf-8", newline="") as f:
7      reader = csv.DictReader(f)
8      first_row = next(reader)
9
10 print(first_row["time"])
11 print(first_row["mag"])
```

先頭データ行の `time` と `mag` が文字列として読み込まれるため、実行結果は次の通りである。

```
2026-04-04T12:00:00Z
3.2
```

`DictReader` は 1 行目を列名として読み、その後の各行を辞書として返す。`next(reader)` は次のデータ行を 1 件取り出す。この例では最初のデータ行を `first_row` に入れる。したがって `row["mag"]` のように列名で値を取り出せる。`newline=""` は、テキスト層で改行変換を行わず、CSV モジュールに改行処理を任せる指定である。

列内のカンマが CSV の規則に従って引用されていれば、`csv` モジュールは一つのフィールドとして返す。上の `place` 列の `"Offshore, Japan"` も分割されない。

5.2.2　文字列から値への変換と結果の出力

多くの表形式データは、1 行の中に複数の列が並ぶ形で保存されている。テキストとして読んだだけでは、まだ単なる文字列である。そこから列に分け、整数や浮動小数点数へ変換して初めて解析に使える。たとえば空白区切りの 1 行を分解する例は次のようになる。

```
line = "17 2026-04-04 12:34:56 15.2 OK"
columns = line.split()

record_id = int(columns[0])
date_text = columns[1]
time_text = columns[2]
value = float(columns[3])
status = columns[4]

print(record_id, date_text, time_text, value, status)
```

`split()` は引数を省略すると連続する空白を一つの区切りとして扱う。`int()` と `float()` が文字列を数値へ変換するため、出力は次のようになる。

```
17 2026-04-04 12:34:56 15.2 OK
```

入力仕様に基づいて、各列の型と単位を決める。識別子の先頭の 0 や、日時文字列の元表現を保持する必要がある場合は、数値へ変換せず文字列として扱う。変換処理を読み込み直後の一か所にまとめると、文字列と数値が混在する範囲を限定できる。

5.2.2.1　ファイルへの書き出し

複数行のテキストを出力ファイルへ順に書き込む例を示す。

```
lines = ["run_id count mean", "1 120 3.42", "2 118 3.37"]

with open("summary.txt", "w", encoding="utf-8") as f:
    for line in lines:
        f.write(line + "\n")
```

このコードの実行後、`summary.txt` には `lines` の 3 要素が 1 行ずつ保存される。`"w"` は、ファイルが存在すれば内容を空にしてから書き、存在しなければ新しく作るモードである。これに対して `"a"` は既存内容の末尾へ追記する。既存ファイルを上書きしてよいのか、別名で保存すべきかは、実験ノートやレポートの再現性にも関わる。出力ファイル名の設計も解析手順に含まれる。

5.2.2.2　文字列整形

f-string（フォーマット済み文字列リテラル）は、文字列内に値と表示書式を埋め込む。

```
mean = 3.14159
sigma = 0.27182

text = f"mean = {mean:.3f}, sigma = {sigma:.3f}"
print(text)
```

実行結果は `mean = 3.142, sigma = 0.272` となる。`f` は固定小数点形式、`.3` は小数点以下の桁数を指定する。これは表示上の丸めであり、変数 `mean` と `sigma` に保存された値そのものは変化しない。

`:.3f` は小数点以下 3 桁の固定小数点形式を指定する。表示桁数は、測定精度や不確かさに基づいて決め、表、図、本文で同じ丸め規則を用いる。

Python 公式チュートリアル [1] では、f-string のほかに `str.format` や `%` 演算子による書式指定も紹介されている。本書では、式と書式を同じ文字列内に明示できる f-string を主に用いる。

5.3　オブジェクトと構造化データの保存

5.3.1　JSON と Pickle / PKL

結果や設定を構造化して保存したいときは、JSON を利用できる。テキストとして内容を確認でき、Python では標準ライブラリ `json` で扱える。保存と読み込みの基本形を次に示す。

```
import json

config = {"threshold": 3.5, "bins": 40}

with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

with open("config.json", "r", encoding="utf-8") as f:
    loaded_config = json.load(f)

print(loaded_config)
```

`indent=2` は、入れ子の階層ごとに 2 個の空白で字下げし、要素ごとに改行した JSON を保存する指定である。読み戻した結果は次のように表示される。

```
{'threshold': 3.5, 'bins': 40}
```

JSON のオブジェクトは Python では辞書として復元される。

JSON は、キーと値、配列、入れ子構造をテキストとして保持する。一方、列ごとの型や圧縮方式を持たないため、大規模な表の反復読み込みには Parquet などの列指向形式も検討する。

5.3.1.1 Pickle / PKL

Python オブジェクトをそのまま保存する形式として、pickle がある。拡張子には慣習的に .pkl が使われる。辞書、リスト、NumPy 配列、学習済みモデル、中間集計結果などを作業用に一時保存する例を次に示す。

```python
import pickle

result = {"mean": 3.14, "sigma": 0.27, "values": [1.0, 2.0, 3.0]}

with open("result.pkl", "wb") as f:
    pickle.dump(result, f)

with open("result.pkl", "rb") as f:
    loaded_result = pickle.load(f)

print(loaded_result["mean"])
print(loaded_result["values"])
```

復元後の出力は次の通りである。

```
3.14
[1.0, 2.0, 3.0]
```

"wb" と "rb" の b はバイナリモードを意味する。JSON と違って内容をテキストとして確認できないが、Python オブジェクトの構造を保存できる。ただし、Python やライブラリのバージョンが異なると復元できるとは限らず、他言語との交換にも向かない。長期保存や共同研究の交換形式ではなく、生成環境と入力を記録したローカルな作業用キャッシュとして用いる。

Pickle の読み込みには、コード実行に関する注意も必要である。pickle.load() は、オブジェクトを復元する過程で任意のコードを実行する可能性がある。出所と生成過程を確認できない .pkl は読み込まず、自分で生成したもの、または信頼できる相手から安全な経路で受け取ったものだけを対象とする。

5.4 日時データの変換と保存形式の選択

5.4.1 日付・時刻の数値表現

日付や時刻を比較し、差を計算する場合は、文字列を時刻オブジェクトまたは数値表現へ変換する。標準ライブラリの datetime を用いた例を次に示す。

```python
from datetime import datetime, timezone

text = "2026-04-04 12:34:56.123456"
dt = datetime.strptime(text, "%Y-%m-%d %H:%M:%S.%f")
```

```
# 入力の時刻が UTC であることを確認した上で指定する
dt = dt.replace(tzinfo=timezone.utc)
unix_time = dt.timestamp()
print(dt)
print(unix_time)
```

`strptime()` の第 1 引数は変換する文字列、第 2 引数は文字列の形式である。`%Y`、`%m`、`%d` は年、月、日を表し、`%H`、`%M`、`%S` は時、分、秒、`%f` はマイクロ秒を表す。`timestamp()` は Unix epoch（1970 年 1 月 1 日 00:00:00 UTC）からの秒数を返すが、タイムゾーン情報を持たない `datetime` の解釈は実行環境のローカルタイムゾーンに依存する。UTC のログでは、タイムゾーンを明示するか、後述する Astropy の `Time` を使う必要がある。

時刻差を保持する単位は、必要な時間分解能と値域に基づいて選ぶ。別の表現へ変換する場合は、基準時刻、時刻系、単位、丸め方法を記録する。

`datetime` オブジェクトは暦に沿った表示と計算を担う。一方、大量の時刻差を配列演算で求める場合は、基準時刻からの秒やナノ秒として表す方が適することもある。表示用の表現と計算用の表現を分ける場合は、基準時刻、単位、時刻系を記録する。

さらに、タイムゾーンが関わるデータでは、見かけの時刻だけを比較すると誤解が生じる。研究用のログや観測データでは UTC を使うことが多いが、比較前に各データの基準時刻を確認する必要がある。

5.4.2　CSV・JSON とバイナリ形式の選択

CSV はテキストとして内容を確認できる一方、読み込み時には文字列から各列型への変換が必要となる。列数が多い場合は、使用しない列の読み込みも所要時間とメモリ使用量を増やす。保存形式は、交換先が利用できる実装、読み込む列と行の範囲、測定した入出力時間を基準に選ぶ。

5.4.2.1　CSV・JSON の可読性と変換コスト

テキスト形式は、UTF-8 などの文字コードを用いて人間が読める形で保存する。CSV/TSV は多くのプログラミング言語や表計算ソフトで読み書きできるため、データ交換に利用しやすい。

計算に使う際には、ディスク上の文字列（例:　「1.2345」）を実数値へ変換する処理（パース）が必要になる。大規模データでは、この変換と区切り文字の解析が読み込み時間の無視できない部分を占めることがある。

5.4.2.2　NPY・HDF5・Parquet の用途

バイナリ形式は、数値を文字列から変換する処理を省ける場合があり、CSV より短時間で読み書きできることが多い。ただし、実際の時間はファイルサイズ、圧縮方式、記憶装置、

列数、読み込む範囲、ライブラリの実装に左右される。バイナリ形式に変えただけで、常に記憶装置の転送速度だけが上限になるわけではない。

代表的なバイナリ形式には用途の違いがあり、データ構造と交換相手に合わせて選ぶ必要がある。

- **NPY / NPZ (NumPy 標準)**: NumPy 配列の形と型を保持する。NPY は 1 配列、NPZ は複数の NPY データを ZIP コンテナへまとめる。交換先が NumPy 形式に対応しているかを確認する。
- **Pickle / PKL**: Python オブジェクトを直列化できるため、同じ管理環境内の一時的な中間結果やキャッシュに用いられる。Python 以外との交換には適さず、読み込み時に任意のコードが実行され得るため、信頼できないファイルを読み込んではならない。
- **HDF5**: 大規模な数値データを階層構造で管理できる形式である。多次元配列を部分的に読み書きし、データと、単位や記録条件などの説明情報（メタデータ）を一つのファイルへまとめたい場合に向く。C/C++、Fortran、R などからも利用できるが、受渡し相手が使うライブラリとデータ構造を事前に確認する必要がある。
- **Parquet**: 列指向のバイナリ形式である。pandas や Polars から表を保存し、列選択や行グループ単位の読み込みを行える。レコード単位の更新ではなく、列の抽出、集計、複数ファイルの走査を前提とする。

解析の初期段階では、内容をテキストとして確認でき、複数のソフトウェアで扱える CSV が検証に役立つ。大量のイベントデータを繰り返し読む時間が長い場合は、変換手順と列型を記録したうえで Parquet や HDF5 を中間形式として導入する。小規模なデータや一度しか読み込まないデータでは、形式を変換・管理する手間に見合う利点が得られない場合がある。`.pkl` は、信頼できる同一環境内の途中結果や作業用キャッシュに用途を限定する。

Parquet は保存形式の名前であり、`pyarrow` は pandas が Parquet の読み書きに利用できる実装の一つである。`read_parquet()` と `to_parquet()` の実行には、pandas が対応するエンジンを利用できる環境が必要となる。

5.5 章末課題：CSV を読み、要約を保存する

`earthquake_practice` を作業フォルダーとする。第 2 章で取得した CSV、または次の人工データを使う。人工データを使う場合は、実カタログと区別して `raw/usgs_example.csv` へ保存する。`raw/` と `results/` は作成しておく。

raw/usgs_example.csv

```
time,latitude,longitude,mag,place
2024-01-01T00:00:00.000Z,35.0,140.0,3.2,"Offshore, Japan"
2024-01-01T00:05:00.000Z,36.0,141.0,,Japan
2024-01-01T00:12:30.000Z,34.5,139.5,2.8,Tokyo
```

1. **CSV の要約を JSON へ保存する。**`csv.DictReader` で読み、件数、最初と最後の時刻、`mag` の空欄数と、空欄を除いた最小値・最大値を求めよ。有効値がなければ最小値と最大値は `None` とする。`results/usgs_summary.json` へ保存し、読み戻した内容を表にせよ。人工データでは 3 件、最小 2.8、最大 3.2、空欄 1 件となる。
2. **途中結果を保存して読み戻す。**先頭 200 行までの `time`、`latitude`、`longitude`、`mag` を辞書のリストにし、`results/usgs_preview.pkl` へ保存せよ。読み戻した件数と先頭 1 件を示し、保存前との一致を確認せよ。人工データでは 3 件となる。Pickle は自分で作った信頼できるファイルだけを読む。
3. **発展：不正な値を分けて記録する。**`mag` に `abc`、`NaN`、`inf` を持つ検証用の行を追加し、空欄、変換できない値、有限でない値の件数を別々に数えよ。有限な値だけで集計し、除外件数を `results/skipped_rows.txt` へ保存せよ。追加前後の集計表を比較し、検出方法を説明せよ。

第 6 章

NumPy による数値計算

NumPy は、複数の数値をまとめて計算するためのライブラリである。まず、記録した時刻から事象の発生間隔を求める。その後で、数値の並べ方、条件に合う値の取り出し、合計や平均、分布の集計へ進む。配列の型やメモリの共有も、実際の値を確かめながら学ぶ。

6.1 配列を作って間隔を求める

6.1.1 隣り合う時刻の差を求める

ある観測の開始から 10 秒、15 秒、23 秒、41 秒の時点で、事象が 4 回記録されたとする。知りたいのは、次の事象が起きるまでに何秒かかったかである。最初の間隔は `15 - 10` で 5 秒、次は `23 - 15` で 8 秒、最後は `41 - 23` で 18 秒になる。

この計算には、複数の数値をまとめて扱う「配列」を使う。`import numpy as np` は、NumPy の機能を `np` という短い名前で使うための準備である。`np.array([...])` で、角括弧の中の数値をまとめた配列を作る。次の例を `intervals.py` へ保存し、端末で `python intervals.py` を実行する。

```
import numpy as np

times = np.array([10, 15, 23, 41])
intervals = np.diff(times)
print(intervals)
```

```
[ 5  8 18]
```

`np.diff(times)` は、後ろの値から一つ前の値を引く計算を順に行う。時刻が 4 個あるので、間隔は 3 個になる。

最後の時刻を `41` から `45` に変えてもう一度実行すると、最後の間隔だけが `22` に変わる。ここでは時刻が小さい順に並んでいる。未整列の入力では、先に並べ替える必要がある。

数式では、この計算を隣り合う時刻の差

$$\Delta t_i = t_{i+1} - t_i$$

と表す。NumPy で配列をまとめて計算する書き方を、ベクトル化という。大量の値を扱うときに Python の要素別ループを減らせるが、速度は実際の入力で測定する。比較方法は第 13 章で扱う。

6.1.2 ndarray と属性

`np.array()` で作った配列の種類を `ndarray` という。配列に付随する情報は、`a.shape` のように名前の後ろへドットと項目名を付けて取り出せる。この情報を属性という。`a.sum()` のように計算を実行するメソッドと異なり、属性を読むときは丸括弧を付けない。次の例で、行数・列数、数値の型、要素数を確認する。

```
import numpy as np

a = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0]])

print(a.shape)     # (2, 3)
print(a.ndim)      # 2
print(a.dtype)     # float64
print(a.size)      # 6
print(a.itemsize)  # 8
```

`shape` は各軸の長さ、`ndim` は次元数、`dtype` は要素型、`size` は総要素数、`itemsize` は 1 要素あたりのバイト数である。値だけでなく `shape` と `dtype` も確認すると、形状不一致と型変換に起因する不具合を切り分けられる。

```
(2, 3)
2
float64
6
8
```

この出力は、`a` が 2 行 3 列、2 次元、6 要素の `float64` 配列であり、1 要素が 8 バイトであることを示す。形状の不一致を調べるときは `shape` と `ndim`、型変換の影響を調べるときは `dtype` と `itemsize` を確認する。

6.2　配列の生成・演算・集計

6.2.1　配列の作成

配列の生成時には、形状、要素型、初期値を定める。np.array は既存の値から配列を作り、np.zeros と np.ones は一定値で初期化し、np.arange と np.linspace は規則的な数列を作る。

```
a = np.array([1, 2, 3], dtype=np.float64)
zeros = np.zeros((2, 3))
ones = np.ones((2, 3))
tmp = np.empty((2, 3))
grid = np.arange(0, 10, 2)
x = np.linspace(0.0, 1.0, 5)

print(a)
print(zeros.shape)
print(grid)
print(x)
```

```
[1. 2. 3.]
(2, 3)
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
```

np.array は既存の Python リストなどから配列を作る。np.zeros と np.ones は、すべての要素をそれぞれ 0 と 1 で初期化する。np.arange は指定した刻み幅の数列、np.linspace は指定区間を等分する数列を作る。後者はビン中心、時間軸、周波数軸の生成に利用できる。

zeros.shape が (2, 3) なので、zeros と ones は 2 行 3 列の配列である。grid は 0 から 8 まで 2 刻み、x は 0 と 1 を両端として等間隔に並べた 5 点の座標である。np.empty は値を初期化せずにメモリを確保するため、生成直後の要素値は未定義である。読み出す前に全要素を代入する処理に用途を限定する。

6.2.2　条件抽出・ソート・差分・ヒストグラム

条件に合う要素の抽出、値のソート、隣接要素の差分、ヒストグラムを、一つの時刻配列へ順に適用する例を示す。

```
times = np.array([41, 10, 23, 15, 42])

selected = times[times > 20]
sorted_times = np.sort(times)
deltas = np.diff(sorted_times)
hist, edges = np.histogram(times, bins=[0, 20, 40, 60])
```

```
7
8  print(selected)
9  print(sorted_times)
10 print(deltas)
11 print(hist)
12 print(edges)
```

```
[41 23 42]
[10 15 23 41 42]
[ 5  8 18  1]
[2 1 2]
[ 0 20 40 60]
```

この例で行っていることは次のとおりである。

- フィルタ: `times > 20` は `[True, False, True, False, True]` に相当する条件配列を作り、その位置にある要素だけを `selected` として取り出す。
- ソート: `np.sort(times)` は値を昇順に並べる。観測時刻やエネルギーのように順序が意味を持つ量では、後続の処理の前に並べ替えることが多い。
- 差分: `np.diff(sorted_times)` は隣り合う値の差を返す。返る配列の長さは元の配列より 1 つ短い。イベント時刻が昇順に並んでいれば、これは到着間隔として読める。
- ヒストグラム: `np.histogram(times, bins=[0, 20, 40, 60])` は、区間ごとの個数を `hist`、区間境界を `edges` として返す。この例では `[0, 20)` に 2 個、`[20, 40)` に 1 個、`[40, 60]` に 2 個入っている。

条件式が返すブール配列は、そのまま添字に指定できる。複数条件には `(times > 20) & (times < 40)` のように要素ごとの論理演算を用いる。`np.diff` の結果は要素の順序に依存するため、時刻差を求める前に入力の順序を確認し、必要ならソートする。

ヒストグラムは、ビン境界 $b_0, b_1, \ldots, b_n$ に対して

$$h_k = \#\{t_i \mid b_k \le t_i < b_{k+1}\}$$

を数える操作と見なせる。ただし、NumPy では最後のビンだけ右端も含むので、最終区間は $[b_{n-1}, b_n]$ と読めばよい。

6.2.3　要素ごとの演算とユニバーサル関数

NumPy の演算子や多くの関数は、原則として要素ごとに働く。NumPy の公式ドキュメント [22] では、`exp`、`sqrt`、`sin` のような要素ごとに作用する関数をユニバーサル関数（ufunc）と呼んでいる。

```
1 a = np.array([1.0, 2.0, 3.0])
2 b = np.array([10.0, 20.0, 30.0])
3
4 print(a + b)
```

```
print(a * 2)
print(a ** 2)
print(np.exp(a))
print(np.sqrt(a))
```

```
[11. 22. 33.]
[2. 4. 6.]
[1. 4. 9.]
[ 2.71828183  7.3890561  20.08553692]
[1.         1.41421356 1.73205081]
```

配列どうしの四則演算は、形がそろっていれば要素ごとに実行される。`np.exp(a)` や `np.sqrt(a)` も同様である。関数が 1 要素に対して何をするかを知っていれば、その関数が配列全体へどう作用するかも追いやすい。

たとえば、配列 $a = (a_i)$、$b = (b_i)$ に対しては

$$(a + b)_i = a_i + b_i, \qquad (a^2)_i = a_i^2, \qquad (\exp(a))_i = e^{a_i}$$

のように、各要素へ同じ規則が適用されると考えればよい。

6.2.4　集計と axis

次の 2 行 3 列の配列で、全体の合計、列ごとの合計、行ごとの合計を求める。`sum()` は合計を求める操作であり、`axis` は、どの方向に並ぶ値を足すかを指定する。まず出力を見て、実際の足し算と対応させる。

```
matrix = np.array([[1.0, 2.0, 3.0],
                   [4.0, 5.0, 6.0]])

total = matrix.sum()
col_sum = matrix.sum(axis=0)
row_sum = matrix.sum(axis=1)
col_max = matrix.max(axis=0)

print(total)
print(col_sum)
print(row_sum)
print(col_max)
```

```
21.0
[5. 7. 9.]
[ 6. 15.]
[4. 5. 6.]
```

`total` は全要素を足した `21.0` である。`axis=0` では、各列を上から下へ足す。1 列目は `1 + 4`、2 列目は `2 + 5`、3 列目は `3 + 6` なので、`[5., 7., 9.]` になる。

`axis=1` では、各行を左から右へ足す。1 行目は `1 + 2 + 3`、2 行目は `4 + 5 + 6` なので、`[6., 15.]` になる。`max(axis=0)` は同じ列の最大値を選ぶので、`[4., 5., 6.]` を返す。この説

明は 2 次元配列についてのものであり、3 次元以上では配列の形と集計する軸を対応させる。

行列 $A = (a_{ij})$ に対しては、

$$(\text{col_sum})_j = \sum_i a_{ij}, \qquad (\text{row_sum})_i = \sum_j a_{ij}$$

に対応する。したがって、`axis=0` の出力は列ごとの和、`axis=1` の出力は行ごとの和である。

6.2.5　添字、スライス、条件抽出

NumPy の添字には、Python リストと同じ整数・スライスに加え、多次元の軸指定とブール配列を使用できる。

```
times = np.array([10, 15, 23, 41, 42])
print(times[0])
print(times[-1])
print(times[1:4])
print(times[times > 20])

grid = np.arange(12).reshape(3, 4)
print(grid[1, 2])
print(grid[:, 1])
print(grid[1, :])
```

```
10
42
[15 23 41]
[23 41 42]
6
[1 5 9]
[4 5 6 7]
```

`times[1:4]` は添字 1 から 3、すなわち 2 番目から 4 番目までを取り出す。`grid[:, 1]` は全行の 2 列目、`grid[1, :]` は 2 行目全体である。多次元配列に対する `for row in grid:` は、第 0 軸に沿って 1 行ずつ反復する。同じ演算を全行へ適用する場合は、明示的な反復と配列演算のどちらが入力と出力の関係を直接表すかを比較する。

スライスを使うと、章の冒頭の間隔計算を次のようにも書ける。途中の配列を表示して、どの値同士を引くかを確認する。

```
times = np.array([10, 15, 23, 41], dtype=np.int64)
later = times[1:]
earlier = times[:-1]
print(later)
print(earlier)
print(later - earlier)
```

```
[15 23 41]
[10 15 23]
```

```
[ 5  8 18]
```

`times[1:]` は先頭を除いた 3 個、`times[:-1]` は末尾を除いた 3 個を取り出す。配列同士の引き算では、同じ位置の値同士を引くため、`15 - 10`、`23 - 15`、`41 - 23` をまとめて計算できる。これが `np.diff(times)` と同じ結果になる。

6.2.6　配列形状の変更

配列の `shape` は、演算時の軸の対応を決める。1 次元配列、2 次元行列、2 次元の列ベクトルでは、同じ演算子を用いてもブロードキャスト後の結果が異なる。

```
a = np.arange(12)
matrix = a.reshape(3, 4)
flat = matrix.ravel()
transposed = matrix.T
column = np.array([1.0, 2.0, 3.0])[:, None]

print(matrix.shape)
print(matrix)
print(flat)
print(transposed)
print(column.shape)
print(column)
```

```
(3, 4)
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
(3, 1)
[[1.]
 [2.]
 [3.]]
```

`reshape` は配列の形状を変える。`ravel` は、可能であればビューのまま 1 次元配列へ変換する。`T` は転置を表す。`[:, None]` は新しい軸を 1 本追加し、1 次元配列を列ベクトルのように扱う書き方である。この軸の追加は、後述するブロードキャストで配列同士の対応関係を指定する役割を持つ。

6.2.7　配列の連結と分割

前処理済みデータを縦方向に連結する場合や、複数の特徴量を列方向に並べる場合には、連結する軸を指定する。逆に、一つの配列を同じ軸に沿って複数部分へ分割できる。

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

joined = np.concatenate([x, y])
stacked = np.stack([x, y], axis=0)

matrix = np.arange(12).reshape(3, 4)
left, right = np.split(matrix, 2, axis=1)

print(joined)
print(stacked.shape)
print(stacked)
print(left)
print(right)
```

```
[1 2 3 4 5 6]
(2, 3)
[[1 2 3]
 [4 5 6]]
[[0 1]
 [4 5]
 [8 9]]
[[ 2  3]
 [ 6  7]
 [10 11]]
```

`concatenate` は既存の軸に沿って配列を連結し、`stack` は新しい軸を追加して配列を並べる。`split` は指定した軸に沿って配列を分割する。これらの操作では、実行後の `shape` が意図した値と一致するかを確認する。

6.2.8　形とブロードキャスト

NumPy では、配列の形状をそろえなくても、自動拡張の規則に従って演算できる場合がある。この規則をブロードキャストという。

```
x = np.array([1.0, 2.0, 3.0])
y = np.array([10.0, 20.0, 30.0])

z = x + y
matrix = x[:, None] + y[None, :]
print("z =")
print(z)
print("\nmatrix =")
print(matrix)
```

後半の `matrix` では、1 次元配列に軸を追加して、3 行 3 列の和の表を作っている。このように、形状が異なる配列を一定の規則で拡張して演算する仕組みをブロードキャストとい

う。二つの 1 次元配列から要素の全組合せを計算できる一方、結果の要素数は両配列の要素数の積になる。この例の結果を次に示す。

```
z =
[11. 22. 33.]

matrix =
[[11. 21. 31.]
 [12. 22. 32.]
 [13. 23. 33.]]
```

この結果から、z は要素ごとの和、matrix は全組み合わせの表であることが分かる。x と y の要素数がそれぞれ N_x と N_y なら、matrix の要素数は N_xN_y になる。大きな配列では、生成前に形状、要素数、dtype から必要なメモリ量を見積もる。

数式で書けば、

$$z_i = x_i + y_i, \qquad M_{ij} = x_i + y_j$$

である。後者の M_{ij} が、ブロードキャストによって作られる「全組み合わせの表」に対応している。

ブロードキャストでは形状を右端の軸から比較し、各軸の大きさが等しいか、一方が 1 なら演算できる。欠けている先頭の軸は大きさ 1 とみなす。たとえば (2, 3) と (3,) は適合するが、(2, 3) と (2,) は末尾の 3 と 2 が適合せず、ValueError になる。

6.2.9　統計量とヒストグラム

平均は値の合計を個数で割ったものである。値が平均の周りにどの程度広がっているかは、分散と標準偏差で表せる。ddof=0 の分散は、各値と平均との差を二乗して平均した値であり、その平方根が標準偏差になる。例えば [1, 3] は平均 2、平均との差が [-1, 1] なので、分散も標準偏差も 1 である。ddof を変えた場合の割り算は、例の後で説明する。NumPy には、これらの統計量や最小値、最大値、ヒストグラムを求める関数がある。代表的な統計量とヒストグラムを計算する例を下に示す。

```
values = np.array([1.0, 2.0, 2.0, 3.0, 4.0])
mean = values.mean()
std = values.std(ddof=0)
hist, edges = np.histogram(values, bins=[0.0, 2.0, 4.0, 6.0])

print(f"mean = {mean:.2f}")
print(f"std = {std:.2f}")
print(hist)
print(edges)
```

np.histogram は、指定したビンごとの要素数とビン境界 edges を返し、図は作成しない。同じビン境界による複数データの比較や、度数に対するモデル計算を作図処理から独立させられる。これらの統計量を出力した結果を次に示す。

```
mean = 2.40
std = 1.02
[1 3 1]
[0. 2. 4. 6.]
```

この例では、`[0, 2)` に 1 個、`[2, 4)` に 3 個、`[4, 6]` に 1 個入っている。平均や標準偏差だけでは分布の形は分からず、ヒストグラムだけでは条件間比較がしにくい。数値要約と分布表示を組み合わせて読むことが、データ解析では基本になる。

`values.mean()` と `values.std(ddof=0)` は、それぞれ

$$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad \sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2}$$

を計算している。`ddof` は分母の自由度補正を指定し、一般に分母は $N - \mathrm{ddof}$ となる [23]。`ddof=0` は手元の N 個の値のばらつきを分母 N で記述する指定であり、未知の母集団の標準偏差が求まったことを意味しない。

ここで、今回の測定値の集まりを標本、その標本を得る元の集まりを母集団という。独立に同じ分布から得た標本では、分母 $N-1$ の標本分散が母分散の不偏推定量となる。不偏とは、同じ条件で標本を繰り返し得たとき、推定値の平均が母集団の値に一致する性質をいう。ただし、その平方根が母標準偏差の不偏推定量になるわけではない。

NumPy の `std()` の既定は `ddof=0`、pandas の `std()` の既定は `ddof=1` なので、比較時には指定をそろえる。

6.3　型・軸・ビューに関する注意点

6.3.1　演算子の意味の取り違え

`*` は要素ごとの積であり、行列積ではない。行列積の計算には `@` または `np.dot` を用いる。

```
A = np.array([[1.0, 2.0],
              [3.0, 4.0]])
B = np.array([[5.0, 6.0],
              [7.0, 8.0]])

print(A * B)
print(A @ B)
```

演算子を取り違えても、形状が適合すれば例外が発生せず、意味の異なる数値が得られる場合がある。線形代数や座標変換では、期待する数式と演算子を対応付け、既知の小さな入力で結果を検証する。

6.3.2　axis と条件式

2 次元配列を集計するとき、axis=0 は第 0 軸を縮約して列ごとの値を返し、axis=1 は第 1 軸を縮約して行ごとの値を返す。また、配列どうしの条件結合では and や or ではなく、要素ごとの論理演算子 & や | を使う。

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
print(matrix.sum(axis=0))
print(matrix.sum(axis=1))

times = np.array([10, 15, 23, 41, 42])
mask = (times > 10) & (times < 40)
print(mask)
print(times[mask])
```

```
[5 7 9]
[ 6 15]
[False  True  True False False]
[15 23]
```

times > 10 & times < 40 のように括弧を省略すると、演算子の優先順位によって意図した比較にならない。各比較を括弧で囲み、その結果を & または | で結合する。

6.3.3　dtype と np.empty

NumPy では、異なる型の値を同じ配列へ入れると、一定の変換規則に従って配列全体の dtype が決まる。その結果、数値計算を意図した配列が文字列配列になる場合がある。

```
a = np.array([1, 2.5])
b = np.array([1, "2"])
tmp = np.empty(5)

print(a.dtype)
print(b.dtype.kind)
print(np.issubdtype(a.dtype, np.number))
print(np.issubdtype(b.dtype, np.number))
print(tmp.shape)
```

```
float64
U
True
False
(5,)
```

a は浮動小数点へそろえられる。これに対して b は dtype.kind == "U"、つまり Unicode 文字列の配列であり、数値演算には使えない。tmp は 0 で初期化されないため、読み出し前に

全要素へ値を代入する。配列の生成方法に応じて、dtype と初期化の有無を確認する。

NumPy の整数型は固定長であり、表現範囲を超えるとオーバーフローし得る。特に符号なし整数の差は負の値を表現できないため、差分を求める前に、値域を確認して astype(np.int64) などで符号付き型へ変換する。Python の通常の int と同じ値域ではない。また、現在時刻付近の Unix 秒を float64 にした値は 1 ns 刻みを保持できない。細かな時刻差が必要なら、範囲内の整数ナノ秒で差を取ってから秒へ換算する。

6.3.4　コピーとビュー

配列のコピーは独立したメモリを使用するため、大きな配列を繰り返しコピーするとメモリ使用量と転送時間が増える。一方、ビューは元配列とメモリを共有する。ビューに値を代入すると、元の配列も変わる。各操作がコピーとビューのどちらを返すかを区別する必要がある。

```
a = np.arange(6)

view = a[1:4]
view[:] = -1

copy = a[[0, 2, 4]]
copy[:] = 99

safe = a.copy()
print("a after view =", a)
print("copy         =", copy)
print("safe         =", safe)
```

この例では、スライスで作った view は元の a とデータを共有するので、view[:] = -1 とすると a も変わる。これに対して、整数配列やブール配列による取り出しは新しい配列を返すので、copy[:] = 99 としても元の a は変わらない。reshape や ravel も可能ならビューを返す。元配列から独立したデータ領域が必要な場合は、copy() を明示する。

```
a after view = [ 0 -1 -1 -1  4  5]
copy         = [99 99 99]
safe         = [ 0 -1 -1 -1  4  5]
```

この出力では、view を書き換えた時点で元配列 a が変化しているのに対し、copy は独立していることが見て取れる。

6.3.5　浮動小数点数の比較

浮動小数点数では、計算手順による丸め誤差のため、数学的に等しい値がビット単位では一致しない場合がある。許容する相対誤差と絶対誤差を定められる場合は、直接比較の代わりに np.isclose または np.allclose を用いる。

```
a = 0.1 + 0.2
print(a == 0.3)
print(np.isclose(a, 0.3))
```

```
False
True
```

有限値に対する `np.isclose(a, b)` は、$|a-b| \le \mathrm{atol} + \mathrm{rtol}|b|$ を判定する。既定の許容差が測定精度に適するとは限らないため、単位と値の大きさに合わせて指定する。`np.allclose()` もブロードキャストするので、配列の同一性を検証する際は形状の一致を先に確認する。欠損値を一致とみなすかどうかも別に定める。

6.3.6　補足：Python・C・NumPy の型とメモリの違い

Python では、変数名へ型を宣言せずに値を代入でき、値ごとに型が決まる。この仕組みを動的型付けという。Python の標準数値型は値ごとに型と管理情報を持つ。これに対して NumPy 配列では、配列全体に共通する固定長の型を `dtype` で指定する。代表的な数値型の対応を表 6.1 に示す。

表 6.1　Python, NumPy, C 言語での代表的な数値型の対応

分類	C 言語の型	Python の標準型	NumPy の型 (`dtype`)
整数型	`int`, `long`	`int` (利用可能なメモリの範囲で可変長)	`int32`, `int64`
単精度浮動小数	`float` (32 ビット)	(標準では存在せず)	`float32`
倍精度浮動小数	`double` (64 ビット)	`float`	`float64`
論理型	`bool`	`bool`	`bool_`

表のビット幅は一般的な処理系の例であり、C 言語の規格がすべての処理系に同じ幅を保証しているわけではない。C 言語では、`float` や `double` のように変数の型を宣言し、処理系が定める大きさの領域へ値を格納する。一般的な CPython では、標準の `int` は必要な桁数に応じて領域が増え、`float` は通常 C 言語の `double` と同じ倍精度形式を使う。また、各数値は型や参照管理の情報を含む Python オブジェクトである。多数の値を Python のリストへ格納すると、リスト中の参照と各オブジェクトの管理領域が必要になり、演算も要素ごとの Python 処理を経由する。

NumPy の `ndarray` は、要素ごとではなく配列全体で一つの `dtype` を共有し、データ領域と、形状・ストライドなどの管理情報を分けて保持する。配列は常に連続とは限らないが、同じ型の値を規則的にたどれるため、要素ごとの Python オブジェクト操作を避け、C や Fortran で実装されたループへまとめて処理を渡せる。この書き方をベクトル化と呼ぶ。

浮動小数点型は、必要な精度と値域、配列の大きさ、入出力元の型、CPU や GPU の特性を基準に選ぶ。NumPy の多くの配列生成関数や Python の `float` から作る配列では `float64` が既定になり、反復計算や非常に近い値どうしの差を求める計算では有力な選択肢になる。一方、`float32` は 1 要素が 4 バイトであり、8 バイトの `float64` と比べて配列本体のメモリ使用量が半分になる。大きな画像、センサデータ、GPU 計算、メモリ転送が支配的な処理では、この差が処理可能な配列サイズを左右する。元データが `float32` なら、必要性を検討せずに倍精度へ変換するとメモリ使用量だけが増える場合もある。

型を変えるときは、速度だけでなく結果が必要な精度を満たすかを確認する。例えば `np.finfo(np.float32).eps` と `np.finfo(np.float64).eps` を表示すると、1 付近で区別できる相対的な刻み幅を比較できる。どちらの型でも、桁落ちや丸め誤差そのものがなくなるわけではない。基準値が決まっていない段階では `float64` から始め、メモリや転送が問題になった時点で `float32` でも解析結果が変わらないかを検証する方法が堅実である。

6.3.7　補足：配列演算を速く実行できる理由

NumPy の配列演算と Python の要素別ループには、主に次の三つの実装上の差がある。

1. 数値を連続したメモリにまとめて持てること
2. Python レベルのループを減らせること
3. 内部で低レベルに最適化された処理を使えること

配列演算は、Python で要素ごとに演算を呼び出す代わりに、配列全体を低水準の実装へ渡す。実行時間は、同じ入力と演算について Python ループと配列演算を測定して比較する。

配列演算では、差分、閾値判定、正規化、平均、分散を、明示的な添字ループを用いずに記述できる。処理対象となる配列と演算の対応が式に現れるため、要素ごとの更新処理と区別できる。

6.4　差分・ヒストグラム・線形代数への応用

6.4.1　解析で NumPy を使うときの視点

大規模データでは、必要以上に大きな型を選ぶとメモリ使用量が増える。たとえば、識別子や時刻差には `int64` が必要となる場合がある一方、範囲の狭いカテゴリ番号は `int16` や `int32` で表せる。要素数を N、1 要素のバイト数を b とすれば、配列本体には少なくとも Nb バイトが必要であるため、型の差は要素数に比例して総使用量へ反映される。

また、平均や分散だけでなく、差分、閾値判定、並べ替え、ヒストグラムまで NumPy の配列操作で一貫して記述すると、解析の流れを追いやすくなる。

6.4.2　差分、ソート、ヒストグラム

解析では、まず時刻順に並べ、次に差分を取り、その後で条件に合う値を選んで分布を見る、という流れがよく現れる。NumPy ではこの流れをほぼそのまま式として書ける。

```
times = np.array([10, 15, 23, 41, 42])
order = np.argsort(times)
sorted_times = times[order]
deltas = np.diff(sorted_times)
mask = (deltas >= 0) & (deltas <= 250)
hist, edges = np.histogram(deltas[mask], bins=50)
peak_bin = hist.argmax()
print("sorted_times =", sorted_times)
print("deltas       =", deltas)
print("peak_bin     =", peak_bin)
```

`np.argsort` は並べ替え順の添字を返す。`np.diff` は隣接差分である。`hist.argmax()` は最も高いビンの位置を返す。イベント時刻、スペクトル、波形特徴量のいずれでも、この種の処理は頻出である。

```
sorted_times = [10 15 23 41 42]
deltas       = [ 5  8 18  1]
peak_bin     = 0
```

6.4.3　`numpy.linalg` による連立一次方程式の求解

`numpy.linalg` は、連立一次方程式、行列分解、固有値などの線形代数演算を提供する。次の例では、2 元連立一次方程式 $A\boldsymbol{x} = \boldsymbol{b}$ を `solve` で解く。

```
A = np.array([[1.0, 1.0],
              [1.0, -1.0]])
b = np.array([3.0, 1.0])
x = np.linalg.solve(A, b)
```

行列演算は、校正係数の決定、座標変換、最小二乗法の前処理に現れる。制約付き最適化や疎行列など、`numpy.linalg` の範囲を越える機能は SciPy の公式資料 [24] を参照するとよい。

6.4.3.1　Python リストで要素ごとに計算する場合

Python の標準リストは異なる型のオブジェクトを同じリストに保持できる。一方、同じ数値演算を大量の要素へ適用する場合は、各要素について Python の演算を呼び出す必要がある。次の例では、100 万個の整数を 2 倍する処理を内包表記で記述している。

```
# 100万要素のリストを作る
py_list = list(range(1000000))
```

```
3
4 # 内包表記の反復で、すべての要素を2倍にする
5 doubled_list = [x * 2 for x in py_list]
```

一般的な CPython のリストは、値そのものではなく Python オブジェクトへの参照を並べて保持する。内包表記では、各参照から値を取り出し、Python の整数演算を呼び出し、新しい整数オブジェクトへの参照を結果リストへ格納する。この反復処理が、固定長の数値配列を低レベルのループで処理する場合との差になる。

6.4.3.2 NumPy 配列によるベクトル化

NumPy 配列では、`int64` や `float64` のような共通の型に従って要素を格納する。`np_array * 2` と書くと、Python で要素ごとのループを記述せず、NumPy 内部のループへ配列全体の処理を渡せる。リストを NumPy 配列へ変換してから計算する例を次に示す。

```
1  import numpy as np
2
3  # 1. Python リストから共通 dtype の NumPy 配列を作る
4  np_array = np.array(py_list)
5
6  # 2. 配列全体の演算を NumPy の内部ループへ渡す
7  doubled_array = np_array * 2
8
9  # 3. 必要な出力形式がリストの場合にだけ変換する
10 back_to_list = doubled_array.tolist()
```

6.4.3.3 pandas DataFrame との変換

後述する pandas DataFrame は、列名と行ラベルを持つ表形式のデータ構造であり、列の内部では NumPy 配列や拡張配列が使われる。解析の途中で NumPy の関数を使いたい場合は、DataFrame や Series を NumPy 配列へ変換できる。pandas の DataFrame・Series と NumPy 配列を相互変換する基本形は次の通りである。

```
1 import pandas as pd
2
3 # NumPy 配列から pandas の DataFrame ( 二次元表 ) に変換
4 df = pd.DataFrame(np_array, columns=["Value"])
5
6 # pandas の列から NumPy 配列を得る
7 values_array = df["Value"].to_numpy()
```

`to_numpy()` がコピーを作るかどうかは、列の型、列数、必要な型変換に依存する。単一の数値列では元データとメモリを共有する場合もあるが、常に共有されるとは限らない。コピーの有無が性能や書き換えに影響する場面では、`np.shares_memory()` で確認し、必要なら `to_numpy(copy=True)` で独立した配列を要求する。

6.4.4　公式資料の使い方

NumPy の基礎を復習したいときは、NumPy quickstart [6] を併読すると整理しやすい。特に、配列生成、添字とスライス、`shape` の変更、ブロードキャスト、コピーとビューは、解析コードを書くたびに戻って確認したくなる項目である。本章は解析で問題になりやすい点を補い、quickstart は関数名や細かな仕様を確認する資料と位置付けられる。

6.5　章末課題：発生間隔を計算して集計する

まず、秒単位の時刻を持つ次の人工データを使う。実カタログを使う場合は、第 5 章の方法で `time` 列を秒へ変換する。

```
import numpy as np

times = np.array([720.0, 0.0, 60.0, 4200.0, 180.0, 780.0])
```

1. **発生間隔を求める。** `np.sort()` と `np.diff()` で、隣り合う時刻の差を求めよ。結果 `[60., 120., 540., 60., 3420.]` と、配列の形状・型を示せ。並べ替える前に差を取った結果と比べよ。
2. **間隔の分布を集計する。** 平均、0 秒を超える個数、$[0, 60)$、$[60, 600)$、$[600, 3600)$ 秒の個数を求めよ。さらに 0 から 3600 秒を 600 秒幅で `np.histogram()` により集計し、表にせよ。検算値は平均 840 秒、正の値 5 個、3 区間は順に 0、4、1 個である。3600 秒を追加し、最後のビンだけ右端を含むことも確かめよ。
3. **発展：ビンの取り方と地域差を比べる。** 実カタログの間隔を、固定幅ビンと 1〜3600 秒の対数ビンで集計せよ。対数ビンは `np.logspace(np.log10(1), np.log10(3600), 30)` とする。空のビン、0 以下の値、範囲外の値の件数を表にせよ。さらに緯度・経度で日本近海を定義し、同じ期間・集計条件の全球データと比較せよ。

第 7 章

pandas による表形式データ解析

pandas は、行と列からなる表形式データを扱うライブラリである。この章では、観測記録を表にし、条件に合う行を選び、検出器ごとの平均を求める。値が空欄のときの扱いと、計算結果の保存も学ぶ。元の表からどの行を計算に使ったかを説明できるようにする。

7.1 DataFrame の構造と列指向処理

7.1.1 Series と DataFrame

まず、検出器名、記録時刻、信号値、記録の有効性を示す値を含む 4 件の記録を表にする。1 行は 1 件の記録、1 列は同じ種類の値を表す。pandas の基本構造は `Series` と `DataFrame` である。`Series` はラベル付き 1 次元配列、`DataFrame` は列名付き 2 次元表である。NumPy 配列が配列全体で一つの `dtype` を持つのに対し、DataFrame は列ごとに異なる型と列名を保持できる。

```
import pandas as pd

events = pd.DataFrame(
    {
        "detector": ["A", "A", "B", "B"],
        "time_ns": [1032, 1188, 1055, 1244],
        "signal": [12.4, 10.1, 14.8, 9.7],
        "valid": [True, True, False, True],
    }
)

print(type(events).__name__)
print(events)
```

このコードを実行すると、次のような DataFrame が表示される。

```
DataFrame
  detector  time_ns  signal  valid
0        A     1032    12.4   True
1        A     1188    10.1   True
2        B     1055    14.8  False
3        B     1244     9.7   True
```

この表では、行番号 0, 1, 2, 3 は index、`"detector"` や `"signal"` は列名である。index は行を識別するラベルであり、列名は各列が表す量を示す。値だけでなく、列名、型、単位、欠損値の有無を合わせて確認する。

DataFrame の利点は、列名によって意味を示しながら、列単位の処理を記述できることにある。列の位置を整数で保持する必要がなくなり、ファイル読み込み、列変換、フィルタ、集計、書き出しまでを一つの表に対する処理として扱える。

DataFrame の列選択は辞書のキーによる値の参照、行の条件抽出はブール配列による NumPy 配列の抽出に対応する。ただし、DataFrame では index と列名のラベルも演算時の対応付けに使われる。

7.1.2　行の内容・表の形状・列の型・欠損値の確認

DataFrame の作成または読み込みの直後には、`head()`、`shape`、`dtypes`、`info()` で行の例、形状、列型、欠損値数を確認する。

```
1 print(events.head())
2 print(events.shape)
3 print(events.dtypes)
```

出力は次の通りである。

```
  detector  time_ns  signal  valid
0        A     1032    12.4   True
1        A     1188    10.1   True
2        B     1055    14.8  False
3        B     1244     9.7   True
(4, 4)
detector        str
time_ns       int64
signal      float64
valid          bool
dtype: object
```

`shape` は行数と列数、`dtypes` は各列の型である。`object` 型は任意の Python オブジェクトを格納できるため、その表示だけでは列が文字列だけで構成されるか、複数の型が混在するかを判定できない。数値として演算する列が `object` 型なら、実際の値と変換条件を確認する。

上の型表示は pandas 3 系の例である。文字列だけの列は既定で `str` と推定され、pandas 2

系の既定では `object` となる [25]。表示の違いと、数値型への変換の要否は分けて確認する。

7.1.3　index と列名の役割

pandas では、整数位置とラベルを区別して行と列を選択する。

```
print(events["signal"])
print(events.loc[1:2, ["detector", "signal"]])
print(events.iloc[1:3, 0:2])
```

`events["signal"]` は列名による選択、`loc` はラベルによる選択、`iloc` は整数位置による選択である。pandas の indexing ガイド [26] でも、選択対象に応じて `[]`、`loc`、`iloc`、`at`、`iat` を使い分けている。解析コードでは、列名・ラベル・整数位置のどれを指定しているかを明確にする。

7.1.4　Python の行反復と列演算の実装差

pandas の列演算は、要素ごとに Python 関数を呼び出すループを避け、配列単位の実装へ処理を渡せる。実行時間の差はデータ型と処理内容に依存するため、対象データを用いて列演算と行反復を測定する。

ただし、すべての処理が pandas の列演算に対応するわけではない。行間の近傍探索や状態遷移のように、前後関係または専用のデータ構造を必要とする処理では、NumPy、探索アルゴリズム、または目的に合わせたデータ構造を検討する。処理を独立した列の変換・抽出・集計として表せるかどうかが、pandas を選択する基準となる。

7.2　抽出・変換・集計・入出力

7.2.1　列選択と行抽出

列選択と条件による行抽出を組み合わせる例を示す。

```
selected = events[["detector", "signal"]]
valid_events = events.loc[events["valid"], :]
strong = events.loc[events["signal"] > 11.0, ["detector", "signal"]]

print(selected)
print(valid_events)
print(strong)
```

各変数に残る行と列は次のとおりである。

```
  detector  signal
0        A    12.4
1        A    10.1
```

```
2        B    14.8
3        B     9.7
  detector  time_ns  signal  valid
0        A     1032    12.4   True
1        A     1188    10.1   True
3        B     1244     9.7   True
  detector  signal
0        A    12.4
2        B    14.8
```

例えば最初の行の信号値は 12.4 なので、`events["signal"] > 11.0` の条件を満たす。2 行目の 10.1 と 4 行目の 9.7 は条件を満たさず、信号が大きい行を選ぶ表には残らない。ブール配列で行を絞る操作は NumPy と似ているが、pandas では列名を介して意味を保ったまま書ける。`events["signal"] > 11.0` の結果は行ごとの真偽値であり、それを `loc` に渡すと条件に合う行だけが残る。

`loc` は行ラベルや列名で選択し、ラベルのスライスでは終点も含む。`iloc` は 0 から始まる位置で選択し、Python の通常のスライスと同じく終点を含まない。また、Series を列へ代入すると、通常は位置ではなく index のラベルで対応付けられる。

7.2.2　列の追加

元の列から比、差、判定結果を求める場合は、列全体を入力とする式を記述する。

```
events["signal_ratio"] = events["signal"] / events["signal"].mean()
events["is_late"] = events["time_ns"] > 1100

print(events)
```

新しい列は、元の表の右端に追加される。出力は次のようになる。

```
  detector  time_ns  signal  valid  signal_ratio  is_late
0        A     1032    12.4   True      1.055319    False
1        A     1188    10.1   True      0.859574     True
2        B     1055    14.8  False      1.259574    False
3        B     1244     9.7   True      0.825532     True
```

`assign()` を使うと、途中の DataFrame を明示しながら複数列をまとめて追加できる。

```
derived = (
    events
    .assign(
        dt_ns=lambda df: df["time_ns"] - df["time_ns"].min(),
        quality=lambda df: pd.Series(["high", "mid", "low", "mid"])
    )
)
```

`assign()` は新しい DataFrame を返すため、メソッドチェーンの途中で列を追加できる。一方、`events["new_col"] = ...` は既存の DataFrame に列を直接追加する。元の表を変更す

るか、新しい表を返すかに応じて選択する。

7.2.3　欠損値と型

実際のデータには、値が記録されていないことがある。pandas では、そのような欠損値を `NaN` や `NA` で表す。

```
df = pd.DataFrame(
    {
        "id": [1, 2, 3, 4],
        "signal": [12.4, None, 14.8, 9.7],
        "comment": ["ok", "bad", None, "ok"],
    }
)

print(df.isna())
print(df.dropna(subset=["signal"]))
print(df["signal"].fillna(df["signal"].mean()))
```

`isna()` の真偽表、`dropna()` の結果、`fillna()` 後の列を順に表示すると、欠損処理の違いが見えやすい。

```
      id  signal  comment
0  False   False    False
1  False    True    False
2  False   False     True
3  False   False    False
   id  signal comment
0   1    12.4      ok
2   3    14.8     NaN
3   4     9.7      ok
0    12.400000
1    12.300000
2    14.800000
3     9.700000
Name: signal, dtype: float64
```

欠損値は `==` で判定しない。`isna()`、`notna()`、`dropna()`、`fillna()` を使う。また、整数列に欠損が入ると `float64` に変わることがあるので、必要なら pandas の nullable integer 型である `Int64` を使う。

```
df["id"] = df["id"].astype("Int64")
```

7.2.4　要約統計と `groupby`

`describe()` は数値列の件数、平均、標準偏差、分位点、最小値、最大値を返す。`groupby()` は、行をカテゴリごとに分け、それぞれを集計する。

```
print(events.describe())

summary = (
    events
    .groupby("detector")
    .agg(
        count=("signal", "size"),
        mean_signal=("signal", "mean"),
        std_signal=("signal", "std"),
        late_fraction=("is_late", "mean"),
    )
)

print(summary)
```

`describe()` は全体の要約、`groupby()` の結果は検出器ごとの集計である。出力は次の通りである。

```
           time_ns     signal  signal_ratio
count     4.000000   4.000000      4.000000
mean   1129.750000  11.750000      1.000000
std     102.613758   2.355844      0.200497
min    1032.000000   9.700000      0.825532
25%    1049.250000  10.000000      0.851064
50%    1121.500000  11.250000      0.957447
75%    1202.000000  13.000000      1.106383
max    1244.000000  14.800000      1.259574
          count  mean_signal  std_signal  late_fraction
detector
A             2        11.25    1.626346            0.5
B             2        12.25    3.606245            0.5
```

`groupby()` は、検出器、日、run などのカテゴリごとに同じ集計を適用する。`agg()` の名前付き集計では、出力列の名前、入力列、集計関数を対応付けて記述できる。上の数値は、この節の `events` を pandas で計算した出力例である。pandas のバージョンによって空白や型の表示形式は変わる場合があるが、数値は同じ入力と計算定義に対応していなければならない。

欠損値を含む列の平均や標準偏差は、既定では欠損値を除いて計算する。`groupby().size()` は行数を、列に対する `count()` は非欠損値の個数を数える。`std()` の既定は `ddof=1` であり、有効値が 1 個だけの群では標準偏差は `NaN` になる。未知の文字列は必ずしも欠損値として認識されない。`pd.to_numeric(..., errors="coerce")` で変換できなかった値を欠損にする場合は、その件数と元の値を記録する。平均値による補完は分散などを変えるため、解析上の根拠なしに適用しない。

7.2.5　表形式の変換

Getting started tutorials [7] でも、表を縦長から横長へ変える操作や、複数表を結合する操作が扱われている。解析では、観測値と較正値を結び付けたり、日別表をまとめたりする場面で用いる。

```
gain = pd.DataFrame(
    {
        "detector": ["A", "B"],
        "gain": [1.02, 0.97],
    }
)

merged = events.merge(gain, on="detector", how="left", validate="many_to_one")
assert len(merged) == len(events)
stacked = pd.concat([events.iloc[:2], events.iloc[2:]], ignore_index=True)

long_df = pd.DataFrame(
    {
        "detector": ["A", "A", "B", "B"],
        "kind": ["raw", "cal", "raw", "cal"],
        "value": [12.4, 12.7, 14.8, 14.4],
    }
)
wide = long_df.pivot(index="detector", columns="kind", values="value")

print(merged[["detector", "signal", "gain"]])
print(wide)
```

次の出力は、`merge()` による較正係数列の追加と、`pivot()` による種別 `kind` の値から列への変換を示す。

```
  detector  signal  gain
0        A    12.4  1.02
1        A    10.1  1.02
2        B    14.8  0.97
3        B     9.7  0.97
kind       cal   raw
detector
A         12.7  12.4
B         14.4  14.8
```

`merge()` はキー列に基づいて表を結合し、`concat()` は表を行方向または列方向に連結する。`pivot()` と `pivot_table()` は縦長データを横長形式へ変換し、`melt()` は横長データを縦長形式へ変換する。

結合先のキーが重複していると、一つの観測行に複数の行が対応し、結合後の行数が増える。上の `validate="many_to_one"` は較正表側のキーの一意性を検査する。結合できなかっ

たキーでは較正値が欠損するため、行数に加えて欠損件数も確認する。`pivot()` は同じ行キーと列キーの組合せが重複するとエラーになる。重複を集計する場合は `pivot_table(..., aggfunc="mean")` のように集計方法を明示する。

7.2.6　時刻、文字列、カテゴリ

時刻や文字列は、そのままでは解析しにくい。pandas には時刻用の `dt` アクセサ、文字列用の `str` アクセサ、カテゴリ型がある。

```
log = pd.DataFrame(
    {
        "timestamp": ["2026-04-01 12:00:00", "2026-04-01 12:00:05", "2026-04-02 09:30:00"],
        "run_name": ["run_A_001", "run_A_002", "test_B_001"],
        "detector": ["A", "A", "B"],
    }
)

# 元のタイムゾーンなし文字列が UTC を表すと確認した場合
log["timestamp"] = pd.to_datetime(log["timestamp"], utc=True)
log["date"] = log["timestamp"].dt.date
log["prefix"] = log["run_name"].str.extract(r"^(run_[A-Z])")
log["detector"] = log["detector"].astype("category")

print(log[["timestamp", "date", "prefix", "detector"]])
```

時刻列と文字列列を分解すると、日付や接頭辞を独立した列として扱える。出力は次の通りである。

```
                  timestamp        date prefix detector
0 2026-04-01 12:00:00+00:00  2026-04-01  run_A        A
1 2026-04-01 12:00:05+00:00  2026-04-01  run_A        A
2 2026-04-02 09:30:00+00:00  2026-04-02    NaN        B
```

`to_datetime()` で日時型へ変換すると、時刻差の計算、年月日の取り出し、時刻順の並べ替えができる。文字列の列には `str.contains()`、`str.extract()`、`str.strip()` などのベクトル化メソッドがある。カテゴリ型は、有限個のカテゴリ値と整数コードを分けて保持し、カテゴリの順序も定義できる。

7.2.7　入出力

pandas は、CSV、Parquet、圧縮テキストなどを DataFrame として読み書きできる。本節では、ファイル形式ごとの型と圧縮方法の指定を扱う。

まず、読み込みを試すための 2 件の記録を圧縮 CSV へ保存する。

```
sample = pd.DataFrame({
    "detector": ["A", "B"],
```

```
    "time_ns": [1000, 1180],
    "signal": [12.4, 9.8],
})
sample.to_csv("events.csv.gz", index=False, compression="gzip")
```

```
df = pd.read_csv(
    "events.csv.gz",
    compression="gzip",
    usecols=["detector", "time_ns", "signal"],
    dtype={"detector": "category", "time_ns": "int64", "signal": "float64"},
)

df.to_parquet("events.parquet")
small = pd.read_parquet("events.parquet", columns=["detector", "signal"])
```

この例は、ヘッダ付きのカンマ区切り CSV を gzip 圧縮した `events.csv.gz` を想定している。第 1 引数は入力パス、`compression="gzip"` は圧縮方式、`usecols` は読み込む列、`dtype` は列ごとの型である。`to_parquet()` は表全体を Parquet へ保存し、`read_parquet(..., columns=...)` は指定した 2 列だけを読み込む。

`read_csv()` では、必要な列、列の型、コメント行の扱いを入力時に指定できる。次は、空白区切りの 2 行を gzip へ保存し、必要な列だけを指定した型で読み込む例である。

```
import gzip

path = "observations.txt.gz"
with gzip.open(path, "wt", encoding="utf-8") as f:
    f.write("1 2026-04-12 12:00:00 12.4\n")
    f.write("2 2026-04-12 12:00:01 9.8\n")

df = pd.read_csv(
    path,
    compression="gzip",
    sep=r"\s+",
    comment="#",
    header=None,
    usecols=[0, 1, 2, 3],
    names=["id", "date", "time", "value"],
    dtype={"id": "int64", "value": "float64"},
)
```

必要な列だけを読み込むと、読み込み時間とメモリ消費を減らせる場合がある。さらに `dtype=` を指定すれば、読み込み後の型変換も減らせる。列数や行数が大きいデータでは、必要な列を入力段階で選ぶことが後続処理の負荷軽減につながる。

2 つ目の例は、ヘッダのない空白区切りファイルを想定している。`sep=r"\s+"` は 1 個以上の空白を区切りとし、`comment="#"` は `#` から始まる行の残りをコメントとして扱う。`header=None` は列名行がないことを示し、`names` が読み込んだ 4 列へ名前を付ける。`usecols=[0, 1, 2, 3]` の番号は入力ファイル中の列位置であり、0 から数える。

大規模なテキストファイルの反復読み込みが処理時間を支配する場合は、一度整形した表を列型と変換条件とともに Parquet へ保存する。`read_parquet()` では必要な列だけを選択できるため、反復解析の読み込み時間とメモリ使用量を削減できる場合がある。効果は入力容量、列数、ストレージによって変わるため、変換前後を測定する。

Parquet はファイル形式の名前であり、`pyarrow` はその読み書きを担うライブラリである。`to_parquet()` や `read_parquet()` がライブラリ不足を示すエラーで失敗した場合は、Parquet ファイルの破損ではなく、`pyarrow` などの対応ライブラリが環境に入っていない可能性がある。

列同士の演算をそのまま式として書ける点も pandas の強みである。

```
df["ratio"] = df["value"] / df["value"].mean()
selected = df[df["ratio"] > 1.5]
```

行ごとの `for` ループを導入する前に、同じ処理を列演算、集計、結合として表せるかを検討する。

Parquet の例には、たとえば `python -m pip install pyarrow` で導入できるエンジンが必要である。後の `to_latex()` には Jinja2 が必要なので、`python -m pip install jinja2` で導入する。uv のプロジェクトではそれぞれ `uv add` を用いる。

7.2.8　大規模 CSV の分割集計

pandas の DataFrame は主としてメモリ上で処理されるが、`read_csv()` の `chunksize` を使えば、CSV を一定行数ずつ読みながら集計できる。例として、次の `events.csv` を考える。

events.csv

```
detector,signal
A,12.4
B,14.8
A,10.1
B,9.7
A,13.0
```

検出器ごとの件数と信号値の合計は、次のように 2 行ずつ読んでも計算できる。

```
counts = pd.Series(dtype="int64")
sums = pd.Series(dtype="float64")

for chunk in pd.read_csv(
    "events.csv",
    usecols=["detector", "signal"],
    dtype={"detector": "string", "signal": "float64"},
    chunksize=2,
):
    grouped = chunk.groupby("detector")["signal"]
    counts = counts.add(grouped.count(), fill_value=0)
    sums = sums.add(grouped.sum(), fill_value=0)
```

```
13
14 summary = pd.DataFrame({"count": counts, "mean": sums / counts})
15 print(summary)
```

実行結果は次の通りである。

```
          count       mean
detector
A           3.0  11.833333
B           2.0  12.250000
```

`chunksize=2` は、反復ごとに最大 2 行の DataFrame を返す指定である。実データでは 10 万行など、利用可能なメモリと処理時間を見ながら値を決める。`Series.add(..., fill_value=0)` は、あるチャンクに片方の検出器しか含まれない場合にも、存在しない集計値を 0 として足す。平均値をチャンク平均の単純平均として求めると、チャンクごとの件数が違う場合に誤るため、全体の合計を全体の件数で割っている。

分割処理が適するのは、総和、件数、最小値、最大値のようにチャンクごとの結果を統合できる計算である。全行を同時に並べ替える処理、全行どうしを結合する処理、全データに依存する順位計算では、単純な分割だけでは同じ結果にならない。その場合は、必要列を減らす、Parquet を使う、次章で説明する Polars の遅延実行やデータベースを使うといった別の方法も検討する。

7.2.9　表の LaTeX 出力

`to_latex()` で集計結果から表を生成すれば、DataFrame の値と LaTeX 表の値を同じ処理から出力できる。

```
1  from pathlib import Path
2
3  Path("tables").mkdir(parents=True, exist_ok=True)
4  tex_table = summary.reset_index().to_latex(
5      index=False,
6      caption="検出器ごとの平均信号",
7      escape=True,
8      label="tab:detector-summary",
9      float_format="%.3f",
10 )
11
12 with open("tables/detector_summary.tex", "w", encoding="utf-8") as f:
13     f.write(tex_table)
```

ここで `index=False` は左端の連番インデックスを出力しない指定であり、`reset_index()` は groupby 結果の index を通常の列へ戻す操作である。ここでは `escape=True` によってセル中の TeX の特殊文字をエスケープする。pandas 2.0 以降では既定値に頼るとエスケープされない場合がある [27]。検証済みの TeX 記法を意図的に埋め込む場合に限り、入力内容を限定したうえで `escape=False` を指定する。

`groupby()`、`agg()`、`to_latex()` を順に適用し、保存したファイルをレポートから `\input` すれば、集計結果をレポート用の表へ直接反映できる。この構成では、集計条件を変更した場合も同じ処理から表を再生成でき、転記による値の不一致を避けられる。

7.2.10　`plot()` による簡易確認

pandas の `plot()` は、DataFrame または Series の内容を Matplotlib へ渡す簡易作図インターフェースである。

```
summary.plot(kind="bar", y="mean")
```

`DataFrame.plot()` は表から概形を確認する場合に用いる。提出図や論文用図では、軸、凡例、余白、保存形式を明示するため、Matplotlib の `Figure` と `Axes` を直接扱う。本書では第 9.3 節以降の方法を基準とする。

7.3　コピー・型・行処理に関する注意点

7.3.1　部分表の変更と `loc`

DataFrame の一部を切り出した後の代入では、元の DataFrame を更新するのか、独立した表を作るのかを明示する必要がある。

```
subset = events[events["signal"] > 11.0]
subset["flag"] = True
```

pandas 2 系の従来の動作では、この書き方で `SettingWithCopyWarning` が出ることがある。独立した部分表が必要なら抽出後に `.copy()` を付け、意図を明示する。pandas 3 系は Copy-on-Write が標準であり、部分表の変更は元の表へ反映されない [28]。元の表の条件に合う行へ代入する場合は、次のように行条件と列名を `loc` の一つの添字指定へまとめる。

```
events.loc[events["signal"] > 11.0, "flag"] = True
```

7.3.2　`object` 型のまま計算しない

数字からなる文字列を含む列でも、空白や数字以外の記号が混在すると `object` 型で読み込まれる場合がある。そのまま演算すると、文字列連結または型に関する例外が生じる。入力仕様が決まっている列は `read_csv()` の `dtype=` で型を指定し、読み込み後の `dtypes` と変換できなかった値を確認する。

7.3.3　行ループを適用する前の検討事項

pandas の DataFrame では、同じ処理でも列演算と行単位の Python 関数で実行時間が大きく異なることがある。特に、`iterrows()`、`apply(axis=1)`、並列化の使い分けは、処理内容と測定結果に基づいて判断する必要がある。

7.3.3.1　`iterrows()` による行ループの負担

DataFrame の行を一つずつ取り出す処理は記述しやすいが、列単位の演算よりオーバーヘッドが大きくなる場合がある。

```
df = pd.DataFrame({"value": [1.0, 2.0, 3.0]})

def heavy_computation(val):
    return val ** 2

results = []
for index, row in df.iterrows():
    results.append(heavy_computation(row["value"]))
df["value_sq"] = results
```

`iterrows()` は各行を `Series` として生成するため、元の列型が行単位の Series に保たれない場合があり、行ごとのオブジェクト生成も生じる。行反復が必要なら `itertuples()` との処理時間を測定する。列ごとに独立して計算できる処理には、列演算または既存のベクトル化メソッドを用いる。

7.3.3.2　`apply(axis=1)` と列演算の比較

行ごとに処理を適用したい場合、pandas が用意している `apply()` メソッドを使うと、コードを DataFrame の処理として記述できる。`axis=1` は「各行へ関数を適用する」という意味であり、`axis=0` なら各列に適用する。

```
df["value_sq"] = df.apply(lambda row: heavy_computation(row["value"]), axis=1)
```

ただし、`axis=1` を付けた `apply()` も各行に対して Python 関数を呼ぶため、列演算とは処理方式が異なる。単に二乗する場合は、次の列演算が演算対象と式を直接表す。実行時間は対象データを用いて測定する。

```
df["value_sq"] = df["value"] ** 2
```

`apply()` は、既存の列演算や NumPy 関数では表しにくい処理に Python 関数を適用する手段である。列演算、`apply()`、並列化の候補について、同じ入力と同じ出力条件で処理時間を測定して選ぶ。

以下の `pandarallel` は追加ライブラリであり、使用する環境に `python -m pip install`

pandarallel などで導入する。二乗程度の軽い処理では並列化の起動・転送コストが支配的になり得るため、この例自体が高速化の推奨例というわけではない。

7.3.3.3 pandarallel による行処理の並列化

通常の apply() は、Python 関数を行ごとに順番に呼び出す。本当に計算が重く、列演算や既存の NumPy 関数へ置き換えられない場合に、マルチプロセスによる並列化が選択肢に入る。

```
from pandarallel import pandarallel

pandarallel.initialize(progress_bar=True)
df["value_sq"] = df.parallel_apply(
    lambda row: heavy_computation(row["value"]),
    axis=1,
)
```

図 7.1 は、iterrows()、apply()、pandarallel の処理時間に生じ得る違いを模式的に示す。実測値ではなく、処理方式を比較するときに確認すべきオーバーヘッドの違いを表した図である。

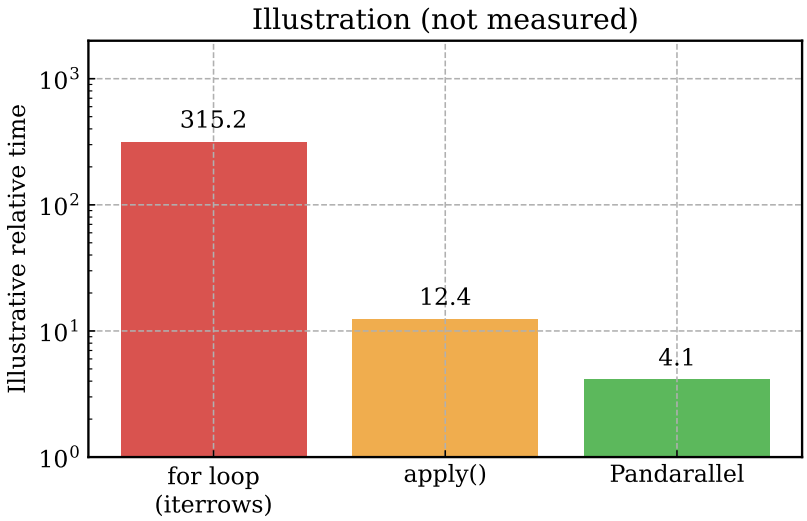


図 7.1　同じ行単位処理に対する Python のループ、apply()、pandarallel の模式的な比較。棒の高さは実測値ではなく、入力サイズや 1 行当たりの計算量によって順位も変わり得る。

pandarallel は、DataFrame を複数プロセスへ分割し、parallel_apply() に渡した関数を各部分へ適用する。プロセスの生成、データの直列化、プロセス間転送の時間も加わるため、対象処理の逐次実行と並列実行を比べ、実行時間とメモリ使用量を測定する。

pandarallel は複数の Python プロセスを起動するため、OS によってプロセスの開始方法や必要な設定が異なる。Windows で本書の Linux 系コマンドと実行条件をそろえる場合は、第 2 章で説明した WSL を利用できる。

pandarallel は DataFrame を複数プロセスへ分配するため、プロセスの起動、データの受け渡し、結果の結合にも時間がかかる。単純な加算のように 1 行当たりの処理が短い場合は、この付加時間が計算時間を上回り、逐次処理より遅くなることがある。並列化の採否は、同じ入力に対する逐次処理と並列処理を測定して判断する。

7.3.4　欠損値と比較演算を混同しない

NaN == NaN は False であるため、欠損判定には == ではなく isna() を用いる。None、NaN、NA の使用可否と表示は列型に依存するので、dtype と欠損値数を確認する。

7.4　観測表の集計と可視化への受け渡し

観測データの解析では、pandas はしばしば次の流れの中心になる。

1. read_csv() によるテキスト・CSV の読み込み
2. head() と dtypes による表構造の確認
3. 必要な列の選択と、欠損値・異常値の整理
4. groupby() と merge() による集計・結合
5. Matplotlib への受け渡しと作図
6. to_latex() による集計表の書き出し

この流れでは、列名と列型を保ったまま抽出、集計、結合、保存を記述している。観測条件、検出器 ID、run 名、欠損フラグのように型の異なる列を一つの記録として扱う場合は、列ごとに型を持つ DataFrame を利用できる。

一方、CSV の読み込みや集計が、実測した処理時間の大部分を占める場合は、次章で扱う Polars も候補となる。行数だけで切り替えを決めず、必要な機能、メモリ使用量、処理時間を同じ入力で比較する。

また、pandas の多くの操作は、DataFrame が 1 台の計算機のメモリに収まることを前提とする。ただし、read_csv(chunksize=...) を使えば、総和や件数のように部分結果を統合できる処理は全件を同時に保持せずに実行できる。必要な表がメモリに収まらない場合は、列と行を早い段階で絞る、分割集計を使う、Parquet に変換する、次章の Polars やデータベースを使う、という順に処理の性質に合う方法を選ぶ。

基本操作の公式資料として Getting started tutorials [7] と 10 minutes to pandas [29] がある。列選択と代入は Indexing [26]、欠損値処理は Missing data [30]、表の結合は Merge, join, concatenate and compare [31]、時刻列は Time series / date functionality [32] に詳しい。本章は、これらの機能を解析手順に沿って再構成した。

7.5　章末課題：検出器ごとの信号を集計する

次を `events.csv` として保存する。各問では、この元の 6 行を読み込む。

events.csv

```
detector,time_ns,signal,valid
A,1000,12.4,true
B,1180,9.8,true
A,1300,15.1,true
B,1510,,false
A,1760,11.0,true
B,2050,13.2,true
```

1. **表の内容を確認する。** `pd.read_csv()` で読み、先頭 3 行、行数・列数、各列の型と欠損数を表示せよ。6 行 4 列、信号の欠損 1 件となることを確かめ、表の特徴を説明せよ。
2. **強い信号を取り出す。** `valid` が真で `signal >= 12.0` の 3 行を取り出し、その 3 行の平均で信号を割った `signal_ratio` 列を追加せよ。抽出前後の件数と結果の表を示せ。
3. **検出器ごとに集計し、較正係数を結合する。** 元の表で、`valid` が真かつ信号が欠損していない行を使い、検出器別の件数、平均、標準偏差（`ddof=1`）を求めよ。件数は A が 3、B が 2、平均は約 12.8333 と 11.5 となる。係数 A: 1.02、B: 0.97 の表を `merge()` で結合し、信号に係数を掛けた結果も示せ。
4. **分割して読んでも同じ平均になるか確かめる。** `chunksize=2` で読み、3 問目と同じ有効条件で件数と平均を求めよ。一括読み込みと値を照合し、比較表を作れ。各分割の合計と件数を使う理由も説明せよ。
5. **集計表を保存して読み戻す。** 3 問目の集計表を CSV と Parquet へ保存し、列名と値が保持されたかを表にせよ。CSV は `index=False` とする。Parquet の読み書きには、対応するエンジン（例えば PyArrow）が必要である。

第 8 章

Polars による式ベースの表処理

Polars は、pandas と同じく行と列からなる表を扱うライブラリである。まず 4 件の記録を表にし、信号値の列に計算を適用する。次に、条件に合う行を選び、検出器ごとに集計する。後半では、行う処理をまとめて記録してから実行する方法を扱う。

8.1 式と遅延実行による表処理

8.1.1 index を持たない DataFrame

Polars でも、1 行を 1 件の記録、1 列を同じ種類の値として表す。pandas と違い、行に index というラベルを持たせず、位置や条件で行を選ぶ。まず次のコードで 4 行 3 列の表を表示する。

```
import polars as pl

df = pl.DataFrame(
    {
        "detector": ["A", "A", "B", "B"],
        "time_ns": [1032, 1188, 1055, 1244],
        "signal": [12.4, 10.1, 14.8, 9.7],
    }
)

print(df)
```

このコードを実行すると、Polars の表は次のように表示される。

```
shape: (4, 3)
┌──────────┬─────────┬────────┐
│ detector ┆ time_ns ┆ signal │
│ ---      ┆ ---     ┆ ---    │
│ str      ┆ i64     ┆ f64    │
╞══════════╪═════════╪════════╡
│ A        ┆ 1032    ┆ 12.4   │
│ A        ┆ 1188    ┆ 10.1   │
│ B        ┆ 1055    ┆ 14.8   │
│ B        ┆ 1244    ┆ 9.7    │
└──────────┴─────────┴────────┘
```

`shape: (4, 3)` は 4 行 3 列であることを示し、各列名の下にはデータ型が表示される。Polars の式は行ラベルではなく列名を参照し、その列に適用する処理を表す。

8.1.2　式による列変換

まず、各信号値に 1 を足す。`pl.col("signal")` は `signal` 列を選ぶ指定であり、`+ 1` を続けると「この列の各値へ 1 を足す」という式になる。

```
1 expr = pl.col("signal") + 1
2 result = df.select(expr.alias("signal_plus_one"))
3 print(result["signal_plus_one"].to_list())
```

```
[13.4, 11.1, 15.8, 10.7]
```

`expr` は計算の指定であり、4 個の答えそのものではない。`df.select(...)` に渡すと表へ適用され、`alias()` で結果の列名を付けられる。ここでは先頭の信号値 `12.4` が `13.4` になったことを確認する。

Polars の基本となるのは expression（式）である。式は、列に対してどの計算を適用するかを表し、その時点の値そのものではない。`pl.col("signal")` は `signal` 列を参照する式であり、四則演算、比較、集計などと組み合わせられる。

```
1 expr = pl.col("signal") / pl.col("signal").mean()
2 print(expr)
```

表示されるのは、概ね `[(col("signal")) / (col("signal").mean())]` のような式であり、4 行分の数値ではない。実際の値を得るには、`df.select(expr.alias("signal_ratio"))` のように、式を DataFrame の処理へ渡す。`alias()` は式の結果へ列名を付ける。Polars の公式ドキュメント [33] でも、expression は列処理を構成する基本概念として説明されている。`select()`、`with_columns()`、`filter()`、`group_by()` の多くは、式を受け取る API で統一されている。

8.1.3　eager と lazy

命令ごとに結果の表を得る方法を eager（即時実行）API という。処理内容を記録し、後でまとめて実行する方法を lazy（遅延実行）API という。例えば、列の選択や行の抽出を先に記録しておけば、必要な部分だけを読むように実行方法を調整できる。以下では、上で作った 4 行の表を実習用の `events.csv` へ保存してから、二つの読み方を比べる。同名ファイルがあれば置き換わるので、この章の実習用フォルダーで実行する。

```
df.write_csv("events.csv")
eager_df = pl.read_csv("events.csv")
lazy_df = pl.scan_csv("events.csv")
```

`read_csv()` は呼び出し時に読み込んで DataFrame を作る。`scan_csv()` は、これから行う処理を記録する `LazyFrame` を返す。`lazy_df.collect()` を呼ぶと、記録した処理が実行され、結果の DataFrame が得られる。遅延実行では、必要な列だけを読む列射影や、不要な行を早い段階で除く述語プッシュダウンなどを適用できる場合がある。処理時間とメモリ使用量への効果は、入力形式と処理内容に依存するため、同じ入力で測定する。

8.2　列変換・集計・結合・入出力

8.2.1　`select()` と `filter()`

列の選択には `select()`、条件に合う行の抽出には `filter()` を用いる。

```
selected = df.select("detector", "signal")
strong = df.filter(pl.col("signal") > 11.0)

print(selected)
print(strong)
```

`select()` で列が減り、`filter()` で条件に合う行だけが残る。出力の例を示す。表の罫線や型の表示は一部簡略化している。

```
shape: (4, 2)
+----------+--------+
¦ detector ¦ signal ¦
+==========+========+
¦ A        ¦ 12.4   ¦
¦ A        ¦ 10.1   ¦
¦ B        ¦ 14.8   ¦
¦ B        ¦ 9.7    ¦
+----------+--------+
shape: (2, 3)
+----------+---------+--------+
```

```
| detector | time_ns | signal |
+==========+=========+========+
| A        | 1032    | 12.4   |
| B        | 1055    | 14.8   |
+----------+---------+--------+
```

`df["signal"]` は列を Series として取り出す。これに対して、`select()` や `filter()` へ渡した式は、複数列の変換や遅延実行の実行計画にも使用できる。値を取り出す操作と、表に適用する式を区別する。

8.2.2 `with_columns()` による列の追加

新しい列は `with_columns()` で追加する。複数の式を一度に渡すと、それぞれの評価結果が列として追加または置換される。

```
derived = df.with_columns(
    (pl.col("signal") / pl.col("signal").mean()).alias("signal_ratio"),
    (pl.col("time_ns") > 1100).alias("is_late"),
)

print(derived)
```

列追加後の表を見ると、式で作った列がどの位置へ入るかを確認できる。

```
shape: (4, 5)
+----------+---------+--------+--------------+---------+
| detector | time_ns | signal | signal_ratio | is_late |
+==========+=========+========+==============+=========+
| A        | 1032    | 12.4   | 1.055319     | false   |
| A        | 1188    | 10.1   | 0.859574     | true    |
| B        | 1055    | 14.8   | 1.259574     | false   |
| B        | 1244    | 9.7    | 0.825532     | true    |
+----------+---------+--------+--------------+---------+
```

`alias()` は式の結果へ列名を付ける操作である。式を複数組み合わせる場合は、出力列名を明示すると後続の選択や集計との対応を追いやすい。

一つの `with_columns()` 内の式は、基本的に呼び出し前の列を参照する。同じ呼び出しで新しく作る列を別の式から使う場合は、`with_columns()` を二段に分ける。

8.2.3 `group_by()` と集計

集計も式として書く。`group_by()` と `agg()` は、pandas の `groupby()` に対応する中心的な道具である。

```
summary = (
    derived
    .group_by("detector")
    .agg(
```

```
        pl.len().alias("count"),
        pl.col("signal").mean().alias("mean_signal"),
        pl.col("signal").std().alias("std_signal"),
        pl.col("is_late").mean().alias("late_fraction"),
    )
    .sort("detector")
)

print(summary)
```

集計結果は、検出器ごとに 1 行の表になる。出力の例を示す。表の罫線や型の表示は一部簡略化している。

```
shape: (2, 5)
┌──────────┬───────┬─────────────┬────────────┬───────────────┐
│ detector ┆ count ┆ mean_signal ┆ std_signal ┆ late_fraction │
╞══════════╪═══════╪═════════════╪════════════╪═══════════════╡
│ A        ┆ 2     ┆ 11.25       ┆ 1.626346   ┆ 0.5           │
│ B        ┆ 2     ┆ 12.25       ┆ 3.606245   ┆ 0.5           │
└──────────┴───────┴─────────────┴────────────┴───────────────┘
```

Polars の `group_by()` は、集計方法も式で指定するため、同じ書き方のまま統計量を増やしやすい。`count`、`mean`、`std` のような基本操作は、式の名前を見れば意味を追いやすい。

`pl.len()` は群の行数、`pl.col("signal").count()` はその列の非 null 値の個数を数える。標準偏差の既定は `ddof=1` である。群の平均を元の各行へ付けるには、`over()` を使う。

```
with_group_mean = derived.with_columns(
    pl.col("signal").mean().over("detector").alias("detector_mean")
)
```

`group_by().agg()` が群ごとに行をまとめるのに対し、この `over()` は元の行数を保つ。

8.2.4　join と表の変形

別表の較正係数や run 情報を結び付けるなら `join()` を使う。縦長と横長の変換も行える。

```
gain = pl.DataFrame(
    {
        "detector": ["A", "B"],
        "gain": [1.02, 0.97],
    }
)

merged = derived.join(gain, on="detector", how="left")
wide = (
    pl.DataFrame(
        {
            "detector": ["A", "A", "B", "B"],
            "kind": ["raw", "cal", "raw", "cal"],
```

```
            "value": [12.4, 12.7, 14.8, 14.4],
        }
    )
    .pivot(index="detector", on="kind", values="value")
    .sort("detector")
)

print(merged.select("detector", "signal", "gain"))
print(wide)
```

2 つの出力は次のようになる。

```
shape: (4, 3)
┌──────────┬────────┬──────┐
│ detector ┆ signal ┆ gain │
╞══════════╪════════╪══════╡
│ A        ┆ 12.4   ┆ 1.02 │
│ A        ┆ 10.1   ┆ 1.02 │
│ B        ┆ 14.8   ┆ 0.97 │
│ B        ┆ 9.7    ┆ 0.97 │
└──────────┴────────┴──────┘
shape: (2, 3)
┌──────────┬──────┬──────┐
│ detector ┆ raw  ┆ cal  │
╞══════════╪══════╪══════╡
│ A        ┆ 12.4 ┆ 12.7 │
│ B        ┆ 14.8 ┆ 14.4 │
└──────────┴──────┴──────┘
```

`join()` の `on="detector"` は両方の表で照合するキー列を指定し、`how="left"` は左側の `derived` の全行を残す結合方法を指定する。`pivot()` では、`index="detector"` が出力表で行を識別する列、`on="kind"` が新しい列名になる値、`values="value"` がセルへ入る値を表す。Polars には pandas のような index がないため、結合や変形に使う列を引数で明示する。

8.2.5　欠損値、`null`、`NaN`

Polars では、欠損値を表す `null` と、浮動小数点の特殊値 `NaN` を区別する。

```
missing = pl.DataFrame(
    {
        "signal": [12.4, None, 14.8, float("nan")],
    }
)

print(missing.with_columns(
    pl.col("signal").is_null().alias("is_null"),
    pl.col("signal").is_nan().alias("is_nan"),
))
```

判定結果のうち、主要部分は次のようになる。

```
signal  is_null  is_nan
12.4    false    false
null    true     null
14.8    false    false
NaN     false    true
```

`null` は値が存在しないこと、`NaN` は浮動小数点の特殊値である。`is_nan()` へ `null` を渡した結果も `null` になる点に注意する。両方を除くなら、例えば `pl.col("signal").is_not_null() & pl.col("signal").is_finite()` をフィルタ条件にする。どちらを除くかによって使う式が変わる。

8.2.6　文字列と条件分岐

式ベースの API では、文字列処理や条件分岐も列全体に対する式として書く。

```
names = pl.DataFrame(
    {
        "run_name": ["run_A_001", "run_A_002", "test_B_001"],
        "signal": [12.4, 10.1, 14.8],
    }
)

classified = names.with_columns(
    pl.col("run_name").str.extract(r"^(run_[A-Z])", 1).alias("prefix"),
    pl.when(pl.col("signal") > 12.0)
      .then(pl.lit("high"))
      .otherwise(pl.lit("normal"))
      .alias("quality"),
)

print(classified)
```

追加した 2 列を含む結果は次の通りである。

```
run_name    signal  prefix  quality
run_A_001   12.4    run_A   high
run_A_002   10.1    run_A   normal
test_B_001  14.8    null    high
```

`str.extract()` の第 1 引数は正規表現、第 2 引数の `1` は取り出すキャプチャグループの番号である。一致しない `test_B_001` の `prefix` は `null` になる。`when`、`then`、`otherwise` は、条件、真の場合の値、偽の場合の値を順に組み立てる。

`when().then().otherwise()` では、Python の `if` のように選ばれた分岐だけが評価されるとは限らない。各分岐の式は、それぞれ単独でも有効でなければならない。条件外の行で不正な型変換が起きる場合も、分岐を書くだけでは回避できない。

8.2.7　遅延実行と scan_parquet()

遅延実行は、複数の読み込み、抽出、列選択、集計を一つの実行計画として最適化する場合に用いる。

```
from pathlib import Path

Path("events").mkdir(exist_ok=True)
df = pl.DataFrame({"detector": ["A", "B", "A"],
                   "signal": [12.4, 9.8, 15.1]})
df.write_parquet("events/sample.parquet")

lf = (
    pl.scan_parquet("events/*.parquet")
    .filter(pl.col("signal") > 0.0)
    .select("detector", "signal")
    .group_by("detector")
    .agg(pl.col("signal").mean().alias("mean_signal"))
)

result = lf.collect()
print(result)
```

scan_parquet() の引数 "events/*.parquet" は、一致する複数の Parquet ファイルを入力にする。返り値は LazyFrame であり、この時点ではデータを実体化しない。collect() を呼ぶとクエリが実行され、返り値として DataFrame が得られる。実行時には、select() で使う列だけを読む列射影や、filter() を早い段階へ移す最適化が適用されることがある。最適化内容を確認したい場合は、実行前に print(lf.explain()) として計画を表示できる。

8.2.8　CSV と Parquet の入出力

直前の例の表を CSV へ保存してから読み直し、読み込み方による型の違いを確認する。

```
df.write_csv("events.csv")
df = pl.read_csv("events.csv")
df.write_parquet("events.parquet")

lf = pl.scan_csv("events.csv")
result = lf.filter(pl.col("signal") > 0).collect()

print(type(df).__name__)
print(type(lf).__name__)
print(type(result).__name__)
```

表示される型名は順に DataFrame、LazyFrame、DataFrame である。read_csv() はその場で読み込み、scan_csv() は読み込み計画を作り、collect() が計画を実行する。write_parquet() の第 1 引数は保存先であり、同名のファイルがある場合は上書きされるため、保存先を確認

してから実行する。

`read_csv()` と `write_parquet()` は呼び出し時に入出力を行う。`scan_csv()` は `LazyFrame` を返し、後続の式を実行計画へ追加する。計画の評価と DataFrame の生成は `collect()` が行う。

8.3 pandas との違いと遅延実行の注意点

8.3.1 pandas の書き方をそのまま持ち込まない

Polars と pandas では、同じ DataFrame という名称を使っていても API と実行モデルが異なる。Polars 公式の migration guide [34] では、index を前提にせず、列式と処理文脈を用いる設計が説明されている。pandas の `loc` や `apply(axis=1)` を機械的に置き換えるのではなく、`select()`、`filter()`、組み込みの式で処理を表現する。

8.3.2 `map_elements()` や Python の `lambda` を多用しない

Polars は組み込み式の内容を調べ、実行計画を最適化できる。Python の `lambda` や `map_elements()` で任意の Python 関数を呼ぶと、最適化や並列実行の範囲が狭まり、呼び出しのオーバーヘッドも加わる場合がある。まず `pl.col()`、`when`、`str`、`dt`、`over` などの組み込み式で表せるかを確認し、表せない処理にだけ Python 関数を用いる。

8.3.3 遅延実行では `collect()` を忘れない

`scan_csv()` や `scan_parquet()` は `LazyFrame` を返す。`print(lf)` が表示するのは実データではなく実行計画である。計画を評価して結果の DataFrame を得るには、`collect()` を呼び出す。

8.3.4 欠損値における `null` と `NaN` の区別

Polars では、各列のデータ型に応じて利用できる式が決まり、値が存在しない `null` と浮動小数点値 `NaN` は別に扱われる。フィルタや集計の前に列型を確認し、`is_null()` と `is_nan()` のどちらを条件に含めるかを決める。

遅延実行は、必要列の選択や条件による絞り込みを実行計画へ反映する仕組みであり、メモリ使用量を無制限に小さくするものではない。`collect()` で最終的に得る表の大きさと、並べ替え・結合の中間データにも注意する。

8.4　大規模表処理と可視化への受け渡し

Polars への移行を検討する条件を次に示す。

- 複数の CSV または Parquet ファイルに対し、同じ列選択と行抽出を適用する場合
- 列変換と集計を一つの実行計画として最適化する場合
- pandas による入出力または集計が、測定した総所要時間の主要部分を占める場合
- 列射影や述語プッシュダウンを適用できる入力形式と処理内容である場合

小規模データで index を用いた対話的操作が必要な場合や、既存コードが pandas の操作に依存している場合は、pandas を維持する方が変更量を抑えられる。切り替えの判断には、行数だけでなく、必要な機能、移行コスト、メモリ使用量、同じ入力に対する処理時間を用いる。

Polars の概念は Concepts [8]、式は Expressions and contexts [33]、遅延実行は Lazy API [35]、入出力は IO [36]、pandas との差は Coming from Pandas [34] に記載されている。本章は、これらの機能を解析手順に沿って再構成した。

8.5　章末課題：表の処理方法と結果を比較する

第 7 章の章末課題に示した `events.csv` を使う。各問では、元の 6 行から始める。

1. **列を選び、行を絞る。** `detector` と `signal` を選び、信号が `null` でなく 12.0 以上の行を取り出せ。結果の 3 行（A が 2 行、B が 1 行）を示し、抽出条件を説明せよ。
2. **検出器ごとの平均からの差を求める。** `mean().over("detector")` で信号から検出器別平均を引いた列を追加せよ。さらに `group_by()` で全行数、有効な信号数、平均、最大値を求めよ。全行数は A、B とも 3、有効数は 3 と 2 となる。列追加後と集計後の表を示し、行数の違いを説明せよ。
3. **処理をまとめてから実行する。** `pl.scan_csv()` から、2 列の選択、欠損除去、検出器別平均を記述せよ。`explain()` の実行計画と `collect()` の結果を示し、どの段階で結果が得られるか説明せよ。
4. **別表を結合し、件数表を作る。** 係数 A: 1.02、B: 0.97 の表を `join()` で結合し、信号に係数を掛けよ。さらに `pivot()` で、検出器を行、`valid` を列、件数を値とする表を作れ。較正後の表と件数表を示せ。
5. **pandas と結果を照合する。** 前章の 3 問目と同じ条件で件数、平均、標準偏差（`ddof=1`）を求め、CSV へ保存せよ。pandas の結果と検出器名で対応付け、数値を許容誤差付きで比較した表を示せ。欠損値と行順序も確認する。

第 9 章

Matplotlib による可視化

Matplotlib は、Python でグラフや画像を描画するライブラリである。この章では、測定値の分布を図にし、軸の名前や単位を付けて保存する。まず 1 枚のヒストグラムを作り、次に点や線を描く方法を学ぶ。後半では複数の図を並べ、色や文字の大きさを整える。保存した図を開き、何を比較した図かを説明できるようにする。

ヒストグラムや散布図には、計算結果の検証に用いる診断図としての役割がある。平均値や標準偏差だけでは識別できない長い裾、二峰性、外れ値などを、分布の形として確認できるためである。したがって、解析途中の検証と最終結果の提示の双方に可視化を組み込む。

9.1 線図・散布図・ヒストグラムの基本

9.1.1 最小限のヒストグラム

最初に、測定値がどの範囲に何個あるかを棒の高さで表すヒストグラムを作る。例えば 0 から 1 の範囲にある値が 10 個なら、その範囲の棒の高さは 10 になる。数える範囲の一つ一つをビンという。ここでは動作確認用の値を 2500 個作り、40 個のビンに分けて数え、ヒストグラムを PDF に保存する。乱数を作るときの番号（シード）を固定し、同じ環境では同じ入力を作れるようにする。入力は `values`、作図処理は `plt.hist()`、出力先は `histogram.pdf` である。

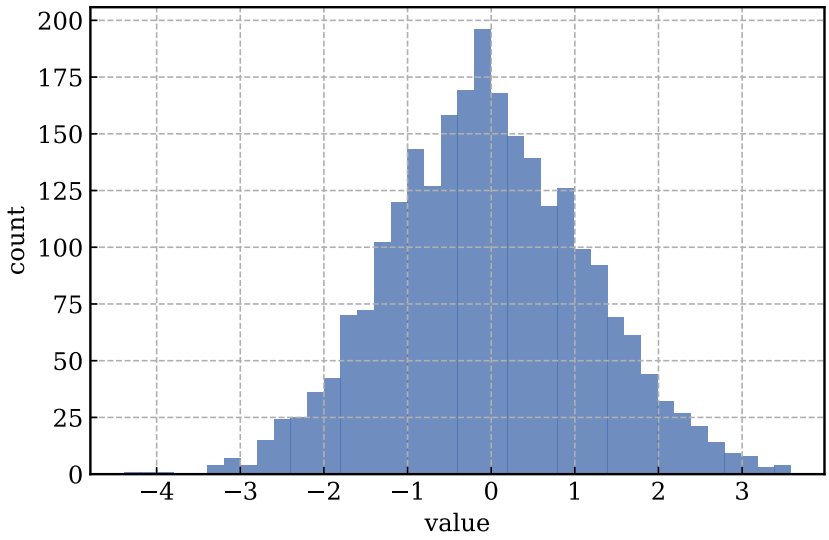


図 9.1　入力配列 `values` を 40 ビンへ数え上げたヒストグラム。横軸は値、縦軸は各ビンに含まれる要素数を表す。

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
values = rng.normal(loc=0.0, scale=1.2, size=2500)

plt.hist(values, bins=40)
plt.xlabel("value")
plt.ylabel("count")
plt.savefig("histogram.pdf")
```

線、文字、マーカーを主体とする図は、ベクタ形式の PDF で保存すると拡大時にも輪郭を保てる。PDF は LaTeX 文書へ直接挿入できる。図 9.1 は、この最小例の出力である。

`plt.show()` は対話的な確認に、`savefig()` は出力条件を指定したファイル保存に用いる。保存先、ファイル形式、解像度、同名ファイルを上書きするかどうかを作図コード内で明示すると、図の生成条件を再現できる。

9.1.2　軸ラベル、凡例、目盛り

次の例では、`plt.subplots()` で図全体を表す `fig` と、点や線を描く領域を表す `ax` を作る。`ax.plot()` はその領域へ線を描き、`fig.savefig()` は図全体を保存する。

図には、各軸が表す量と単位、系列を識別する凡例、比較に必要な目盛りを記す。これらが欠けると、数値の物理的意味や系列間の対応を図だけから判定できない。

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0.0, 10.0, 60)
y = np.exp(-x / 4.0) * (1.0 + 0.08 * np.sin(2.5 * x))
```

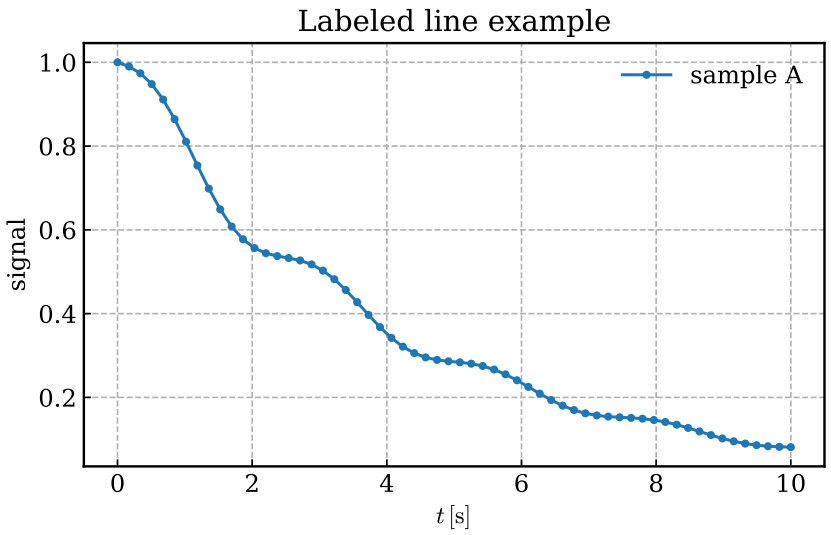


図 9.2　ラベル、凡例、目盛りを付けた線図。横軸は秒単位の時刻、縦軸は信号値、凡例は系列名を表す。

```
6
7  fig, ax = plt.subplots(figsize=(6, 4))
8  ax.plot(x, y, marker="o", label="sample A")
9  ax.set_xlabel(r"$t\,[\mathrm{s}]$")
10 ax.set_ylabel("signal")
11 ax.set_title("Labeled line example")
12 ax.set_xticks([0, 2, 4, 6, 8, 10])
13 ax.legend(loc="upper right")
14 fig.savefig("labeled_plot_example.pdf")
```

図 9.2 には、横軸の量と単位、縦軸の量、系列名、比較に必要な目盛りが示されている。配色を調整する前に、これらの情報が図だけから読み取れるかを確認する。

9.1.3　plot 関数へ渡すデータ

Matplotlib の多くの作図関数は、NumPy 配列や Python リストのような「配列に変換できるもの」を受け取る。加えて、辞書や DataFrame のように列名で参照できるオブジェクトなら、`data=` を使って列名指定で描くこともできる。

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  times = [0, 1, 2, 3]
5  values = [1.0, 2.0, 1.5, 3.0]
6
7  fig, ax = plt.subplots(figsize=(6, 4))
8  x = np.asarray(times)
9  y = np.asarray(values)
10 ax.plot(x, y)
11
```

```
table = {"time": x, "value": y}
ax.plot("time", "value", data=table)
```

np.asarray は、入力を NumPy 配列として扱うことを明示する。data= を用いる場合は列名を文字列で指定するため、存在しない列名を渡したときのエラーも確認する。独立した配列を渡す場合は、x と y を明示する。

9.1.4 線・点・参照線・誤差棒の描画関数

線図には plot()、散布図には scatter()、鉛直および水平の参照線には axvline() と axhline()、誤差付きの点には errorbar() を用いる。次の例は、これらの関数と出力図の対応を示す。

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(1234)
t = np.linspace(0.0, 10.0, 80)
y = np.sin(t) * np.exp(-t / 12.0)
y_noise = y + 0.12 * rng.normal(size=t.size)
y_err = 0.10 + 0.03 * np.cos(t / 2.0) ** 2

fig, axes = plt.subplots(2, 2, figsize=(10, 7))

axes[0, 0].plot(t, y, marker="o")
axes[0, 1].scatter(t, y_noise, c=t, cmap="viridis")
axes[1, 0].plot(t, y_noise)
axes[1, 0].axvline(3.0, linestyle="--")
axes[1, 0].axhline(0.0, linestyle=":")
axes[1, 1].errorbar(t[::4], y_noise[::4], yerr=y_err[::4], fmt="o")

fig.savefig("basic_plot_types_example.pdf")
```

図 9.3 は、主要な描画関数の出力を並べたものである。線図は順序を持つ系列や関数形、散布図は 2 変量の対応とばらつき、参照線は閾値や基準値、エラーバーは測定値の不確かさを表す。

plot() は既定で点を線で結ぶため、順序を持つ系列または連続関数の表示に用いる。scatter() は点どうしを結ばず、個々のイベントや標本の 2 変量分布を表示する。axvline() と axhline() は、閾値や基準値を対応する軸上へ示す。

9.1.5 ヒートマップとカラーバー

格子上で定義された 2 次元配列は imshow() で色に対応させる。これに対して、点群を格子ごとに数え上げる場合は hist2d() を用いる。いずれの場合も、色が表す量と尺度をカラーバーに記す。

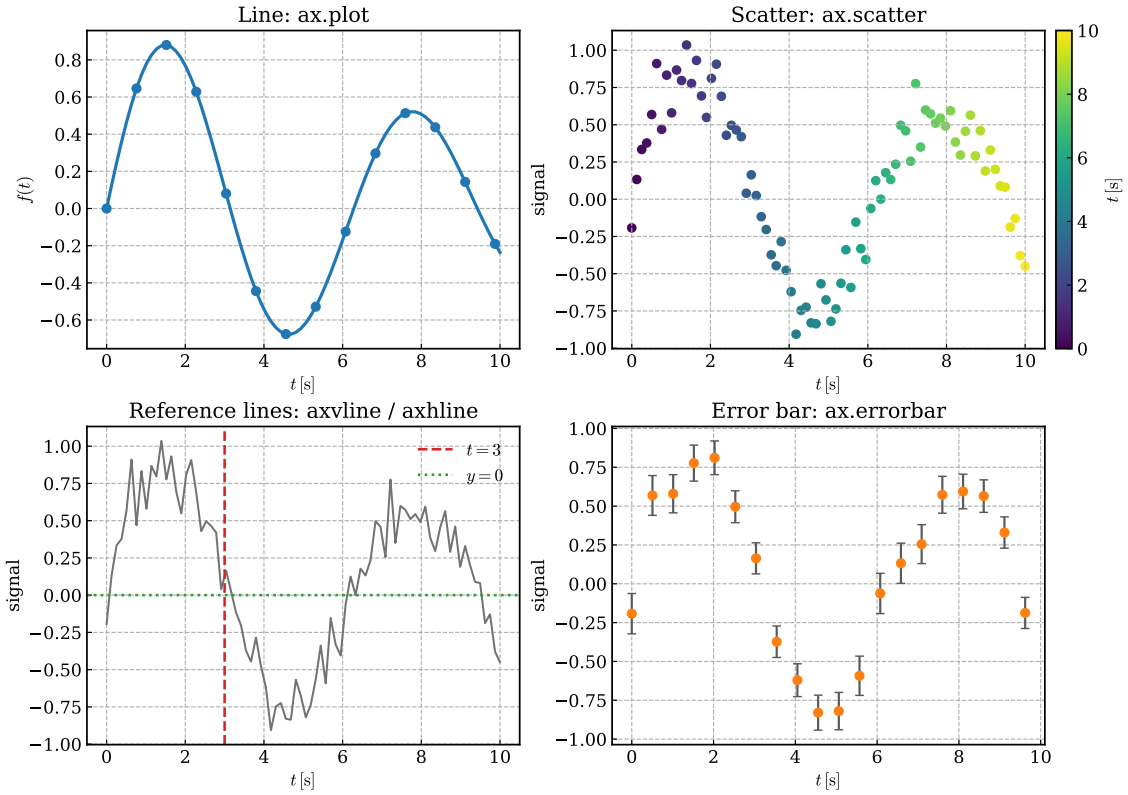


図 9.3　主要な描画関数の出力。左上は `plot()`、右上は `scatter()`、左下は `axvline()` と `axhline()`、右下は `errorbar()` の例である。

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm

x = np.linspace(-3.0, 3.0, 160)
y = np.linspace(-2.5, 2.5, 140)
xx, yy = np.meshgrid(x, y)
image = np.exp(-(xx**2 + 1.6 * yy**2)) + 0.02
rng = np.random.default_rng(1234)
sample_x = rng.normal(loc=0.0, scale=1.0, size=80000)
sample_y = 0.7 * sample_x + rng.normal(loc=0.0, scale=0.8, size=80000)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
im = axes[0].imshow(image, origin="lower", cmap="viridis", aspect="auto")
fig.colorbar(im, ax=axes[0], label="intensity")

h = axes[1].hist2d(sample_x, sample_y, bins=90, norm=LogNorm(), cmap="magma")
fig.colorbar(h[3], ax=axes[1], label="counts")
```

図 9.4 の左は `imshow()` によるヒートマップ、右は `hist2d()` と `LogNorm` を使った 2 次元ヒストグラムである。`cmap=` は値から色への対応を定め、`fig.colorbar(...)` は色に対応する値と、その量の名前を図中に示す。

`imshow()` は格子上に並んだ値を描くため、シミュレーションで得た場や画像データをその

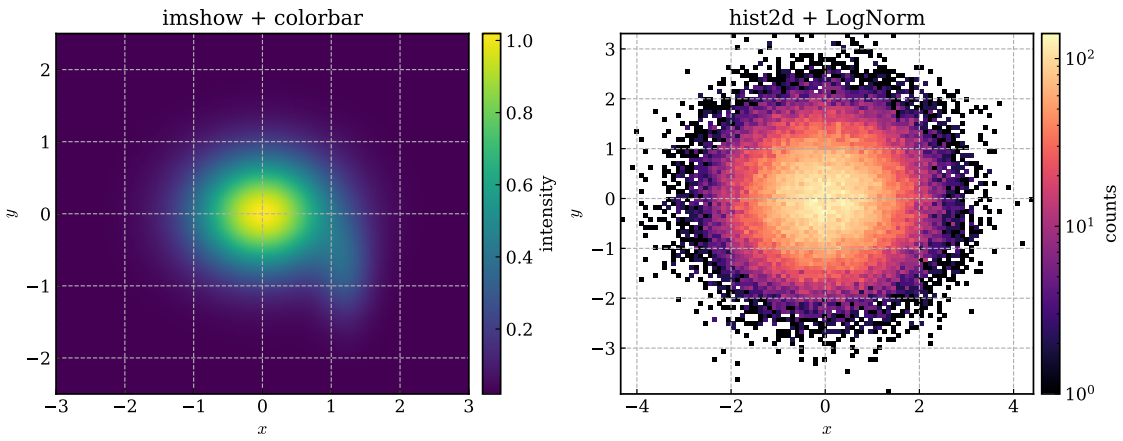


図 9.4　ヒートマップとカラーバーの例。左は `imshow()`、右は `hist2d()` と `LogNorm` を使った例であり、どちらもカラーバーによって色の意味を示している。

まま表示できる。一方、`hist2d()` は点群を格子ごとに数え上げる。格子ごとの点数が数桁にわたる場合は、`LogNorm()` による対数的な色割り当てによって、高密度領域と低密度領域を同じ図に表示できる。

9.1.6　全天図と投影: Hammer 図法と Mollweide 図法

経度と緯度のような球面上のデータを平面へ描くには、投影法が必要になる。Matplotlib には、地図ライブラリなしでも使える全天図向けの投影として、Hammer 図法や Mollweide 図法が用意されている。

```
import numpy as np
import matplotlib.pyplot as plt

lon_deg = np.linspace(-180.0, 180.0, 241)
lat_deg = 25.0 * np.sin(np.deg2rad(2.0 * lon_deg))
lon = np.deg2rad(lon_deg)
lat = np.deg2rad(lat_deg)

fig = plt.figure(figsize=(10, 4))
ax1 = fig.add_subplot(1, 2, 1, projection="hammer")
ax2 = fig.add_subplot(1, 2, 2, projection="mollweide")

ax1.scatter(lon, lat)
ax2.scatter(lon, lat)
```

図 9.5 は、同じ点群を Hammer 図法と Mollweide 図法で投影した例である。どちらも経度の全範囲と両極を有限の領域に写すため、全天イベント分布や銀河座標図に利用できる。`projection="hammer"` と `projection="mollweide"` の座標値にはラジアンを渡す。

国境、海岸線、地理座標系の変換を必要とする地図には Cartopy を用いる。球面座標上の点分布だけを表示する場合は、Matplotlib の標準投影を利用できる。入力座標の単位、経度の範囲、周期境界における処理を作図前に確認する。 Cartopy は追加のライブラリであり、

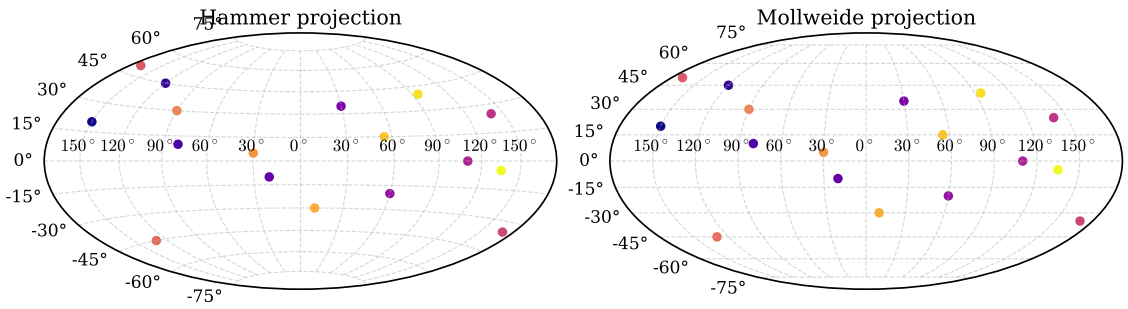


図 9.5　Hammer 図法と Mollweide 図法の例。どちらも経度の全範囲と両極を一つの楕円領域内に投影する。

導入方法は公式資料 [37] を参照する。海岸線などの地図データを初めて使う際には取得が必要になる場合がある。

9.2　分布・相関・密度に応じた図の選択

2 変数の関係を調べるときに、データ量を確認せず散布図を描くだけでは、点の重なりによって密度構造を見落とす場合がある。この節の対象は、データ点数と確認したい性質に応じた散布図、透過表示、2 次元ヒストグラムの選択である。

9.2.1　度数と確率密度の区別

`hist()` の既定の縦軸はビン内の度数である。`density=True` は、幅 Δx_i のビンの度数 n_i を $n_i/(N\Delta x_i)$ に正規化する。ここで N は表示範囲内で数えた総度数であり、全ビンの面積の和が 1 になる。棒の高さの和が 1 になるとは限らない。

不等幅ビンや対数間隔のビンでは、広いビンほど多くの点が入り得る。分布の形を比較する場合は、度数、単位幅当たりの度数、確率密度のどれを示すかを軸に明記する。`density=True` は横軸の値そのものの幅で割る指定であり、対数軸にしただけで対数値の確率密度にはならない。確率密度関数を重ねるときも、正規化する範囲と単位をそろえる。

9.2.2　点の重なりに応じた表示方法の選択

9.2.2.1　散布図による点の重なりの確認

2 変数の対応を点ごとに示す基本的な方法が散布図（`scatter`）である。ただし、データ点が多い場合は同じ領域に点が重なり、局所的な密度差を判別できなくなる。

```
import numpy as np
import matplotlib.pyplot as plt

# 約 60 万点を不透明な点として描画する
rng = np.random.default_rng(19)
```

```
x = rng.normal(loc=0.0, scale=1.0, size=600_000)
y = 0.65 * x + rng.normal(loc=0.0, scale=0.55, size=x.size)

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(x, y, s=10, c='black')
fig.savefig("scatter_bad.png", dpi=150)
```

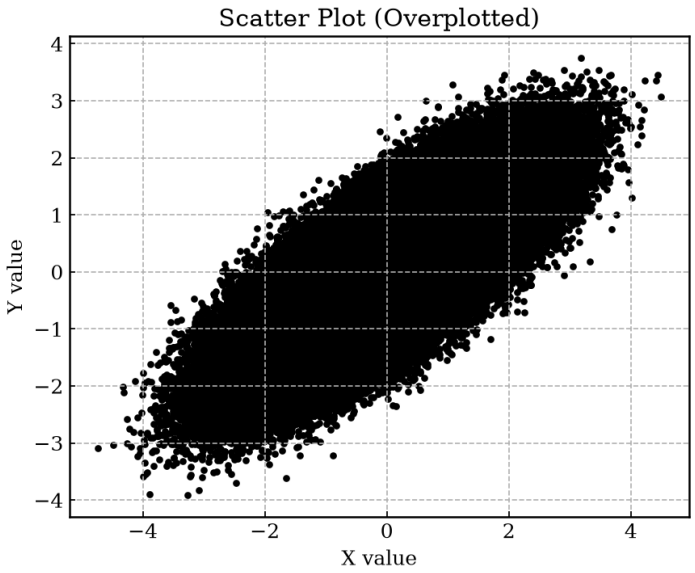


図 9.6　約 60 万点を不透明な点で描いた散布図。点の重なりによって、中心部の密度差を判別できない。

図 9.6 からは点が存在する範囲は分かるが、各領域の点数は読み取れない。領域間の密度は、点数をビンごとに集計する 2 次元ヒストグラムで比較する。

9.2.2.2　2 次元ヒストグラムによる密度の計数

2 次元ヒストグラム（`hist2d`）は、領域ごとの点数を色で表すため、点の重なりがある場合にも密度を定量的に示せる。

```
fig, ax = plt.subplots(figsize=(6, 5))
h_lin = ax.hist2d(x, y, bins=50, cmap='viridis')
cbar = fig.colorbar(h_lin[3], ax=ax, label="Counts (Linear)")
fig.savefig("hist2d_linear_bad.pdf")
```

図 9.7 では中心部の密度を読み取れるが、中心部と周辺部の度数が大きく異なるため、周辺の少数カウントの領域には近い色が割り当てられる。この線形カラースケールでは、周辺部の密度差や裾の広がりを識別しにくい。

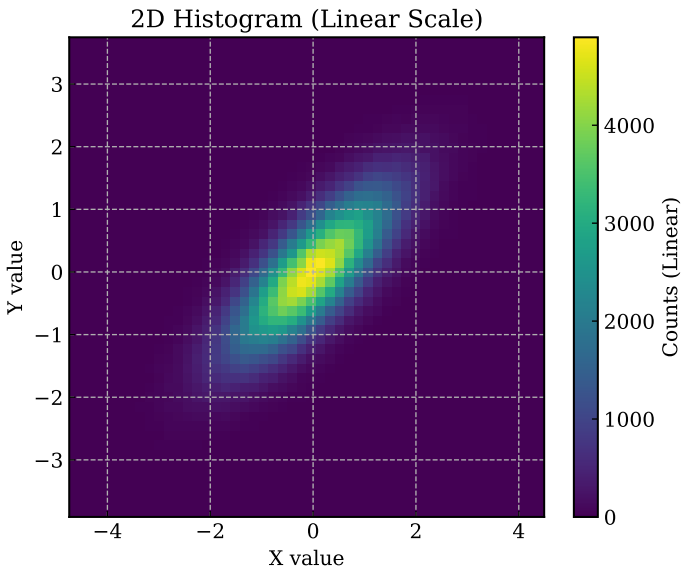


図 9.7　線形カラースケールを用いた 2 次元ヒストグラム。中心部の高密度領域に色範囲の大半が割り当てられ、低密度領域の差を識別しにくい。

9.2.2.3　LogNorm による広い密度範囲の表示

物理イベントの頻度が 1、10、100、1000 のように数桁にわたる場合には、線形カラースケールだけでは低密度領域の差を表しにくい。LogNorm は、正のカウントを対数尺度で色へ割り当てる。

```
import matplotlib.colors as mcolors

fig, ax = plt.subplots(figsize=(6, 5))
# カウントから色への割り当てを対数尺度にする
h_log = ax.hist2d(x, y, bins=50, norm=mcolors.LogNorm(), cmap='viridis')
cbar = fig.colorbar(h_log[3], ax=ax, label="Counts (Log Scale)")
fig.savefig("hist2d_lognorm_good.pdf")
```

図 9.8 では、単一のピークを取り囲む低密度領域も判別できる。線形尺度では近い色に対応していた低密度領域が、対数尺度では異なる色に割り当てられるためである。

9.2.3　subplots() による複数パネルの配置

関連する分布を同じ尺度で比較する場合は、1 枚のキャンバス（Figure）の中に複数のパネル（Axes）を配置する。

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(23)
data_a = rng.normal(loc=0.0, scale=1.0, size=2000)
data_b = rng.normal(loc=0.4, scale=1.3, size=2000)

```

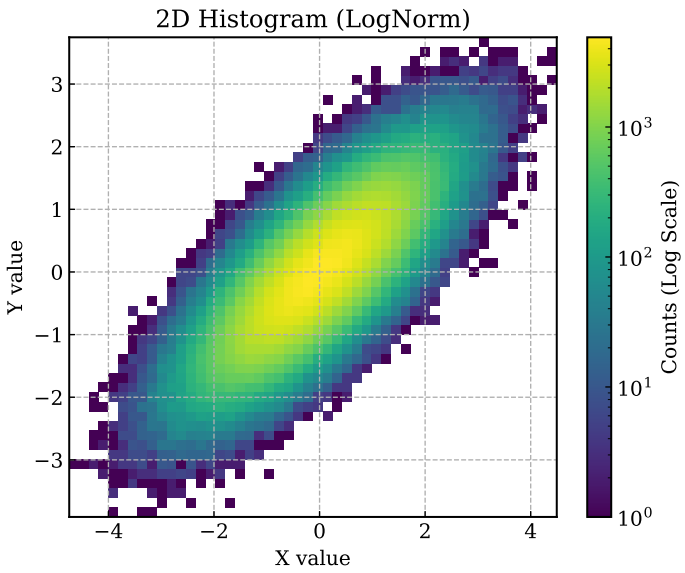


図 9.8　`LogNorm` を用いた 2 次元ヒストグラム。中心の高密度領域と周囲の低密度領域を同じ色尺度で表す。

```
# 1 行 2 列 ( 左右に 2 つ ) のパネルを作成する。
# fig は Figure 全体、axes は二つの Axes を持つ配列である。
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# 左のパネルには axes[0] を使って描画する
axes[0].hist(data_a, bins=30)
axes[0].set_title("Data A")

# 右のパネルには axes[1] を使って描画する
axes[1].hist(data_b, bins=30)
axes[1].set_title("Data B")

# 複数のパネルを描くと軸の数字やラベルが隣の図と重なりやすいため、
# 描画後に余白を調整する。
fig.tight_layout()
fig.savefig("histogram_pair_example.pdf")
```

複数パネルの図では、`axes[0]` のように描画先の `Axes` を明示する。これにより、各処理がどのパネルに作用するかをコード上で追跡できる。`Figure` と `Axes` の役割は、第 9.3 節以降で詳しく説明する。

9.2.4　単純な横並び以外: `subplot_mosaic` と `GridSpec`

上段の主結果に広い領域を割り当て、下段に補助図を 2 枚置くなど、パネルごとに面積を変える場合は、`subplot_mosaic()` または `GridSpec` を用いる。

```
import numpy as np
import matplotlib.pyplot as plt

```

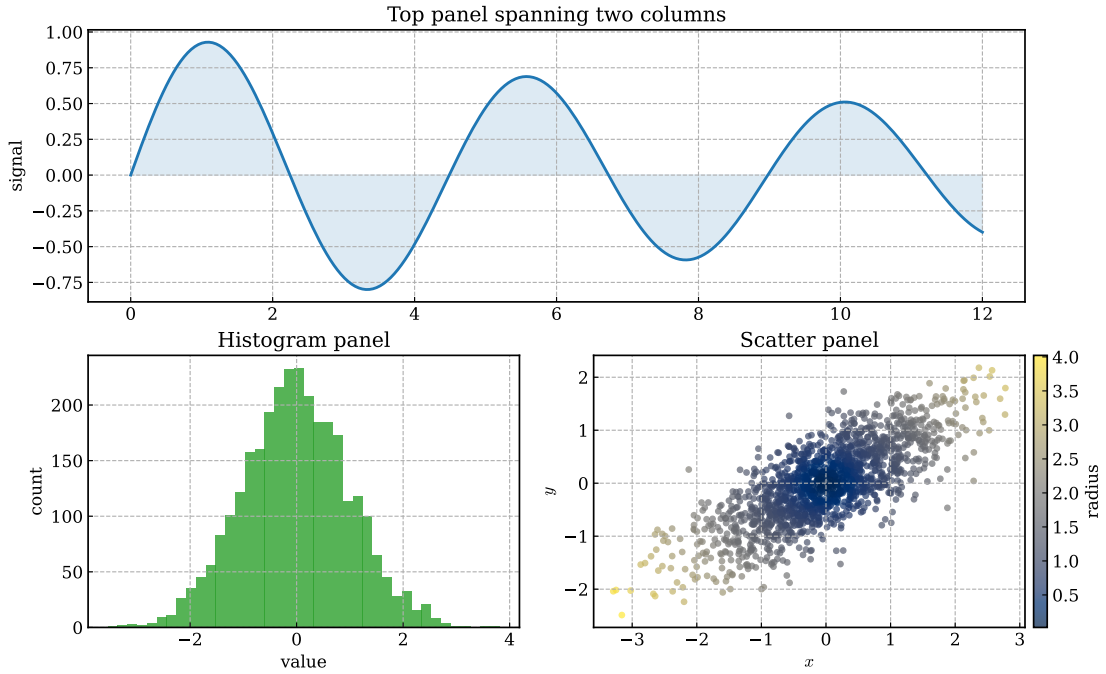


図 9.9　`subplot_mosaic()` を使った複合レイアウトの例。上段は 2 列をまたぐ主図、下段は補助図 2 枚であり、単純な `plt.subplots(1, 2)` では作れない配置になっている。

```
values = np.random.default_rng(42).normal(size=2000)
x = np.linspace(0.0, 10.0, 150)
y_top = np.exp(-x / 5.0) * np.cos(2.0 * x)
x_sc = np.linspace(0.0, 1.0, 25)
y_sc = x_sc**2 + 0.05 * np.sin(20.0 * x_sc)

layout = [["A", "A"], ["B", "C"]]
fig, axes = plt.subplot_mosaic(layout, figsize=(10, 6))

axes["A"].plot(x, y_top)
axes["B"].hist(values, bins=40)
axes["C"].scatter(x_sc, y_sc)
```

図 9.9 のように、`"A"` を 2 回書くと、上段の 2 列を一つの Axes が占める。主図と補助図に異なる面積を割り当てる場合に利用できる。

`subplot_mosaic()` は文字列の名前で Axes を参照できる。`GridSpec` は、列幅、行高、結合するセルを数値で指定する。Axes の参照方法と、行や列の大きさをどこまで調整したいかに応じて選ぶ。

`LogNorm` は 0 以下の値を対数へ写せないため、空のビンなどは通常マスクされる。背景と同じ色で表示された領域を、微小な正の度数と混同しないようにする。また、`imshow()` の既定の座標は画像の画素位置である。物理的な座標で表示する場合は、`origin` と `extent=(xmin, xmax, ymin, ymax)` で上下の向きと画像の外縁を明示する。

9.2.5　凡例とレイアウト

複数の系列やフィット曲線を同じ図へ重ねる場合は、各系列を識別する凡例が必要である。凡例の位置、枠の有無、項目の順序は、データとの重なりと系列間の対応を考慮して定める。

次の例では、同じ物理量を条件 A と B で測定した状況を正規乱数で模擬する。両パネルに同じビン境界と共有した軸を用い、分布の位置と幅を比較する。

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(29)
values1 = rng.normal(loc=0.0, scale=1.0, size=3000)
values2 = rng.normal(loc=0.5, scale=1.2, size=3000)

bin_edges = np.linspace(min(values1.min(), values2.min()),
                        max(values1.max(), values2.max()), 41)
fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharex=True, sharey=True)
axes[0].hist(values1, bins=bin_edges, alpha=0.7, label="sample A")
axes[0].set_title("sample A")
axes[0].set_xlabel("value")
axes[0].set_ylabel("count")

axes[1].hist(values2, bins=bin_edges, alpha=0.7, label="sample B")
axes[1].set_title("sample B")
axes[1].set_xlabel("value")

for ax in axes:
    ax.legend(loc="upper right")
    ax.set_axisbelow(True)

fig.tight_layout()
fig.savefig("subplots_example.pdf")
```

この例では、`axes` は二つの `Axes` を並べた配列であり、`for ax in axes:` の部分で凡例位置をそろえる処理を両パネルへ適用している。`sharey=True` を付けると図 9.10 のように縦軸の尺度がそろうため、左右の高さを直接比較できる。各パネルの系列が一つだけでも凡例を用意しておけば、後からフィット曲線や参照線を追加しても同じ構成を保てる。

`tight_layout` は余白を自動調整するが、すべての文字やパネルの重なりを解消するとは限らない。タイトルや長いラベルが多い場合には、図のサイズや余白を追加で調整する。複数パネルの配置は、比較対象の対応関係が読み取れるかどうかで判断する。

凡例がデータを隠す場合は、`loc="best"` で重なりの少ない位置を選ばせるか、`bbox_to_anchor` で Axes の外側に配置する。自動調整後も、データ、ラベル、凡例の重なりを保存ファイル上で確認する。

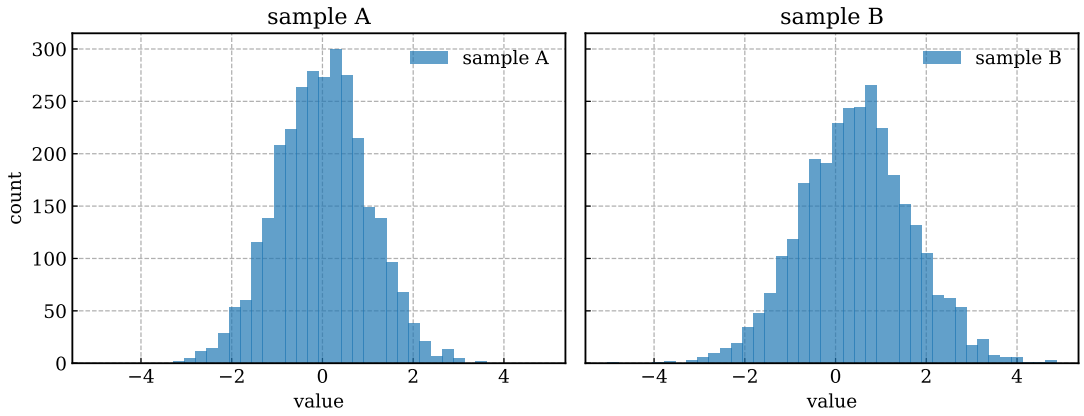


図 9.10　左右に並べたヒストグラムの出力例。両パネルで縦軸の尺度を共有しているため、同じ高さが同じカウントに対応する。

9.2.6　エラーバーと両対数軸

測定値の不確かさを図に示すには、点とともにエラーバーを描く。値が数桁にわたって変化する場合には、対数軸を用いると分布形状を比較しやすい。ただし、対数軸では 0 以下の値を表示できないため、データ範囲を事前に確認する必要がある。

図 9.11 にエラーバーと対数プロットの例を示す。

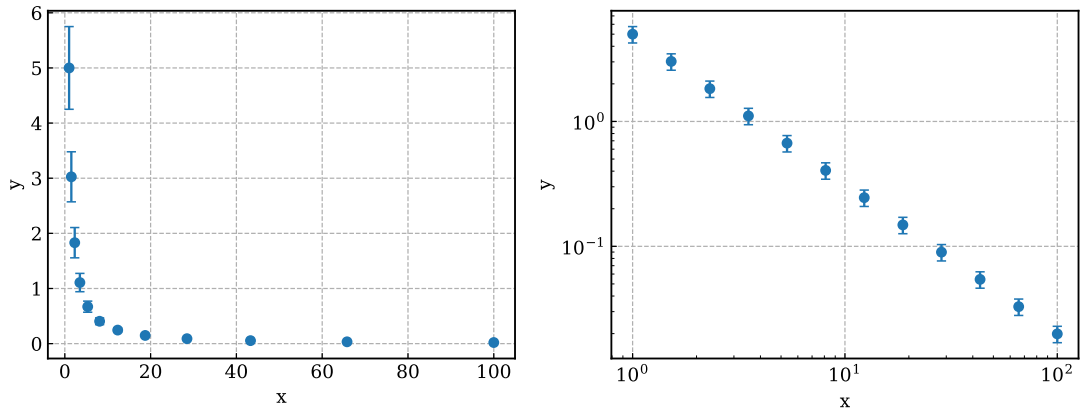


図 9.11　エラーバーと対数軸の設定例。左は線形軸、右は x 軸と y 軸の両方を対数尺度にした図。

```
import numpy as np
import matplotlib.pyplot as plt

x_val = np.geomspace(1.0, 100.0, 12)
y_val = 5.0 * x_val ** (-1.2)
y_err = 0.15 * y_val

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# fmt='o' は点を丸で描き、capsize=3 は端部の横棒を 3 point にする
axes[0].errorbar(x_val, y_val, yerr=y_err, fmt='o', capsize=3, label="data")
```

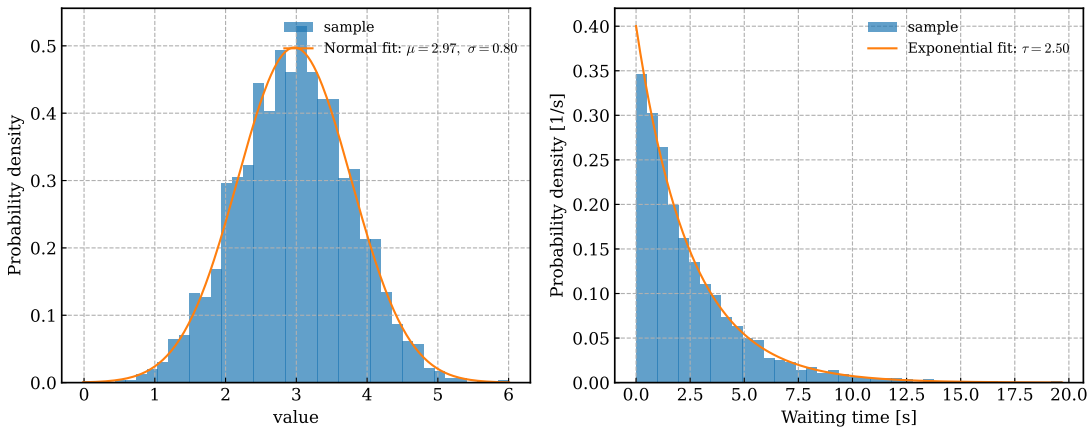


図 9.12　Matplotlib による分布図とフィットの例。左はガウシアン型、右は指数分布であり、どちらもヒストグラムの上にフィット曲線を重ね、凡例に分布名と推定パラメータを示している。

```
axes[0].set_xlabel("x")
axes[0].set_ylabel("y")

axes[1].errorbar(x_val, y_val, yerr=y_err, fmt='o', capsize=3, label="data")
axes[1].set_yscale('log')
axes[1].set_xscale('log')
axes[1].set_xlabel("x")
axes[1].set_ylabel("y")

fig.tight_layout()
fig.savefig("errorbar_example.pdf")
```

9.2.7　分布図における軸・色・密度の読み取り

分布図を解釈するときは、軸が表す量と単位を特定したうえで、分布の中心と幅、裾、外れ値の順に確認する。

例えば、図 9.12 の左パネルでは、ヒストグラムとフィット曲線の残差、および分布の左右対称性を調べる。右パネルでは、短時間側の立ち上がりと長時間側の減衰を同じ時定数で説明できるかを調べる。レポートには、図から読み取った特徴と、それがどのように結論を支えるかを記述する。

9.2.8　画像フォーマットの選択：ラスタとベクタ

解析図の保存形式は、図の要素数、出力先、必要な拡大率に応じて選ぶ。

- **ベクタ形式**（`.pdf`, `.svg`, `.eps`）：線や文字を図形要素として保存するため、拡大しても輪郭がぼやけない。線図や要素数の少ない散布図を論文やレポートへ掲載するときは、`savefig("plot.pdf")` のようにベクタ形式で保存する。
- **ラスタ形式**（`.png`, `.jpg`）：図をピクセルの集合として保存するため、拡大率によって

は輪郭に画素が見える。ベクタ形式では各点が個別要素となるので、密集した散布図や各セルを図形として描くヒートマップではファイルサイズと描画負荷が増える。`imshow()` の画像は PDF 内にもラスタ画像として埋め込まれる。多数の点やセルだけに `rasterized=True` を指定すれば、文字をベクタのまま残すこともできる。図全体を画像にする場合は、`savefig("plot.png", dpi=150)` のように必要な解像度を指定して PNG で保存する。

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, y)
ax.set_xlabel("time [ns]")
ax.set_ylabel("count")

fig.savefig("spectrum.pdf")
fig.savefig("spectrum.png", dpi=150)
```

同じ図でも、`spectrum.pdf` は線と文字をベクタ要素として保持し、`spectrum.png` は指定した画素数のラスタ画像として保存する。線図や要素数の少ないヒストグラムには PDF を用いる。点や格子の数が多く、ベクタ形式ではファイルサイズと描画負荷が増える散布図やヒートマップには、必要な解像度を指定した PNG を用いる。

9.3　オブジェクト指向 API による複雑な図の管理

API は、プログラムから機能を呼び出すための関数名や使い方を指す。ここでは、図全体の `fig` と描画領域の `ax` を分け、`ax.plot()` のように描画先を指定する書き方を扱う。これをオブジェクト指向 API と呼ぶ。

9.3.1　Figure, Axes, Axis, Artist の役割

Matplotlib では、`Figure`、`Axes`、`Axis` が異なる階層を表す。`Axes` は複数形に見えるが、Matplotlib では一つの作図領域を表すクラス名である。各オブジェクトの対応を次に示す。

`Figure`　図全体である。1 つの PDF や PNG、1 つの表示ウィンドウに相当する。`savefig`、`suptitle`、図全体の凡例や colorbar などは Figure のメソッドまたは構成要素である。

`Axes`　データを描く 1 つのパネルである。`plot`、`hist`、`scatter`、`set_xlabel`、`legend` などは Axes のメソッドとして呼び出す。

`Axis (x Axis, y Axis)`　目盛り、目盛りラベル、スケールを管理する部品である。通常は `ax.set_xscale("log")` や `ax.set_xlim(...)` のように Axes のメソッドで設定するが、内部には `ax.xaxis` と `ax.yaxis` がある。

`Artist`　図の中に描かれる要素の総称である。線は `Line2D`、文字は `Text`、凡例は `Legend`、ヒストグラムの棒は `Rectangle` という Artist として管理される。代表的な要素を次に

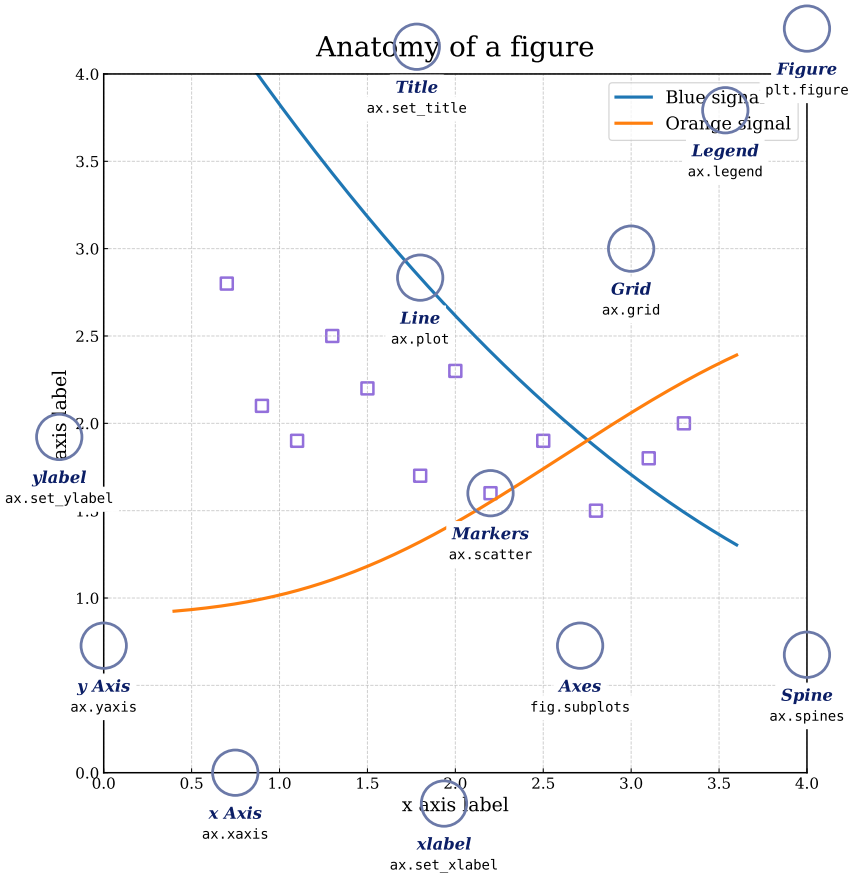


図 9.13 Figure、Axes、Axis、Artist の関係を示した例。1 つの Figure の中に Axes があり、その Axes の中で線、文字、凡例などの Artist が描かれている。

示す。

- `Title`, `Legend`: 図のタイトル、および各系列を示す凡例。
- `Grid`, `Spine`: グリッド線と、座標系を囲む枠線。
- `Line`, `Markers`: 折れ線や散布図の点。
- `xlabel`, `ylabel`: x 軸・y 軸のラベル。

図の構成要素の関係を木構造で示すと、次のようになる。

```
Figure
¦-- Axes
¦   ¦-- Title
¦   ¦-- Spine, Grid
¦   ¦-- xaxis, yaxis (x Axis, y Axis / xlabel, ylabel)
¦   ¦-- Line2D, Markers
¦   `-- Legend
`-- Figure-level Text / Colorbar / ...
```

図 9.13 に、Figure、Axes、Axis、Artist の位置関係を示す。Figure は一つ以上の Axes を持ち、各 Axes は x 軸と y 軸を表す Axis、および線、文字、凡例などを持つ。Figure、Axes、Axis 自体も Artist の一種であり、四者が互いに独立した分類というわけではない。

この対応を実際の Python オブジェクトの型で示すコードを次に掲げる。

```
import matplotlib.pyplot as plt

x = [0, 1, 2, 3]
y = [1.0, 2.0, 1.5, 3.0]

fig, ax = plt.subplots(figsize=(6, 4))
line, = ax.plot(x, y, label="signal")
ax.set_title("Axes title")
ax.set_xlabel("time")
ax.set_ylabel("signal")
legend = ax.legend()

print(type(fig).__name__)
print(type(ax).__name__)
print(type(ax.xaxis).__name__)
print(type(line).__name__)
print(type(legend).__name__)
print(len(fig.axes))
fig.savefig("line.pdf")
```

このコードを実行すると、次の出力が得られる。

```
Figure
Axes
XAxis
Line2D
Legend
1
```

最後の `1` は、この Figure の中に Axes が 1 枚だけ入っていることを表す。もし `plt.subplots(2, 2)` と書けば、Figure は 1 つのままで、Axes が 4 枚入る。

`Artist` は線だけを指す名称ではない。タイトルや軸ラベルは `Text`、ヒストグラムの各棒は `Rectangle` として管理される。次のコードは、それぞれの返り値の型を表示する。

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4))
title = ax.set_title("Histogram")
xlabel = ax.set_xlabel("value")
bars = ax.hist([1, 1, 2, 3], bins=[0.5, 1.5, 2.5, 3.5])[2]

print(type(title).__name__)
print(type(xlabel).__name__)
print(type(bars).__name__)
print(type(bars[0]).__name__)
```

このコードの出力は次のとおりである。

```
Text
Text
BarContainer
Rectangle
```

ここで bars 全体は BarContainer であり、その中の 1 本 1 本は Rectangle である。したがって、「ヒストグラムの棒だけ色を変える」といった細かい調整は、個々の Artist の設定を変えて行う。

Figure 側の要素も、コード上の型を見ると役割を区別しやすい。図全体のタイトル suptitle や colorbar は、特定の Axes だけでなく Figure 全体の構成に関わる。次のコードは、これらの型を表示する。

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  fig, ax = plt.subplots(figsize=(5, 4))
5  img = ax.imshow(np.arange(4).reshape(2, 2))
6  suptitle = fig.suptitle("Figure title")
7  cbar = fig.colorbar(img, ax=ax)
8
9  print(type(suptitle).__name__)
10 print(type(cbar).__name__)
```

このコードでは、次の出力が得られる。

```
Text
Colorbar
```

suptitle は Figure 全体に付く Text であり、colorbar は Figure に追加される補助要素である。複数パネルに共通するタイトルや colorbar は、Figure のメソッドを通じて追加する。

Axis は目盛り、目盛りラベル、表示範囲、尺度を管理する。次の操作は、Axes のメソッドを通じて x Axis の設定を変更する。

```
1  fig, ax = plt.subplots(figsize=(6, 4))
2  ax.plot([1, 10, 100], [2, 3, 5], marker="o")
3  ax.set_xlim(1, 100)
4  ax.set_xscale("log")
```

set_xlim は表示範囲を、set_xscale("log") は座標値から表示位置への変換を対数尺度へ変更する。多くの設定は、ax.xaxis を直接操作せず、Axes のメソッドから変更できる。

操作対象とメソッドの対応を次に示す。

- 図全体を保存する、図全体のタイトルを付ける: Figure
- 線やヒストグラムを描く、軸ラベルや凡例を付ける: Axes
- 対数軸、目盛り位置、目盛りラベルを変える: Axis 関連。ただし実際の操作は Axes メソッド経由が多い
- 特定の線だけ色や太さを変える、凡例や文字を細かく調整する: Artist

保存には `fig.savefig(...)`、凡例には `ax.legend()`、対数軸には `ax.set_xscale("log")` を用いるのは、それぞれの操作対象が Figure または Axes だからである。各メソッドの呼び出し元が `fig` と `ax` のどちらかを追うと、処理対象を判別しやすい。

9.3.2　オブジェクト指向 API による描画

Matplotlib には、現在の作図状態へ描画する `pyplot` API と、`Figure` と `Axes` を明示的に操作するオブジェクト指向 API がある。複数パネルや再利用する作図関数では、描画先を引数として渡せる後者を用いる。

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
values = rng.normal(loc=0.0, scale=1.2, size=2500)

fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(values, bins=40)
ax.set_xlabel("value")
ax.set_ylabel("count")
fig.savefig("histogram.pdf")
```

描画先の `ax` を明示すると、同じ処理を複数の Figure や Axes に適用できる。また、関数の引数として Axes を受け取れるため、作図処理を再利用できる。

`Figure` と `Axes` の区別は、設定の適用範囲にも対応する。図全体のタイトルや余白は Figure、軸ラベルや凡例は各 Axes に設定する。

9.3.3　補助関数による作図処理の集約

同じ体裁の図を反復して作成する場合は、Axes を引数に取る補助関数へ作図処理をまとめる。

```
import numpy as np
import matplotlib.pyplot as plt

def plot_hist(ax, values, label):
    ax.hist(values, bins=40, alpha=0.7, label=label)
    ax.set_xlabel("value")
    ax.set_ylabel("count")
    ax.legend(loc="upper right")

rng = np.random.default_rng(0)
values1 = rng.normal(loc=0.0, scale=1.0, size=2500)
values2 = rng.normal(loc=0.8, scale=1.3, size=2500)

bin_edges = np.linspace(min(values1.min(), values2.min()),
                        max(values1.max(), values2.max()), 41)
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharex=True, sharey=True)
plot_hist(axes[0], values1, "sample A")
plot_hist(axes[1], values2, "sample B")
fig.savefig("histogram_pair_helper.pdf")
```

この補助関数は描画先の Axes を引数として受け取るため、比較図の各パネルや条件ごとの反復描画に同じ処理を適用できる。

9.4　ラベル・凡例・色・スタイルの管理

図の可読性に関わる主な要素は次のとおりである。

- 量と単位を含む軸ラベル
- 簡潔なタイトル
- 適切な余白
- 統一された色と線幅
- レポート全体で一貫した表記

色、フォント、線幅、軸ラベル、単位、凡例の規則を文書全体で統一すると、同じ量や条件を一貫した表現で示せる。系列や条件の区別には、文書内で一貫した色、マーカー、線種を割り当てる。

9.4.1　マーカー、線種、色の指定

Matplotlib では、点の形、線種、色をそれぞれ独立に指定できる。対応する主な引数は次の 5 つである。

```
ax.plot(
    x,
    y,
    marker="o",
    markersize=5,
    linestyle="--",
    linewidth=2.0,
    color="C3",
)
```

- `marker`: 点の形を決める。
- `markersize`: 点の大きさを point 単位で指定する。`scatter()` の `s` は point の二乗単位であり、同じ数値でも意味が異なる。
- `linewidth`: 線の太さを point 単位で指定する。
- `linestyle`: 実線、破線、点線などを決める。
- `color`: 線や点の色を決める。

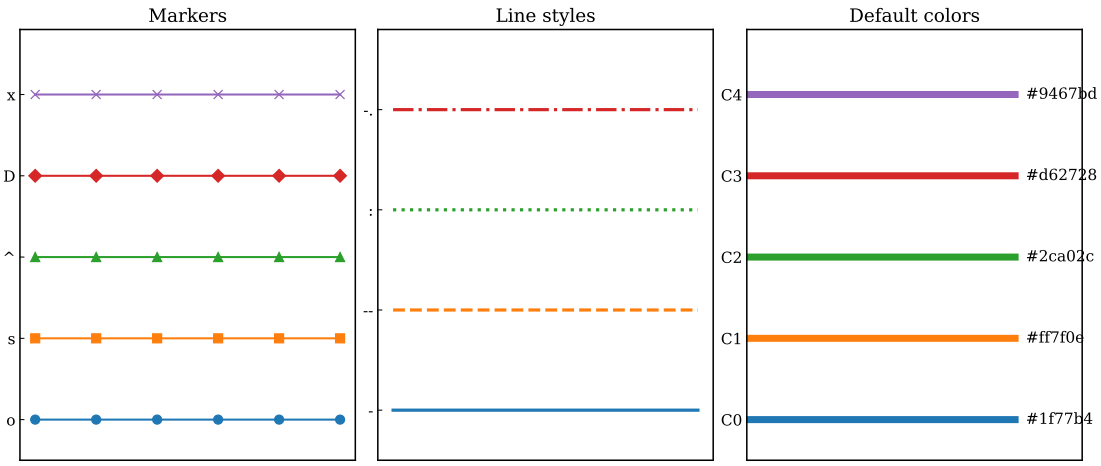


図 9.14　マーカー、線種、既定色の代表例。各指定と描画結果の対応を示す。

表 9.1　marker の指定例

指定	形状	用途の例
`"o"`	丸	測定点、既定の散布図
`"s"`	四角	条件の違いを強調したい点
`"^"`	上向き三角	系列の区別
`"D"`	ひし形	参照データや補助系列
`"x"`	バツ	強調した外れ値や選択点

図 9.14 は、マーカー、線種、既定の色循環の指定と描画結果を並べたものである。

図 9.14 では、マーカー、線種、色の各指定と描画結果を比較できる。マーカーの指定方法と用途は表 9.1、線種は表 9.2、色は表 9.3 にまとめた。系列の識別を色だけに依存させず、マーカーまたは線種も併用すれば、白黒印刷でも系列間の差を保持できる。

色だけによる系列の区別は、白黒印刷や色覚特性によって失われる場合がある。系列が 2 つ以上ある場合は、色に加えて marker または line style を変え、色以外からも系列を識別できるようにする。

表 9.2　line style の指定例

指定	線種	用途の例
`"-"`	実線	主結果、連続関数
`"--"`	破線	フィット曲線、比較対象
`":"`	点線	補助線、閾値
`"-."`	一点鎖線	参照モデル、別条件

表 9.3　色指定の基本

指定	例	備考
`"C0"` など	`"C3"`	既定の color cycle を順に使う
名前付き色	`"black"`, `"tab:blue"`	手早く書ける
16 進数	`"#d62728"`	RGB の色を明示的に固定する場合

9.4.2　rcParams による描画設定の統一

Matplotlib では、図のフォントや線幅などの既定値を `rcParams` という辞書オブジェクトで管理する。複数の図に共通する設定は、作図スクリプトの冒頭でこの辞書へ指定する。

```
import matplotlib as mpl

mpl.rcParams.update(
    {
        "font.family": "serif",
        "mathtext.fontset": "cm",
        "font.size": 12,
        "axes.grid": True,
        "grid.linestyle": "--",
        "xtick.direction": "in",
        "ytick.direction": "in",
        "axes.linewidth": 1.2,
        "legend.frameon": False,
    }
)
```

- `font.family`: 既定フォント族を指定する。本文と図で同じフォント族を使う場合に設定する。
- `font.size`: 文字の基準サイズを指定する。出力媒体と図の寸法に応じて設定する。
- `axes.grid`, `grid.linestyle`: グリッドを既定で表示するか、その線種をどうするかを決める。
- `xtick.direction`, `ytick.direction`: 目盛り線を内向きにするか外向きにするかを決める。
- `axes.linewidth`: Axes を囲む枠線の太さを指定する。
- `legend.frameon`: 凡例の枠を出すかどうかを決める。

`rcParams` で設定した値は、その後に作成する図の既定値になる。図ごとに異なる項目は、`ax.set_*` などのメソッドで上書きする。

表 9.4　カラーマップの分類と指定例

種類	例	対応させる量
Sequential	`viridis`, `plasma`, `cividis`	小さい値から大きい値へ連続的に変化する量
Diverging	`coolwarm`, `RdBu_r`	0 や基準値をはさんで符号が変わる量
Grayscale	`Greys`	印刷や下地としての濃淡表示
Categorical	`tab10`	離散的な系列の識別

9.4.3　cmap の選び方

ヒートマップや散布図で色を値へ対応付けるときは、`cmap=` でカラーマップを選ぶ。どの色を選んでもよいわけではなく、データの意味に応じて使い分ける必要がある。代表例を表 9.4 に示す。

カウント数や強度のように順序と大小を持つ量には、`viridis` などの sequential map を対応させる。正負のずれや差分のように基準値の両側を区別する量には、diverging map を対応させる。値の大小を持たない系列の分類には categorical map を用いる。

9.4.4　設定ファイル (.mplstyle) による描画設定の共通化

解析テーマごとに作図スクリプトが増えると、各 Python ファイルの先頭へ同じ設定辞書をコピーする方法は保守しにくい。Matplotlib では、共通の描画設定を `.mplstyle` 形式のスタイル設定ファイルへ分離し、まとめて読み込める。

作業ディレクトリに置く `my_style.mplstyle` の記述例を次に示す。

```
font.family: serif
mathtext.fontset: cm
font.size: 12
axes.grid: True
grid.linestyle: --
xtick.direction: in
ytick.direction: in
axes.linewidth: 1.2
legend.frameon: False
```

各解析スクリプトの先頭でこの設定ファイルを読み込むと、以降の図に共通の設定が適用される。

```
import matplotlib.pyplot as plt

# スクリプトの冒頭で自分のスタイルシートを読み込む
plt.style.use("./my_style.mplstyle")

# 以降の描画はすべて統一されたスタイル（Serif フォント等）が反映される
fig, ax = plt.subplots(figsize=(6, 4))
```

設定をファイルへ分離すると、解析スクリプトごとのフォントやグリッドの不統一を防げる。学会発表用に文字を大きくした別の設定へも容易に切り替えられる。

9.4.5　共通スタイルの部分的な上書き

共通スタイルを直接変更すると、そのスタイルを使うすべての図に変更が反映される。発表用に文字だけを拡大する場合や、下書きだけグリッドを消す場合は、基準スタイルを読み込んだ後に必要な項目だけを上書きする。

```
# presentation_style.mplstyle
font.size: 16
axes.grid: False
legend.fontsize: 11
```

```
import matplotlib as mpl
import matplotlib.pyplot as plt

plt.style.use(["./my_style.mplstyle", "./presentation_style.mplstyle"])

# スタイルファイルを増やしたくないなら、その場で個別に上書きしてもよい
mpl.rcParams["axes.titlesize"] = 14

fig, ax = plt.subplots(figsize=(6, 4))
ax.grid(False)  # この Axes だけグリッドを消す
```

二つの `.mplstyle` を配列で渡すと、後ろに書いたファイルの設定が優先される。したがって、基準となる `my_style.mplstyle` を保ったまま、文字サイズやグリッドの有無だけを差し替えられる。さらに、`mpl.rcParams` の変更はその後に作成する図へ、`ax.grid(False)` のような Axes メソッドは指定したパネルへ適用される。スタイルファイル、スクリプト内の既定値、個別の Axes の順に、適用範囲が狭くなる。

9.5　図中の数式表記

9.5.1　Matplotlib による数式表記

Matplotlib では、軸ラベルや凡例に数式を書ける。既定では LaTeX そのものではなく、TeX に似た `MathText` という仕組みが使われる。

```
plt.xlabel(r"$\Delta t\,[\mathrm{ns}]$")
plt.ylabel(r"$N$")
```

単位を数式の中へ入れる場合は、上の例のように `\mathrm` を使って立体で表す。図中の記号、字体、単位は、本文での定義と一致させる。

MathText は、Matplotlib 内部で数式の一部を組版する機能であり、LaTeX の処理系そのものではない。分数、上付き・下付き、ギリシャ文字、`\mathrm` などを扱える一方、任意の LaTeX マクロやパッケージ依存の記法には対応しない。本文と同じ LaTeX 処理系による組版が必要な場合は、外部依存関係を記録したうえで `text.usetex=True` を検討する。

本文で τ を時定数として定義したなら、図の凡例とキャプションでも同じ記号を用いる。解析中の量と図中の表記を一対一に対応させる。

9.6 pyplot と Axes・カテゴリ軸・画像保存の注意点

9.6.1 plt と ax の混在による参照先の混乱

`plt.xlabel(...)` のような状態依存 API は、現在選択されている Axes を変更する。複数の Axes や Figure を扱うコードでは、`fig, ax = plt.subplots(...)` で対象を受け取り、`ax.set_xlabel(...)` のようなメソッドで変更先を明示する。

9.6.2 文字列データによるカテゴリ軸への変換

数字に見える列でも、実際には文字列として読まれていると、Matplotlib は数値軸ではなくカテゴリ軸として解釈する。すると、`1`, `2`, `10`, `20` が数値間隔ではなく、単なる 4 つのラベルとして等間隔に並ぶ。

```
import matplotlib.pyplot as plt

x_str = ["1", "2", "10", "20"]
y = [1.0, 4.0, 2.0, 5.0]
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x_str, y, marker="o")
```

図 9.15 の左では文字列がカテゴリ軸として扱われ、右では数値への変換によって数値軸になっている。CSV の読み込み後に軸間隔が数値に比例しない場合は、作図に渡した配列の `dtype` を確認する。

9.6.3 savefig と余白調整の順序

長いラベルや図外の凡例は、保存領域からはみ出す場合がある。`tight_layout()`、`constrained_layout=True`、または `bbox_inches="tight"` の効果を保存したファイルで確認する。再現可能なスクリプトでは、描画、余白調整、保存の順序を固定する。対数軸へ渡す値は正でなければならない。

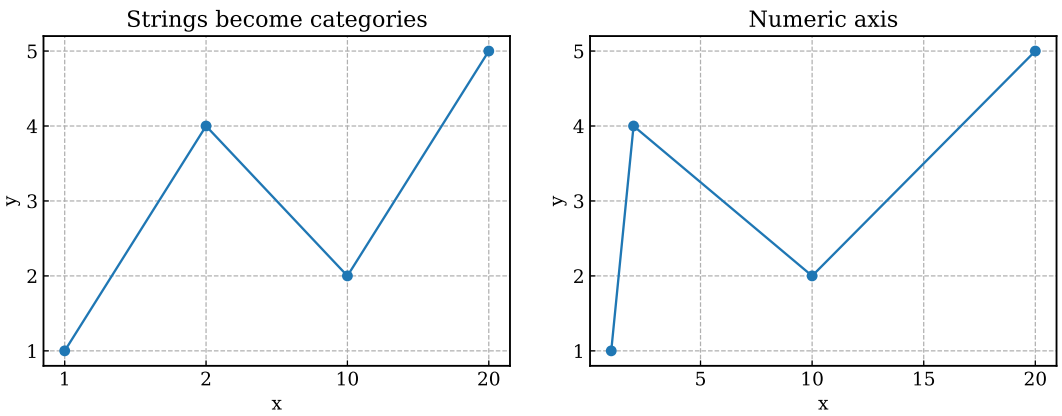


図 9.15　文字列軸と数値軸の違い。左は "1", "2", "10", "20" をカテゴリとして並べており、右は数値として扱っている。

9.6.4　公式資料における作図例の検索

ここまでの各節では、基本的な描画、レイアウト、オブジェクトの役割、保存方法を説明した。実際の作図では、目的に近い例を公式資料から探すと、利用できる図種や引数を把握しやすい。特に次の 4 つは参照頻度が高い。

- Quick start [9]
- Plot types [38]
- Gallery [39]
- Cheatsheets [40]

`Plot types` は図種ごとの選択肢、`Gallery` は完成例と実装、`Cheatsheets` は主要な引数と書式、`Quick start` は `Figure` と `Axes` の基本構造を示す。目的に近い公式例を選び、入力データ、軸、単位、保存形式を解析条件に合わせて変更する。

9.7　章末課題：分布の特徴が伝わる図を作る

まず次の人工データを使う。実カタログと第 6 章の発生間隔があれば、同じ手順で作図する。`figures/` を作り、保存した図を開いて確認する。

```
import numpy as np

longitude = np.array([140.0, 141.2, 139.5, 142.0, 138.7])
latitude = np.array([35.0, 36.1, 34.8, 37.0, 33.9])
magnitude = np.array([3.2, 4.1, 2.8, 4.8, 3.7])
dt = np.array([1.0, 4.0, 12.0, 60.0, 180.0, 900.0, 3600.0])
```

1. **位置とマグニチュードを図にする。**横軸を経度、縦軸を緯度、点の色をマグニチュードとする散布図を作れ。軸ラベル、カラーバー、対象範囲が分かるタイトルを付け、`figures/usgs_global_scatter.pdf` へ保存せよ。図を示し、位置と色から読み取れることを説明せよ。
2. **発生間隔の見せ方を比較する。**固定幅ビン・線形横軸と、対数ビン・対数横軸のヒストグラムを並べよ。縦軸はいずれも件数の対数表示とし、単位、ビン幅、範囲を記す。0 以下と範囲外の値は別に数え、`figures/interval_comparison.pdf` へ保存せよ。短時間側と長時間側の見え方の違いを説明せよ。
3. **発展：海岸線を重ねる。**Cartopy を利用できる場合は、同じ散布図に海岸線を加え、`figures/usgs_map.pdf` へ保存せよ。下の例を使い、地図と入力データの座標系をそれぞれ指定する理由を説明せよ。Cartopy は初回に海岸線データを取得する場合がある。

発展課題の出発点を次に示す。全球図では `set_extent()` を `set_global()` に替える。

```
import matplotlib.pyplot as plt
import cartopy.crs as ccrs

fig, ax = plt.subplots(
    subplot_kw={"projection": ccrs.PlateCarree()}
)
ax.set_extent([122, 154, 20, 50], crs=ccrs.PlateCarree())
ax.coastlines()
points = ax.scatter(longitude, latitude, c=magnitude,
                    transform=ccrs.PlateCarree())
ax.set_xlabel("longitude [deg]")
ax.set_ylabel("latitude [deg]")
fig.colorbar(points, ax=ax, label="magnitude")
fig.savefig("figures/usgs_map.pdf")
```

第 10 章

SciPy による数値計算とフィット

SciPy は、データに式を当てはめる計算や、分布の推定などに使うライブラリである。この章では、まず 5 個の点に合う直線を求める。その後で分布、重み付け、補間、波形処理を学ぶ。求めた値を図へ重ね、データとのずれと適用条件を確かめられるようにする。

10.1 データに直線を当てはめる

10.1.1 データに合う直線を求める

横軸を x、縦軸を y としてデータを描くと、点がおおむね直線に沿って並ぶ場合がある。そこで、直線を表す式 y = a * x + b を使い、点の並びに合う a と b を求める。

a は直線の傾き、b は x = 0 のときの y の値である。このように、データに合わせて式の中の値を求めることを「フィット」という。ここでは、各点の y と直線の y の差を二乗し、その合計が小さくなるように a と b を求める。

式の中で調整する a と b をパラメータという。観測値から式による値を引いた差を残差という。求めた直線をデータと同じ図に描き、各点とのずれを確かめる。次の curve_fit() の例で、この 2 つの値を計算する。

直線を使ってよいかは、データが得られた過程や図の形に基づいて判断する。推定値の解釈には、使った範囲、残差、推定値の不確かさの確認も必要である。重み付けや共分散による評価は、基本のフィットを試した後に説明する。

10.1.2 curve_fit によるパラメータ推定

次の動作確認用のデータでは、傾きと切片を持つ直線を当てはめる。linear_model(x, a, b) に x と候補の a、b を渡すと、直線上の y が返る。curve_fit() はこの関数を繰り返し呼び、データとの差の二乗和が小さくなる a と b を探す。

```
import numpy as np
from scipy.optimize import curve_fit

def linear_model(x, a, b):
    return a * x + b

xdata = np.array([0.0, 1.0, 2.0, 3.0, 4.0])
ydata = np.array([1.1, 2.0, 3.2, 4.1, 5.1])

linear_params, linear_cov = curve_fit(linear_model, xdata, ydata)
linear_errors = np.sqrt(np.diag(linear_cov))

print(linear_params)
print(linear_errors)
```

最初の行が傾きと切片、次の行がそれぞれの標準誤差である。標準誤差は、仮定した誤差モデルの下で推定値がどの程度変わり得るかの目安であり、その計算は後で説明する。まず傾きが約 1.01、切片が約 1.08 になったことを確認する。

```
[1.01 1.08]
[0.02516611 0.06164414]
```

`linear_params` は推定した傾きと切片である。`linear_cov` は、それらの不確かさを 2 行 2 列にまとめた共分散行列である。左上と右下を対角成分と呼び、それぞれ傾きと切片の分散を表す。`np.diag(linear_cov)` はこの 2 個を取り出し、`np.sqrt(...)` は平方根を求めるため、各パラメータの標準誤差が得られる。上の例では `linear_params[0]` が傾き、`linear_params[1]` が切片である。非対角成分はパラメータ間の共分散を表すため、相関を調べる場合は各パラメータの標準偏差で規格化する。`curve_fit()` の出力は、仮定したモデル、残差の定義、`sigma` の指定に依存する。

上の直線モデル $f(x;a,b)=ax+b$ なら、`curve_fit()` は概ね

$$S(a,b)=\sum_{i}\left[y_i-(ax_i+b)\right]^2$$

を小さくする a と b を探していると考えればよい。より一般には、モデル $f(x;\mathbf{p})$ に対して

$$S(\mathbf{p})=\sum_{i}\left[y_i-f(x_i;\mathbf{p})\right]^2$$

という残差二乗和を最小化している。

得られた推定値は、データとモデルを重ねた図、別条件のデータ、単位、残差と照合する。SciPy の各関数についても、入力、返り値、仮定、失敗条件を確認する。

10.2　分布と要約統計

10.2.1　NumPy 配列を入力とする専門計算機能

SciPy の多くの関数は NumPy 配列を入力とし、NumPy 配列またはスカラーを返す。したがって、入力の `shape` と `dtype`、返り値の意味を NumPy の配列操作と対応付ける。SciPy 内で目的別に分けた機能のまとまりをサブパッケージといい、次のように選ぶ。

- `scipy.stats`: 分布、検定、要約統計
- `scipy.optimize`: 最適化、非線形最小二乗、`curve_fit`
- `scipy.interpolate`: 補間
- `scipy.signal`: フィルタ、ピーク検出、相関、スペクトル処理
- `scipy.integrate`: 数値積分
- `scipy.constants`: 物理定数

要素ごとの演算や基本的な集計には NumPy を用い、確率分布、最適化、補間、信号処理などの専用アルゴリズムが必要な場合に、対応する SciPy のサブパッケージを選ぶ。

10.2.2　正規分布と指数分布

多数の独立な微小変動の和は、条件を満たせば正規分布で近似できる。事象が互いに独立に一定の発生率で起こる Poisson（ポアソン）過程では、事象間の待ち時間が指数分布に従う。対象データがこれらの仮定を満たすかは、生成過程、残差、代替モデルとの比較によって検証する。

たとえば、平均 μ、標準偏差 σ のガウシアンは

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$

で書ける。また、時定数 τ の指数分布は

$$p(t) = \frac{1}{\tau} \exp\left(-\frac{t}{\tau}\right) \qquad (t \geq 0)$$

である。SciPy の分布オブジェクトは、確率密度、累積分布、分位点、乱数生成を共通のインタフェースで提供する。

10.2.3　`scipy.stats` による要約統計と分布

配列の要素数、中心、広がり、形状を数値で確認する方法を示す。

```
1 import numpy as np
```

```
from scipy import stats

values = np.array([1.2, 0.9, 1.5, 1.1, 0.8])

summary = stats.describe(values)
print(summary.nobs)
print(f"{summary.mean:.3f}")
print(f"{summary.variance:.3f}")
```

この例では、要素数、平均、分散が順に表示される。

```
5
1.100
0.075
```

`stats.describe()` は、要素数、平均、分散、最小値、最大値、歪度、尖度を名前付きタプルとして返す。分散の既定は `ddof=1` であり、NumPy の `var()` の既定とは異なる。各統計量の定義と単位を確認してから利用する。

SciPy の分布オブジェクトは、確率密度関数や累積分布関数を同じ API で扱える。

```
x = np.linspace(-3.0, 3.0, 7)
pdf = stats.norm.pdf(x, loc=0.0, scale=1.0)
cdf = stats.norm.cdf(x, loc=0.0, scale=1.0)

print(pdf)
print(cdf)
```

`pdf` と `cdf` を同じ点列で評価すると、次のような値が得られる。

```
[0.00443185 0.05399097 0.24197072 0.39894228 0.24197072 0.05399097
 0.00443185]
[0.0013499  0.02275013 0.15865525 0.5        0.84134475 0.97724987
 0.9986501 ]
```

`pdf` は確率密度関数、`cdf` は累積分布関数、`rvs` は乱数生成、`ppf` は累積確率から分位点を求める逆関数に対応する。正規分布では `loc` は平均、`scale` は標準偏差であるが、これらの引数の意味は分布ごとに確認する。

ここで `pdf` は確率密度 $p(x)$、`cdf` はその積分

$$F(x) = \int_{-\infty}^{x} p(t)\,dt$$

に対応する。したがって、`stats.norm.cdf(x)` は「x 以下に入る確率」を返している。

10.3　モデル選択・重み付け・残差評価

モデル推定では、次の項目を順に記録する。

1. 作図に基づく候補関数の検討

2. フィット範囲の決定
3. 必要に応じた妥当な初期値の設定
4. データとフィット曲線の重ね描き
5. 推定値と残差の確認

モデルの定義とデータの確認を省いて `curve_fit()` を実行しても、推定値の妥当性を判断できない。フィットは、モデルという仮説をデータと残差によって検証する手続きである。

10.3.1　初期値、制約、残差

モデルによっては、初期値や制約を与えないと不安定になる。

```
def exp_model(x, a, tau, c):
    return a * np.exp(-x / tau) + c

x_decay = np.linspace(0.0, 8.0, 40)
rng = np.random.default_rng(31)
y_decay = 4.0 * np.exp(-x_decay / 2.5) + 0.3
y_decay += rng.normal(loc=0.0, scale=0.05, size=x_decay.size)

exp_params, exp_cov = curve_fit(
    exp_model,
    x_decay,
    y_decay,
    p0=[5.0, 2.0, 0.0],
    bounds=([0.0, 0.0, -np.inf], [np.inf, np.inf, np.inf]),
)
```

`p0` は初期値、`bounds` はパラメータ範囲である。どちらも収束を保証するものではないが、物理的にあり得ない範囲を除外する際に有効である。モデルによっては、初期値によって収束先が変わる。また、物理的に非負であるパラメータには、0 を下限とする制約を指定できる。

フィット結果の図には、関数形、推定パラメータ、データ系列を識別できる情報を記載する。凡例を用いる場合は、データや曲線を隠さない位置を選び、複数の図で同じ書式を用いる。

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(37)
values = rng.normal(loc=3.0, scale=0.8, size=3000)
mean, sigma = stats.norm.fit(values)
x = np.linspace(values.min(), values.max(), 300)

fig, ax = plt.subplots()
ax.hist(values, bins=40, density=True, alpha=0.7, label="sample")
```

```
ax.plot(
    x,
    stats.norm.pdf(x, loc=mean, scale=sigma),
    linewidth=2.0,
    label=(
        "fit: "
        r"$f(x)=\frac{1}{\sqrt{2\pi}\sigma}"
        r"\exp\!\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$"
        "\n"
        + fr"$\mu={mean:.2f},\ \sigma={sigma:.2f}$"
    ),
)
ax.legend(loc="upper right")
```

図 9.12 では、フィットした分布名と推定パラメータを凡例へ記載している。左パネルは上の正規乱数と同じ入力を用い、右パネルは同じ乱数生成器で続けて生成した時定数 2.5 s、3000 個の指数乱数を用いる。本文には推定方法と解釈、キャプションには図の対象と条件、凡例には系列の識別情報を置く。

```
model_y = linear_model(xdata, *linear_params)
residual = ydata - model_y
print(residual)
```

最適化が終了して推定値を返したことは、モデルの妥当性を保証しない。残差を横軸に対して表示し、符号や大きさに系統的な構造が残っていないか、特定の範囲だけ偏差が大きくないかを確認する。

外れ値やモデルの適用範囲外にある領域を含めると、推定値が変化する場合がある。フィット範囲と除外条件を推定値とともに記録する。

```
mask = (xdata >= 0.0) & (xdata <= 3.0)
range_params, range_cov = curve_fit(
    linear_model,
    xdata[mask],
    ydata[mask],
)
```

フィット範囲は、モデルを適用する領域を定義する解析条件である。範囲をデータ確認後に変更した場合は、その判断基準と変更前後の結果も報告する。

10.3.2　`sigma` を使った重み付きフィット

物理の実データでは、各データ点の不確かさが等しいとは限らない。点ごとの標準偏差が既知であり、その大きさが異なる場合は、`sigma` を使って残差を標準偏差で割る。このように比較の基準となる大きさで割る操作を、ここでは規格化と呼ぶ。同じ 1 のずれでも、標準偏差が 0.5 なら規格化した残差は 2、標準偏差が 2 なら 0.5 となる。`sigma` を与えないフィットはすべての点を等しい重みで扱うため、不確かさが点ごとに異なるデータでは目的と一致

しない場合がある。

```
y_err = np.array([0.2, 0.2, 1.5, 0.2, 0.2])

weighted_params, weighted_cov = curve_fit(
    linear_model,
    xdata,
    ydata,
    sigma=y_err,
    absolute_sigma=True,
)
```

この定義では、`sigma` が大きい点ほど規格化残差が小さくなり、目的関数への寄与が小さくなる。図 10.1 は、その差を模式的に示している。

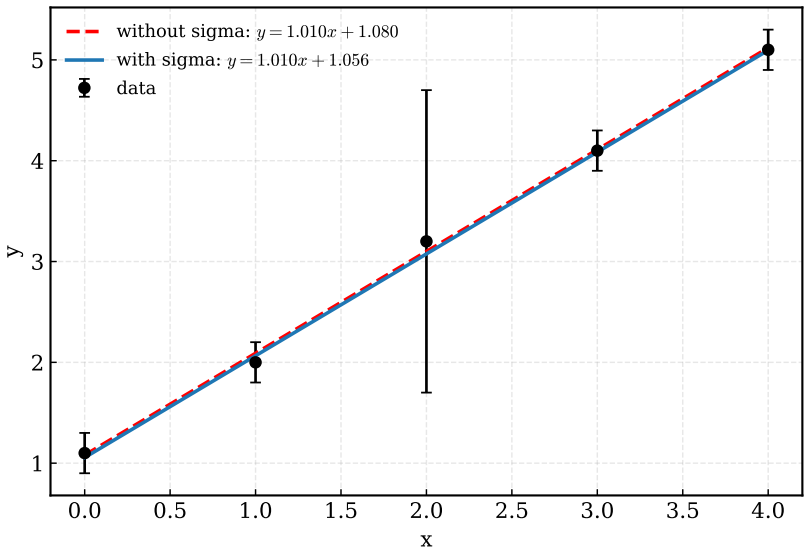


図 10.1　`sigma` の有無によるフィット結果の違い。不確かさが点ごとに異なる場合、`sigma` の指定によって各残差の寄与が変わる。

`sigma` には、通常 `ydata` の各点に対応する 1 次元配列を渡す。このとき最小化される量は

$$\chi^2(\mathbf{p}) = \sum_i \left(\frac{y_i - f(x_i; \mathbf{p})}{\sigma_i} \right)^2$$

と書けるので、σ_i が大きい点ほど目的関数への寄与は小さく、σ_i が小さい点ほど大きくなる。独立な Poisson 計数で平均値を観測数 N によって推定し、正規近似が妥当な範囲では、標準偏差を $\sqrt{N}$ で近似する場合がある。低計数や重み付き計数には、その近似を機械的に適用しない。

`absolute_sigma=True` は、共分散の計算に `sigma` の絶対的な大きさを使う指定である。渡した値が適切な標準偏差かどうかを、この設定が検証するわけではない。これを `False` のままにすると、`sigma` は相対的な重みとしては使われるが、共分散行列の大きさは残差のばらつきに合わせてあとから再スケーリングされる。そのため、推定値の重み付けだけでなく誤差

評価まで行いたいなら、sigma の意味を明確にした上で absolute_sigma の設定も意識する必要がある。

図 10.1 は、点ごとの不確かさを重みに反映した場合と、全点を等しい重みで扱った場合の違いを示している。どちらを採用するかは、sigma が測定の標準偏差を表すか、各点を同じ重みで比較する解析なのかによって決まる。

curve_fit() の共分散とその対角成分から求める標準誤差は、解の近傍での線形近似に依存する [41]。強い非線形性がある場合や、複数のパラメータを変えてもほぼ同じ曲線になる場合（パラメータ間の縮退）には、信頼できないことがある。1 次元の sigma による上の式は各点の誤差を独立とする。相関を扱うには共分散行列を渡す方法があり、通常の呼び出しは横軸 xdata の測定誤差を考慮しない。

10.3.3　scipy.interpolate による補間

補間は、測定点の間における値を仮定した関数形から推定する操作である。ここでは、1 次元データに対する区分線形補間を扱う。

```
from scipy.interpolate import make_interp_spline

x = np.array([0.0, 1.0, 2.0, 4.0])
y = np.array([0.0, 2.0, 1.0, 3.0])

linear_interp = make_interp_spline(x, y, k=1)
x_new = np.array([0.5, 1.5, 3.0])
y_new = linear_interp(x_new, extrapolate=False)

print(y_new)
```

補間点 x_new に対しては、次の値が返る。

```
[1.  1.5 2. ]
```

入力の x は重複がなく、昇順に並んでいる必要がある。区分線形補間は測定点を通る連続関数だが、接続点で傾きが不連続になり得る。上の extrapolate=False は入力範囲の外で NaN を返し、外挿を行わない指定である。範囲外まで同じ傾向が続くとは限らない。

線形補間では、各区間 $x_i \le x \le x_{i+1}$ で

$$y(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i)$$

という一次式を使っている。したがって、測定点の間では直線でつながるが、測定範囲の外ではこの式をそのまま信じる根拠はない。

10.3.4　scipy.signal による平滑化とピーク検出

波形や時系列では、ピークの検出や軽い平滑化がよく必要になる。

```python
from scipy.signal import savgol_filter, find_peaks

x = np.linspace(0.0, 20.0, 200)
signal = np.sin(x) + 0.15 * np.random.default_rng(7).normal(size=x.size)

smooth = savgol_filter(signal, window_length=11, polyorder=3)
peaks, props = find_peaks(smooth, height=0.7, distance=15)

print(peaks)
print(np.round(props["peak_heights"], 3))
```

この例では、検出されたピーク位置とその高さが次のように表示される。

```
[ 13  76 138]
[0.977 1.048 0.994]
```

`savgol_filter()` は局所多項式によって平滑化する。`find_peaks()` は高さ、試料点間の距離、prominence などの条件でピーク候補を抽出する。検出条件をサンプリング間隔と対応付け、既知の合成波形で検出率と誤検出を確認してから実データへ適用する。

10.3.5　`scipy.integrate` と `scipy.constants`

`scipy.integrate` は離散点または関数の数値積分を、`scipy.constants` は物理定数の数値と単位情報を提供する。

```python
from scipy import constants
from scipy.integrate import simpson

x = np.linspace(0.0, np.pi, 101)
y = np.sin(x)

area = simpson(y, x=x)
print(area)
print(constants.c)
```

この積分値と光速定数は、次のように出力される。

```
2.0000000108245044
299792458.0
```

`simpson()` は、ここでは

$$\int_0^{\pi} \sin x \, dx = 2$$

という積分を、離散点 (x_i, y_i) から数値的に評価している。したがって、`simpson(y, x=x)` は「$y(x)$ をサンプル点から積分する関数」と読めばよい。また、`constants.c` は

$$c = 299792458 \ \mathrm{m\,s^{-1}}$$

を表す定数である。`constants.c` 自体は単位を保持しない浮動小数点数なので、単位を演算中も保持する必要がある場合は、次章の Astropy の `Quantity` と定数オブジェクトを用いる。

10.4　モデル・制約・外挿に関する注意点

10.4.1　モデル選択の理由がないままフィットしない

ガウシアン、指数、直線の選択には、物理的または統計的な仮定が必要である。曲線がデータ点の近くを通ることだけでは、モデルの生成過程やパラメータの解釈を正当化できない。

10.4.2　初期値と制約を無視しない

非線形フィットは、初期値によって異なる極小値へ収束したり、収束しなかったりする。`curve_fit()` が値を返したからといって、それが唯一の解だとは限らない。複数の初期値で試す、図へ重ねて見る、残差を見る、といった確認が必要である。

10.4.3　補間と外挿を混同しない

`make_interp_spline()` などの補間関数は、観測点の間を埋めるための道具である。測定範囲の外まで同じ関係が続く根拠は補間からは得られない。外側を予測する場合は、外挿に用いるモデルと適用範囲を別に検討する。

10.4.4　信号処理におけるサンプリング間隔

`find_peaks()` の `distance` やフィルタ長は、サンプル数で指定することが多い。したがって、時間軸が 1 サンプル 1 ns なのか 10 ns なのかで、同じ数値でも意味が変わる。コード中の数字をそのまま流用せず、軸の単位へ戻して考える必要がある。

10.5　発生間隔分布のモデル比較

分布を比べる前に、解析に使う測定値の集まりを標本、その分布を決める調整値をパラメータ（母数）と呼ぶことを確認する。例えば、位置を 0 に固定した指数分布では、平均待ち時間が一つのパラメータになる。

図では、ある値以下に入る標本の割合を階段状に描くと、分布の形を比較できる。例えば 5 個の間隔のうち 3 個が 100 秒以下なら、100 秒での割合は $3/5 = 0.6$ である。この図を経験累積分布関数（ECDF）と呼び、モデルの累積分布関数（CDF）と重ねる。次は、5 個の人工

的な間隔を指数分布と比べる例である。

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

intervals = np.array([30.0, 60.0, 90.0, 140.0, 360.0])
ordered = np.sort(intervals)
fraction = np.arange(1, len(ordered) + 1) / len(ordered)
loc, scale = stats.expon.fit(intervals, floc=0)
print(scale)

fig, ax = plt.subplots()
ax.step(ordered, fraction, where="post", label="data")
ax.plot(ordered, stats.expon.cdf(ordered, loc=loc, scale=scale),
        label="exponential model")
ax.set_xlabel("interval [s]")
ax.set_ylabel("cumulative fraction")
ax.legend()
fig.savefig("interval_cdf.pdf")
```

平均待ち時間の推定値は 136 秒となる。`fraction` は `[0.2, 0.4, 0.6, 0.8, 1.0]` であり、100 秒以下の割合は 0.6 である。`floc=0` は分布の開始位置を 0 秒へ固定する指定である。保存した図では、同じ横軸の値に対するデータとモデルの縦軸の差を比べる。少数の値で描画方法を確かめた後、実データでは標本数や独立性も考慮してモデルを判断する。

10.5.1　補足：尤度と AIC による比較

個々の値が互いに独立で同じ分布から得られたと仮定すると、各観測値での確率密度を掛け合わせ、パラメータによってデータとの合い方を比べられる。この積を尤度という。連続値では密度を使うので、尤度を「その値が出る確率」とは呼ばない。尤度を最大にするパラメータを探す方法が最尤推定であり、SciPy の連続分布の `fit()` は既定でこの方法を使う [43]。計算では自然対数を取り、密度の積を `logpdf(...).sum()` という和へ変える。

調整するパラメータを増やすと、データへ合わせやすくなる。その複雑さも含めて候補を比べる指標の一つが AIC（赤池情報量規準）であり、$\mathrm{AIC} = 2k - 2\log L$ と定義される [44]。k はデータから推定したパラメータ数、L は最大尤度、log は自然対数である。同じ標本と同じ観測範囲に対して求めた候補の間で、小さい値を選ぶ目安にする。AIC が小さいことだけでは、モデルの正しさは保証されない。地震間に相関があれば、上の独立性の仮定も別に検討する。

SciPy は、入力配列の物理的意味や、採用すべきモデルを自動では決めない。特にフィットでは、データ範囲、重み、残差、推定値の不確かさを確認し、採用した条件を結果とともに記録する必要がある。

本文を読み終えた後で関数名や引数を確認したいときは、SciPy User Guide [10] を起点にすると探しやすい。分布や検定は Statistics [42]、フィットや最適化は Optimization [45]、補

間は Interpolation [46]、波形処理は Signal processing [47]、数値積分は Integration [48] が対応している。

10.6　章末課題：モデルを当てはめて妥当性を確かめる

最初の 2 問では、次の人工データを使う。

```
import numpy as np

rng = np.random.default_rng(20260412)
synthetic_dt = rng.exponential(scale=120.0, size=5000)
x = np.linspace(0.0, 10.0, 80)
y_true = 2.5 * x + 1.2
sigma = np.full_like(x, 0.5)
y_obs = y_true + rng.normal(scale=sigma)
```

1. **平均待ち時間を推定する。** `stats.expon.fit(synthetic_dt, floc=0)` の `scale` を求めよ。標本平均と比べ、生成時に指定した平均 120 秒に近い値（115〜125 秒）になることを `assert` で確認せよ。生成時の設定値、標本平均、推定値を表にせよ。
2. **直線を求め、残差を描く。** `y_obs` を直線へフィットし、`sigma=sigma, absolute_sigma=True` を指定せよ。傾き・切片と標準不確かさを表にし、上段にデータと直線、下段に残差を描け。生成時に指定した傾き 2.5、切片 1.2 と比較せよ。検算の目安は傾き 2.3〜2.7、切片 0.7〜1.7 である。
3. **実データの分布モデルを検討する。** 実カタログの発生間隔について、ヒストグラムと経験累積分布を作れ。指数分布を含む候補を比べ、採用するモデルと範囲を選べ。図と残差を示し、選択理由を説明せよ。期間、地域、件数、地震の種別とマグニチュード尺度を記録する。実データがなければ `synthetic_dt` で方法を確かめる。
4. **解析条件を変えて比較する。** 3 問目のビン幅、推定範囲、重み、モデルのいずれかを変え、推定値と不確かさを表で比較せよ。どのデータの寄与が変わったかを説明する。範囲を切って推定する場合は、その範囲で規格化した条件付き分布を使う。指数分布で $t \geq c$ だけを使う場合は、同モデルの無記憶性に基づき $t - c$ へ当てはめる。
5. **発展：地震規模と累積件数の関係を調べる。** 実カタログから、マグニチュード M の分布と累積件数 $N(\geq M)$ を描け。記録の取りこぼしが少ないと考える下限 M_c を選び、$\log_{10} N(\geq M) = a - bM$ の関係 [50] を検討せよ。M_c、件数、推定法、不確かさ、図と残差を示す。累積件数の点は独立でないため、推定には個々の値の尤度か、重ならない区間の計数モデルを用いる。

第 11 章

Astropy による時刻・単位・天体データ処理

Astropy は、単位や時刻、天体の座標を扱うライブラリである。この章では、数値と単位を一組にして計算し、秒とナノ秒などの換算を確かめる。次に時刻差や天体の座標を求め、単位を残したまま表へ保存する。数値だけを扱う場合との違いを説明できるようにする。

11.1 数値と単位・時刻・座標の一元的な管理

11.1.1 数値への単位・時刻・座標情報の付与

Astropy の中心的な発想は、「数値そのもの」ではなく「何の単位で、どの時刻系で、どの座標系なのか」までオブジェクトへ持たせることである。例えば 100 という値だけでは、100 ns なのか 100 ms なのか分からない。数値と単位を一組にした `Quantity` を使えば、この曖昧さを減らせる。ns（ナノ秒）は 1 秒の 10 億分の 1、ms（ミリ秒）は 1 秒の 1000 分の 1 である。

```
import astropy.units as u

time_window = 250 * u.ns
speed = 299792458 * u.m / u.s
distance = (speed * time_window).to(u.m)

print(f"{distance.to_value(u.m):.7f} m")
print(f"{distance.to_value(u.km):.10f} km")
```

この計算では、単位付きのまま距離が評価され、次のように表示される。

```
74.9481145 m
0.0749481145 km
```

この例では、`250 * u.ns` という時点で「250 ns」という意味が値に埋め込まれている。`u` は単位を使うための短い名前であり、`u.ns` はナノ秒、`u.m` はメートル、`u.s` は秒を表

す。`.to(u.m)` は単位付きのままメートルへ換算し、`.to_value(u.m)` はメートルへ換算した数値だけを取り出す。式の途中で単位を落とさない限り、単位換算はライブラリが引き受ける。

数式で書けば、ここで行っているのは

$$d = v\Delta t$$

である。特に $v = c$ とみなすなら、250 ns の時間窓に対応する距離 $d = c\Delta t$ をそのまま安全に計算できる。

11.1.2 時刻系・座標系を属性として保持する理由

観測データでは、UTC、TAI、TT、MJD、JD、Unix time が混在することがある。文字列の分解だけで相互変換すると、時刻系のオフセットとうるう秒が計算に反映されず、時刻差を誤る可能性がある。Astropy の `Time` は、時刻形式と時刻系を属性として保持する。

UTC（協定世界時）、TAI（国際原子時）、TT（地球時）は時刻系であり、同じ瞬間に付ける数値の基準が異なる。一方、JD（ユリウス日）、MJD（修正ユリウス日）、Unix time は時刻の表現形式である。MJD = JD − 2400000.5 であり、どの時刻系で表した値かを併記する。時刻系の指定と、日時を文字列や日数で表示する形式の指定は別の操作である [51]。

11.2 単位・時刻・座標・表・FITS の操作

11.2.1 `units` と `Quantity`

単位付き計算の基本は `Quantity` である。

```
import astropy.units as u

length = 3.2 * u.m
time = 12.5 * u.ns
velocity = length / time

print(velocity)
print(f"{velocity.to_value(u.m / u.s):.1f} m / s")
```

単位変換の結果は次の通りである。

```
0.256 m / ns
256000000.0 m / s
```

`to()` は、指定した単位へ換算した `Quantity` を返す。`.value` は単位情報を除いた数値を返すため、単位を受け取らない関数へ渡す直前など、数値だけが必要な箇所に使用を限定する。

この計算は

$$v = \frac{L}{t}$$

に対応しており、具体的には

$$v = \frac{3.2\ \mathrm{m}}{12.5\ \mathrm{ns}} = 0.256\ \mathrm{m/ns} = 2.56 \times 10^{8}\ \mathrm{m/s}$$

である。コードの出力をこの換算式と照合すれば、桁と SI 接頭語の対応を検証できる。

11.2.2　物理定数

Astropy には物理定数も入っている。SciPy の `constants` と似ているが、Astropy では値が単位付きで返る。

```
from astropy.constants import c, k_B

print(c)
print(k_B)
```

Astropy の定数オブジェクトを表示すると、値だけでなく単位と参照元も確認できる。

```
Name   = Speed of light in vacuum
Value  = 299792458.0
Uncertainty  = 0.0
Unit  = m / s
Reference = CODATA 2022
Name   = Boltzmann constant
Value  = 1.380649e-23
Uncertainty  = 0.0
Unit  = J / K
Reference = CODATA 2022
```

この表示から、定数の値、単位、不確かさ、参照する定数体系を確認できる。

11.2.3　`Time` による時刻表現

Astropy の `Time` は、複数の時刻形式と時刻系を扱い、相互に変換できる。

```
from astropy.time import Time

t = Time("2026-04-12 12:34:56.123456789", format="iso", scale="utc")

print(t.mjd)
print(t.unix)
print(t.tai.iso)
```

同じ時刻を複数の表現へ変換すると、次の値が得られる。

```
61142.52426068816
1775997296.1234567
2026-04-12 12:35:33.123
```

`format="iso"` は入力形式、`scale="utc"` は時刻系を表す。`mjd` と `unix` は同じ時刻の別形式を返し、`tai.iso` は時刻系を TAI へ変換して ISO 形式で返す。

たとえば、修正ユリウス日 MJD は

$$\mathrm{MJD} = \mathrm{JD} - 2400000.5$$

で定義される。Astropy の `Time` は、時刻形式と時刻系を属性として保持し、それらの変換を同じオブジェクト上で行う。

11.2.4 時刻差と `TimeDelta`

差分計算も `Time` のまま行える。`TimeDelta` によって、差分の単位変換も一貫して扱える。

```
from astropy.time import Time

t1 = Time("2026-04-12T12:00:00.000000100", format="isot", scale="utc")
t2 = Time("2026-04-12T12:00:00.000000320", format="isot", scale="utc")
dt = t2 - t1

print(f"{dt.to_value('s'):.9e}")
print(f"{dt.to_value('ns'):.3f}")
```

時刻差を秒と ns へ変換した結果は次の通りである。

```
2.200000182e-07
220.000
```

`TimeDelta` の既定の内部形式や表示は、入力形式と同じ単位とは限らない。そのため、出力では `to_value("s")` や `to_value("ns")` によって単位を明示する。秒境界や日付境界をまたぐ場合も、文字列を分解せずに差を計算できる。

ここで計算している時刻差は

$$\Delta t = t_2 - t_1 = 220 \text{ ns}$$

である。`Time` どうしの減算は `TimeDelta` を返すため、時刻系に基づく差分と単位情報が結果に保持される。

Astropy の `Time` は内部で時刻を二つの浮動小数点数に分けて保持する。`t.unix` などを単一の浮動小数点数として取り出すと、元の時間分解能が失われる場合がある。ns 程度の差を調べるときは `Time` のまま減算してから単位変換する。また、表示桁数は計算精度とは別であり、`t.precision = 9` などで変更できる。

11.2.5 `SkyCoord` による座標変換

地上の位置を経度と緯度で示すように、空の方向も二つの角度で表せる。赤経は天の赤道に沿って測る角度、赤緯は天の赤道から南北へ測る角度である。次の例では赤経 150 度、赤

緯 20 度の方向を指定する。SkyCoord は、この二つの角度と、その基準となる座標系をまとめて保持する。frame="icrs" は国際天球基準座標系（ICRS）を使う指定である。

```
from astropy.coordinates import SkyCoord
import astropy.units as u

coord = SkyCoord(ra=150.0 * u.deg, dec=20.0 * u.deg, frame="icrs")
gal = coord.galactic

print(f"{gal.l.deg:.6f}")
print(f"{gal.b.deg:.6f}")
```

同じ空の方向を、銀河面を基準とした銀河座標へ表し直す。変換後の銀河経度と銀河緯度は次の通りである。

```
213.800977
50.263718
```

ra は赤経（right ascension）、dec は赤緯（declination）を表し、icrs は国際天球基準座標系を指定する。ra と dec には単位を付ける。これにより、入力角度が度またはラジアンのどちらで与えられたかが SkyCoord に保持される。

この操作は、赤道座標 (α, δ) から銀河座標 (l, b) への変換に対応する。天球上の回転そのものは自分で式を書かなくても、入力座標系と出力座標系を明示すれば Astropy が一貫して扱える。

11.2.6　Table と QTable

Astropy には表処理のために Table と QTable がある。Table も列の単位をメタデータとして保持できるが、QTable は単位付きの列を Quantity として保持し、単位を考慮した演算に使える。

```
from astropy.table import QTable
import astropy.units as u

table = QTable()
table["time"] = [0.0, 1.0, 2.0] * u.s
table["signal"] = [12.4, 10.1, 14.8] * u.mV
table["detector"] = ["A", "A", "B"]

print(table)
```

QTable を表示すると、列名だけでなく単位も一緒に確認できる。

```
time signal detector
 s    mV
---- ------ --------
0.0   12.4        A
1.0   10.1        A
2.0   14.8        B
```

QTable は Quantity 列を保持するため、単位付きの列をそのまま演算できる。一方、複雑な groupby、join、列演算を中心とする処理には pandas や Polars の機能を利用できる。単位とメタデータの保持、および必要な表操作の双方から中心となる表形式を選ぶ。

11.2.7 Astropy Table と pandas の行き来

Astropy の表と pandas の DataFrame は相互変換できるが、単位や時刻の情報が完全には残らないことがある。

```
df = table.to_pandas()
print(df)
```

to_pandas() で得た通常の pandas 列には、Quantity の単位属性はそのまま保持されない。変換前に列単位を記録し、変換後の数値へどの単位を対応させるかを明示する。マスク付き列と時刻列についても、欠損表現と型が変換前後で一致するかを確認する。

11.2.8 astropy.io.fits による FITS の読み書き

天文データでは FITS（Flexible Image Transport System） が広く用いられる。HDU（Header/Data Unit）はヘッダとデータをまとめた単位であり、一つの FITS ファイルは複数の HDU を持てる。Astropy なら NumPy 配列やヘッダをそのまま扱える。

```
import numpy as np
from astropy.io import fits

image = np.arange(9).reshape(3, 3)
hdu = fits.PrimaryHDU(image)
hdu.header["OBJECT"] = "test"
hdu.writeto("image.fits", overwrite=True)

with fits.open("image.fits") as hdul:
    data = hdul[0].data.copy()
    header = hdul[0].header.copy()

print(data.shape)
print(header["OBJECT"])
```

FITS を書いてから読み直すと、配列形状とヘッダ値は次のように確認できる。

```
(3, 3)
test
```

`fits.open()` は HDU のリストを返す。画像は `PrimaryHDU`、表は `BinTableHDU` などに格納される。`hdul.info()` で各 HDU の型、名前、次元を確認してから、対象のデータを読み込む。

11.2.9　ASCII・ECSV 形式の読み込み

Astropy の表入出力は FITS だけではない。`Table.read()` は ASCII、CSV、ECSV などにも対応している。

```
from astropy.table import Table

table.write("events.ecsv", format="ascii.ecsv", overwrite=True)
tbl = Table.read("events.ecsv", format="ascii.ecsv")
print(tbl.colnames)
print(len(tbl))
```

`Table.read()` の第 1 引数は入力パス、`format="ascii.ecsv"` は ECSV として解釈する指定である。上で作った 3 行・3 列の表を保存して読み直すと、`['time', 'signal', 'detector']` と `3` が表示される。ECSV は、列の単位、データ型、表のメタデータをテキストとともに保存できるため、中間生成物の受け渡しに利用できる。保存時は `table.write("events.ecsv", format="ascii.ecsv", overwrite=True)` と書く。`overwrite=True` は同名ファイルの置換を許可する指定である。

11.3　単位情報・時刻系・座標系を失わないための注意

11.3.1　`.value` を早く使い過ぎない

`Quantity.value` は、単位を除いた NumPy 配列またはスカラーを返す。以後の計算では単位整合性が検査されないため、外部ライブラリへ渡す直前まで `Quantity` を保持し、`.to_value("ns")` のように変換先の単位を明示する。

11.3.2　時刻系を省略しない

`Time("2026-04-12 12:00:00")` のように `format` と `scale` を省略すると、Astropy の推定と既定値に依存し、作成者が意図した入力形式と時刻系がコード上に残らない。観測データでは `format=` と `scale=` を明示し、UTC、TAI、TT などを区別する。

11.3.3　座標変換に必要な観測条件

赤道座標から銀河座標への変換だけなら比較的単純だが、地平座標へ変換するには観測地点や観測時刻が必要になる。座標系だけを変えているつもりでも、背後では時刻と場所が意味を持つことがある。

11.3.4　DataFrame への変換に伴う情報の欠落

Astropy Table を pandas や Polars へ変換すると、単位、マスク、時刻系、メタデータの一部は対応する表現を持たず、失われたり別の型へ変換されたりする。変換前後の列型とメタデータを比較し、必要な情報は別の列または付随するメタデータとして保存する。集計のために変換する場合も、計算後に単位を復元できる情報を残す。

11.4　SciPy・pandas とのデータ受け渡し

Astropy を使うべき場面は、数値とともに単位、時刻系、座標系、天文形式の情報を保持する必要がある場合である。代表的な用途を次に示す。

- 観測時刻を UTC、MJD、JD、Unix time の間で変換したいとき
- ns、μs、mV、MeV などの単位を混ぜて計算したいとき
- 天球座標を別の座標系へ変換したいとき
- FITS 画像や FITS テーブルを直接読み書きしたいとき
- 表データの単位やメタデータを保ったまま受け渡したいとき

大規模表の集計や複雑な join では、pandas や Polars が必要な API と性能を備えている場合がある。その場合は、Astropy で単位と時刻を解釈し、変換時に失われる情報を記録してから必要な列だけを表処理ライブラリへ渡す。集計結果を Matplotlib や SciPy へ渡すときも、数値に対応する単位と時刻系を併記する。

全体の導入は Getting Started [52] と User Guide [11]、単位は Units and Quantities [53]、時刻は Time and Dates [51]、座標は Coordinates [54]、表は Tables [55]、FITS は FITS File Handling [56] に記載されている。本章は、そのうち観測データ解析で用いる機能を扱った。

11.5　章末課題：単位と時刻の意味を保って計算する

1. **時間窓に入る値を選ぶ。**`[120, 220, 250, 251, 800]` ns から、250 ns 以下の値を単位付きで選べ。ns と秒の両方で比較し、どちらも 3 個になることを表に示せ。境界の 250 ns を含めること。

2. **秒をまたぐ短い時刻差を求める。** `2026-04-12T11:59:59.999999900` と `2026-04-12T12:00:00.000000120` を `Time(format="isot", scale="utc")` で作れ。差を ns で求め、約 220 ns であることと、250 ns 以内かどうかを示せ。
3. **同じ瞬間を UTC と TAI で表す。** 2 問目の最初の時刻を UTC と TAI で表示せよ。表示文字列と、時刻オブジェクト同士の差を並べ、表示が違っても同じ瞬間を表すことを説明せよ。
4. **単位付きデータを保存する。** 時刻 `[0, 120, 250]` ns、信号 `[12.4, 9.8, 15.1]` mV、検出器 `["A", "B", "A"]` の `QTable` を ECSV へ保存せよ。同じ信号を FITS 画像へ保存し、ヘッダに `BUNIT="mV"` を付ける。読み戻した行数、値、単位を表で照合せよ。`overwrite=True` は既存の実習ファイルを置き換える指定である。
5. **天体の座標を変換する。** 赤経 150 度、赤緯 20 度を ICRS の `SkyCoord` とし、銀河座標へ変換せよ。銀河経度約 213.801 度、銀河緯度約 50.264 度と照合し、変換前後の座標系と値を表にせよ。地平座標への変換に追加で必要な観測条件も挙げよ。

第 12 章

自作モジュールの作成と再利用

平均の計算や図の作成を複数の解析で繰り返すときは、その処理を一つのファイルへまとめておくと、使うたびに書き直さずに済む。まず短い平均の関数を保存し、別のファイルから呼び出してみる。その後で、他のプロジェクトへも配れるパッケージの構成と、インストール方法を扱う。

12.1 平均の関数を別のファイルから使う

まず、2 つの測定値の平均を別のファイルから求める。新しい作業用フォルダーを作り、次の内容を `signal_stats.py` という名前で保存する。

signal_stats.py

```
def mean_signal(values):
    """数値列の算術平均を返す。空列ではValueErrorを出す。"""
    if len(values) == 0:
        raise ValueError("入力列が空です。")
    return sum(values) / len(values)
```

同じフォルダーに `analyze.py` を作り、次を保存する。

analyze.py

```
from signal_stats import mean_signal

values = [12.4, 15.1]
print(mean_signal(values))
```

端末で 2 つのファイルを保存したフォルダーへ移動し、実行する。

```
$ python analyze.py
13.75
```

`from signal_stats import mean_signal` は、`signal_stats.py` に書いた関数を読み込む命令

である。`mean_signal(values)` がその関数を呼び、2 つの測定値の平均を返す。このように読み込んで使う Python ファイルをモジュールという。

複数のモジュールをディレクトリへまとめ、読み込んで使う単位をパッケージという。同じフォルダーの別のスクリプトからも、この関数を読み込める。他のフォルダーやプロジェクトからも安定して使うには、後で説明するパッケージにまとめ、使用する環境へインストールする。次の節からは、その構成を作る理由と手順を扱う。

12.2　解析処理を再利用可能な部品へ分ける理由

処理を役割ごとのモジュールへ分けると、各公開関数に対して独立したテスト、型情報、docstring を用意できる。解析コードで複数の入力やプロジェクトから共有する対象には、次のものがある。

- データ読み込み関数
- フィルタ条件やカットの判定関数
- 図を作成して保存する関数
- フィット関数やモデル関数

同じ処理を複数のスクリプトへコピーすると、それぞれが独立した実装となり、修正内容が一致しなくなる場合がある。自作モジュールから `from histfit_tools import save_histogram_with_fit` のように import すれば、処理のコードを複製せずに一つの実装を共有できる。

12.3　引数・設定ファイルとコードを分ける

第 4.4.2 節では、実行時にファイル名などを指定する方法を学んだ。ここでは、繰り返し使う解析条件をどこへ置くかを整理する。

値の配置は、その値が実行ごとに変わるか、複数の実行で共有されるか、プログラムの定義として固定されるかによって決める。すべてをコマンドライン引数にすると、同じ条件を反復する際にも毎回指定が必要となる。

コマンドライン引数　実行ごとに変わる値を置く。入力ディレクトリ、出力先、日付範囲、並列数などが典型である。

設定ファイル　プロジェクト単位では固定だが、将来変更し得る値を置く。較正ファイルのパス、検出器 ID、閾値、図の保存形式などが当てはまる。

環境変数　計算機ごとに異なるデータの置き場所などを渡す。例えば `DATA_ROOT` に保存先を設定する。認証情報を扱う場合はコードや共有ファイルへ直接書かず、利用するサービスの案内に従って管理する。

コード内の定数　定義として固定したい値や単位変換を書く。`NS_PER_SECOND` や物理定数のように、意味が仕様として決まっているものを置く。

入力ファイルのパス、出力先、しきい値、作業ディレクトリなどの実行条件は、コードへ直接固定せず、外から指定できるようにする。コードを変更せずに条件を切り替えられれば、同じ処理の再利用と実行条件の再現が容易になる。

12.3.1　設定ファイルに解析条件をまとめる

実行のたびには変えないものの、解析条件として変更する可能性がある値は、設定ファイルへまとめられる。形式には JSON や TOML などがある。ここでは標準ライブラリだけで扱える JSON の例（`config.json`）を示す。

config.json

```
{
  "calibration_path": "calib/detector_a.csv",
  "coincidence_window_ns": 250,
  "figure_format": "pdf"
}
```

この JSON ファイルを Python のスクリプトから読み込む場合のコード例は次のようになる。

```
import json
from pathlib import Path

config = json.loads(Path("config.json").read_text(encoding="utf-8"))
window_ns = config["coincidence_window_ns"]
print(window_ns)
```

実行結果は次のようになる。

```
250
```

この `250` は、JSON の `coincidence_window_ns` から読み込んだ値であり、Python コードには直接記述されていない。そのため、コードを変更せずに設定ファイルだけを差し替えられる。Python 3.11 以降では、標準ライブラリの `tomllib` で TOML も読み込める。入力パス、出力先、閾値など、実行ごとに変わり得る条件はコードから設定ファイルまたは引数へ分離する。

`250` や `3600` のような数値が説明や名前なしにコードへ直接記述されていると、その意味や単位を判断しにくい。このような数値はマジックナンバーと呼ばれる。名前の付け方は第 4.5.1.3 節で扱った。値が仕様上の定数なら意味と単位を表す名前を付け、実行条件なら引数または設定ファイルへ分離する。

12.4　uv によるパッケージ開発の初期化

12.4.1　作業用プロジェクトの作成

本書では、他のコードから import するライブラリを作るため、uv init --lib を用いる。次のコマンドによって、ヒストグラム作図用ライブラリの初期構成と依存関係を作成する。

```
$ mkdir histfit_tools
$ cd histfit_tools
$ uv init --lib
$ uv add numpy matplotlib scipy
$ uv add --dev pytest ruff
$ uv sync
```

uv add で追加した numpy、matplotlib、scipy は、このパッケージの実行時依存関係である。pytest と ruff は開発時にだけ使うため、--dev 付きで追加する。pytest は公開関数の返り値と例外を検証し、ruff は uv run ruff check . によって未使用 import や規則違反を変更記録前に検出する。本章では、公開関数のテストをパッケージ構成の一部として扱う。

この例で作成するディレクトリ構成を次に示す。

histfit_tools/ の例

```
.gitignore
.python-version
README.md
pyproject.toml
src/
`-- histfit_tools/
    ¦-- __init__.py
    ¦-- py.typed
    ¦-- models.py
    `-- plotting.py
tests/
`-- test_models.py
```

ここで src/ の下にパッケージを置く構成を src レイアウトという。プロジェクトのルートをカレントディレクトリにしただけでは src/histfit_tools を直接 import できないため、インストールされていない作業ツリーを誤って読み込む問題を検出しやすい。py.typed は、配布したパッケージの型情報を型検査器が利用できることを示す。

12.4.2　pyproject.toml の役割

パッケージ開発では、pyproject.toml が中心になる。ここにはパッケージ名、Python バージョン条件、依存関係、ビルド方法などを書く。次は各項目の役割を示す簡略例であり、uv

`init --lib` と `uv add` が生成する下限バージョンは実行時の解決結果によって異なる。

pyproject.toml

```
[project]
name = "histfit_tools"
version = "0.1.0"
description = "Histogram plotting and fitting utilities."
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
  "matplotlib",
  "numpy",
  "scipy",
]

[build-system]
requires = ["uv_build"]
build-backend = "uv_build"

[dependency-groups]
dev = [
  "pytest",
  "ruff",
]
```

`dependencies` に列挙したライブラリは、このパッケージをインストールするときに実行時依存関係として解決される。実際に生成された `pyproject.toml` の `[build-system]` には、`uv` と互換性のある `uv_build` のバージョン範囲も記録されるため、その範囲を理由なく削除しない。

`uv.lock` は、リポジトリ内の開発・検証環境で解決した依存関係を記録する。`uv sync` は `pyproject.toml` と `uv.lock` に基づいて環境を同期する。一方、一般的なビルドフロントエンドがライブラリをインストールするときには、`pyproject.toml` のプロジェクト情報と依存関係を参照し、`uv.lock` を配布パッケージの依存関係としては用いない。

12.5　具体例: ヒストグラムとフィットをまとめたパッケージ

本節では、1 次元データを受け取り、`numpy.histogram` でヒストグラムを作成し、`matplotlib` の `errorbar` で描画したうえで、必要に応じて `scipy.optimize.curve_fit` のフィット曲線を重ねて保存する関数の実装を扱う。要求仕様は次のとおりである。

- 入力: データ列と保存先
- ビン指定: `bins` または `bin_width`
- フィットの切替え: `fit=False` ではヒストグラムのみを保存
- モデル選択: `linear`、`quadratic`、`power_law`、`gaussian`

- ユーザー定義関数の受付
- 引数 `p0` によるフィット初期値の指定

12.5.1　型ヒントと結果をまとめるクラス

以降の例には型ヒントを付ける。`x: float` は引数の想定型、`-> np.ndarray` は返り値の想定型を示すが、記述するだけで実行時の型検査が行われるわけではない。`float | None` は浮動小数点数か `None`、`tuple[float, float]` は二つの浮動小数点数の組を表す。これらの書式を使う例は Python 3.10 以降を前提とする。

クラスは、関連するデータや操作をまとめたオブジェクトの型を定義する。ここでは `@dataclass` を使い、保存先、度数、ビン境界、推定値を一つの結果として返す。`result.counts` のように属性名で取り出せる。`frozen=True` は属性の再代入を禁止するが、属性が参照する NumPy 配列の要素まで変更不能にするものではない。呼び出し可能な関数やオブジェクトを callable といい、`Callable[..., np.ndarray]` は引数構成を限定せず NumPy 配列を返す呼び出しを表す型ヒントである。

12.5.2　モデル関数の分離

まず、フィット関数群を `models.py` へ切り出す。

src/histfit_tools/models.py

```python
from collections.abc import Callable

import numpy as np

ModelFunc = Callable[..., np.ndarray]


def linear(x: np.ndarray, a: float, b: float) -> np.ndarray:
    """一次関数 y = a x + b を返す。"""
    return a * x + b


def quadratic(x: np.ndarray, a: float, b: float, c: float) -> np.ndarray:
    """二次関数 y = a x^2 + b x + c を返す。"""
    return a * x**2 + b * x + c


def power_law(x: np.ndarray, amplitude: float, alpha: float) -> np.ndarray:
    """べき関数 amplitude * x^alpha を返す。x > 0 を仮定する。"""
    return amplitude * np.power(x, alpha)


def gaussian(
```

```
    x: np.ndarray,
    amplitude: float,
    mean: float,
    sigma: float,
    offset: float,
) -> np.ndarray:
    """ガウス関数と定数オフセットを返す。"""
    z = (x - mean) / sigma
    return amplitude * np.exp(-0.5 * z**2) + offset


NAMED_MODELS: dict[str, ModelFunc] = {
    "linear": linear,
    "quadratic": quadratic,
    "power_law": power_law,
    "gaussian": gaussian,
}


def resolve_model(model: str | ModelFunc) -> ModelFunc:
    """名前または callable からフィット関数を返す。"""
    if callable(model):
        return model

    try:
        return NAMED_MODELS[model]
    except KeyError as exc:
        names = ", ".join(sorted(NAMED_MODELS))
        raise ValueError(f"Unknown model: {model}. Choose from {names}.") from exc
```

モデル関数を別ファイルへ置くと、作図処理と数学的なモデルが分離される。指数減衰やローレンツ関数を追加する場合も、`plotting.py` の変更箇所をモデルの選択と呼び出しに限定できる。

12.5.3　pytest による最小テスト

`tests/test_models.py` は、`models.py` の公開関数が最低限期待通りに振る舞うかを確認するためのファイルである。ここでは、式の値そのもの、ガウス関数の基本的な形、未知のモデル名に対する例外の 3 点だけを確かめておく。

tests/test_models.py

```
import numpy as np
import pytest

from histfit_tools.models import gaussian, linear, resolve_model


def test_linear_returns_expected_values() -> None:
```

```
    x = np.array([0.0, 1.0, 2.0])
    y = linear(x, 2.0, 1.0)
    assert np.allclose(y, [1.0, 3.0, 5.0])


def test_gaussian_has_peak_at_mean() -> None:
    x = np.array([4.0, 5.0, 6.0])
    y = gaussian(x, amplitude=10.0, mean=5.0, sigma=1.0, offset=2.0)
    assert y[1] == pytest.approx(12.0)
    assert y[1] > y[0]
    assert y[0] == pytest.approx(y[2])


def test_resolve_model_rejects_unknown_name() -> None:
    with pytest.raises(ValueError):
        resolve_model("lorentzian")
```

最初のテストは一次関数の出力を既知の値と比較する。2 つ目は、ガウス関数が平均値の位置で最大となり、平均値から等距離にある 2 点で同じ値を返すことを検証する。3 つ目は、登録されていないモデル名に対して `ValueError` が発生することを検証する。

テストは次のコマンドで実行できる。

```
$ uv run pytest
$ uv run ruff check .
```

`pytest` は関数の出力と例外条件を検証し、`ruff` は未使用 import や設定した規則への違反を検出する。実行結果と静的検査を分けることで、テストされていない動作と、実行前に検出できる記述上の問題を区別できる。

12.5.4　ヒストグラムの作成・保存関数

ヒストグラムの作成、フィット、保存を担う `save_histogram_with_fit` の実装を `plotting.py` に置く。

src/histfit_tools/plotting.py

```
from collections.abc import Sequence
from dataclasses import dataclass
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from numpy.typing import ArrayLike
from scipy.optimize import curve_fit

from .models import ModelFunc, resolve_model


@dataclass(frozen=True)
```

```
class HistogramFitResult:
    """保存先、ヒストグラム、フィット結果を保持する。"""

    output_path: Path
    counts: np.ndarray
    bin_edges: np.ndarray
    parameters: np.ndarray ¦ None
    covariance: np.ndarray ¦ None


def _make_bin_edges(
    values: np.ndarray,
    bins: int ¦ Sequence[float] ¦ None,
    bin_width: float ¦ None,
    data_range: tuple[float, float] ¦ None,
) -> np.ndarray:
    if bins is not None and bin_width is not None:
        raise ValueError("Specify either bins or bin_width, not both.")

    if data_range is None:
        low = float(np.min(values))
        high = float(np.max(values))
    else:
        low, high = map(float, data_range)
        if not np.isfinite([low, high]).all() or low >= high:
            raise ValueError("data_range must contain finite low < high.")

    if low == high:
        high = low + 1.0

    if bin_width is not None:
        if not np.isfinite(bin_width) or bin_width <= 0:
            raise ValueError("bin_width must be finite and positive.")
        n_bins = int(np.ceil((high - low) / bin_width))
        return low + np.arange(n_bins + 1) * bin_width

    if bins is None:
        bins = 40

    if isinstance(bins, int):
        if bins <= 0:
            raise ValueError("bins must be positive.")
        return np.linspace(low, high, bins + 1)

    bin_edges = np.asarray(bins, dtype=float)
    if bin_edges.ndim != 1 or bin_edges.size < 2:
        raise ValueError("bins must contain at least two edges.")
    if not np.isfinite(bin_edges).all() or np.any(np.diff(bin_edges) <= 0):
        raise ValueError("bin edges must be finite and strictly increasing.")
```

```
    return bin_edges


def save_histogram_with_fit(
    data: ArrayLike,
    output_path: str | Path,
    *,
    bins: int | Sequence[float] | None = None,
    bin_width: float | None = None,
    data_range: tuple[float, float] | None = None,
    fit_model: str | ModelFunc = "gaussian",
    fit: bool = True,
    p0: Sequence[float] | None = None,
    xlabel: str = "x",
    ylabel: str = "Counts",
    title: str | None = None,
) -> HistogramFitResult:
    """ヒストグラムと誤差棒を描き、必要ならフィット線も重ねて保存する。

    Args:
        data: 1 次元データ列。
        output_path: 保存先ファイル名。
        bins: ビン数またはビン境界列。幅も未指定なら 40 ビン。
        bin_width: ビン幅。bins と同時には指定しない。
        data_range: 範囲の下限と上限。幅指定時は上端を切り上げる。
        fit_model: モデル名または自作 callable。
        fit: False ならフィットを実行しない。
        p0: curve_fit へ渡す初期値。
        xlabel: x 軸ラベル。
        ylabel: y 軸ラベル。
        title: 図タイトル。

    Returns:
        保存先、ヒストグラム、フィット結果をまとめたオブジェクト。
    """
    values = np.asarray(data, dtype=float)
    if values.ndim != 1:
        raise ValueError("data must be one-dimensional.")
    if values.size == 0:
        raise ValueError("data must not be empty.")
    if not np.isfinite(values).all():
        raise ValueError("data must contain only finite values.")

    bin_edges = _make_bin_edges(values, bins, bin_width, data_range)
    if fit and not np.allclose(np.diff(bin_edges), np.diff(bin_edges)[0]):
        raise ValueError("Fitting counts requires equal-width bins.")
    counts, bin_edges = np.histogram(values, bins=bin_edges)
    centers = 0.5 * (bin_edges[:-1] + bin_edges[1:])
    errors = np.sqrt(counts)
```

```
    fig, ax = plt.subplots()
    try:
        ax.errorbar(centers, counts, yerr=errors, fmt="o", capsize=3, label="data")

        fit_params = None
        fit_cov = None
        if fit:
            model = resolve_model(fit_model)
            sigma = np.where(errors > 0, errors, 1.0)
            fit_params, fit_cov = curve_fit(
                model,
                centers,
                counts,
                p0=p0,
                sigma=sigma,
                absolute_sigma=True,
                maxfev=10000,
            )
            x_fit = np.linspace(bin_edges[0], bin_edges[-1], 400)
            y_fit = model(x_fit, *fit_params)
            ax.plot(x_fit, y_fit, label="fit")

        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)
        if title is not None:
            ax.set_title(title)
        ax.legend()

        output_path = Path(output_path)
        output_path.parent.mkdir(parents=True, exist_ok=True)
        fig.savefig(output_path, dpi=150, bbox_inches="tight")
    finally:
        plt.close(fig)
    return HistogramFitResult(
        output_path=output_path,
        counts=counts,
        bin_edges=bin_edges,
        parameters=fit_params,
        covariance=fit_cov,
    )
```

この関数では、ビン境界の決定と作図本体を分けている。こうしておくと、ビン幅指定のロジックだけを単独にテストしやすい。また、`fit_model` が文字列なら既定のモデル群から選び、callable ならそのまま使うので、利用者は既定モデルと自作モデルを同じ引数で扱える。

この関数のフィットは、等幅ビンの中心における度数を最小二乗法で近似する学習用の実

装である。不等幅ビンでは幅の違いが度数に入るため、フィットを拒否する。$\sqrt{N}$ の誤差棒と、空のビンに便宜上与える $\sigma = 1$ は、低計数に対する信頼区間を保証しない。精密な推定にはビン内で積分した期待計数や Poisson 尤度などを検討する。幅を指定すると上端が指定幅に合わせて切り上がるので、実際の範囲は返された `bin_edges` で確認する。

12.5.5　`__init__.py` による公開インタフェースの整理

他のプログラムから使ってよい関数名や呼び出し方を、公開 API という。パッケージの公開 API に含める関数とクラスは、`__init__.py` で再公開する。

src/histfit_tools/__init__.py

```python
"""Histogram plotting and fitting utilities."""

from .models import gaussian, linear, power_law, quadratic
from .plotting import HistogramFitResult, save_histogram_with_fit

__all__ = [
    "gaussian",
    "HistogramFitResult",
    "linear",
    "power_law",
    "quadratic",
    "save_histogram_with_fit",
]
```

こうしておけば、利用側は `from histfit_tools import save_histogram_with_fit` と短く書ける。内部ファイル構成を後で整理し直しても、公開 API を保っている限り、利用者側のコードを壊しにくい。

12.5.6　使い方の例

実際にこのパッケージを使うスクリプトは、たとえば次のようになる。

```python
from pathlib import Path

import numpy as np

from histfit_tools import save_histogram_with_fit

rng = np.random.default_rng(42)
data = rng.normal(loc=5.0, scale=1.2, size=4000)

fit_result = save_histogram_with_fit(
    data,
    Path("figures/gaussian_fit.png"),
    bins=35,
    fit_model="gaussian",
```

```
    p0=[250.0, 5.0, 1.0, 1.0],
    xlabel="signal [mV]",
    title="Gaussian fit example",
)

if fit_result.parameters is not None and fit_result.covariance is not None:
    parameter_errors = np.sqrt(np.diag(fit_result.covariance))
    print(fit_result.parameters)
    print(parameter_errors)

save_histogram_with_fit(
    data,
    Path("figures/hist_only.png"),
    bin_width=0.2,
    fit=False,
    xlabel="signal [mV]",
)
```

1 つ目はガウス関数でフィットし、2 つ目は `fit=False` としてヒストグラムだけを保存している。ビン数指定とビン幅指定の両方が動くことも、この 2 例で確認できる。

1 つ目の呼び出しで保存される図の例を図 12.1 に示す。ヒストグラムは入力した 4000 個の信号値を表し、曲線はビンごとの度数へ当てはめたガウス関数である。この入力は平均 5.0 mV、標準偏差 1.2 mV の正規分布から生成している。返り値の `parameters` と `covariance` を調べれば、推定した中心と幅、および各パラメータの標準誤差を生成条件と比較できる。図の保存だけでなく、既知の入力に対して推定結果が妥当かどうかも検証する。

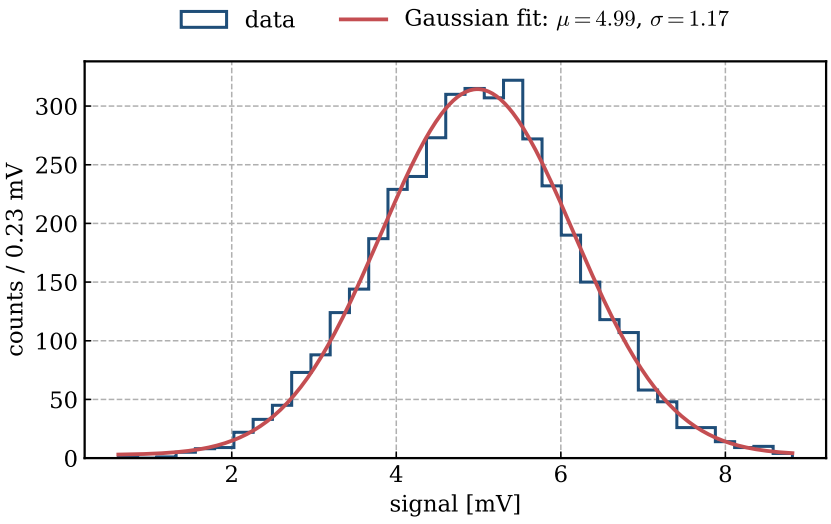


図 12.1　`save_histogram_with_fit()` の出力例。ヒストグラムは乱数シード 42 で生成した 4000 個の信号値、実線はガウス関数によるフィットを表す。

`fit_model` には、あらかじめ登録した名前だけでなく、同じ呼び出し規約を持つ自作関数も渡せる。指数分布から生成したデータのヒストグラムへ、指数減衰関数を当てはめる例を

次に示す。

```python
import numpy as np

from histfit_tools import save_histogram_with_fit


def exponential_decay(
    x: np.ndarray,
    amplitude: float,
    tau: float,
    offset: float,
) -> np.ndarray:
    return amplitude * np.exp(-x / tau) + offset


rng = np.random.default_rng(42)
decay_data = rng.exponential(scale=3.0, size=4000)

save_histogram_with_fit(
    decay_data,
    "figures/custom_fit.png",
    bin_width=0.25,
    data_range=(0.0, 15.0),
    fit_model=exponential_decay,
    p0=[300.0, 3.0, 0.0],
)
```

`fit_model` は、登録済みのモデル名に加えて、同じ呼び出し規約を持つ callable を受け取る。これにより、パッケージ本体を変更せずに利用者定義のモデルを渡せる。

12.6　docstring、型ヒント、README

自作モジュールには、入力、返り値、例外条件、導入方法を記述する必要がある。関数単位の情報は docstring と型ヒント、パッケージ全体の情報は README が担う。

12.6.1　docstring と型ヒント

docstring には、引数の意味と既定値、返り値、例外条件を記述する。`help()` と、docstring 表示に対応したエディタの補完機能は、この情報を参照する。

`save_histogram_with_fit` には、`data: ArrayLike`、`output_path: str | Path`、`fit: bool = True` のような型ヒントが付いている。VS Code などのエディタでは、関数名にカーソルを合わせたときや補完候補を開いたときに、これらの型ヒントと docstring の先頭が表示されることが多い。したがって、docstring は文書であると同時に、コーディング中に参照できる説明でもある。

docstring の第 1 行には、関数の役割を一文で記述する。たとえば「ヒストグラムと誤差棒を描き、必要ならフィット線も重ねて保存する。」と書き、その後に引数、返り値、例外条件を続ける。

12.6.2 README に書くべきこと

README.md には、パッケージ全体の目的と導入方法を記述する。この例では次の 4 点を含める。

- このパッケージが何をするか
- どうインストールするか
- 最小の使用例
- 主要な依存関係

たとえば最小限の README は次のようになる。

README.md

````
# histfit_tools

Histogram plotting and fitting utilities for analysis scripts.

## Installation

```bash
uv venv
uv pip install .
```

## Example

```python
from histfit_tools import save_histogram_with_fit

result = save_histogram_with_fit([1, 2, 2, 3], "example.pdf", bins=3, fit=False)
print(result.counts)
```

## Requirements

Python >= 3.11; NumPy, Matplotlib, SciPy.
Dependencies are installed from pyproject.toml.
````

README はパッケージ全体の目的、導入方法、最小使用例を示し、docstring は関数の引数、返り値、例外条件を示す。両者の対象範囲を分け、同じ情報を重複して保守しない。

12.7 uv pip による通常インストールと editable install

12.7.1 通常インストール

現在のプロジェクト用仮想環境へパッケージを通常インストールするには、パッケージのルートディレクトリで `uv pip install .` を実行する。`uv pip` は pip 互換のインターフェースであり、仮想環境内に `pip` コマンドがインストールされていない場合にも利用できる。

```
$ uv pip install .
```

これは現在のソースからビルドしたパッケージを環境へインストールする操作である。後から `src/histfit_tools/plotting.py` を編集しても、その変更はインストール済みのパッケージへ自動では反映されない。変更後に `uv pip install .` を再実行する。

12.7.2 editable install

開発中は、`uv pip install -e .` による editable install を利用できる。

```
$ uv pip install -e .
```

`-e` は editable の略であり、インストール先から手元のソースディレクトリを参照できる形でパッケージを登録する。したがって、Python ソースの変更を再インストールせずに import 側へ反映できる。関数定義や docstring を反復して変更する開発環境で用いる。

ただし、editable install が自動的に反映する範囲は、ビルドバックエンドと変更内容に依存する。`pyproject.toml` の依存関係を変更した場合や、後述する pybind11 のような C++ 拡張を編集した場合には、環境の同期、再インストール、再ビルドが必要になる。

12.7.3 インストール確認

インストール後は、その仮想環境の Python を直接起動して読み込み元を確認する。`uv run` は環境を再同期してプロジェクトを editable install へ戻す場合があるため、通常インストールの確認にはここでは使わない。読み込み済みモジュールはプロセス内に保持されるので、変更後の確認には新しい Python プロセスを使う。

```
$ .venv/bin/python -c "import histfit_tools; print(histfit_tools.__file__)"
```

この import テストは、パッケージ名と公開関数を解決できることを確認する。依存関係、計算、ファイル出力を含む動作は、既知の小規模入力で関数を実行し、返り値と保存ファイルを検証する統合テストで確認する。

12.8 Git リポジトリによるパッケージの共有

研究室内や共同研究で共有する場合は、Git リポジトリによってソース、テスト、設定の変更履歴を管理できる。`uv init --lib` は、外側に既存の Git リポジトリがなく、`--vcs none` も指定していなければ、初期化対象に `.git/` を作る。

GitHub で共有する場合は、GitHub の Web 画面でコミットを持たない空のリポジトリを作る。リポジトリ名と公開範囲を指定し、手元にある README と `.gitignore` は GitHub 側で重複して作成しない。

空のリモートリポジトリを作成した後、記録対象を確認して次のコマンドを実行する。

```
$ git add pyproject.toml uv.lock README.md src tests .python-version .gitignore
$ git commit -m "Create histfit_tools package"
$ git branch -M main
$ git remote add origin git@github.com:username/histfit_tools.git
$ git push -u origin main
```

`uv init --vcs none` のように Git 初期化を明示的に無効化して始めた場合には、最初に `git init` が必要になる。

`.venv/` や `__pycache__/` を追跡対象から除外する規則は、`.gitignore` に記述する。Git の状態確認、ステージング、コミット、リモート接続は付録 C で扱う。

GitHub などに置いたリポジトリを別環境から直接インストールするコマンドは、次のとおりである。

```
$ uv pip install "git+ssh://git@github.com/username/histfit_tools.git"
```

タグやコミットを固定したいなら、URL の末尾に `@v0.1.0` や `@9f3c1a2` を付ける。

```
$ uv pip install "git+ssh://git@github.com/username/histfit_tools.git@v0.1.0"
```

タグまたはコミットハッシュを URL に含めると、後日の再インストールでも同じリビジョンを指定できる。研究ノートには、インストールに用いた URL とリビジョンを記録する。

12.9 実行時依存関係と開発用依存関係

依存関係については、利用者に必要な実行時依存関係と、開発者だけが使う開発用依存関係を区別する。開発・検証環境の再現にはロックファイルを使う。

実行時依存関係 パッケージを import して使うだけで必要なもの。今回なら `numpy`、`matplotlib`、`scipy` がこれに当たる。

開発用依存関係 テスト、整形、静的解析にだけ必要なもの。今回なら `pytest` や `ruff` がこれに当たる。

ローカル環境の固定　開発者どうしで完全に同じ環境を再現したいときの情報。これは `uv.lock` が担う。

ライブラリの `dependencies` には、実装とテストによって確認した互換範囲を記述する。根拠なく `numpy==2.0.0` のような完全固定を行うと、利用者側の他の依存関係と両立しない場合がある。下限だけを検証した場合は `numpy>=2.0` のように記述し、非互換の上限が判明している場合はその上限も記述する。リポジトリ内の開発・検証環境は `uv.lock` で再現する。

開発中には、テストや説明文の作成に使う道具も必要になる。これらをすべて実行時依存関係へ含めると、利用者のインストール量と依存関係の衝突リスクが増す。公開 API と実行時依存関係は必要最小限に保ち、開発用の道具は開発用依存関係へ分ける。

12.10　章末課題：平均を求めるパッケージを作る

1. **パッケージを準備する。** `uv init --lib signal_tools` で作成し、`pyproject.toml` と `src/signal_tools/` を確認せよ。ファイル構成を図にし、設定ファイルとコードの役割を説明せよ。
2. **平均の関数とテストを作る。** `statistics.py` に `mean_signal(values)` を作り、`from signal_tools import mean_signal` で使えるようにせよ。`[1.0, 2.0, 3.0]` では 2.0、空の入力では `ValueError` となる 2 件のテストを書く。`uv add --dev pytest ruff`、`uv run pytest` を実行し、関数と `2 passed` の結果を示せ。
3. **使い方を説明する。** 型ヒントと docstring を付け、`help(mean_signal)` で表示せよ。引数、返り値、空の入力の扱いが分かる使用例を一つ作り、説明文と実行結果を示せ。
4. **発展：インストール方法を比べる。** 別々の仮想環境へ通常形式と `-e` 付きでインストールせよ。実習用に関数の返り値を変更し、再インストール前に反映されるかを表で比較せよ。確認後は正しい関数へ戻し、テストを再実行する。
5. **共有するファイルを選ぶ。** NumPy を実行時、pytest と ruff を開発用の依存関係へ追加せよ。`.venv/`、`__pycache__/`、`.pytest_cache/`、`dist/` を `.gitignore` へ書く。共有するファイルと生成物を表にし、`uv.lock` の扱いも理由とともに説明せよ。

第 13 章

Python 解析の高速化

この章では、同じ入力を処理する二つの方法を比べ、結果を保ったまま速くする方法を学ぶ。まず処理時間を測り、次に入力が増えたときの処理回数と、メモリの使用量を調べる。改善前後の結果と実測時間を示し、どこを変えた効果かを説明できるようにする。

13.1 処理時間の基準測定

高速化を始める前に、処理時間の内訳を測定する必要がある。データの読み込み、探索、図の作成のどこに時間を要しているかを区別しなければ、変更による効果を適切に評価できない。

`time.perf_counter` を使うと、処理の前後で取得した時刻の差から経過時間を求められる。解析全体の所要時間に加えて、データの読み込み、前処理、探索、書き出し、作図という段階ごとの時間も記録すると、変更すべき箇所を特定しやすい。

```
import time

def run_analysis():
    total = 0
    for value in range(10000):
        total += value * value
    return total

t0 = time.perf_counter()
result = run_analysis()
t1 = time.perf_counter()
print(result)
print(f"elapsed = {t1 - t0:.3f} s")
```

キャッシュ、OS のスケジューリング、同時に動作するプロセスによって測定値は変動する。入力と実行条件をそろえて複数回測定し、中央値や最小値など、あらかじめ定めた代表

値を比較する。

この例の `run_analysis()` は、0 から 9999 までの数を二乗し、合計する関数である。最初に `333283335000`、次に所要時間が秒で表示される。`t0` と `t1` の差は、その間の関数呼び出しにかかった時間を表す。手元の解析を測るときは、この関数を測定したい処理に置き換える。

13.2　計算量の見積もり

処理時間の測定に加えて、入力サイズに対する演算回数の増加率を見積もる。入力サイズを N としたとき、演算回数が N、$N \log N$、N^2 のいずれに比例するかによって、大規模入力で必要となる処理量が異なる。

大きな入力での増え方を数式で詳しく調べる前でも、入力サイズを 2 倍にしたときの演算回数を見積もれば、実測値の増加がアルゴリズムから予想される範囲かを検証できる。個々の実装技法を選ぶ前に、問題が必要とする比較・探索・集計の回数を調べる。

変数名を短くしたり、関数を減らしたりしても、通常は実行時間にほとんど影響しない。一方、比較回数の削減、探索範囲の限定、配列演算への置換は、計算量または定数倍の処理時間を変え得る。

13.2.1　オーダーとは何か

ここでいう「オーダー」は、入力サイズ N が大きくなったときの処理回数の増え方を表す。Big-O は、入力が十分に大きいときに、処理回数がその式のある定数倍を超えないという上限（漸近的な上界）を表す。厳密な比例関係や実測時間そのものを表す記法ではない。最悪の場合、平均的な場合など、どの条件の計算量かも区別する。通常は Big-O 記法で書き、定数倍や低次の項を落として、最終的に支配的になる項だけを見る。

$$3N + 20 \to O(N), \qquad 5N \log N + 100N \to O(N \log N), \qquad N^2 + N \to O(N^2)$$

解析コードに現れる計算量の例を次に示す。

- $O(\log N)$: 範囲を半分ずつ絞る処理。二分探索など
- $O(N)$: 1 回走査する処理。総和、平均、閾値判定、隣接差分など
- $O(N \log N)$: ソートを含む処理。高速なソート、ソート後の探索など
- $O(N^2)$: すべての組み合わせを調べる処理。全ペアの逐次比較など

たとえば $N = 10^6$ なら、$\log_2 N$ はおよそ 20、$N \log_2 N$ はおよそ 2×10^7、N^2 は 10^{12} である。この場合、N^2 と $N \log_2 N$ の間には約 5 万倍の差がある。小さなテストデータでは判別しにくい差も、入力が大きくなると処理時間を支配し得る。

ただし、オーダーが同じ実装どうしでも、1 要素あたりの演算数、メモリアクセス、関数呼び出しなどの違いによって処理時間は変わる。まず計算量を確認し、次に測定結果に基づ

いて定数倍の差を調べる。

$O(\log N)$ は問題サイズを反復ごとに一定比率で縮小する処理に現れ、$O(N \log N)$ は各要素に対する処理をその段数だけ行う場合に現れる。以下では、二分探索とマージソートを例として両者を説明する。

13.2.1.1　二分探索ではなぜ $O(\log N)$ になるか

$\log N$ は、反復ごとに候補数を一定の比率で減らす処理に現れる。昇順に並んだ列から値を探す二分探索では、中央の値との比較によって、次に調べる範囲を左半分または右半分へ限定する。

```
def binary_search(values, target):
    left = 0
    right = len(values) - 1

    while left <= right:
        mid = (left + right) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

要素数が N 個の場合、候補数は $N \to N/2 \to N/4 \to \cdots \to 1$ と減少する。反復回数 k は $N/2^k \leq 1$ を満たすため、$k \geq \log_2 N$ である。したがって、二分探索の計算量は $O(\log N)$ となる。

13.2.1.2　ソートではなぜ $O(N \log N)$ になるか

挿入ソートやバブルソートの最悪計算量は $O(N^2)$ である。一方、マージソートやヒープソートの最悪計算量は $O(N \log N)$ である。Python の `sorted()` は Timsort を用い、最悪計算量は $O(N \log N)$、すでに整列した入力では $O(N)$ となる。

計算量に $N \log N$ が現れる理由を考える。典型例であるマージソートでは、まず列を半分に分け、さらにその半分へ分ける、という分割を繰り返す。この分割の深さは、およそ $\log_2 N$ 段である。一方、各段では、分割した短い列どうしを併合する過程で、全体として各要素を 1 回ずつ見るので $O(N)$ かかる。

$$\text{各段の仕事量 } O(N) \quad \times \quad \text{段数 } O(\log N) \quad = \quad O(N \log N)$$

ソートを前処理として行うと、後続の探索や近傍判定に必要な比較回数を減らせる場合がある。$O(N \log N)$ のソート後に線形走査を行う実装は、全ペアを比較する $O(N^2)$ の実装より、入力サイズの増加に対する演算回数を抑えられる。

データ数が 10 倍になったとき、$O(N^2)$ のソートの演算回数はおよそ 100 倍になる。一方、$O(N \log N)$ では、$N = 10^6$ から 10^7 へ増える場合の比は約 11.7 である。この増加率の差が、入力規模に応じてアルゴリズムを選ぶ根拠となる。

13.2.2　オーダーが改善する例: 重複判定

長さ N の整数列について、同じ値が 2 回以上現れるかを判定する問題を考える。

最も素朴な方法は、すべての組 (i, j) を比べることである。

```
def has_duplicate_slow(values):
    for i in range(len(values)):
        for j in range(i + 1, len(values)):
            if values[i] == values[j]:
                return True
    return False
```

この方法では、最悪の場合ほぼすべてのペアを比べるので $O(N^2)$ になる。データ量が 10 倍になると、比較回数はおおよそ 100 倍に増える。

列をソートすると、同じ値は隣接して並ぶ。したがって、隣接要素の比較だけで重複を判定できる。

```
def has_duplicate_fast(values):
    values_sorted = sorted(values)
    for i in range(len(values_sorted) - 1):
        if values_sorted[i] == values_sorted[i + 1]:
            return True
    return False
```

`sorted()` は最悪 $O(N \log N)$、その後の走査は $O(N)$ なので、全体の最悪計算量は $O(N \log N)$ である。全ペア比較を隣接要素の比較へ置き換えたことにより、計算量が $O(N^2)$ から $O(N \log N)$ へ変わる。

Python の `set(values)` を使えば、重複判定は平均的に $O(N)$ で実行できる。ここでは、ソート後の走査によって処理量を減らす考え方を説明するため、ソートを用いた例を示している。

未整列の列に対する線形探索は $O(N)$ であり、整列済みの列に対する二分探索は $O(\log N)$ である。探索が 1 回だけならソート時間が支配的になり得るが、同じ列を反復して探索する場合は、ソートと全探索を含む総演算回数を比較して前処理の採否を決められる。

13.2.3　オーダーは同じで定数倍だけ改善する例

今度は、$n \times n$ 行列 A が対称行列かどうか、すなわち $A_{ij} = A_{ji}$ が成り立つかを調べる問題を考える。

ここでは入力が有限な実数だけを含む正方行列であるとする。NaN は自分自身とも等しくないため、対角成分を省略する前に除外する必要がある。最も素朴な方法は、すべての (i, j)

について比較することである。

```
def is_symmetric_slow(A):
    n = len(A)
    for i in range(n):
        for j in range(n):
            if A[i][j] != A[j][i]:
                return False
    return True
```

これは n^2 回程度の比較を行うので $O(n^2)$ である。

対称性の判定では、$A_{ij} = A_{ji}$ と $A_{ji} = A_{ij}$ が同じ比較に当たる。そこで、対角成分を除く上三角だけを調べれば、判定結果を変えずに重複を除ける。

```
def is_symmetric_fast(A):
    n = len(A)
    for i in range(n):
        for j in range(i + 1, n):
            if A[i][j] != A[j][i]:
                return False
    return True
```

こちらも計算量のオーダーは $O(n^2)$ だが、比較回数は $n(n-1)/2$ 回であり、素朴な実装のおよそ半分である。入力サイズに対する増加率は変わらないものの、重複する比較を除いた分だけ実行時間の定数因子が小さくなる。

この変更では計算量のオーダーは変わらず、重複の除去によって定数因子だけが改善する。図 13.1 の隣接差分フィルタも同じ分類に属する。比較回数の次数を下げたのではなく、Python が要素ごとに処理する回数を減らしたことが時間差の要因である。

13.3　ベクトル化・型・逐次処理による改善

13.3.1　NumPy と pandas による高速化

アルゴリズムを見直した後は、NumPy や pandas へ処理を移すことで、Python レベルのループを減らせる。特に差分計算、ソート、列抽出、フィルタは、これらのライブラリが得意とする。図 13.1 からも、ベクトル化が速度に与える効果を視覚的に確認できる。

NumPy や pandas への置換だけでは、アルゴリズムの計算量は変わらない。全ペア比較などの不要な演算を削減した後、残った要素ごとの処理を配列演算へ置き換える。

13.3.2　メモリとコピーの管理

大きな中間配列を反復して生成すると、メモリ使用量とデータ転送時間が増える。処理によっては、演算時間よりもメモリ確保と転送が総所要時間を支配する。

反復回数に加えて、配列のコピー回数とコピーした総バイト数も測定対象となる。必要な列と範囲だけを読み込み、中間配列の生成を減らせば、メモリ使用量と転送時間を抑えられる。

NumPy では、スライスが元配列とメモリを共有するビューになる一方、条件抽出は新しい配列を作る。次のコードで両者を確認できる。

```
import numpy as np

values = np.arange(1_000_000, dtype=np.float64)
view = values[100:200]
copied = values[values % 2 == 0]

print(values.nbytes)
print(view.nbytes, np.shares_memory(values, view))
print(copied.nbytes, np.shares_memory(values, copied))
```

実行結果は次の通りである。

```
8000000
800 True
4000000 False
```

`nbytes` は配列要素が占めるバイト数を表す。`np.shares_memory(a, b)` は 2 配列が同じメモリ領域を共有するかを調べる。この例では、`view` 自体が参照する要素は 800 バイト分だが、元配列のメモリを共有するため、`view` の要素を書き換えると対応する `values` の要素も変わる。`copied` は 50 万要素の独立した配列であり、新たに 4 MB を使う。なお、`nbytes` は Python オブジェクト自体の管理領域を含まないため、プロセス全体のメモリ使用量と完全には一致しない。

13.3.3　ファイル分割、並列処理、逐次集計

入力全体が利用可能なメモリを超える場合は、ファイルまたはチャンク単位で読み込み、次の処理に必要な情報だけを保持する。互いに独立した単位へ分割できる処理は、並列実行の候補となる。

逐次処理では、現在の計算に必要な状態だけを保持し、件数、合計、最小値などを入力ごとに更新する。保持する状態の大きさが入力件数に依存しなければ、集計用メモリを一定に保てる。

例えば、1 行に 1 個の測定値が書かれた `values.txt` の平均は、全行をリストへ保存せずに求められる。

values.txt

```
1.5
2.0
4.0
2.5
```

```
def streaming_mean(path):
    count = 0
    total = 0.0

    with open(path, "r", encoding="utf-8") as f:
        for line_number, line in enumerate(f, start=1):
            text = line.strip()
            if text == "":
                continue
            try:
                value = float(text)
            except ValueError as exc:
                raise ValueError(f"line {line_number}: {text!r}") from exc
            count += 1
            total += value

    if count == 0:
        raise ValueError("input contains no numeric values")
    return total / count


print(streaming_mean("values.txt"))
```

出力は `2.5` となる。引数 `path` は入力ファイルの場所であり、`enumerate(..., start=1)` はエラー箇所をファイルと同じ 1 始まりの行番号で報告するために使う。この関数が保持するのは現在行、件数、合計だけなので、入力行数が増えても集計用メモリはほぼ一定である。ただし、中央値や全体のソートのように全データの順序が必要な統計量は、この単純な方法だけでは正確に求められない。

13.4　処理時間の内訳と発展的な改善

13.4.1　プロファイラによるボトルネックの特定

全体の所要時間を左右する、特に時間のかかる箇所をボトルネックという。その場所を探すために、関数ごとの呼び出し回数と所要時間を測定する。この測定に使う道具がプロファイラである。例えば `cProfile` は、関数自身の処理時間と、呼び出した関数を含む累積時間を出力する。

```
import cProfile

cProfile.run("run_analysis()", sort="cumtime")
```

先に定義した `run_analysis()` を使うと、その関数が 1 回呼ばれたことを確認できる。表示

される列のうち、ncalls は呼び出し回数、tottime は関数自身の処理時間、cumtime は呼び出し先も含めた時間である。次は列の読み方を示す抜粋例であり、秒数は実行環境によって変わる。

```
ncalls  tottime  cumtime  function
     1    0.001    0.001  run_analysis
```

この関数は中で別の Python 関数を呼ばないため、関数自身の時間と累積時間はほぼ同じになる。別の関数を呼ぶ解析では、累積時間にその呼び出し先の時間も入る。親と子の累積時間を足すと同じ処理を二重に数えるため、どの列を比較しているかを確かめる。ファイルの読み書きでは待ち時間も入るので、解析全体の経過時間も併せて測定する。

13.4.2　発展：C や C++ への書き換え

Python で作成した解析コードでも、計算量の多い部分だけを C や C++ で実装できる。移植する範囲は、アルゴリズムを整理し、プロファイルによって処理時間を支配する箇所を特定してから決める。

処理時間の主因がファイル入出力、不要なデータ変換、またはアルゴリズムにある場合、実装言語だけを変えても大きな改善は期待できない。低レベル言語への移行は、Python 側の測定によって計算コアを限定し、入出力を含む総所要時間に対する効果を見積もった後に検討する。

```
for (std::size_t i = 0; i < n; ++i) {
    for (std::size_t j = i + 1; j < n; ++j) {
        if (std::abs(times[j] - times[i]) <= window) {
            ++count;
        }
    }
}
```

C++ への書き換えによって 1 回の比較に要する時間が減っても、上の二重ループの計算量は $O(N^2)$ のままである。アルゴリズム変更による演算回数の削減と、実装言語変更による 1 演算あたりの時間短縮を分けて測定する。

13.4.3　発展：Python から C/C++ を呼ぶ

Python から低水準の実装を呼び出す方法には、ctypes、Python C API、pybind11、Cython がある。それぞれ、既存共有ライブラリの利用、C API による拡張、C++ インタフェースの公開、型付きコードのコンパイルという異なる用途を持つ。

```
cdef int i
cdef double total = 0.0

for i in range(n):
    total += values[i]
```

この断片は Python の `.py` ファイルへそのまま書いて実行するコードではなく、Cython の `.pyx` ファイル内で、配列と長さを定義した関数に組み込んでコンパイルする例である。配列の要素アクセスも低水準化するには、配列側の型指定も必要になる。ここで `cdef` は Cython 特有の宣言であり、変数を Python オブジェクトではなく C の整数や浮動小数点数として扱わせるために使う。これにより、ループ内部の型判定やボックス化のコストを減らせる。

実装方法は、既存コードの言語、必要なインタフェース、測定された処理時間、保守とビルドの負担に基づいて選ぶ。

- `ctypes`: 既存の C 共有ライブラリを、追加の拡張モジュールをコンパイルせずに呼び出す。引数型と返り値型は Python 側で宣言する
- Python C 拡張: Python の C API を直接使用する。参照カウントと例外処理を含め、C 側で Python オブジェクトを管理する
- `pybind11`: C++ の関数やクラスに対する Python バインディングを C++ で記述する
- Cython: Python に近い構文へ静的な型情報を追加し、C または C++ を経由して拡張モジュールを生成する

既存の C 共有ライブラリを呼ぶ場合は `ctypes`、C++ の関数やクラスを Python API として公開する場合は pybind11、Python に近いコードへ型を付けてコンパイルする場合は Cython が候補となる。言語境界のインタフェースを限定し、測定で特定した計算コアだけを分離する。pybind11 の最小構成は付録 D で説明する。

付録 D の `count_pairs_within_window()` は、このように C++ の関数を Python から呼ぶ実装例である。

Python 側の引数、返り値、例外条件をテストで確認すれば、それらを保ったまま内部実装を置き換えられる。Cython や C++ へ移植しても計算量は自動では改善しないため、プロファイルによって処理時間を支配するループを特定し、アルゴリズムと実装言語の効果を分けて測定する。

13.4.4　外部実装におけるアルゴリズムとデータ受渡しの確認

外部の C や C++ 実装を参照する場合は、対象とする問題、採用したアルゴリズム、低レベル言語へ移した範囲、Python 側とのデータ受け渡しを確認する。

高速化の章では、このような例を参照しながら、次の二つを常に区別して考える。

- アルゴリズムを変えたから速くなったのか
- 実装言語を変えたから速くなったのか

両者を同時に変更すると効果を分離できないため、同じ入力と出力条件の下で個別に比較する。

高速化のレポートでは、「NumPy にした」「C++ にした」という実装手段だけでなく、計算量、比較回数、メモリ配置、Python レベルのループ回数がどのように変化したかを説明

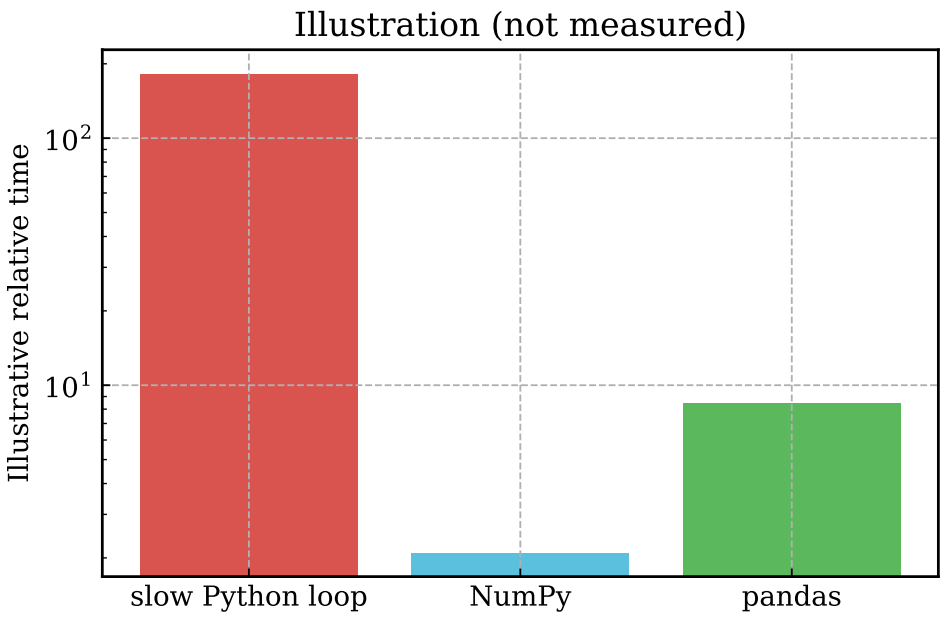


図 13.1　実行時間の比較方法を示す模式例。値は説明用であり、特定の計算機で得たベンチマーク結果ではない。縦軸には対数目盛を用いている。

する。

13.5　高速化前後の比較と測定条件の報告

高速化の結果は、変更前後の所要時間、比、測定条件を同じ表または出力欄へ記録する。次は報告形式を示すための仮想的な数値例である。

```
slow version : 182.4 s
vectorized   :  12.7 s
streaming    :   4.8 s
```

報告には、この形式で測定した値を記載し、続けて「比較回数を減らした」「Python ループを配列演算へ移した」「全件保持をやめた」など、変化を生じさせた実装上の差を示す。測定条件、測定値、変更内容を一組として記録する。

高速化の報告では、$O(N^2)$ から $O(N \log N)$ または $O(N)$ へ計算量を変えた改善と、同じオーダーのまま NumPy、Cython、C++ などで定数倍を変えた改善を区別する。前者はアルゴリズム、後者は実装効率の変更である。

```
import numpy as np

data = [0.0, 0.1, 0.5, 0.6, 1.0]
values = np.array(data, dtype=np.float64)
diffs = np.diff(values)
count = np.count_nonzero(diffs > 0.25)
```

図 13.1 は実行時間の大小を比較するための図であり、結果の正しさまでは保証しない。高

速化の前後で同じ `count` が得られることをテストし、その上で入力サイズ、反復回数、実行環境、代表値をそろえて時間を比較する必要がある。

13.6　章末課題：正しさと処理時間を比較する

測定には同じ入力と計算機を使う。各方法を 5 回ずつ測り、中央値または最小値のどちらを使ったかを記す。

1. **ループと配列演算を比べる。** `np.arange(1_000_000, dtype=np.float64)` の二乗和を、`for` と NumPy で求めよ。`np.isclose()` で結果の一致を確かめ、実行時間と速度比を表にせよ。
2. **入力が増えたときの時間を比べる。** 本文の重複判定 2 関数を、`np.random.default_rng(42).permutation(N).tolist()` に適用せよ。$N = 1000, 2000, 4000$ の時間と、N を倍にしたときの時間比を図または表にせよ。$O(N^2)$、$O(N \log N)$ の予想と比較する。入力は整列させずに使う。
3. **ビューとコピーを比べる。** 本文の `values`、`view`、`copied` を作り、`view[0] = -1` と `copied[0] = -2` を実行せよ。元配列の変化と `np.shares_memory()` の結果を表にせよ。同じ要素数の `float32` と `float64` の `nbytes` も比較する。
4. **1 行ずつ読んで平均を求める。** 1〜100000 を 1 行ずつ書いたファイルを作り、本文の `streaming_mean()` と全件読み込みで平均を求めよ。両方が 50000.5 になることを確かめ、保持するデータ量が行数に対してどう増えるか説明せよ。
5. **測定結果から改善対象を選ぶ。** 1 問目の 2 実装を関数にし、`python -m cProfile -s cumtime script.py` で測定せよ。累積時間の上位 3 関数を表にし、改善すべき箇所を一つ選ぶ。選択理由を処理時間と呼び出し回数から説明せよ。

第 14 章

総合課題：同時観測イベントの探索

14.1　入力データと実行環境の準備

この課題では、異なる検出器の記録から、時刻差が 250 ns 以内の組を探す。配布コードの誤りを修正して処理を速くし、件数、処理時間、2 種類の時間差の分布を発表する。

`events_data.zip` と `slow_event_analysis.py` を、配布リポジトリ（https://github.com/i-komae/python_textbook）から同じ作業フォルダーへ保存する。`unzip events_data.zip` で展開すると、年・月ごとのフォルダーに 500 日分の gzip ファイルが現れる。

配布コードには修正対象の誤りが含まれる。元のファイルを残し、`cp slow_event_analysis.py event_analysis.py` で作業用のコピーを作る。

14.1.1　配布コードによる基準測定

まず本節の「小規模入力による検証」に示す 3 日分の入力を用意し、配布コードを変更せずに実行せよ。必要なライブラリは `matplotlib` と `tqdm` であり、使用する仮想環境に導入する。

```
$ python slow_event_analysis.py events_data_small --outdir slow_output
```

全期間の処理時間は、データ量、ストレージ、計算機によって変わる。初回の対象は数日分とし、入力形式と出力の確認後に対象期間を広げるのがよい。

配布コードの処理時間と出力を変更前に記録し、その後は小さな修正と検証を繰り返す。各修正の効果は、変更前の基準値との差によって説明できる。

14.1.2　入力ファイルの構造

解析を始める前に、gzip ファイルを一つ開き、各列の意味を確認せよ。

```
$ gzip -cd events_data/2024/01/events_2024_01_01.txt.gz | head
```

各行の列は次のようになっている。

```
event_id year month day hhmmss ns detector_id signal
```

各列の型と役割を言語化すると、整数として扱う列、時刻を構成する列、識別子としてのみ使う列、物理量として解釈する列を区別できる。この区別が曖昧なままでは、型変換や時刻比較の誤りを招く。

この課題では、異なる検出器で記録された 2 事象の時刻差を ns 単位まで含めて評価し、$|\Delta t| \le 250\,\mathrm{ns}$ を満たすものを「同時観測」と定義する。したがって、`hhmmss` 列だけを見ればよいわけではなく、`ns` 列も含めて時刻を一つの量として組み立てる必要がある。秒境界や日付境界をまたぐ場合にどう振る舞うかも、この定義を満たす形で確認しなければならない。

イベント対は順序を区別せず 1 組につき 1 回だけ数える。一つのイベントが複数の相手と条件を満たせば、それぞれ別の対として数える。`hhmmss` は 6 桁の文字列、`ns` は秒未満の部分を表す $0 \le \mathrm{ns} < 10^9$ の整数として扱う。日付を含む共通の基準からの整数ナノ秒で比較し、現在時刻付近の Unix 秒を浮動小数点数へ変換してから ns の差を取らない。配布データは学習用であり、`signal` を特定の物理単位の測定値とみなす必要はない。

14.1.3　小規模入力による検証

500 日分の全データを処理する前に、数日分を別ディレクトリへコピーし、短時間で再実行できる検証用入力を用意せよ。

```
$ mkdir -p events_data_small/2024/01
$ cp events_data/2024/01/events_2024_01_01.txt.gz events_data_small/2024/01/
$ cp events_data/2024/01/events_2024_01_02.txt.gz events_data_small/2024/01/
$ cp events_data/2024/01/events_2024_01_03.txt.gz events_data_small/2024/01/
```

小規模入力は、修正後の結果を短時間で検証するために用いる。上記の 3 日分について正しい実装から得られる同時観測イベント対は 66 組である。小規模入力でこの件数と個々のイベント対を確認した後に、500 日分の処理を実行する。

```
n_pairs_total = 66
analysis_sec = <measured value>
```

この段階では、期待件数との一致、個々のイベント対の正しさ、再実行時の一致をそれぞれ確認する。件数が毎回変わる、出力ファイルの形式が揺れる、といった状態では、その先の高速化に進むべきではない。

14.2　検証・修正・高速化の要件

次の順に進め、結果を保存する。

1. 3 日分で配布コードを実行し、件数と処理時間を記録する。
2. 時刻差の定義と照らして誤りを特定し、作業用コードを修正する。
3. 小さな入力で各組の正しさを確かめ、3 日分・全期間の件数を基準値と比べる。

4. 同じ入力で処理を高速化し、結果の一致と時間の変化を確認する。
5. 以下に定義する 2 枚の分布図を作り、モデルを当てはめて解釈する。

高速化の前後で、件数だけでなくイベント対の内容も照合する。

14.2.1　同時観測を判定する時刻差

まず、2 つの検出器が記録した時刻を比べる。検出器 A が 1000 ns、検出器 B が 1180 ns に記録していれば、時刻差の絶対値は $1180 - 1000 = 180\,\mathrm{ns}$ である。異なる検出器の記録であり、差が 250 ns 以内なので、この 2 件を同時観測の 1 組として数える。250 ns ちょうどの差も含める。

見つかった組の一覧を `coincidence_pairs.csv` へ保存する。両方のイベント ID、検出器 ID、元のファイル名、時刻差を記録する。別ファイルのイベントが組になる場合もあるので、両方のファイル名を残す。

図に描く時刻差については、引き算の順序も決める。例えば、検出器番号の小さい側を第 1 事象、大きい側を第 2 事象とし、$\Delta t = t_2 - t_1$ と定義する。この定義では正負の値が現れ、同時観測の判定には $|\Delta t| \le 250\,\mathrm{ns}$ を使う。この分布を `pair_dt_hist.pdf` へ保存せよ。

14.2.2　同時観測の組どうしの間隔

次に、見つかった各組へ、その組を代表する時刻を付ける。ここでは 2 件の時刻の平均を使う。1000 ns と 1180 ns の組の代表時刻は $(1000 + 1180)/2 = 1090\,\mathrm{ns}$ になる。

別の組の代表時刻が 2000 ns なら、2 組の間隔は $2000 - 1090 = 910\,\mathrm{ns}$ になる。180 ns は一つの組を作る 2 件の間の差であり、910 ns は二つの組の代表時刻の間の差である。別々の図に描き、軸ラベルで何の差かを示す。

全組を代表時刻の小さい順に並べ、隣り合う組の間隔を求める。この分布を `pair_mean_gap_hist.pdf` へ保存せよ。日ごとに差を求めると日付をまたぐ組どうしの間隔が欠けるので、全期間の代表時刻をまとめて並べる。

整数ナノ秒の平均には 0.5 ns が現れ得る。例えば 1000 ns と 1181 ns の平均は 1090.5 ns である。丸め規則を定めるか、平均の 2 倍に当たる和を整数で保持し、和どうしの差を取ってから 2 で割る。一つのイベントを共有する複数の組は独立とは限らないので、組どうしの間隔が指数分布に従うとは仮定せず、分布を見て検討する。

14.2.3　図とフィットの確認

2 枚の分布図には、横軸が表す差の定義、単位、ビン幅、表示範囲を明記する。フィットを行うときは、使ったモデルと範囲も記録する。

同時観測の選択では $|\Delta t| \le 250\,\mathrm{ns}$ の値だけを残している。確率分布を当てはめる際は、モデル側もこの窓内の確率の合計が 1 になるように規格化する。これが、選択した範囲の条件

付き分布を使うという意味である。ヒストグラムと比べる場合は、各ビン内でモデルを積分した確率に全組数を掛け、期待される件数を求める。

件数と実行時間の要約を summary.txt へ保存する。必須出力をまとめると、次の 4 ファイルになる。

```
coincidence_pairs.csv
pair_dt_hist.pdf
pair_mean_gap_hist.pdf
summary.txt
```

14.2.4　正しさを検証する基準値

今回の配布データは 500 日分であり、前述の定義に従って全期間から得られる同時観測イベント対は

```
n_pairs_total = 12033
```

となる。この値を全期間処理の基準値とする。

ただし、この値に合ったからといって、それだけで実装が正しいとは限らない。偶然件数だけ一致している可能性もある。実装の検証には、可能な範囲で個々のイベント対の内容と、ファイル境界付近の振る舞いも含める必要がある。

```
n_pairs_total = 12033
analysis_sec = <measured value>
plot_sec = <measured value>
```

このような要約ファイルを保存しておくと、コード変更の前後で結果を比較しやすい。件数と処理時間を同じファイルへ記録すれば、正しさと速度を同時に追跡できる。処理時間は計算機と実行時の状態に依存するため、例示した欄には各自の実測値を記入する。

14.3　結果のまとめと発表

解析コードと、LaTeX で作成したレポート（ソースと PDF）を提出する。レポートと発表では、次の 4 点を順に示せ。

1. **問題と修正：** 元のコードの誤り、原因、修正内容を説明する。
2. **正しさ：** 小規模入力の検算と、3 日分 66 組・全 500 日分 12033 組との照合結果を示す。境界の検証も含める。
3. **処理時間：** 修正前、正しく修正した後、高速化後の処理時間を、同じ入力・測定条件で測り、表で比較する。変更点と速度比を記す。
4. **分布の解釈：** 2 枚の図、モデル、範囲、推定値と不確かさを示し、図から分かることと判断できないことを説明する。

14.3.1　発展課題

読み込み、探索、作図、フィットを含む全体の時間を測り、時間のかかる箇所を一つ改善せよ。方法は測定結果に基づいて選び、変更前後の時間、メモリ使用量、結果の一致を表で示せ。

付録 A

実行環境の補足

第 3 章で用意した環境を使って、本文の基本例を実行できる。本付録では、既に入っている Python の利用、必要なライブラリの版の記録、複数の Python の切り替えを扱う。自分の作業に必要な節を選んで参照する。Notebook を使う場合の手順も、第 A.4 節にまとめた。

A.1 既存の Python から仮想環境を作る

使いたい Python が既に入っている場合は、その Python から標準ライブラリの venv でもプロジェクト専用の仮想環境を作れる。ここでは別の作業用フォルダーを作り、第 3 章の環境と区別する。仮想環境が有効な場合は、`deactivate` で解除してから進める。

```
$ mkdir -p analysis_venv
$ cd analysis_venv
$ python3 -m venv .venv
$ source .venv/bin/activate
$ command -v python
$ command -v pip
$ python --version
$ pip --version
```

仮想環境を有効にすると、プロンプトの先頭に `(.venv)` が付くことが多い。ターミナルの表示は次のように変わる。

```
(.venv) student@host analysis_venv $
```

`command -v python` 以降の結果は、たとえば次のようになる。

```
/Users/student/analysis_venv/.venv/bin/python
/Users/student/analysis_venv/.venv/bin/pip
Python 3.11.11
pip 25.0 from /Users/student/analysis_venv/.venv/lib/python3.11/site-packages/pip
```

`command -v` の結果から、`python` と `pip` が `.venv/bin` で見つかることが分かる。これは、`source .venv/bin/activate` が `$PATH` の先頭に `.venv/bin` を追加するためである。Python 本体の確認には、`python -c "import sys; print(sys.executable)"` も利用できる。表示したバージョンとパスは説明用の例である。

この方式で作った環境にも、第 3.3 節の `python -m pip install ...` でライブラリを追加できる。作業を終えたら `deactivate` で環境の利用を終える。

A.2　uv による環境の記録と再現

第 3.1 節では `uv` で Python と仮想環境を用意した。この節では、作業名や必要なライブラリをファイルへ記録し、同じ環境を用意しやすくする方法を扱う。必要なライブラリと版の条件を依存関係といい、実際に解決した版はロックファイルへ記録できる。

A.2.1　uv の導入を確認する

`uv --version` を実行して番号が表示されることを確認する。第 3.1 節で導入してあれば、そのまま次へ進める。未導入の場合は、第 3.1 節の保存・確認・実行の手順を使う。以降では、作る構成を明示するため `uv init --package` を用いる。uv 0.12 より前と以降で既定のひな形が異なる場合も、この指定でビルド可能な構成を作れる。

A.2.2　プロジェクトの初期化

基本的なプロジェクトは `uv init` で作成でき、追加のオプションは必要に応じて指定する。別の仮想環境が有効な場合は `deactivate` で解除する。新しいディレクトリを作成して初期化する手順は次のとおりである。

```
$ mkdir analysis_uv
$ cd analysis_uv
$ uv init --package --python 3.11
```

現在の `uv init` は、既定でコマンドラインアプリケーションとして配布できる構成を作る。`ls -a` と `find src -maxdepth 2 -type f` で確認できる主要な項目を次に示す。外側に既存の Git リポジトリがある場合など、Git 関連項目の有無は実行場所によって異なる。

analysis_uv/ の初期構成

```
.git/
.gitignore
.python-version
README.md
pyproject.toml
src/
`-- analysis_uv/
    `-- __init__.py
```

src/analysis_uv/__init__.py には、pyproject.toml の [project.scripts] から呼び出す関数が置かれる。uv init の直後には、通常 .venv と uv.lock はまだない。これらは、最初の uv add、uv sync、または uv run によって依存関係を解決し、環境を同期するときに作られる。

- uv init --bare: pyproject.toml のみを作成
- uv init --vcs none: Git 初期化を省略
- uv init --no-pin-python: .python-version の作成を省略
- uv init --no-package: ビルドシステムを持たない平坦なスクリプト構成
- uv init --lib: 他のプロジェクトから読み込むライブラリ向けの構成

既定のアプリケーション構成はコマンドとして実行するプログラムに、--lib は他のコードから読み込む関数やクラスの配布に向く。ビルドシステムを必要としない単純なスクリプトには --no-package を指定できる。

A.2.3　依存関係の追加と環境の同期

uv add は依存ライブラリを追加し、pyproject.toml とロック情報を更新する。次の例では、7 個の解析用ライブラリをまとめて追加する。

```
$ uv add numpy pandas polars matplotlib scipy astropy tqdm
```

uv add を実行すると、依存関係の解決結果が要約表示される。たとえば次のようになる。

```
Resolved 18 packages in 0.42s
Installed 7 packages in 0.31s
+ numpy==2.3.0
+ pandas==2.3.0
+ scipy==1.15.0
```

Resolved ... packages は、直接指定したライブラリだけでなく、その依存先も含めて必要な集合を決めたことを表す。Installed ... packages は実際に環境へ追加した数であり、その下の + 付き行は今回入った主なパッケージ名とバージョンである。

uv sync は、pyproject.toml とロックファイルに基づいて環境をそろえる。uv init 後にまだ環境を同期していなければ、この実行時に .venv と uv.lock が作られる。

```
$ uv sync
```

uv sync の後は、たとえば次のような行が見える。

```
Creating virtual environment at: .venv
Installed 18 packages in 0.54s
```

Creating virtual environment at: .venv と表示された場合、プロジェクト用の仮想環境が .venv/ に作られている。uv sync は、pyproject.toml と uv.lock に基づいて依存関係をその

環境へ同期する。リポジトリの取得と変更履歴の確認に必要な Git の操作は、付録 C で説明する。

A.2.4　コマンドの実行とバージョン固定

`uv run` は、そのプロジェクトの環境でコマンドを実行する。スクリプトのパスを指定する場合と、Python 本体へ引数を渡す場合では、コマンドの構成が異なる。両者の例を次に示す。

```
$ uv run analysis-uv
$ uv run python --version
```

たとえば `uv run python --version` の結果は次のようになる。

```
Python 3.11.11
```

この 1 行は、現在のプロジェクト設定に従って起動された Python のバージョンを示す。既定構成では、`pyproject.toml` の `[project.scripts]` に登録された `analysis-uv` を `uv run analysis-uv` で実行する。`-m` を使う場合や Python 本体へオプションを渡す場合には、`uv run python ...` と記述する。ここに示したバージョン番号は説明用の例である。

ここまでの説明では、プロジェクト環境でコマンドを実行する方法を扱った。次項では、プロジェクトが用いる Python のバージョンを固定する。

`uv` は、プロジェクトで使用する Python の版を `.python-version` に記録できる。`uv` はプロジェクト設定とコマンドの `--python` 指定に基づいて実行環境を選択する。プロジェクトの既定値と、1 回のコマンドだけに適用する指定を次に示す。

```
$ uv python pin 3.12
$ uv run python --version
$ uv run --python 3.11 python --version
```

結果の例は次の通りである。

```
Python 3.12.10
Python 3.11.11
```

1 行目は、`uv python pin 3.12` によってプロジェクトの既定が 3.12 系になっていることを示す。2 行目は、`--python 3.11` を指定した実行に限って 3.11 系を使用した結果である。`pin` はプロジェクトの既定値を設定し、`--python` は個別のコマンドに対して使用する Python を指定する。

`3.12` の指定は 3.12 系を選ぶものであり、パッチ版まで固定する指定ではない。必要なら `uv python pin 3.12.10` のように版を完全に指定する。また、`--python 3.11` は `pyproject.toml` の `requires-python` が 3.11 を許す場合にだけ使用できる。異なる版で実行すると、プロジェクトの仮想環境が作り直される場合がある。ロックファイルとともに、実際の Python と OS の版も記録する。

A.3 pyenv による複数バージョンの管理

Python 本体の導入と切り替えには pyenv を利用できる。pyenv は、複数の Python を利用者の領域へ導入し、シェルから python などを実行したときに選ばれるバージョンを切り替える。切り替えの範囲には、利用者の既定値、ディレクトリごとの設定、一つのシェルセッションだけに適用する設定がある。

A.3.1 macOS での準備

pyenv は Python 本体をビルドして入れるので、macOS では Xcode Command Line Tools が必要になる。すでに Homebrew が正常に動いているなら、Command Line Tools は入っていることが多い。インストール済みかどうかは、次のコマンドで確認できる。

```
$ xcode-select -p
```

すでにインストールされていれば、結果の例は次のようになる。

```
/Library/Developer/CommandLineTools
```

このように開発ツールのパスが表示されれば、選択された場所を確認できる。Xcode 本体を導入した環境では、そのアプリケーション内のパスが表示されることもある。パスの代わりにエラーが出る場合は、未導入または設定不備の可能性がある。未導入の場合のインストールコマンドは次のとおりである。

```
$ xcode-select --install
```

以下の macOS の手順は Homebrew を利用する。未導入なら、Homebrew の公式案内 [16] に従って導入し、brew --version が表示されることを確認する。pyenv による Python のビルドには追加のライブラリも必要になるため、pyenv のビルド環境の案内 [17] に従って用意する。代表的な macOS 用の指定は次のとおりである。

```
1 brew install openssl@3 readline sqlite3 xz tcl-tk@8 libb2 zstd zlib pkgconfig
```

必要なパッケージや追加設定は、導入する Python と OS の版によって異なる。

A.3.2 WSL / Ubuntu での準備

WSL の Ubuntu では、ビルドに必要なライブラリを先に入れておく。次のコマンドで、パッケージ一覧の更新と必要なライブラリの導入を続けて行える。

```
$ sudo apt update
$ sudo apt install -y build-essential curl git libssl-dev zlib1g-dev \
  libbz2-dev libreadline-dev libsqlite3-dev libffi-dev liblzma-dev tk-dev
```

`apt update` の出力は長いが、末尾でたとえば次のように `Done` が見えれば、パッケージ一覧の更新は完了している。

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

`apt install` でも多くの行が流れる。このコマンド例には `-y` を付けているため、通常は確認なしで続行する。`-y` を外した場合は、途中で次のような確認が出ることがある。

```
Need to get 48.2 MB of archives.
After this operation, 201 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

この表示が出たら、示されているパッケージや使用量に問題がないことを確認し、続けるなら `y` を入力して `Enter` を押す。`n` を入力すると処理は中止され、インストールは行われない。

インストールの終盤では、次のように `Setting up ...` が並ぶことがある。

```
Setting up libffi-dev:amd64 (3.4.4-1) ...
Setting up libsqlite3-dev:amd64 (3.40.1-2ubuntu0.2) ...
Setting up build-essential (12.9ubuntu3) ...
```

これは各パッケージの設定を進めている段階であり、この表示自体は正常である。最後まで流れたあとでエラーが出ずにシェルのプロンプトへ戻れば、インストールは完了している。

A.3.3　pyenv の導入とシェルへの組み込み

macOS では、次のコマンドで `pyenv` を導入できる。

```
$ brew install pyenv
```

導入後の確認には、後述する `pyenv --version` と `command -v pyenv` を用いる。Homebrew の途中経過や保存先はバージョンによって変わるため、特定の表示文字列を成功条件にはしない。

一方、WSL / Ubuntu では通常 Homebrew を前提にしないので、前節の依存パッケージを入れたあとにインストーラスクリプトを使う。

```
$ curl -fsSL -o pyenv-installer.sh https://pyenv.run
$ less pyenv-installer.sh
$ bash pyenv-installer.sh
```

1 行目は公式のインストール用スクリプトを保存し、2 行目は実行前に内容を表示する。配布元と内容を確認したうえで、3 行目によって実行する。取得した内容を直接 `bash` へ渡す方法では、実行前の確認ができない。

導入後は、現在のシェルで pyenv を使うための初期化コードを、起動時の設定ファイルへ追加する。Homebrew で導入した場合と、上のスクリプトで導入した場合とでは、最初のコマンドだけが異なる。

```
# Homebrew で導入した場合
$ pyenv init --install

# インストール用スクリプトで導入した場合
$ ~/.pyenv/bin/pyenv init --install

$ exec "$SHELL"
```

pyenv init --install は、実行中のシェルを判別し、推奨される初期化コードを対応する起動時設定ファイルへ追加する。実行前に変更内容が表示された場合は、対象のファイルと追記内容を確認する。bash で環境変数 BASH_ENV が .bashrc を指している特殊な構成では自動設定を避け、pyenv の公式文書にある手動設定を用いる。

exec "$SHELL" は、いま動いているシェルを、$SHELL が示すシェルプログラムで置き換えるコマンドである。SHELL には通常、ログインシェルとして設定されたプログラムのパスが入り、zsh なら /bin/zsh、bash なら /bin/bash となる。ただし、SHELL と現在実行中の対話シェルが常に一致するとは限らない。現在のシェルは ps -p $$ -o comm= でも確認できる。ここでは ~/.zshrc に追記した内容をすぐ反映させるために実行している。ターミナルをいったん閉じて開き直しても同じ目的は果たせる。

シェルを起動し直した後、次の 2 コマンドで pyenv のバージョンと、シェルが解決したコマンドを確認する。

```
$ pyenv --version
pyenv 2.x.x
$ command -v pyenv
/Users/student/.pyenv/bin/pyenv
```

1 行目でバージョン番号が出れば、シェルから pyenv を実行できる状態にある。2 行目のパスは導入方法によって異なり、Homebrew で導入した場合は Homebrew の bin ディレクトリが表示されることもある。command not found になる場合は、pyenv init --install が変更したファイル、現在のシェル、再起動後の PATH を順に調べる。表示したバージョン番号とパスは説明用の例であり、同じ文字列になる必要はない。

A.3.4　Python バージョンの導入と切り替え

pyenv を使う場合は、pyenv install でバージョンを追加し、pyenv global、pyenv local、pyenv shell で切り替える。次の例では、2 系統の Python を追加してから一覧を表示する。

```
$ pyenv install 3.11.11
$ pyenv install 3.12.10
$ pyenv versions
```

pyenv install は数分かかることがあり、途中でビルドログが多数流れる。完了後に pyenv versions を実行すると、たとえば次のように表示される。

```
* system (set by /Users/student/.pyenv/version)
  3.11.11
  3.12.10
```

`system` は、pyenv の shim より後ろにある `PATH` から選ばれる実行ファイルを表す。OS に付属する Python とは限らず、Homebrew などで導入した Python が先に見つかる場合もある。`3.11.11` と `3.12.10` が一覧にあれば、そのバージョンは pyenv の管理下に導入されている。先頭の `*` は現在選ばれている設定を示す。

その利用者がディレクトリ固有の設定を持たない場合に使う既定バージョンは、`pyenv global` で指定する。他の利用者や OS 全体の設定は変更しない。

```
$ pyenv global 3.12.10
$ python --version
$ command -v python
$ pyenv which python
```

結果の例は次のようになる。

```
Python 3.12.10
/Users/student/.pyenv/shims/python
/Users/student/.pyenv/versions/3.12.10/bin/python
```

`command -v python` が示す `shims/python` は Python 本体ではなく、pyenv が選んだ実行ファイルへ処理を渡す中継用プログラムである。`pyenv which python` は、中継後に実行される Python のパスを表示する。これに対して `pyenv local` は、あるディレクトリ以下で優先するバージョンを指定する。特定のプロジェクト用ディレクトリでバージョンを固定する手順を次に示す。

```
$ mkdir analysis311
$ cd analysis311
$ pyenv local 3.11.11
$ python --version
$ cat .python-version
```

結果の例は次のようになる。

```
Python 3.11.11
3.11.11
```

`pyenv local` を実行すると `.python-version` が作られ、指定した `3.11.11` が記録される。この設定は、そのディレクトリと下位のディレクトリに適用される。ただし、さらに下位に別の `.python-version` があれば、そちらが優先される。同じマシンでも、プロジェクトごとに異なる Python を選択できる。

`pyenv shell 3.11.11` は、現在のシェルセッションだけに適用される一時的な切り替えである。特定の Python バージョンによる動作確認に用いる。

A.4　Notebook でコードと結果をまとめる

コードと説明、結果や図を一つの文書に保存したいときは、Notebook（ノートブック）という形式も使える。これは任意の補助環境であり、本書の基本例と課題は Python スクリプトで実行できる。Notebook を使う場合は、その編集・実行用アプリケーションである JupyterLab を導入する。

`analysis_practice` へ移動し、仮想環境を有効にした端末で次を実行する。

```
$ python -m pip install jupyterlab
$ python -m jupyterlab
```

ブラウザが開いたら「Python 3」の Notebook を作る。コードを入力する枠をコードセルという。例えば `print(3 + 2)` を入力して Shift+Enter を押すと、セルの下に `5` が表示される。保存するときは Ctrl+S を押し、ファイル名を付ける。説明文は Markdown という種類のセルへ書ける。

ブラウザが開かない場合は、端末に表示された `http://localhost:.../lab?token=...` を同じコンピュータのブラウザで開く。`localhost` は同じコンピュータを示す。`token` を含む URL は接続用の認証情報なので、他人へ送ったり公開したりしない。

セルのコードを実行する Python をカーネルという。セルは表示順と違う順序でも実行できるため、結果を確認するときはカーネルを再起動し、全セルを上から順に実行する。終了時は、JupyterLab を起動した端末で Ctrl+C を押し、終了確認に従う。

配布リポジトリの `notebooks/` には章末課題の解説 Notebook もある。本文の説明を読み、自分で課題を試した後に参照できる。対応するファイルの一覧は `notebooks/README.md` にある。

A.4.1　Jupyter の `%timeit` による計測

Jupyter の `%timeit` と `%%timeit` は、短い式またはセルを複数回実行し、所要時間の統計量を表示する。短い処理の比較に用いる。次の例では、別のセルで `import numpy as np` と `array = np.arange(1000000)` を実行してから、計測セルを実行する。

```
1 %%timeit
2 result = array * 2
```

ただし、`%timeit` はノートブックや IPython で短い処理を測定する機能であり、スクリプト全体の読み込みや入出力を含めた時間の測定には適さない。短い式や関数には `%timeit`、解析全体には `perf_counter` を使う。

A.5　演習：環境の作成方法と切り替えを比較する

環境管理を詳しく学ぶ場合に取り組む。版や保存先は自分の環境の値を記録する。

1. **uv で環境を記録する。**`earthquake_practice/usgs_project` を `uv init --package` で作り、NumPy、Matplotlib、SciPy を `uv add` で追加せよ。生成ファイルの役割と、`uv run python` が使う版・場所を表にせよ。
2. **venv と比較する。**別の `usgs_venv` に `python -m venv .venv` で環境を作れ。有効化の前後と解除後の Python の版・場所、pip の場所を表にし、uv の方式との違いを説明せよ。
3. **発展：Python の版を切り替える。**2 系統の Python が使える場合は、別々のフォルダーに `pyenv local` を設定せよ。`command -v python`、`pyenv which python`、`sys.executable` を比較し、中継プログラムと Python 本体の違いを説明せよ。uv の環境での結果も併記する。

付録 B

SSH，鍵認証，ファイル転送

B.1　サーバーへの SSH 接続と鍵認証

SSH（Secure Shell）は、離れた計算機との通信を暗号化し、その計算機上でコマンドを実行するための仕組みである。研究室や計算サーバーに接続し、大規模なデータ処理や長時間ジョブを実行する場合に利用する。GitHub への SSH 接続も同じ認証方式を用いるが、GitHub は対話的なシェルの接続先ではなく、Git 操作の認証先である。

B.1.1　サーバーへの接続

初回の SSH 接続では、接続先のホスト鍵の指紋（fingerprint）を確認することがある。これは接続先サーバーを識別するためのもので、自分の認証用公開鍵とは異なる。管理者など信頼できる経路から得た指紋と照合して登録する。ホスト鍵が変わったという警告では、再構築などの理由を確認してから更新する。詳細は OpenSSH の資料 [57] を参照する。

最も基本的な形は次である。

```
$ ssh user@server.example.jp
```

`user` はサーバー上の自分のユーザー名、`server.example.jp` はホスト名である。ポート番号が既定の 22 でないなら `-p` を付ける。

```
$ ssh -p 2222 user@server.example.jp
```

接続後に入力したコマンドは、原則として接続先の計算機上で実行される。表示されるファイルも接続先にあるため、ローカルファイルとの取り違えは意図しない変更や削除につながる。作業を始める前に `hostname`、`whoami`、`pwd` を実行すれば、接続先、利用者名、作業ディレクトリを確認できる。

B.1.2　SSH 鍵の作成

公開鍵認証では、公開鍵と秘密鍵の対を使用する。公開鍵をサーバーへ登録し、秘密鍵は接続元の計算機から外部へ渡さない。

```
$ ssh-keygen -t ed25519 -C "your_email@example.com"
```

既定の保存先を選ぶと、`~/.ssh/id_ed25519` が秘密鍵、`~/.ssh/id_ed25519.pub` が公開鍵になる。サーバーへ登録するのは `.pub` が付いた公開鍵だけである。`.pub` が付かない秘密鍵は共有しない。

```
$ ls -l ~/.ssh
```

秘密鍵を他の利用者も読める設定にしていると、SSH 接続が拒否されることがある。その場合は次のように直す。

```
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/id_ed25519
$ chmod 644 ~/.ssh/id_ed25519.pub
```

B.1.3　サーバーへの公開鍵登録

`ssh-copy-id` は、公開鍵を接続先の `authorized_keys` へ追加するコマンドである。

```
$ ssh-copy-id user@server.example.jp
```

特定の公開鍵を使いたいなら `-i` で指定できる。

```
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server.example.jp
```

`ssh-copy-id` は、指定した公開鍵をサーバー側の `~/.ssh/authorized_keys` へ追加する。公開鍵の転送と追記を一つのコマンドで実行できる。

B.1.3.1　`ssh-copy-id` が使えない場合

`ssh-copy-id` が使えない環境では、公開鍵を手動で登録する。登録対象は、次のコマンドで表示される 1 行全体である。

```
$ cat ~/.ssh/id_ed25519.pub
```

表示された 1 行全体を、サーバー側の `~/.ssh/authorized_keys` へ追記する。サーバーへ通常のパスワードログインができるなら、自分でログインして追記してもよい。

B.1.4　`ssh-agent` による秘密鍵の管理

パスフレーズ付きの鍵を使う場合は、`ssh-agent` に読み込ませておくと毎回の入力を減らせる。

```
$ eval "$(ssh-agent -s)"
$ ssh-add ~/.ssh/id_ed25519
```

B.2　接続設定と GitHub 連携

B.2.1　~/.ssh/config による接続設定

~/.ssh/config には、接続先ごとのホスト名、利用者名、ポート番号、秘密鍵ファイルを記録できる。次の設定では、これらの接続条件に labserver という名前を付ける。

~/.ssh/config

```
Host labserver
    HostName server.example.jp
    User student
    Port 2222
    IdentityFile ~/.ssh/id_ed25519
```

この設定を保存すると、次のコマンドが同じ接続条件を参照する。

```
$ ssh labserver
```

この設定ファイルには接続先、利用者名、秘密鍵ファイルの場所が記録されるが、秘密鍵そのものは含まれない。ただし、第三者による書き換えを防ぐため、所有者だけが読み書きできる権限を設定する。

```
$ chmod 600 ~/.ssh/config
```

B.2.2　GitHub への SSH 鍵登録

通常のサーバーとは違って、GitHub には ssh-copy-id を使わない。GitHub はシェル接続先ではなく、アカウント設定に公開鍵を登録する相手だからである。

GitHub の設定画面にある SSH and GPG keys へ公開鍵を登録する。GitHub CLI gh を使う場合は、認証手順の中で既存鍵の登録または新規鍵の作成を選択できる。

```
$ gh auth login --git-protocol ssh --web
```

GitHub CLI を使う場合は、既存の鍵を検出してアップロードするか、新しい鍵を作るかを対話的に選べる。GitHub CLI を使わない場合は、cat /.ssh/id_ed25519.pub で表示した公開鍵を Web 画面へ貼り付ければよい。

B.3　scp・rsync によるファイル転送

scp は指定したファイルまたはディレクトリを転送する。rsync は送信元と転送先の差分を調べ、除外条件や中断後の部分ファイルの保持を指定できる。単発の転送には scp、同じディレクトリの反復同期には rsync を用いる。

B.3.1　scp の最小例

scp では、ローカルとリモートのどちらを送信元に書くかによって転送方向が決まる。両方向の最小例を次に示す。

```
$ scp result.csv user@server.example.jp:~/work/
$ scp user@server.example.jp:~/work/result.csv .
```

1 行目はローカルからサーバーへ送信し、2 行目はサーバーからローカルへ受信する。ディレクトリ全体を転送する場合は -r を付ける。

B.3.2　差分同期に rsync を用いる理由

rsync は、送信元と転送先の差分を転送し、--partial で中断時の部分ファイルを保持し、--exclude で転送対象を除外できる。解析用ディレクトリやコード一式を反復同期する場合は、変更のないファイルも再転送する scp より転送量を抑えられる。

```
$ rsync -aP ./analysis/ user@server.example.jp:~/work/analysis/
```

主なオプションの意味を次に示す。

-a　アーカイブ（archive）モードである。再帰コピーを行い、時刻や権限などもまとめて保つ。

-P　--partial --progress の省略形である。途中ファイルを残しつつ進捗を表示する。

-v　何を転送しているか詳しく表示する。

-z　転送時に圧縮する。低速回線では有利だが、既に圧縮済みの大きなデータでは効果が薄いこともある。

--exclude　特定のファイルやディレクトリを転送対象から除外する。

次の例では、アーカイブモードと進捗表示を有効にし、転送先で再作成できる仮想環境、キャッシュ、ビルド成果物を除外する。

```
$ rsync -aPv \
  --exclude='.venv/' \
  --exclude='__pycache__/' \
  --exclude='build/' \
  --exclude='.mypy_cache/' \
```

```
./analysis/ user@server.example.jp:~/work/analysis/
```

この例では、依存関係の定義やソースコードから転送先で再生成できる仮想環境、キャッシュ、ビルド成果物を除外し、転送量を減らしている。

B.3.2.1　末尾のスラッシュによる転送結果の違い

rsync では、送信元パスの末尾に / を付けるかどうかで意味が変わる。

```
$ rsync -aP analysis user@server:~/work/
$ rsync -aP analysis/ user@server:~/work/
```

1 行目は analysis というディレクトリごと送り、2 行目は analysis の中身を送る。末尾のスラッシュを取り違えると転送先の階層が 1 段ずれる。初回の転送前には、--dry-run を加えた実行で転送対象を確認できる。

転送条件やオプションの詳細は rsync の公式マニュアル [58] で確認できる。

付録 C

Git と GitHub の基本

C.1 リポジトリ・ステージング・コミットの関係

Git は、手元のファイルの変更履歴を記録するためのソフトウェアである。これに対して GitHub は、Git リポジトリを置いて共有するためのサービスである。

- Git: ローカルマシンでも単独で使えるバージョン管理システム
- GitHub: Git リポジトリをホストし、共有、Issue、Pull Request などを提供するサービス

したがって、`git init`、`git add`、`git commit` は Git の操作であり、GitHub 上のリポジトリを `clone` で取得したり、`push` で送ったりする段階で GitHub が関係する。

C.1.1 リポジトリと公開範囲

リポジトリは、コード、データ、履歴をまとめて管理する単位である。GitHub へ置く場合は、公開範囲も意識する必要がある。

- 公開（`public`）: 誰でも閲覧できる
- 非公開（`private`）: 明示的に許可された利用者だけが閲覧できる
- 内部（`internal`）: 主に GitHub Enterprise で、同じ Enterprise に属するメンバーへ公開する。単一の Organization 内だけに限定する設定とは異なる

授業用の配布物、研究中のコード、機密データを含むリポジトリでは、公開範囲の誤設定が情報漏えいにつながる。GitHub で新しいリポジトリを作成する際には、公開範囲（visibility）の確認が必要である。

C.1.2　作業ツリー・ステージングエリア・コミット履歴

Git では、作業中のファイルと履歴へ記録する内容を分けて管理する。変更を履歴へ記録する手順は、次の 3 段階からなる。

1. 作業ファイルの編集
2. `git add` による記録対象の選択
3. `git commit` による履歴の確定

この 2 段階目にある、次のコミットへ記録する内容を保持する領域をステージングエリアという。`git add` は作業ツリーの内容をステージングエリアへ登録し、`git commit` は登録済みの内容を履歴へ記録する。

Git が管理する状態は、次の 3 層に分けられる。

- 作業ツリー（working tree）：エディタで直接編集しているファイル
- ステージングエリア（staging area、index）：次のコミットへ入れる変更を一時的に集めた領域
- コミット履歴（commit history）：確定済みの変更が記録された履歴

たとえば、ファイルを編集した直後の変更は作業ツリーだけに存在する。`git add` を実行すると、その時点の内容がステージングエリアへ登録される。`git commit` は、ステージングエリアの内容を履歴として確定する操作である。`git add` の後に同じファイルを編集した場合、追加分は作業ツリーだけに残り、ステージ済みの内容には含まれない。

C.2　ローカル履歴の作成

C.2.1　コミットへ記録する名前とメールアドレスの設定

コミットへ記録する作成者名とメールアドレスを設定する。

```
$ git config --global user.name "Your Name"
$ git config --global user.email "your_email@example.com"
```

これをしておかないと、コミット時に誰の変更なのかが分からなくなる。

C.2.2　新しいリポジトリの作成

まだ Git で管理されていないディレクトリを初期化するコマンドが `git init` である。

```
$ mkdir analysis_work
$ cd analysis_work
$ git init
```

```
$ git status
```

`git status` は、未追跡、変更済み、コミット待ちのファイルを一覧できる。編集やコミットの前後で実行すると、各変更がどの段階にあるかを確認できる。

C.2.3　編集、確認、記録の流れ

日常の作業では、ファイルを編集し、差分を確認してから、必要な変更をコミットする。

```
$ git status
$ git diff
$ git add analyze.py
$ git diff --cached
$ git commit -m "Add first analysis script"
$ git log --oneline
```

`git diff` は、作業ツリーとステージングエリアの差分を表示する。`git diff --cached` は、ステージングエリアと直前のコミットの差分を表示する。`git log --oneline` は、コミット履歴を 1 件 1 行で表示する。

C.2.3.1　`git status` の読み方

`git status` の表示では、少なくとも次の 3 種類を区別する。

- 未追跡（untracked）：まだ Git が追跡していない新規ファイル
- 変更済み（modified）：追跡中だが、変更内容をまだ `add` していないファイル
- コミット予定（changes to be committed）：`add` 済みで、次のコミットへ入る変更

`git status` の出力から、各ファイルが未追跡、変更済み、コミット予定のいずれに該当するかを判別できる。

C.2.3.2　`git add` による変更内容のステージング

`git add` は「変更を次のコミット候補として選ぶ」コマンドである。保存ではない。したがって、`git add` した後でも、コミット前にさらに編集すれば、その追加分はまだコミット対象へ入っていない。

```
$ git diff
$ git add analyze.py
$ git diff --cached
```

この 2 種類の `diff` を区別すると、作業ツリーと次のコミット候補の差を把握できる。

C.2.4 .gitignore による除外設定

生成物まで毎回コミットすると、履歴が見にくくなる。Python 解析なら、仮想環境、キャッシュ、ビルド成果物などは除外することが多い。

C.2.4.1 .gitignore の記述

.gitignore は、Git に「このパターンのファイルは追跡しなくてよい」と教えるためのファイルである。リポジトリのルートに置くことが多い。Python 解析用の最小例は次のようになる。

.gitignore

```
.venv/
__pycache__/
*.pyc
build/
dist/
.pytest_cache/
```

このファイルを作ると、まだ追跡されていない .venv/ や build/ は無視され、通常の git status には表示されない。すでに追跡中のファイルには、この設定は適用されない。

C.2.4.2 すでに追跡済みなら git rm --cached が必要

.gitignore は「これから追跡しない」ための設定である。すでに一度コミットしたファイルには、そのままでは効かない。

```
$ git rm --cached -r .venv __pycache__ build
$ git commit -m "Stop tracking generated files"
```

git rm --cached は、作業ディレクトリの実ファイルを残したまま、インデックスに登録されたパスだけを削除する。実行後の削除をコミットすると、そのパスは Git の追跡対象から外れる。

C.2.4.3 uv プロジェクトにおけるコミット対象

uv を使うプロジェクトでは、通常 .venv/ はコミットしない。依存関係を記述する pyproject.toml と除外規則を記述する .gitignore はコミットする。アプリケーションや再現可能な解析環境では、解決済みの依存関係を記録する uv.lock も通常はコミットする。ライブラリでは、リポジトリ内のテストや開発環境をロックファイルで再現するかどうかを運用方針として定める。.python-version は、プロジェクトで使用する Python の版を共有する場合にコミットする。

C.3　GitHub への接続

C.3.1　既存リポジトリの取得

`git clone` は、GitHub 上のプロジェクトや教材を履歴ごと手元へ複製するコマンドである。

```
$ git clone git@github.com:example/project.git
$ cd project
$ git remote -v
```

`git clone` は、リポジトリのファイルと履歴をローカルへ複製する。`git remote -v` は、接続先として登録された URL を表示する。本書の例では、取得と push の双方に SSH URL を用いる。公開鍵の準備は付録 B で説明する。

C.3.2　ローカルリポジトリの GitHub への初回公開

既存リポジトリの取得とは逆に、ローカルで作成した履歴を新しい GitHub リポジトリへ送る場合がある。まず GitHub 上に空のリポジトリを作成し、次にローカルリポジトリへ接続先を登録して push する。

C.3.2.1　GitHub 上の空リポジトリの作成

GitHub の Web 画面で `New repository` を開き、リポジトリ名と公開範囲を決める。ローカルにすでに履歴がある場合は、README、`.gitignore`、ライセンスを GitHub 側で追加せず、コミットを持たない空のリポジトリを作成する。

README や `.gitignore` を GitHub 側で追加すると、リモート側にローカルとは別のコミットが作られる。その場合、双方の履歴を統合してから push する必要がある。空のリモートリポジトリであれば、この統合は不要である。

C.3.2.2　README や LICENSE を先に作っていた場合

GitHub 側で README や LICENSE を作成した場合、リモートリポジトリにはローカルと異なるコミットが存在する。リモート側の履歴をローカルへ取り込んで統合した後に push する。

```
$ git branch -M main
$ git remote add origin git@github.com:owner/project.git
$ git fetch origin
$ git merge --allow-unrelated-histories origin/main
$ git push -u origin main
```

`--allow-unrelated-histories` が必要なのは、手元のリポジトリと GitHub 側のリポジトリが別々に作られており、履歴の出発点が一致していないからである。この `fetch` と `merge` によって、GitHub 側の README や LICENSE のコミットを自分の履歴へ取り込んでから、その続きとして push できるようになる。

ローカルとリモートの双方で同じ名前の README や LICENSE を作成している場合、`git merge` によって競合が生じることがある。競合箇所を編集し、`git add` と `git commit` で統合結果を記録してから、`git push -u origin main` を実行する。

C.3.2.3　接続先の登録と最初の push

GitHub が表示する URL を確認し、ローカルリポジトリへ `origin` という名前で登録する。既定ブランチを `main` とする例を次に示す。

```
$ git status
$ git branch --show-current
$ git branch -M main
$ git remote add origin git@github.com:owner/project.git
$ git remote -v
$ git push -u origin main
```

`git branch --show-current` は現在のブランチ名を表示する。`git branch -M main` は現在のブランチ名を `main` へ変更する。既存のブランチ名を維持する場合は、以後のコマンドにその名前を指定する。

`git remote add origin ...` は、GitHub の URL を `origin` という名前で登録する。`git remote -v` は登録後の取得用 URL と送信用 URL を表示する。`git push -u origin main` の `-u` は、ローカルの `main` に対する上流ブランチを `origin/main` に設定する。この設定後は、接続先とブランチ名を省略した `git push` と `git pull` を利用できる。

C.3.2.4　`origin` がすでにある場合

すでに `origin` が登録されている状態で `git remote add origin ...` を実行すると、エラーになる。その場合は、いま何が設定されているかを確認してから、必要なら URL を差し替える。

```
$ git remote -v
$ git remote set-url origin git@github.com:owner/project.git
$ git remote -v
```

教材を別のリポジトリから複製して流用した場合や、HTTPS から SSH へ切り替えたい場合に、この操作が必要になることがある。

C.3.3　GitHub との同期

ローカルでコミットしただけでは、まだ GitHub には反映されない。GitHub 側から最新を取り込むのが `pull`、手元のコミットを送るのが `push` である。

```
$ git pull --ff-only
$ git push origin main
```

`git pull --ff-only` は、ローカルブランチをリモートブランチまで早送りできる場合にだけ更新する。ローカルとリモートの両方に異なるコミットがあり履歴が分岐している場合は停止するため、意図しないマージコミットを作らない。

C.3.4　GitHub で利用する HTTPS URL と SSH URL

GitHub のリポジトリ URL には、少なくとも HTTPS と SSH の 2 通りがある。

```
https://github.com/owner/repo.git
git@github.com:owner/repo.git
```

HTTPS URL は HTTPS 認証、SSH URL は SSH 鍵認証を用いる。一つのリポジトリでは、登録したリモート URL と認証情報の対応を記録する。本書では、`git clone`、`pull`、`push` に用いる認証方式を SSH に統一する。公開リポジトリの取得には HTTPS URL も利用できるが、push や非公開リポジトリの取得には認証が必要になる。SSH URL に統一すれば、操作ごとに認証方式を切り替える必要がない。

C.4　認証と最小ワークフロー

C.4.1　GitHub の認証: SSH 鍵を基本にし、必要なら PAT

GitHub では、HTTPS 経由の Git 操作にアカウントのパスワードを使用できない。SSH URL では SSH 鍵、HTTPS URL では個人用アクセストークン（PAT）または認証情報ヘルパーを用いる[1)]。本書では SSH 鍵を標準とし、HTTPS と PAT の組み合わせを代替手段として扱う。

C.4.1.1　SSH 鍵による認証

SSH を使う場合は、付録 B の手順で公開鍵を用意し、それを GitHub の `SSH and GPG keys` へ登録する。GitHub CLI の `gh auth login --git-protocol ssh --web` は、認証と公開鍵の登録に必要な操作を対話形式で案内する。

1)　GitHub Docs, “About authentication to GitHub”（2026 年 8 月 25 日閲覧）。

C.4.1.2 PAT による認証

SSH 鍵を使えない事情があり、HTTPS で GitHub を使う場合は、プライベートリポジトリへの push や pull で PAT を使う。

1. GitHub の `Settings` への移動
2. `Developer settings` から `Personal access tokens` への移動
3. Fine-grained token または classic token の作成
4. 対象リポジトリと必要最小限の権限の選択
5. 発行済みトークンの安全な保管

認証時には、ユーザー名として GitHub のアカウント名を入力し、パスワードを求められた欄へ PAT を入力する。組織がトークンの利用制限や承認を設定している場合には、その方針にも従う必要がある。

Fine-grained PAT では、対象となるリポジトリと権限を個別に限定できる。GitHub は、可能であれば personal access token (classic) よりも fine-grained PAT を推奨している[2)]。ただし、組織の方針によって管理者の承認が必要な場合があり、利用する機能が fine-grained PAT に対応しているかも確認する。

SAML SSO を利用する組織では、SSH 鍵や PAT に組織リソースへのアクセス許可が必要になる場合がある。Personal access token (classic) は作成後に SSO への承認が必要であり、fine-grained PAT は作成時に組織を対象として指定する[3)]。実際の承認手順と利用可能な認証方式は、対象組織の方針を確認する。

C.4.2 変更確認から同期までの最小ワークフロー

変更の確認から GitHub との同期までは、次の順序で行う。

1. `git status` による状況の確認
2. `git diff` による変更内容の確認
3. `git add` による記録対象の選択
4. `git diff --cached` によるコミット内容の再確認
5. `git commit -m "..."` による履歴への記録
6. GitHub との同期に用いる `git pull --ff-only` と `git push`

この流れは、教材の取得、`uv` プロジェクトの再現、共同研究でのコード共有に共通する。`git add` と `git commit` の前後で `status` と `diff` を確認し、記録対象と実際の差分を一致させる。

公開範囲やアクセス条件の詳細は、GitHub のリポジトリに関する公式資料 [59] を参照する。

2) GitHub Docs, “Managing your personal access tokens”（2026 年 8 月 25 日閲覧）。
3) GitHub Docs, “Authorizing a personal access token for use with single sign-on”（2026 年 8 月 25 日閲覧）。

付録 D

pybind11 による C++ 拡張モジュール

Python のプロファイル結果から計算量の多い関数を特定した後、その関数だけを C++ で実装する方法がある。pybind11 は、C++ の関数やクラスを Python モジュールとして公開するためのライブラリである。本付録では、最小構成の C++ 拡張モジュールを作り、`uv pip install -e .` で開発用環境へインストールする手順を示す。

D.1 C++ 拡張を選ぶ判断基準

プロファイルによって少数の関数が処理時間を支配すると確認でき、NumPy などの既存実装では置き換えられない場合に、その関数を C++ で実装することを検討する。入出力、可視化、ワークフロー制御を Python に残し、計算量の多い関数だけを pybind11 で公開すれば、C++ との境界を限定できる。

具体例として、時刻列 `times` に対して、差の絶対値が `window` 以下となる全ペアの数を C++ で数える。この二重ループの計算量は、時刻数を N とすると $O(N^2)$ である。

D.2 pybind11 パッケージの最小構成

ディレクトリ構成は次のようになる。

fastcount_cpp/ の例

```
pyproject.toml
CMakeLists.txt
cpp/
`-- bindings.cpp
src/
`-- fastcount_cpp/
    `-- __init__.py
```

pyproject.toml はビルド方法と Python パッケージの情報、CMakeLists.txt は C++ 拡張のコンパイル方法を記述する。cpp/bindings.cpp に C++ の処理と Python への公開方法を書き、src/fastcount_cpp/__init__.py から利用者向けの関数を再公開する。以下では、各ファイルの役割と内容を構成順に説明する。

D.2.1 pyproject.toml

C++ 拡張には、Python パッケージの情報に加えてコンパイル方法を定めるビルドバックエンドが必要である。ここでは、scikit-build-core、pybind11、CMake を組み合わせる。

pyproject.toml

```
[build-system]
requires = [
  "pybind11>=2.13",
  "scikit-build-core>=0.10",
]
build-backend = "scikit_build_core.build"

[project]
name = "fastcount_cpp"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["numpy>=2.0"]

[tool.scikit-build]
wheel.packages = ["src/fastcount_cpp"]
```

build-system に書いたパッケージは、拡張モジュールのビルド環境で用いられる。インストール後の実行に必要なパッケージは project.dependencies に分けて記述する。

D.2.2 CMakeLists.txt

CMakeLists.txt には、使用する C++ 規格、コンパイル対象、インストール先を記述する。最小構成は次の通りである。

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(fastcount_cpp LANGUAGES CXX)

find_package(pybind11 CONFIG REQUIRED)

pybind11_add_module(_core cpp/bindings.cpp)
target_compile_features(_core PRIVATE cxx_std_17)
install(TARGETS _core DESTINATION fastcount_cpp)
```

pybind11_add_module が、Python から import できる共有ライブラリを作る中心になる。こ

こで作るモジュール名を `_core` にしておくと、Python 側では `fastcount_cpp._core` として参照できる。

D.2.3　C++ の実装

`cpp/bindings.cpp` では、時刻差を数える C++ 関数を定義し、同じファイル内で pybind11 による公開名と引数名を指定する。

cpp/bindings.cpp

```cpp
#include <cmath>
#include <cstddef>
#include <stdexcept>
#include <vector>

#include <pybind11/pybind11.h>
#include <pybind11/stl.h>

std::size_t count_pairs_within_window(
    const std::vector<double>& times,
    double window
) {
    if (window < 0.0) {
        throw std::invalid_argument("window must be non-negative");
    }

    std::size_t count = 0;
    const std::size_t n = times.size();

    for (std::size_t i = 0; i < n; ++i) {
        for (std::size_t j = i + 1; j < n; ++j) {
            if (std::abs(times[j] - times[i]) <= window) {
                ++count;
            }
        }
    }
    return count;
}

namespace py = pybind11;

PYBIND11_MODULE(_core, m) {
    m.doc() = "Count close pairs in C++.";
    m.def(
        "count_pairs_within_window",
        &count_pairs_within_window,
        py::arg("times"),
        py::arg("window"),
        "Return the number of pairs whose time difference is within window."
```

```
    );
}
```

`pybind11/stl.h` をインクルードすると、Python の list から `std::vector<double>` への変換が有効になる。したがって、この関数の呼び出し側に明示的な変換コードは不要である。負の時間窓は定義に反するため、C++ 側で `std::invalid_argument` を送出する。この例の時刻は浮動小数点数で表せる範囲の相対時刻である。総合課題の Unix 時刻付近で ns の差を扱う場合は、整数ナノ秒による API と検出器の条件を別に実装する。`double` へ変換してから差を取る方法では、元の時間分解能を失い得る。

D.2.4 Python から公開する API

C++ 拡張を内部モジュール `_core` として直接公開すると、利用者が実装上の名前を知る必要が生じる。そこで、パッケージの `__init__.py` から公開関数だけを再公開する。

src/fastcount_cpp/__init__.py

```python
"""Python interface for the C++ pair counter."""

from ._core import count_pairs_within_window

__all__ = ["count_pairs_within_window"]
```

この `__init__.py` に公開関数を再エクスポートすると、利用側は `from fastcount_cpp import count_pairs_within_window` と記述できる。C++ 側の内部モジュール名 `_core` は、公開 API に現れない。

D.3 拡張モジュールのビルドとインストール

開発中は、プロジェクトの仮想環境へ editable install する。Python 側のファイルはインストール元を参照し、C++ ソースは再ビルドによってコンパイル済み拡張へ反映される。

```
$ uv sync
$ uv pip install -e .
$ uv run python -c "from fastcount_cpp import count_pairs_within_window; print(
    count_pairs_within_window([0.0, 0.1, 0.4], 0.25))"
```

最後の出力は次のようになる。

```
1
```

これは、`(0.0, 0.1)` だけが窓 `0.25` の中に入っていることを表している。

純粋な Python ファイルとは異なり、C++ ソースの変更はコンパイル済み拡張へ自動では反映されない。`bindings.cpp` を編集した後は、`uv pip install -e .` を再実行して拡張モジュールを再ビルドする。

D.4　Python からの C++ 関数の呼び出し

インストール後の拡張モジュールは、Python モジュールと同じ import 文で読み込める。

```
from fastcount_cpp import count_pairs_within_window

times = [0.0, 0.1, 0.4, 0.55]
window = 0.2

count = count_pairs_within_window(times, window)
print(count)
```

Python へ公開する関数を限定し、その関数の引数、返り値、例外条件をテストで固定する。C++ への移植後もこの契約を保てば、利用側のコードを変更せずに内部実装を置き換えられる。

D.5　C++ へ移す処理の選び方

入出力、可視化、例外処理、引数処理まで C++ へ移すと、Python と C++ の境界が広がり、ビルドと保守が複雑になる。次のように、ワークフローを担う Python と計算コアを担う C++ の役割を分ける。

- Python: ファイルの読み込み、引数処理、図の保存、ワークフロー全体
- C++: 処理時間を支配するループ、組み合わせ探索、反復数値計算

Python 側には引数検証を含む公開 API とワークフローを置き、C++ 側には測定上のボトルネックとなった計算コアを置く。このように役割を分けると、C++ のビルドが必要な範囲と、Python から検証する関数を限定できる。

ビルド設定と Python/C++ 間の型変換の詳細は、pybind11 [60] と scikit-build-core [61] の公式資料を参照する。

付録 E

正規表現の基本

正規表現は、文字列が満たすパターンを記述する記法である。grep、awk、Python の re モジュールでは、ログの抽出、ファイル名の検査、検出器 ID の選別、時刻文字列の検証などに正規表現を用いる。ここでは、本文で触れた grep や awk の延長として、解析に必要な構文をまとめる。

以下のメタ文字の説明は、主に Python の re と grep -E の拡張正規表現を対象とする [62, 63]。grep の既定の基本正規表現とは、+、?、括弧などの扱いが異なる。Python の . は既定では改行に一致せず、\d は ASCII の数字以外の Unicode の十進数字にも一致する。半角数字だけが対象なら [0-9] と明示する。

E.1 文字列パターンを表す正規表現

E.1.1 文字どおりの一致とメタ文字

まず、文字どおりの検索と正規表現による検索を区別する。det_A という文字列だけを探す場合は、各文字をそのまま並べる。

```
$ grep "det_A" events.txt
```

これに対して、. や * のような特別な意味を持つ記号を使うと、「どの文字でもよい」「0 回以上くり返す」といった条件を書けるようになる。こうした特別な意味を持つ文字をメタ文字という。

```
.
*
+
?
^
$
```

```
[]
()
¦
```

たとえば `det_.` は `det_A` にも `det_B` にも一致する。`.` は「任意の 1 文字」を表すからである。ピリオドそのものとの一致を指定するには、前にバックスラッシュを付け、`\.` のようにエスケープする。`+` や `?` を文字どおりに照合する場合も同様である。

E.1.2　ワイルドカードとの違い

シェルのワイルドカードと正規表現は似て見えるが、同じものではない。`ls *.txt` の `*` は「任意の文字列」を表すが、正規表現の `*` は「直前の要素の 0 回以上のくり返し」を表す。したがって、正規表現で「任意の文字列」を書きたいときは `.*` と書く。

表 E.1　ワイルドカードと正規表現の違い

場面	記法	意味
シェル	`*.txt`	任意の名前で `.txt` で終わるファイル
正規表現	`.*\.txt$`	任意の文字列で始まり `.txt` で終わる文字列

表 E.1 は、二つの記法における `*` の意味を比較している。`grep "*.txt"` では、引用符内の `*` をシェルではなく `grep` の正規表現エンジンが解釈する。正規表現の `*` は直前の要素の 0 回以上の反復を表すため、この位置では「任意の文字列」にならず、エラーとなる場合もある。シェルによるファイル名展開にはワイルドカード、文字列内容の照合には正規表現を用いる。

正規表現は文字列の形を検査する。日付や時刻のパターンに一致しても、2 月 30 日や 99 時などが有効な日時になるとは限らない。暦や時刻の値域は `datetime` などでも検証する。

E.2　アンカー・文字クラス・量指定子

E.2.1　アンカー（行頭と行末）

`^` は行頭、`$` は行末を表す。したがって、文字列の一部ではなく「行全体がその形である」ことを確かめたいときに使う。

```
$ grep '^ERROR' run.log
$ grep 'txt$' filelist.txt
$ grep '^det_A$' detectors.txt
```

1 行目は `ERROR` で始まる行、2 行目は `txt` で終わる行、3 行目は行全体がちょうど `det_A` である行だけを取る。特に 3 行目のように両端をアンカーで挟むと、`det_AB` や `xdet_A` が紛れ込むのを防げる。

E.2.2　文字クラス [...] と代表的な省略記法

[abc] は a、b、c のいずれか 1 文字、[A-Z] は英大文字 1 文字を表す。検出器名やセンサー番号のように、限られた候補から 1 文字を選ぶパターンに用いる。

```
^det_[AB]$
^run_[0-9][0-9][0-9]$
^[A-Za-z_][A-Za-z0-9_]*$
```

1 行目は det_A または det_B、2 行目は run_001 のような 3 桁番号付きの名前、3 行目は Python 風の識別子に近い形を表す。範囲指定の [0-9] は、数字を含むパターンで頻出する。

Python の re では、数字 1 文字を \d、空白 1 文字を \s と書ける。

```
1 import re
2
3 re.fullmatch(r"det_[AB]", "det_A")
4 re.fullmatch(r"\d\d:\d\d:\d\d", "12:34:56")
5 re.search(r"\s+", "det_A   15.2")
```

ただし、標準的な grep の基本・拡張正規表現と awk では、\d や \s を Python と同じ省略記法として使用できない。コマンドラインでは [0-9] や [[:space:]] を用いる。

E.2.3　量指定子 *, +, ?

量指定子は、直前の要素が何回くり返されるかを表す。* は 0 回以上、+ は 1 回以上、? は 0 回または 1 回である。

```
ab*c
ab+c
colou?r
```

ab*c は ac、abc、abbbc に一致する。ab+c は少なくとも 1 個の b が必要なので ac には一致しない。colou?r は color と colour の両方に一致する。

量指定子を使うと、1 個以上の空白、任意指定の符号、任意指定の小数部など、要素の出現回数をパターンに記述できる。

```
1 pattern = re.compile(r"^[+-]?\d+(\.\d+)?$")
2
3 for text in ["12", "-3.5", "+0.25", "1.2.3"]:
4     print(text, bool(pattern.fullmatch(text)))
```

このパターンは、符号付き整数や小数を表し、1.2.3 のように形式に合わない文字列には一致しない。

E.3　run 名・時刻・数値ログの抽出

E.3.1　ログからの対象行抽出

実験ログやジョブログでは、異常行や特定の run だけを抽出する場合がある。アンカーと文字クラスを組み合わせると、行全体に対する条件を具体的に記述できる。

```
$ grep '^ERROR' run.log
$ grep '^2026-04-04 .* det_[AB] ' run.log
$ grep -E '^(WARN¦ERROR)' run.log
```

3 行目の `grep -E` は、拡張正規表現として ¦ による選択を解釈する。拡張正規表現では、括弧、+、?、¦ をバックスラッシュなしで記述できる。

E.3.2　検出器 ID・センサー名の選別

検出器名が `det_A`, `det_B`, `det_C` のように並んでいるなら、文字クラスや選択を使って対象を絞れる。

```
$ grep '^det_[AB]$' detector_ids.txt
$ awk '$1 ~ /^det_[AB]$/ {print $1, $3}' summary.txt
```

`awk` の `$1 ~ /.../` は、第 1 列が正規表現に一致するかを判定する。列ごとに異なる照合条件や出力列を指定する場合は、列を直接参照できる `awk` を用いる。

E.3.3　Python による抽出と変換

正規表現の括弧で囲んだ部分は、照合結果から個別に取り出せる。Python では、`re.search` や `re.fullmatch` の返り値に対して `group()` を呼び出す。

```
1 import re
2
3 line = "2026-04-04 12:34:56 det_A signal=15.2"
4 match = re.search(r"signal=(\d+\.\d+)", line)
5
6 if match:
7     signal = float(match.group(1))
8     print(signal)
```

括弧 `(...)` で囲んだ部分はキャプチャされ、`group(1)`、`group(2)` のように取り出せる。ログ 1 行から run 番号、検出器名、信号値を個別に抽出する場合に用いる。

E.4 Python の re と grep/awk の使い分け

E.4.1 Python の raw string によるパターン記述

Python の通常の文字列リテラルでは、バックスラッシュが Python と正規表現の双方で特別な意味を持つ。正規表現は、Python 側でバックスラッシュを解釈しない `r"..."` 形式の raw string で記述する。

```
import re

pattern = re.compile(r"^det_[A-Z]\d+$")
print(bool(pattern.fullmatch("det_A12")))
```

通常の Python 文字列では、`"\\d+"` のようにバックスラッシュを二重に記述する必要がある。正規表現のパターンは、原則として raw string で記述する。

E.4.2 search, match, fullmatch の使い分け

`re.search` は文字列中の任意の位置、`re.match` は先頭、`re.fullmatch` は文字列全体に対して一致を調べる。ファイル名や識別子の全体を検証する場合は、接頭辞または接尾辞に余分な文字があれば不一致となる `fullmatch` を用いる。

```
import re

print(re.search(r"\d+", "run_42").group())
print(bool(re.match(r"run", "run_42")))
print(bool(re.fullmatch(r"run_\d+", "run_42")))
```

E.4.3 コマンドラインで十分な場面と、Python に移すべき場面

条件に合う行の表示、件数の確認、特定列の抽出は、`grep` または `awk` だけで記述できる。一致部分を数値へ変換して計算する場合、複数の条件分岐を組み合わせる場合、または抽出結果から DataFrame を作る場合は、変換、分岐、テストを同じコードに記述できる Python を用いる。

- `grep`: 行全体の抽出や件数確認をすばやく行いたいとき
- `awk`: 列条件と正規表現を組み合わせたいとき
- Python `re`: 抽出した結果をそのまま解析処理へつなげたいとき

正規表現は必要な条件を小さく組み合わせて作る。受理する文字列と拒否する文字列のテストを用意し、変更時に再確認する。

付録 F

シェルスクリプトによる自動化と一括処理

第 2 章では、シェルを対話的に操作する方法を説明した。同じコマンド列を複数のファイルや条件に対して実行する場合は、操作をシェルスクリプトへ記述すると、処理順序と引数を記録したまま再実行できる。

本付録では、引数の受取り、変数の計算、ファイルの存在確認、条件分岐、配列、`for` ループを扱う。最後に、設定ファイルのテンプレートを `sed` で置換して複数条件を実行する方法を説明する。

F.1 シェルスクリプトの基礎構造

F.1.1 シバンと実行権限

シェルスクリプトの先頭には、どのシェルで解釈するかを指定するシバン（shebang）を書く。本付録の例は Bash の配列と算術式を使うため、`env` で `bash` の所在を検索する。

```
#!/usr/bin/env bash

echo "Hello, Shell Scripting!"
```

`run.sh` という名前で保存した場合は、実行権限を付けてから直接実行する。

```
$ chmod u+x run.sh
$ ./run.sh
Hello, Shell Scripting!
```

`chmod u+x` はファイル所有者に実行権限を追加し、`./run.sh` は現在のディレクトリにあるファイルを実行する。最後の行がスクリプトの標準出力である。

F.1.2　コマンドライン引数

スクリプトに外から値（ファイル名や設定値）を渡すには、実行時の引数を利用する。スクリプト内からは `$1`, `$2` といった特殊な変数で受け取れる。

- `$0`: 実行されたスクリプトのファイル名（例: `./run.sh`）
- `$1`, `$2`, ...: 1 番目、2 番目以降の引数
- `$#`: 渡された引数の総数
- `$@`: 渡されたすべての引数。個々の引数を保ったまま展開する場合は `"$@"` と書く

次のスクリプトは、二つの位置引数を検査してから入力名と出力名を表示する。

```
#!/usr/bin/env bash

# 二つの位置引数が渡されたことを確認する
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <input_file> <output_file>" >&2
    exit 1
fi

INPUT="$1"
OUTPUT="$2"
echo "Reading from $INPUT, writing to $OUTPUT"
```

`./run.sh input.csv result.csv` と実行すると、`Reading from input.csv, writing to result.csv` と表示される。引数が二つでなければ使用法を表示し、終了状態 1 で処理を打ち切る。

F.2　変数、演算、条件式

F.2.1　変数の定義と配列

シェルで変数を定義する際は、イコールの前後に空白を入れてはならない。また、変数の値を参照するときは先頭に `$` を付ける。

```
COUNT=100
NAME="data_analysis"

echo "The count is $COUNT."       # 展開される
echo 'The count is $COUNT.'       # シングルクォートでは展開されず "$COUNT" と表示される
```

Bash では、要素を空白で区切って括弧 `()` で囲むと配列を定義できる。各要素を引用符で囲めば、要素内の空白やワイルドカードが意図せず展開されることを防げる。

```bash
# 配列の定義
targets=("background" "signal" "noise")

# 配列サイズの取得
echo "Number of targets: ${#targets[@]}"

# 配列要素へのアクセス
echo "First element: ${targets[0]}"

# 配列全体を展開する（for ループでよく使う）
echo "All elements: ${targets[@]}"
```

F.2.2　数値計算（算術式展開）

Bash で整数の加減乗除を行う場合は、算術式展開 `$((...))` または算術評価 `((...))` を用いる。この中では変数名の `$` を省略できる。Bash の算術評価は浮動小数点計算に対応しないため、小数を扱う場合は Python、`bc`、`awk` などを用いる。

```bash
A=10
B=5

# 計算式は (( )) を用いる
SUM=$((A + B))
MUL=$((A * B))

# 変数を直接加算・インクリメントする
(( A++ ))
(( B += 20 ))
```

F.2.3　条件式（test コマンド）

数値の大小比較や、ファイルが存在するかどうかの確認には、`[ ]` を用いる。大括弧の内側には必ずスペースが必要である。

数値の比較:

- `-eq`: 等しい (equal)
- `-ne`: 等しくない (not equal)
- `-gt`: より大きい (greater than)
- `-ge`: 以上 (greater than or equal)
- `-lt`: 未満 (less than)
- `-le`: 以下 (less than or equal)

文字列の比較:

- `=`, `==`: 一致する

- `!=`: 一致しない
- `-z "$STR"`: 文字列が空である

ファイルやディレクトリの確認:

- `-e "$FILE"`: ファイル（またはディレクトリ）が存在する
- `-f "$FILE"`: 通常のファイルとして存在する
- `-d "$DIR"`: ディレクトリとして存在する

これらの条件式は、後述する `if` 文の `[]` の中身として頻繁に登場する。

F.3　条件分岐 (if, case)

F.3.1　if 文を用いた分岐

前節の条件式を組み合わせて処理を分岐する。`then` と `fi` でブロックを囲むのがシェルの特徴である。

```
#!/usr/bin/env bash

if [ "$#" -ne 1 ]; then
    echo "使用法: $0 <file>" >&2
    exit 1
fi

FILE="$1"

if [ -z "$FILE" ]; then
    echo "エラー: ファイル名が指定されていません" >&2
    exit 1
fi

if [ -f "$FILE" ]; then
    echo "ファイル $FILE を発見しました。処理を開始します。"
elif [ -d "$FILE" ]; then
    echo "エラー: $FILE はディレクトリです。ファイルを指定してください。" >&2
    exit 1
else
    echo "エラー: $FILE が存在しません。" >&2
    exit 1
fi
```

F.3.2　case 文による分岐

条件が特定の文字列パターンとの一致で表せる場合は、case 文を用いると elif を重ねずに分岐を列挙できる。拡張子の判定や、-h、--version などのオプション処理に利用できる。

```
#!/usr/bin/env bash

if [ "$#" -ne 1 ]; then
    echo "Usage: $0 <start¦stop¦halt>" >&2
    exit 1
fi

MODE="$1"

case "$MODE" in
    "start")
        echo "Starting process..."
        ;;
    "stop" ¦ "halt")
        echo "Stopping process..."
        ;;
    *)
        echo "Unknown mode: $MODE"
        echo "Available modes: start, stop, halt"
        exit 1
        ;;
esac
```

各区分の終わりは ;; で締めくくる。アスタリスク * は「上記以外のすべて」にマッチするワイルドカードとして機能する。

F.4　反復処理と一括自動化 (for)

F.4.1　配列やリストの要素に対する for ループ

指定した値のリストや、配列の全要素に対して処理を繰り返す for 文の例を次に示す。

```
#!/usr/bin/env bash

# 配列の全要素に対して
targets=("run_01" "run_02" "run_03")

for t in "${targets[@]}"; do
    echo "Processing target: $t"
done
```

```
9
10 # 単純な数値の手書きリストやコマンド展開に対して
11 for num in 10 20 30; do
12     echo "Number is $num"
13 done
```

if は直後のコマンドの終了ステータスが 0 なら条件成立とする。[...] も条件を調べるコマンドであり、括弧の内側の空白が必要である。command1 && command2 は前段が成功した場合だけ、command1 || command2 は失敗した場合だけ後段を実行する。エラーを表示するだけでは処理は止まらないため、必要な箇所で exit 1 を使う。詳細は Bash の公式資料 [14] を参照する。

F.4.2　複数ファイルの入出力パス生成と一括処理

特定のディレクトリにある複数のファイルを順次処理する場合にも、for ループを利用できる。次の例では、入力ファイル名（data/input/run01.csv など）から、対応する出力ファイル名（data/output/run01_result.png など）を生成する。

basename コマンドや、変数の末尾から指定した部分を除く記法（%...）を用いることでこれを簡潔に実装できる。

```
1  #!/usr/bin/env bash
2
3  INPUT_DIR="data/input"
4  OUTPUT_DIR="data/output"
5
6  # 出力先が存在しなければ作成する
7  if [ ! -d "$OUTPUT_DIR" ]; then
8      mkdir -p "$OUTPUT_DIR" || exit 1
9  fi
10
11 # INPUT_DIR の中にあるすべての .csv ファイルに対して反復
12 for filepath in "$INPUT_DIR"/*.csv; do
13     # 一致がない場合や、名前が .csv で終わるディレクトリは処理しない
14     if [ ! -f "$filepath" ]; then
15         continue
16     fi
17
18     echo "Found raw file: $filepath"
19
20     # basename コマンドでディレクトリパスを取り除き、ファイル名だけにする
21     # 例: "data/input/run01.csv" -> "run01.csv"
22     filename=$(basename "$filepath")
23
24     # 文字列置換: 後方の ".csv" を取り除く
25     # 例: "run01.csv" -> "run01"
26     name_no_ext="${filename%.csv}"
27
```

```
    # 新しい出力パスを組み立て直す
    output_path="$OUTPUT_DIR/${name_no_ext}_result.png"

    echo "Running analysis Python code for: $name_no_ext"
    # 実際の処理コマンド（例: python analyze.py <in> <out>）
    # python analyze.py "$filepath" "$output_path"
done
```

この例では、入力ファイルごとに拡張子を除いた名前を作り、対応する出力パスを組み立てている。解析コマンドを有効にする前に、echo で表示される入出力の対応を確認すれば、意図しない上書きを防げる。

F.5 sed による条件別設定ファイルの生成

外部プログラムには、--input XXX のようなコマンドライン引数ではなく、config.ini などの設定ファイルから実行条件を読み込むものもある。複数のシード値や閾値を比較する場合は、条件ごとの設定ファイルをテンプレートから生成すると、各出力に対応する実行条件を保存できる。

sed を使うと、テンプレートとして用意した設定ファイルの一部を置換し、条件ごとの実行用設定ファイルを生成できる。

F.5.1 テンプレートファイルの準備

テンプレートでは、書き換えるパラメータをファイル内の他の場所に現れないプレースホルダーへ置き換える。次の例では、__THRESHOLD__ と __OUTFILENAME__ を用いる。

config_template.ini

```
[Simulation]
Events = 10000
Threshold = __THRESHOLD__
OutputFile = __OUTFILENAME__
```

F.5.2 sed による置換の実行

シェルスクリプト内で条件値を順に取り出し、sed で置換した設定ファイルを生成して実行プログラムへ渡す。

基本形は sed 's/置換前文字列/置換後文字列/g' 入力ファイル > 出力ファイル である。sed の置換式でシェル変数を展開するには、シングルクォートではなく**ダブルクォート**("...") で囲む必要がある。

```
#!/usr/bin/env bash

# 閾値を 3 つ変えてシミュレーション用の設定ファイルを動的生成し、実行する例
```

```
thresholds=(0.5 1.0 2.5)

for thresh in "${thresholds[@]}"; do

    out_name="result_th_${thresh}.dat"
    run_config="config_run_${thresh}.ini"

    echo "Generating config for Threshold = $thresh"

    # sedコマンドを用いて __THRESHOLD__ と __OUTFILENAME__ の 2 箇所を同時に置換する。
    # 変数（$thresh など）を展開させるためダブルクォートで 's/.../.../g' 全体を囲む
    sed -e "s/__THRESHOLD__/$thresh/g" \
        -e "s/__OUTFILENAME__/$out_name/g" \
        config_template.ini > "$run_config"

    # 置換された設定の内容を表示して確認する
    cat "$run_config"

    # 生成した設定ファイルを実行プログラムへ渡す
    # ./my_simulation_app --config "$run_config"

done
```

この方法は、設定ファイルを入力とする Python プログラムだけでなく、コンパイル済みのシミュレーションプログラムにも適用できる。各反復で生成した設定ファイルを別名で保存すれば、どの条件で各結果を得たかを後から確認できる。ただし、置換値に `/` や `&` が含まれる場合は、`sed` の区切り文字やエスケープ方法を調整する必要がある。

参考文献

[1] Python Software Foundation, *The Python Tutorial*, https://docs.python.org/3/tutorial/, 閲覧日: 2026-09-14.

[2] Python Software Foundation, *The Python Standard Library*, https://docs.python.org/3/library/, 閲覧日: 2026-09-14.

[3] Project Jupyter, *JupyterLab Documentation*, https://jupyterlab.readthedocs.io/, 閲覧日: 2026-09-14.

[4] Astral, *uv Documentation*, https://docs.astral.sh/uv/, 閲覧日: 2026-09-14.

[5] pyenv contributors, *pyenv README / Documentation*, https://github.com/pyenv/pyenv, 閲覧日: 2026-09-14.

[6] NumPy Developers, *NumPy Quickstart*, https://numpy.org/doc/stable/user/quickstart.html, 閲覧日: 2026-09-14.

[7] pandas development team, *Getting started tutorials*, https://pandas.pydata.org/docs/getting_started/intro_tutorials/, 閲覧日: 2026-09-14.

[8] Polars project, *Concepts*, https://docs.pola.rs/user-guide/concepts/, 閲覧日: 2026-09-14.

[9] Matplotlib Development Team, *Quick start*, https://matplotlib.org/stable/users/explain/quick_start.html, 閲覧日: 2026-09-14.

[10] SciPy community, *SciPy User Guide*, https://docs.scipy.org/doc/scipy/tutorial/, 閲覧日: 2026-09-14.

[11] Astropy Project, *User Guide*, https://docs.astropy.org/en/stable/index_user_docs.html, 閲覧日: 2026-09-14.

[12] Free Software Foundation, *GNU tar Manual*, https://www.gnu.org/software/tar/manual/tar.html, 閲覧日: 2026-08-25.

[13] Zsh Development Group, *Zsh Manual: Expansion*, https://zsh.sourceforge.io/Doc/Release/Expansion.html, 閲覧日: 2026-09-14.

[14] Free Software Foundation, *Bash Reference Manual*, https://www.gnu.org/software/bash/manual/bash.html, 閲覧日: 2026-09-14.

[15] U.S. Geological Survey, *Earthquake Catalog: API Documentation*, https://earthquake.usgs.gov/fdsnws/event/1/, 閲覧日: 2026-09-14.

[16] Homebrew contributors, *Homebrew*, https://brew.sh/, 閲覧日: 2026-09-14.

[17] pyenv contributors, *Suggested build environment*, https://github.com/pyenv/pyenv/wiki#suggested-build-environment, 閲覧日: 2026-09-14.

[18] Astral, *uv: Installation*, https://docs.astral.sh/uv/getting-started/installation/, 閲覧日: 2026-09-14.

[19] Python Software Foundation, *Floating-Point Arithmetic: Issues and Limitations*, https://docs.python.org/3/tutorial/floatingpoint.html, 閲覧日: 2026-09-14.

[20] Python Software Foundation, *PEP 8 — Style Guide for Python Code*, https://peps.python.org/pep-0008/, 閲覧日: 2026-09-14.

[21] Python Software Foundation, *sys — System-specific parameters and functions*, https://docs.python.org/3/library/sys.html#sys.stdin, 閲覧日: 2026-09-14.

[22] NumPy Developers, *NumPy Documentation*, https://numpy.org/doc/stable/, 閲覧日: 2026-09-14.

[23] NumPy developers, *numpy.std*, https://numpy.org/doc/stable/reference/generated/numpy.std.html, 閲覧日: 2026-09-14.

[24] SciPy community, *SciPy Documentation*, https://docs.scipy.org/doc/scipy/, 閲覧日: 2026-09-14.

[25] pandas development team, *Migration guide for the new string data type*, https://pandas.pydata.org/docs/user_guide/migration-3-strings.html, 閲覧日: 2026-09-14.

[26] pandas development team, *Indexing and selecting data*, https://pandas.pydata.org/docs/user_guide/indexing.html, 閲覧日: 2026-09-14.

[27] pandas development team, *pandas.DataFrame.to_latex*, https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_latex.html, 閲覧日: 2026-09-14.

[28] pandas development team, *Copy-on-Write*, https://pandas.pydata.org/docs/user_guide/copy_on_write.html, 閲覧日: 2026-09-14.

[29] pandas development team, *10 minutes to pandas*, https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html, 閲覧日: 2026-09-14.

[30] pandas development team, *Working with missing data*, https://pandas.pydata.org/docs/user_guide/missing_data.html, 閲覧日: 2026-09-14.

[31] pandas development team, *Merge, join, concatenate and compare*, https://pandas.pydata.org/docs/user_guide/merging.html, 閲覧日: 2026-09-14.

[32] pandas development team, *Time series / date functionality*, https://pandas.pydata.org/docs/user_guide/timeseries.html, 閲覧日: 2026-09-14.

[33] Polars project, *Expressions and contexts*, https://docs.pola.rs/user-guide/concepts/expressions-and-contexts/, 閲覧日: 2026-09-14.

[34] Polars project, *Coming from Pandas*, https://docs.pola.rs/user-guide/migration/pandas/, 閲覧日: 2026-09-14.

[35] Polars project, *Lazy API*, https://docs.pola.rs/user-guide/concepts/lazy-api/, 閲覧日: 2026-09-14.

[36] Polars project, *IO*, https://docs.pola.rs/user-guide/io/, 閲覧日: 2026-09-14.

[37] Met Office, *Cartopy Documentation*, https://cartopy.readthedocs.io/stable/, 閲覧日: 2026-09-14.

[38] Matplotlib Development Team, *Plot types*, https://matplotlib.org/stable/plot_types/index.html, 閲覧日: 2026-09-14.

[39] Matplotlib Development Team, *Gallery*, https://matplotlib.org/stable/gallery/index.html, 閲覧日: 2026-09-14.

[40] Matplotlib Development Team, *Cheatsheets*, https://matplotlib.org/cheatsheets/, 閲覧日: 2026-09-14.

[41] SciPy developers, *scipy.optimize.curve_fit*, https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html, 閲覧日: 2026-09-14.

[42] SciPy community, *Statistics*, https://docs.scipy.org/doc/scipy/tutorial/stats.html, 閲覧日: 2026-09-14.

[43] SciPy developers, *scipy.stats.rv_continuous.fit*, https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.rv_continuous.fit.html, 閲覧日: 2026-10-22.

[44] statsmodels developers, *statsmodels.tools.eval_measures.aic*, https://www.statsmodels.org/stable/generated/statsmodels.tools.eval_measures.aic.html, 閲覧日: 2026-10-22.

[45] SciPy community, *Optimization*, https://docs.scipy.org/doc/scipy/tutorial/optimize.html, 閲覧日: 2026-09-14.

[46] SciPy community, *Interpolation*, https://docs.scipy.org/doc/scipy/tutorial/interpolate.html, 閲覧日: 2026-09-14.

[47] SciPy community, *Signal processing*, https://docs.scipy.org/doc/scipy/tutorial/signal.html, 閲覧日: 2026-09-14.

[48] SciPy community, *Integration*, https://docs.scipy.org/doc/scipy/tutorial/integrate.html, 閲覧日: 2026-09-14.

[49] U.S. Geological Survey, *Earthquake Catalog: CSV Format*, https://earthquake.usgs.gov/earthquakes/feed/v1.0/csv.php, 閲覧日: 2026-09-14.

[50] B. Gutenberg and C. F. Richter, “Frequency of Earthquakes in California,” *Bulletin of the Seismological Society of America*, 34(4), 185–188 (1944), https://doi.org/10.1785/BSSA0340040185.

[51] Astropy Project, *Time and Dates*, https://docs.astropy.org/en/stable/time/index.html, 閲覧日: 2026-09-14.

[52] Astropy Project, *Getting Started*, https://docs.astropy.org/en/stable/index_getting_started.html, 閲覧日: 2026-09-14.

[53] Astropy Project, *Units and Quantities*, https://docs.astropy.org/en/stable/units/index.html, 閲覧日: 2026-09-14.

[54] Astropy Project, *Coordinates*, https://docs.astropy.org/en/stable/coordinates/index.html, 閲覧日: 2026-09-14.

[55] Astropy Project, *Tables*, https://docs.astropy.org/en/stable/table/index.html, 閲覧日: 2026-09-14.

[56] Astropy Project, *FITS File Handling*, https://docs.astropy.org/en/stable/io/fits/index.html, 閲覧日: 2026-09-14.

[57] OpenBSD project, *ssh(1) — OpenSSH remote login client*, https://man.openbsd.org/ssh, 閲覧日: 2026-09-14.

[58] rsync contributors, *rsync(1) Manual Page*, https://download.samba.org/pub/rsync/rsync.1, 閲覧日: 2026-09-14.

[59] GitHub, *About repositories: Repository visibility*, https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories, 閲覧日: 2026-09-14.

[60] pybind11 developers, *pybind11 Documentation*, https://pybind11.readthedocs.io/en/stable/, 閲覧日: 2026-09-14.

[61] scikit-build-core developers, *scikit-build-core Documentation*, https://scikit-build-core.readthedocs.io/en/latest/, 閲覧日: 2026-09-14.

[62] Python Software Foundation, *re — Regular expression operations*, https://docs.python.org/3/library/re.html, 閲覧日: 2026-09-14.

[63] Free Software Foundation, *GNU grep Manual*, https://www.gnu.org/software/grep/manual/grep.html, 閲覧日: 2026-09-14.

[64] pandas development team, *pandas Documentation*, https://pandas.pydata.org/docs/, 閲覧日: 2026-09-14.

[65] Polars project, *Polars Documentation*, https://docs.pola.rs/, 閲覧日: 2026-09-14.

[66] Matplotlib Development Team, *Matplotlib Documentation*, https://matplotlib.org/stable/, 閲覧日: 2026-09-14.

[67] Astropy Project, *Astropy Documentation*, https://docs.astropy.org/, 閲覧日: 2026-09-14.

[68] Git project, *Git Documentation*, https://git-scm.com/doc, 閲覧日: 2026-09-14.

[69] GitHub, *GitHub Docs*, https://docs.github.com/, 閲覧日: 2026-09-14.

索引

A

append() 42
Astropy 153
awk 20
axis 73

B

Bash
- case 文 239
- for ループ 239
- if 文 238
- 配列 236

Big-O 記法 182
bounds 145
broadcast ..... → ブロードキャスト

C

CamelCase 53
cat 18
cd 11
chmod 30
CMake 223
CMakeLists.txt 224
cp 14
C++ 拡張モジュール 223
cProfile 187
CSV 61
curl 22
Cython 188

D

dataclass 168
DataFrame 87
datetime 64
ddof 78
dict ..... → 辞書
dictionary ..... → 辞書
DictReader 61
docstring 53, 176
dtype 70

E

editable install 178, 226
elif 44
else 44
except 55
expression 104

F

FileNotFoundError 52
filter 105
FITS 158
fitting ..... → SciPy, curve_fit
f 文字列 40

G

Git 215
git add 217
git clone 219
git commit 217
git config 216
git diff 217
git init 216
git pull 221
git push 221
git remote 219

git rm --cached 218
git status 217
GitHub 179, 215
HTTPS 221
SSH 221
認証 221
.gitignore 218
glob 59
grep 20
GridSpec 122
group_by 106
groupby 91
gzip 23

H

Hammer projection 118
head 19
help() 53

I

if 44
iloc 89
import 50

J

join 107
JSON 63
JupyterLab 207

L

lambda 52
LazyFrame 105
len() 42
less 18
linestyle 132
list ..... → リスト
loc 89
ls 9

M

marker 132
MathText 136
Matplotlib 113
.mplstyle 135
Axes 121
axhline() 116
axvline() 116
errorbar() 116
Figure 121
hist2d() 116
imshow() 116
LogNorm 121
plot() 116
savefig() 126
scatter() 116
mkdir 13
Mollweide projection 118
mv 15

N

NaN 108
None 49
NumPy 69
np.arange 71
np.array 71
np.diff 72
np.histogram 72
np.linspace 71
np.mean 77
np.ones 71
np.sort 72

np.std 77
np.zeros 71

O

object 型 **98**
open **57**

P

p0 ..... **145**
pandas **87**
category 94
chunksize 96
concat() 93
datetime 94
describe() 88
dropna() 91
fillna() 91
index 89
isna() 91
iterrows() 99
itertuples() 99
merge() 93
Parquet 94
pivot_table() 93
read_csv() **95**
to_latex() 97
Parquet **94**, **110**
個人用アクセストークン（PAT） **221**
PATH **25**
pathlib **58**
PEP 8 ..... **53**
time.perf_counter **181**
pickle **64**
pip
install **178**
install -e **178**
Polars **103**
collect() 110
eager API 105
遅延実行 105
map_elements() 111
null 108
Parquet 110
pivot() 107
scan_parquet() 110
pop() **42**
pwd **9**
pybind11 ..... **188**, **223**
PYBIND11_MODULE **225**
pyenv **203**
pyproject.toml **166**, **201**

Q

QTable **157**
Quantity **153**

R

raw string **233**
rcParams 134
re モジュール **233**
fullmatch 233
match 233
search 233
rm **16**
rsync **212**
末尾のスラッシュ 213

S

scikit-build-core **223**
SciPy **141**
scipy.constants **143**
curve_fit **143**

scipy.integrate 143
scipy.interpolate 143
scipy.signal 143
scipy.stats 143
scp 212
sed 241
select 105
Series 87
set 42
SettingWithCopy 98
shape 70
sigma 146
SkyCoord 157
slice ..... → スライス
snake_case 53
sort() 42
sorted() 42
src レイアウト 166
SSH 209
ssh 209
~/.ssh/config 211
ssh-agent 210
ssh-copy-id 210
ssh-keygen 210
stderr ..... → 標準エラー出力
stdin ..... → 標準入力
stdout ..... → 標準出力
subplot_mosaic 122
sys 60

T

Table 157
tail 19
tar 24
tee 29
test 237
Time 155
TimeDelta 156
%timeit 207
try 55
tuple ..... → タプル
TypeError 52

U

UPPER_CASE 53
uv 200
uv.lock 201

V

ValueError 52
venv 199

W

wc 19
wget 22
with_columns 106
WSL 8

Z

zip 24

ア

アクセス権 30
アルゴリズム 182
アンカー（正規表現） 230
アンパック 52
依存関係 166, 200
インデックス 43
エラーバー ..... → Matplotlib, errorbar
演算子 39
オーダー ..... → Big-O 記法

カ

鍵認証 **209**
確率密度 **119**
仮想環境 **34**, **199**
型 **37**
型ヒント **168**, **176**
カラーバー ..... → Matplotlib, colorbar
カラーマップ 117
環境変数 **25**
関数 **48**
　返り値 48
　引数 48, **49**
キーワード引数 **49**
クラス **168**
計算量 **182**
欠損値 **91**
公開 API **174**
公開鍵 **210**
公開範囲 **215**
高速化 **181**
コピー 185
　リスト 42
コマンドライン引数 **236**

サ

算術式展開 **237**
参照線 ..... → Matplotlib, axvline
散布図 ..... → Matplotlib, scatter
シェル **7**
シェルスクリプト **235**
時間窓 **194**
時刻差 **194**
辞書 **41**
　値 41
　キー 41
四則演算 39
シバン **235**
集合 **42**
終了ステータス **30**
条件分岐 **44**
剰余 39
初期値 ..... → `p0`
真偽値 37
ステージングエリア **216**
ストリーミング **186**
スライス **43**, **74**
正規表現 **229**
整数 37
整数除算 39
線図 ..... → Matplotlib, plot
添字 ..... → インデックス

タ

ターミナル **7**
対数軸 **125**
正しさの検証 **196**
タプル **41**
デフォルト引数 **49**
投影法 118
同時観測 **193**
トレースバック **52**

ナ

入出力 **57**

ハ

パイプ **21**
配列 ..... → `ndarray`
パス **12**
パッケージ **164**
バッチ処理 **240**
凡例 ..... → Matplotlib, legend
ヒートマップ ..... → Matplotlib, imshow

比較演算子 44
ヒストグラム **71**, 113
　正規化 **119**
秘密鍵 **210**
ビュー **185**
標準エラー出力 **17**
　Python 60
標準出力 **17**
　Python 60
標準入力 **17**
　Python 60
標準偏差 **78**
フィッティング ..... → SciPy, curve_fit
浮動小数点数 37
負の添字 43
ブロードキャスト **76**
プロファイラ **187**
プロンプト **7**
分割読み込み **96**, **186**
分散 **78**
並列化 **186**
べき乗 39
ベクトル化 **185**
変数 **37**
ベンチマーク 197
ボトルネック **187**

マ

マジックナンバー **54**, **165**
メソッド **42**
メタ文字 **229**
メモリ管理 **185**
文字クラス **231**
モジュール **50**, **164**
文字列 37, **40**
　f 文字列 **40**
　`replace()` **40**
　`split()` **40**
　`strip()` **40**

ヤ

ユニバーサル関数 (ufunc) **72**

ラ

リスト **41**
リスト内包表記 **47**
リポジトリ **215**
リモートリポジトリ **219**
量指定子 **231**
例外 **52**
　例外処理 **55**
ロックファイル **201**
論理演算子 44

ワ

ワーキングツリー **216**
ワイルドカード **17**, 230